

The Backend Engineer's Library

JavaScript, Node.js, and Frontend Frameworks

Volume 19

Contents

JavaScript Engines: V8, SpiderMonkey, and JavaScriptCore

The Event Loop, Microtasks, Macrotasks, and Timers

Node.js Internals: libuv, the Thread Pool, and Native Addons

The JavaScript Type System, Prototypes, Proxies, and the Module System

Async JavaScript: Promises, async/await, Generators, and the Promise Job Queue

Build Tooling and Bundlers: RSPack, Vite, esbuild

React Internals: The Fiber Reconciler, Hooks, Suspense, and Concurrent Features

Vue and Svelte Internals: Reactivity (Proxy vs Signals vs Compile-Time), Virtual DOM vs Compiled Output

Rendering at Scale: SSR, SSG, ISR, Streaming SSR, Hydration, Islands, and Partial Hydration

State Management, Data Fetching, and Caching (TanStack Query, SWR, Zustand/Jotai, Cache Invalidation)

Testing, Linting, and Tooling for Frontend at Scale (Vitest, Playwright, ESLint, TypeScript)

Production Frontend: Observability, Performance (Core Web Vitals, Lighthouse), and Deployment (CDN, Edge, RSC)

Chapter 1 — JavaScript Engines: V8, SpiderMonkey, and JavaScriptCore

What this chapter covers: The anatomy of a modern JavaScript engine — from source text to machine code — and the concrete mechanisms that make dynamic JavaScript fast. We dissect the shared pipeline (parser, AST, bytecode, inline caches, optimizing compilation), then go engine-by-engine through V8 (Ignition / Maglev / TurboFan), SpiderMonkey (Baseline / Ion / Warp), and JavaScriptCore (LLInt / Baseline / DFG / FTL). Along the way you will trigger real hidden-class transitions, watch inline caches move through their states with `d8`, and trace a deoptimization bailout from optimized code back to the interpreter.

Learning goals:

- Explain the full engine pipeline — character stream, tokenizer, parser, AST, bytecode, feedback vectors, and tiered optimizing compilers — and why each stage exists.
 - Describe V8's hidden classes (Maps), transition trees, and property-storage strategies, and predict when an object goes dictionary mode.
 - Trace inline-cache states (uninitialized → monomorphic → polymorphic → megamorphic) and explain how ICs feed speculative optimization.
 - Compare V8, SpiderMonkey, and JSC tiering architectures side-by-side, including what triggers tier-up and tier-down.
 - Use `d8` and engine flags (`--print-ast`, `--print-bytecode`, `--trace-ic`, `--trace-opt`, `--print-opt-code`, `--allow-natives-syntax`) to observe engine internals on real code.
 - Diagnose deoptimization: why it happens, what it costs, and how to avoid it in hot paths at fleet scale.
 - Connect engine behavior to distributed-systems concerns: Node.js fleet performance variance, tail-latency amplification, and reproducible benchmarking.
-

1. Why a Backend Engineer Should Care About Engine Internals

JavaScript is no longer a browser-only language. Every Node.js service, every Cloudflare Worker, every edge function, and every server-side-rendered page runs on one of three engines. At scale — hundreds of services, thousands of containers, rolling deploys — engine behavior becomes infrastructure behavior.

A single polymorphic property load in a hot middleware path can keep V8 from inlining a function, adding microseconds per request. Multiply by 50,000 requests per second across a fleet and you have measurable CPU and tail-latency regression. A deoptimization storm after a deploy that changes object shape can spike p99 latency for minutes until the new code stabilizes in the optimizing compiler. Understanding *why* requires understanding the engine, not just the profiler.

This chapter treats the engine as a distributed-systems component: it is a JIT-compiled runtime with adaptive optimization, speculative assumptions, and fallback paths — not unlike a database query planner that picks execution strategies based on observed statistics.

2. The Universal Pipeline

Every modern engine follows the same high-level pipeline. Implementations differ in naming and number of tiers, but the stages are structurally identical.



Key idea: the engine does not compile JavaScript once. It compiles it *repeatedly*, each time with more information and more aggressive assumptions. The interpreter is not a legacy fallback — it is the profiling tier that makes optimization safe.

Stage	Input	Output	Cost	Optimism
Scanner	characters	tokens	O(n)	none
Parser	tokens	AST	O(n)	none

Stage	Input	Output	Cost	Optimism
PreParser (lazy)	tokens	function skeleton	O(n)	none
Bytecode gen	AST	bytecode + constant pool	O(n)	none
Interpreter	bytecode	results + feedback	low startup	none
Baseline JIT	bytecode + feedback	unoptimized machine code	medium	little
Optimizing JIT	bytecode + stable feedback	highly optimized machine code	high	speculative

The scanner and parser run on the main thread and are latency-sensitive — they block first execution. Bytecode generation is where lazy parsing pays off: V8's preparser skips inner function bodies until they are first called, reducing parse cost on large bundles.

3. From Characters to AST

3.1 Tokenization and Parsing

The scanner converts a UTF-16 character stream into tokens (`Ident`, `Number`, `String`, `Punctuator`, `Keyword`). The parser — a recursive-descent parser in all three engines — builds an Abstract Syntax Tree. V8 has two parsers: the full parser and the preparser. SpiderMonkey and JSC use similar two-pass strategies.

Consider this minimal program:

```
// example.js
function add(a, b) {
  return a + b;
}
const r = add(2, 3);
```

Dumping its AST with V8's `d8` shell (built from the V8 source with `gm x64.debug`):

```
# Build d8 (once – from the v8/ checkout)
# tools/dev/gm.py x64.release

# Print the AST for a snippet
./d8 --print-ast example.js
```

Trimmed output:

```
--- AST ---
FUNC at 0
. KIND 0
. SUSPEND COUNT 0
. NAME "add"
. PARAMS
. . VAR (0x123) "a"
. . VAR (0x124) "b"
. BLOCK NOCOMPLETIONS at -1
. . RETURN at 14
. . . ADD at 14
. . . . VAR PROXY "a" (0x123)
. . . . VAR PROXY "b" (0x124)
...
```

For `add(2, 3)`, the AST encodes `CALL` with `VAR PROXY` for `add` and two `LITERAL 2` / `LITERAL 3` arguments. Every node carries source position for stack traces and scope info for closures.

The raw AST is not executed directly. It is lowered to bytecode (V8 Ignition), or to a structured IR (SpiderMonkey's `BytecodeEmitter`, JSC's `BytecodeGenerator`). Bytecode is compact, position-independent, and easy to interpret — a good profiling substrate.

3.2 Bytecode

V8's Ignition bytecode for `add` looks like this (via `--print-bytecode`):

```
./d8 --print-bytecode --print-bytecode-filter=add example.js
```

[generated bytecode for function: add (0x1a2b...)]

Parameter count 3

Register count 0

Frame size 0

```
12 S> 0x1a2b00001234 @    0 : 0b 02          LdaSmi [2]
      0x1a2b00001236 @    2 : 26 fb          Star r0
      0x1a2b00001238 @    4 : 0b 03          LdaSmi [3]
      ...
14 E> 0x1a2b00001240 @    6 : 0d 02 00        Ldar a1
      0x1a2b00001243 @    9 : 4b fa 01        Add a1, [1]
      0x1a2b00001246 @   12 : ab            Return
```

Each handler is a small native function. `Ldar` loads an argument register, `Add` performs the addition — but not naively. `Add` consults the *feedback vector* for that bytecode offset: has this addition always been `Smi + Smi` (small integers)? If so, the optimizing compiler can emit a single integer add with an overflow check rather than a generic `ToNumber + NumberAdd` call.

SpiderMonkey and JSC bytecode are structurally similar — stack-based in JSC's LLInt, register-based in V8's Ignition — but all three share the feedback-vector idea: every property load, store, call, and arithmetic operation has a slot that records what types were seen. For `Add` at slot 1, the feedback vector might record `{ SmiAdd: 1200, NumberAdd: 0, Generic: 0 }` — if stable and monomorphic, TurboFan can emit a single checked `Int32Add`; otherwise the engine stays in the interpreter and keeps profiling.

4. Hidden Classes, Maps, and Shapes — Making Dynamic Objects Fast

JavaScript objects are hash maps in the spec. Real engines do not implement them as hash maps in the fast path. Instead, they assign every object a *hidden class* — called **Map** in V8, **Shape** in SpiderMonkey, **Structure** in JSC — that describes the object's layout: property names, offsets, and attributes.

4.1 The Core Idea

When you write:

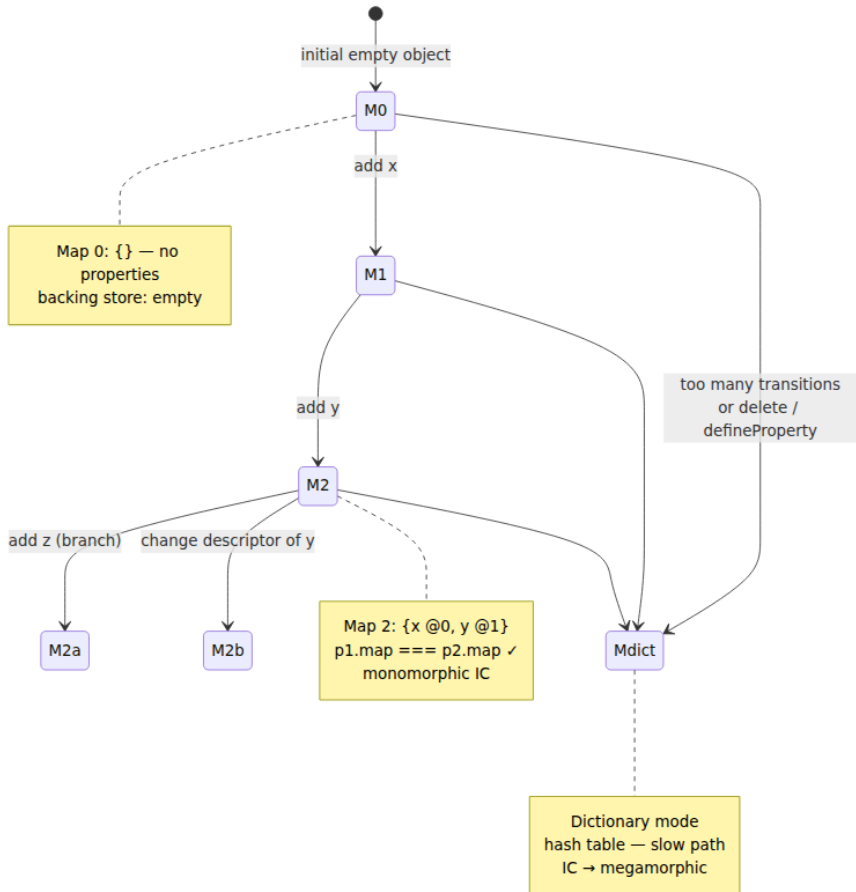
```

function Point(x, y) {
  this.x = x;
  this.y = y;
}

const p1 = new Point(1, 2);
const p2 = new Point(3, 4);

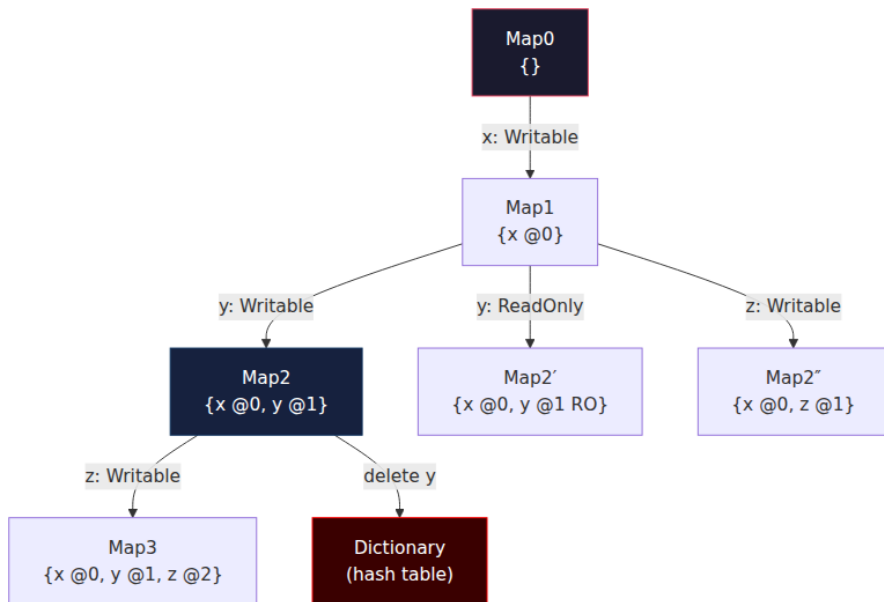
```

Both `p1` and `p2` share the same Map because they added properties in the same order. The Map says: offset 0 is `x`, offset 1 is `y`, both writable data properties, stored inline. Property access `p1.x` becomes a single indexed load — no hash lookup — guarded by a Map check.



4.2 Transition Trees

Maps form a *transition tree* (a trie). Each edge is labeled by the property being added and its attributes.



Critical property: objects that follow the same transition path *share* Maps. The engine can then optimize for a specific Map — if `p1.map === p2.map`, the offset of `x` is a compile-time constant. If a third object `p3` adds `y` before `x`, it takes a different branch and gets a different Map. Code that handles both `p1` and `p3` becomes polymorphic.

Concrete example showing the cost:

```

// monomorphic – one Map
function sumPoints(points) {
  let s = 0;
  for (const p of points) s += p.x + p.y; // IC sees one Map
  return s;
}

// polymorphic – two Maps alternating
const a = { x: 1, y: 2 }; // Map2
const b = { y: 2, x: 1 }; // Map2" – different order!
sumPoints([a, b, a, b, a, b]); // IC becomes polymorphic

// megamorphic – many Maps
const many = [];
for (let i = 0; i < 200; i++) {
  const o = {};
  o["prop" + i] = i; // each object has a distinct Map
  many.push(o);
}

```

Observe the transition with `--allow-natives-syntax` (requires `d8` built with natives syntax):

```
// map-transitions.js – run with: d8 --allow-natives-syntax map-
transitions.js

function Point(x, y) { this.x = x; this.y = y; }

const p1 = new Point(1, 2);
console.log(%HaveSameMap(p1, new Point(3, 4))); // true – same
transitions

const q = {};
q.y = 2;
q.x = 1;
console.log(%HaveSameMap(p1, q));                // false – y before x

// Inspect maps directly
%DebugPrint(p1);
%DebugPrint(q);

// Adding a property out of order forces a new branch
const r = new Point(5, 6);
r.z = 7;    // Map2 → Map3 (x,y → x,y,z)
console.log(%HaveSameMap(r, p1));                // false – r has extra
transition

// Deleting forces dictionary mode
delete r.z;
%DebugPrint(r); // → map transitions to dictionary / hash table
```

```
./d8 --allow-natives-syntax --trace-maps map-transitions.js 2>&1 |
head -40
```

```
[TraceMaps: InitialMapCreation Map 0x1a2b... for Point]
[TraceMaps: Transition from Map 0x1a2b... to Map 0x1a2c... for "x"]
[TraceMaps: Transition from Map 0x1a2c... to Map 0x1a2d... for "y"]
[TraceMaps: HaveSameMap true]
[TraceMaps: Transition from Map 0x... to Map 0x... for "y"] // q.y
[TraceMaps: Transition from Map 0x... to Map 0x... for "x"] // q.x –
different branch
[TraceMaps: HaveSameMap false]
```

4.3 Storage Details

- **Inline vs. out-of-line:** V8 stores the first N properties (typically 4–8 depending on object size) directly in the object header (in-object properties). Overflow goes to a separate `PropertyArray`.

- **Elements vs. properties:** Integer-indexed properties (`arr[0]`) live in the *elements* backing store, which has its own kind lattice (`PACKED_SMI_ELEMENTS` → `PACKED_DOUBLE_ELEMENTS` → `PACKED_ELEMENTS` → `HOLEY_*` → `DICTIONARY_ELEMENTS`). A single `arr[1000000] = 1` or `delete arr[0]` can hole-ify or dictionary-ify an array permanently.
- **Attributes matter:** Making a property non-writable, non-enumerable, or accessor-based creates a distinct Map branch. `Object.defineProperty(obj, "x", { value: 1, writable: false })` is a different transition than `obj.x = 1`.

```
// Elements kind transitions – observable via %HasFastElements /
%DebugPrint
const arr = [1, 2, 3];
%DebugPrint(arr);           // PACKED_SMI_ELEMENTS

arr.push(4.5);
%DebugPrint(arr);           // PACKED_DOUBLE_ELEMENTS (widened)

arr.push("hello");
%DebugPrint(arr);           // PACKED_ELEMENTS

arr[100] = 99;               // hole
%DebugPrint(arr);           // HOLEY_ELEMENTS – deoptimization risk
for loops
```

4.4 Why This Matters at Scale

Hidden classes are a *global* resource. Two teams adding properties to the same shared object type in different orders — say, two microservices constructing the same event envelope — create polymorphism that slows every consumer. Enforcing a single factory function or using `class` syntax (which guarantees consistent property order) is not a style preference; it is a performance contract.

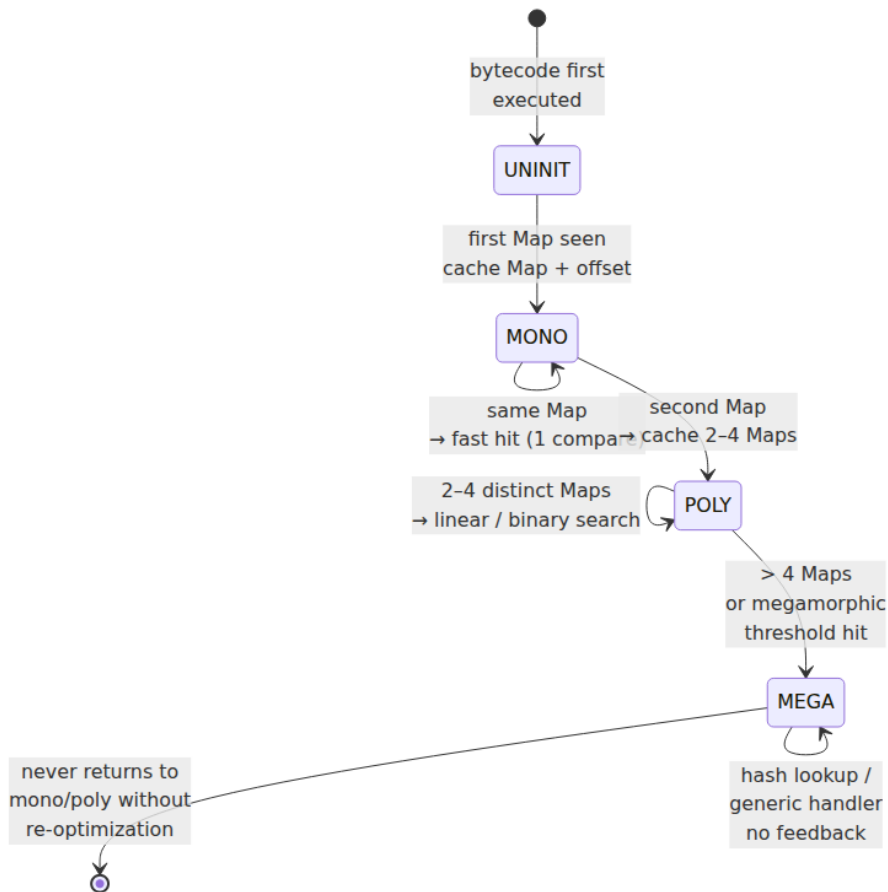
5. Inline Caches

Hidden classes make property access fast *if* the engine knows which Map to expect. Inline caches are the mechanism that remembers.

5.1 How an IC Works

Every property access site in bytecode has an IC slot. The first time the code runs, the slot is **uninitialized**. On the first execution it records the

Map it saw and the offset. On subsequent executions, it compares the object's Map against the cached one — a single pointer comparison — and if it matches, loads at the cached offset immediately.



V8's thresholds (as of mid-2025, subject to tuning): monomorphic is 1 Map, polymorphic caches up to 4 Maps (configurable), beyond that → megamorphic. SpiderMonkey and JSC have similar cutoffs but different constants.

5.2 Observing ICs with d8

```
// ic-demo.js
function getX(o) { return o.x; }

const a = { x: 1, y: 2 };
const b = { y: 2, x: 1 }; // different Map

// Warm up - monomorphic
for (let i = 0; i < 10000; i++) getX(a);

// Now introduce a second Map - polymorphic
for (let i = 0; i < 10000; i++) getX(i % 2 === 0 ? a : b);

// Megamorphic - many distinct Maps
for (let i = 0; i < 10000; i++) {
  const o = {};
  o["k" + (i % 20)] = i; // 20 different Maps - over threshold
  getX(o);
}
```

```
./d8 --trace-ic ic-demo.js 2>&1 | grep -E "GetX|LoadIC|StoreIC|
megamorphic"
```

Representative trimmed output:

```
[TraceIC] LoadIC at getX:0 (uninitialized -> monomorphic)
map=0x1a2b... offset=8
[TraceIC] LoadIC at getX:0 monomorphic hit (map=0x1a2b... count=9999)
[TraceIC] LoadIC at getX:0 monomorphic miss -> polymorphic (maps=[0x1a2
b..., 0x1a2c...])
[TraceIC] LoadIC at getX:0 polymorphic hit
[TraceIC] LoadIC at getX:0 polymorphic -> megamorphic (exceeded max pol
ymorphic maps)
[TraceIC] LoadIC at getX:0 megamorphic (generic handler, no map check)
```

Additional flags for deeper inspection:

```

# Verbose IC + map transitions + optimization history
./d8 --trace-ic --trace-maps --trace-opt --trace-deopt ic-demo.js 2>&1
| less

# Dump the feedback vector for a function
./d8 --allow-natives-syntax --print-feedback-vectors ic-demo.js
# %DebugPrint(getX) also shows feedback vector slots

# Per-site IC state via internal method (when available)
./d8 --allow-natives-syntax -e '
  function getX(o){ return o.x; }
  const a={x:1}, b={x:2};
  %DebugPrint(getX);
'

```

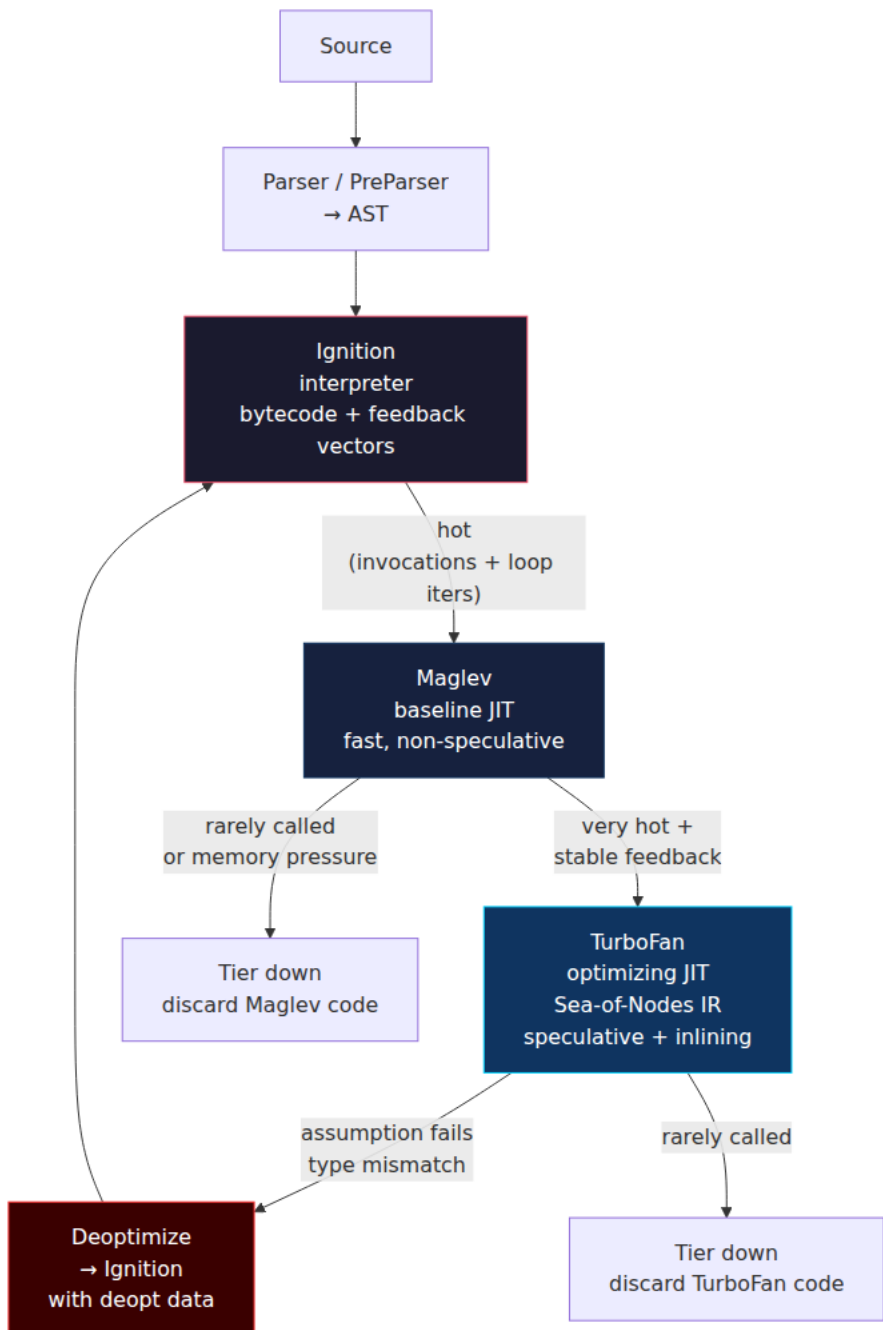
5.3 ICs and Speculative Optimization

IC feedback is not just an optimization itself — it is the *input* to the optimizing compiler. TurboFan, Ion, and FTL all read feedback vectors to decide what types to speculate on. A monomorphic `o.x` load becomes "guard that `o` has Map M, then load at offset 8" — no hash, no branch on type. A megamorphic site becomes "call the generic runtime handler" — no speculation possible, no inlining.

This is why a single megamorphic site in a hot function can prevent the entire function from being optimized, or force it to deoptimize repeatedly. The IC state is the canary.

6. V8 — Ignition, Maglev, and TurboFan

V8 (Chrome, Node.js, Deno, Cloudflare Workers) has the most documented and most tiered architecture. As of 2024–2025, production V8 uses three execution tiers plus the parser:



6.1 Ignition — The Interpreter

Ignition is a register-based bytecode interpreter with an accumulator model. Bytecode handlers are hand-written in a DSL called `CodeStubAssembler` and compiled ahead of time. Every bytecode dispatches through a jump table; feedback is collected inline in `FeedbackVector` slots.

- **Startup:** Ignition code is generated synchronously on first parse — no background thread needed. This keeps first-paint latency low.
- **Feedback:** Each `LdaNamedProperty`, `CallProperty`, `Add`, etc. has a slot recording Maps, call targets, and arithmetic types.
- **On-Stack Replacement (OSR):** Hot loops can OSR from Ignition directly mid-iteration without waiting for the function to be re-entered.

6.2 Maglev — Mid-Tier Baseline JIT (Since V8 11.x)

Maglev sits between Ignition and TurboFan. Before Maglev, there was Sparkplug (a non-optimizing baseline compiler). Maglev replaced Sparkplug in most configurations because it is faster to compile *and* faster to execute — it uses faster register allocation and copies Ignition's bytecode with light optimization, without speculative guards.

- **When it triggers:** After a function has been executed N times or a loop has iterated M times (heuristics tuned per platform; roughly single-digit invocations on desktop, higher on mobile).
- **What it does:** Translates Ignition bytecode to machine code one-to-one, performs simple constant folding, and retains IC feedback collection — but does not speculate on types. The resulting code is faster than interpretation but not yet near peak.
- **Why it matters:** Maglev reduces the time spent in the interpreter before TurboFan kicks in. For short-lived Node.js request handlers that never get hot enough for TurboFan, Maglev is the peak tier.

6.3 TurboFan — The Optimizing Compiler

TurboFan is a sea-of-nodes optimizing compiler. It consumes Ignition bytecode plus the feedback vector, builds a graph IR, and applies ~100 optimization passes: inlining, escape analysis, loop unrolling, range analysis, load elimination, and type specialization.

The critical mechanism is *speculation*: TurboFan emits guards — cheap Map checks, type checks, bounds checks, overflow checks — and assumes they pass. If they do, the optimized code is dramatically faster (inlined property loads, unboxed numbers, eliminated bounds checks). If any guard fails, the entire optimized frame is deoptimized. The pipeline is Bytecode + FeedbackVector → Sea-of-Nodes IR graph → inlining and escape analysis → type/range narrowing → optimized machine code with guards; if all guards pass the fast path runs with no runtime calls, otherwise the engine bails out to Ignition via the deoptimization table.

Example of what TurboFan can elide:

```
// Before optimization - generic
function sum(arr) {
  let total = 0;
  for (let i = 0; i < arr.length; i++) total += arr[i];
  return total;
}
```

With stable feedback (`arr` always `PACKED_SMI_ELEMENTS`, `total` always `Smi`):

- `arr.length` load → Map guard + inline offset load (no dictionary lookup).
- `arr[i]` → bounds check + single indexed load (no elements-kind check in the hot loop if hoisted).
- `total += ...` → `Int32Add` with overflow check; overflow bails to `BigInt`/double path.
- Loop → unrolled or strength-reduced if profitable.

If any assumption is wrong — `arr` arrives with `PACKED_DOUBLE_ELEMENTS`, or `total` overflows `Smi` range — the guard fails and `sum` deoptimizes.

6.4 Observing Tiers with d8

```
# Trace optimization, deoptimization, and tier transitions
./d8 --trace-opt --trace-deopt --trace-maglev --trace-turbo ic-demo.js

# Print optimized code (Intel syntax) for a function named sum
./d8 --print-opt-code --print-opt-code-filter=sum --code-comments sum-
demo.js

# Force optimization / deoptimization via natives syntax
./d8 --allow-natives-syntax -e '
  function sum(arr) {
    let t = 0;
    for (let i = 0; i < arr.length; i++) t += arr[i];
    return t;
  }
  const a = [1,2,3,4,5];
  // Warm up through Ignition → Maglev → TurboFan
  for (let i = 0; i < 20000; i++) sum(a);
  %OptimizeFunctionOnNextCall(sum);
  sum(a); // next call triggers TurboFan compilation
  console.log(%GetOptimizationStatus(sum)); // bitmask: optimized?
  // Break the assumption – holey array forces deopt
  a[100] = 1;
  sum(a);
  console.log(%GetOptimizationStatus(sum));
'
```

`%GetOptimizationStatus` returns a bitmask (V8 version-dependent — consult `runtime.h`):

Bit	Meaning
1	Function is optimized
2	Function is optimized by TurboFan
4	Function was never optimized
16	Function may have been deoptimized

Printed optimized-code snippet (trimmed, x64):

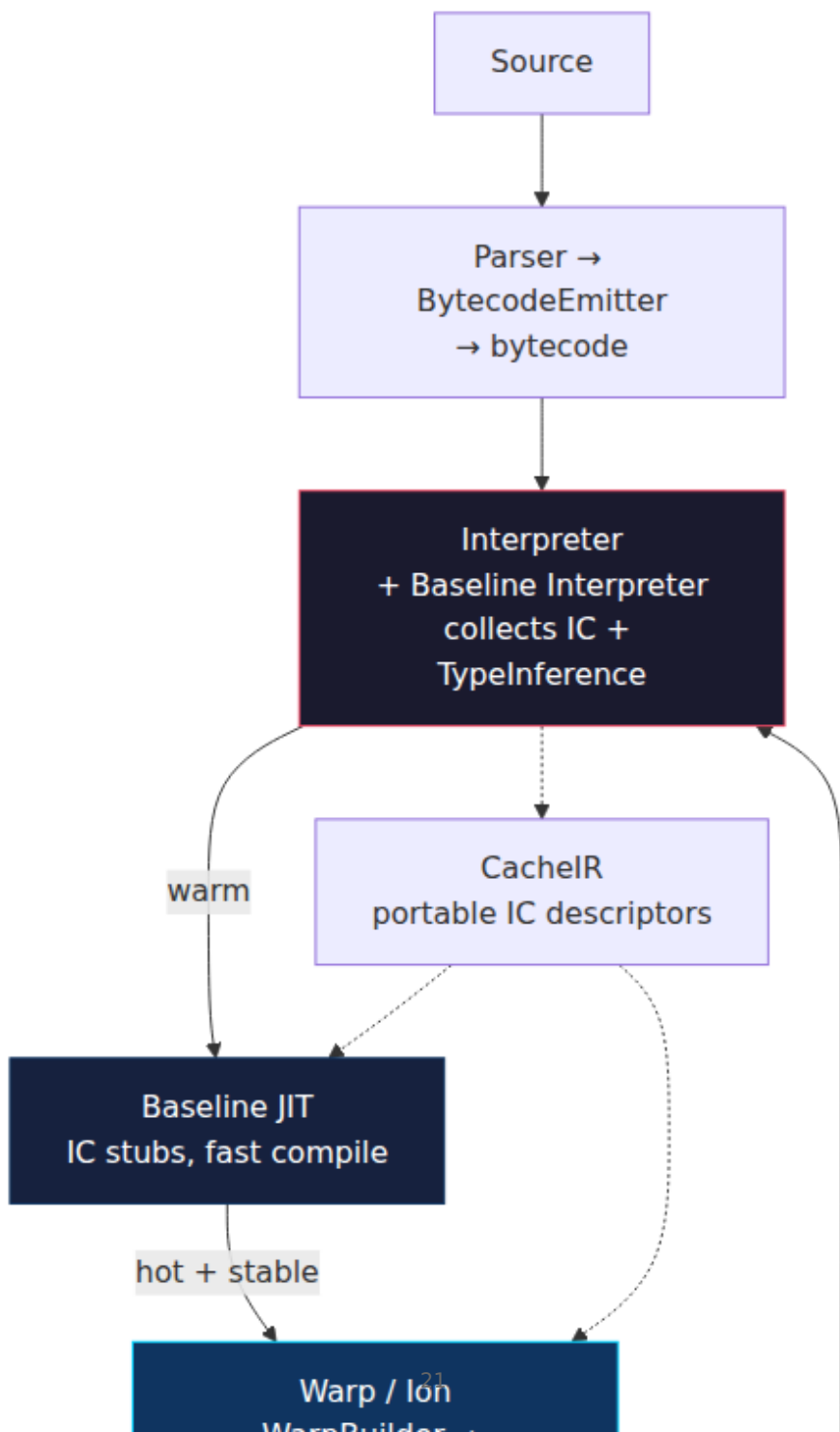
```

; TurboFan code for sum - prologue
0x1a2b0000b000    cmp    [rdi+0x18], MapForPackedSmiElements ; Map
guard
0x1a2b0000b008    jnz    DeoptimizeBailout                      ; bailout
if wrong Map
0x1a2b0000b00e    mov    rbx, [rdi+0x10]                                ; load
elements backing store
0x1a2b0000b012    mov    ecx, [rdi+0x14]                                ; load
length
; ... loop body: single add, no runtime call ...
0x1a2b0000b040    jo     DeoptimizeBailout                                ; overflow check

```

7. SpiderMonkey — Baseline, Ion, and Warp

SpiderMonkey (Firefox, SpiderNode) has converged on a similar shape but with different naming and history.



7.1 Baseline Interpreter and Baseline JIT

SpiderMonkey's Baseline tier is split: a Baseline Interpreter (the default execution tier since ~2022) and a Baseline JIT that compiles bytecode to machine code using generated IC stubs. The key abstraction is **CacheIR**: instead of emitting machine code directly for each IC, the engine emits a portable CacheIR sequence (a linear IR describing the cache logic). CacheIR is then compiled to machine code by the Baseline JIT or consumed by Warp/Ion for optimization. This makes IC logic shareable across tiers.

7.2 Warp and IonMonkey

Warp (shipped ~2021) replaced the older Ion Builder. Warp transpiles bytecode + CacheIR + type inference data into IonMonkey's MIR (Middle-level IR). The WarpBuilder runs on a background thread; when it finishes, the function is patched to jump to Ion code.

- **MIR** — SSA-based, high-level, speculative (type guards, shape guards).
- **LIR** — Low-level IR after register allocation.
- **Bailouts** — Ion frames capture snapshot state so they can reconstruct interpreter/Baseline frames on guard failure. SpiderMonkey bails to Baseline, not directly to the interpreter, which makes re-optimization faster.

Warp's advantage is compile speed: by reusing CacheIR as its input, it avoids re-analyzing every IC from scratch. This was critical for large web apps where Ion's old builder was a compile-time bottleneck.

7.3 Tuning

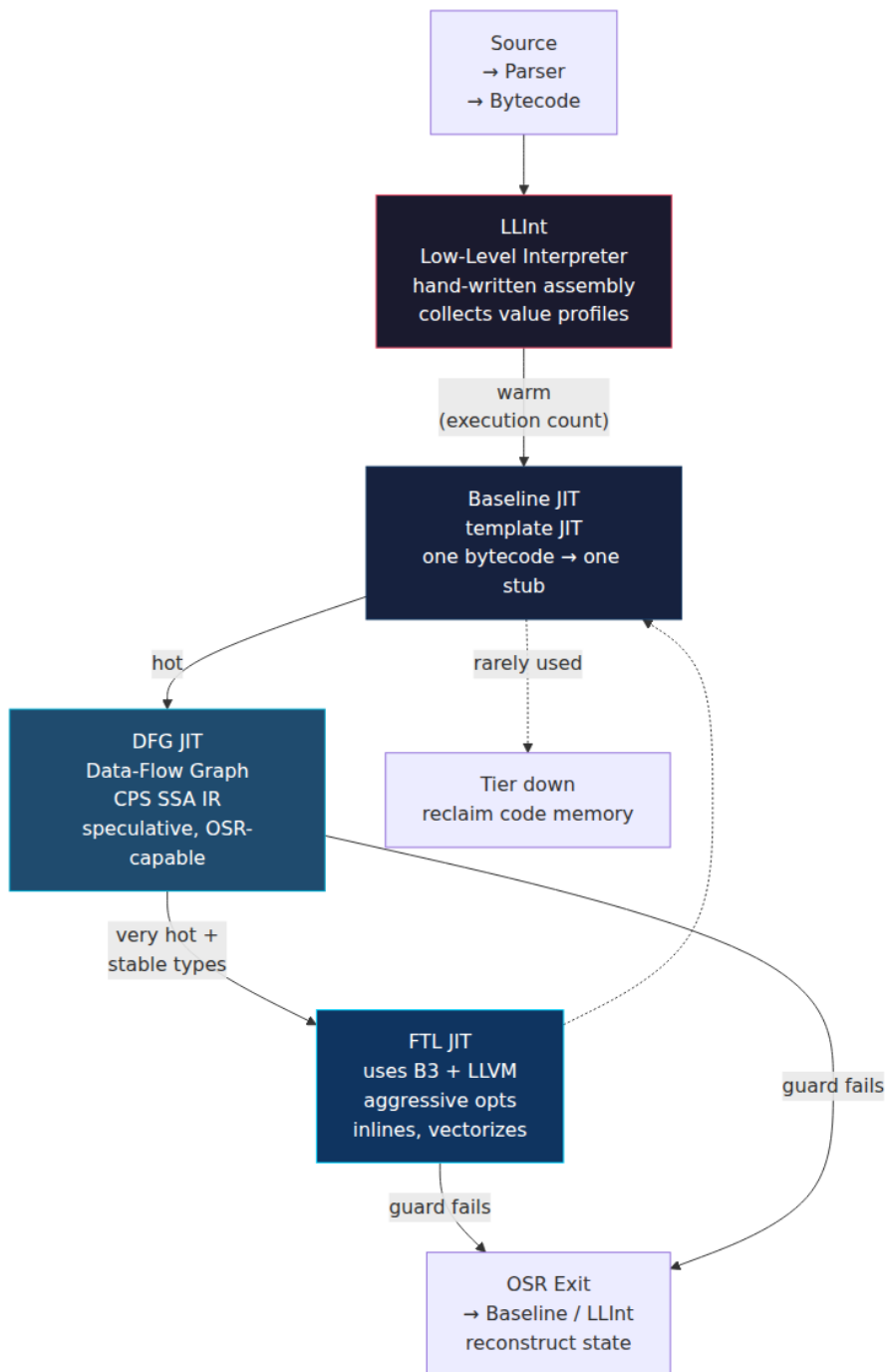
SpiderMonkey exposes tiering decisions via prefs and env vars, but unlike `d8`, there is no single `--trace-opt` flag for SpiderMonkey's `js` shell — instead:

```
# SpiderMonkey js shell (built from mozilla-central)
./js --help | grep -i baseline

IONFLAGS=help ./js script.js      # Ion optimization trace
CACHEIR_LOGS=1 ./js script.js     # CacheIR stub generation
```

8. JavaScriptCore — LLInt, Baseline, DFG, and FTL

JavaScriptCore (Safari, Bun) has *four* execution tiers — the most of any engine — betting that finer granularity wins on heterogeneous Apple hardware (high-efficiency vs. high-performance cores).



8.1 LLInt — The Assembly Interpreter

LLInt is not a bytecode loop in C — it is hand-written assembly (generated from `offlineasm` templates) that dispatches bytecode with computed `goto` / jump tables. Its job is to start executing immediately (no JIT compile delay) and to profile values flowing through each bytecode: what types appear at each `GetById`, what callees appear at each `Call`.

8.2 Baseline JIT

The Baseline JIT is a *template JIT*: each bytecode has a pre-written assembly template that is stamped out and patched together. It is very fast to compile (no IR, no optimization) and produces code roughly 2-3× faster than LLInt. It continues to collect profiling data.

8.3 DFG JIT

DFG (Data Flow Graph) builds a CPS-style SSA graph from bytecode + profiling data. It is speculative: it emits `CheckStructure` / `CheckType` guards and optimizes under the assumption they pass. DFG supports OSR entry (compile a hot loop without waiting for the function to re-enter) and OSR exit (bail mid-function back to Baseline). A key JSC strength is DFG's handling of *polyvariant inlining* — it can inline the same callee at multiple call sites with different speculative types.

8.4 FTL JIT

FTL (Faster Than Light) is the top tier. DFG code that remains hot is recompiled via two backends: **B3** (Bare Bones Backend, JSC's own optimizing IR) and optionally **LLVM** for the most aggressive optimizations (loop vectorization, heavy inlining, global value numbering). FTL can do optimizations that DFG cannot afford because FTL compilation runs on a background thread with fewer latency constraints. The tradeoff is compile time and memory — FTL code is large, so JSC is aggressive about discarding it when functions cool down.

8.5 Choosing a Tier

JSC's tiering is driven by execution counters + profiling stability + memory pressure. On iOS, where memory is constrained, JSC may never promote to FTL; on macOS with abundant memory, it does eagerly. This makes performance *device-dependent* — a hot path that is FTL-optimized

on a developer's MacBook may remain in DFG or Baseline on a low-end device or in a memory-constrained container.

8.6 Cross-Engine Tier Comparison

Dimension	V8	SpiderMonkey	JSC
Interpreter	Ignition (bytecode)	Baseline Interpreter	LLInt (assembly)
Baseline JIT	Maglev	Baseline JIT (CacheIR)	Baseline JIT (template)
Optimizing JIT(s)	TurboFan	Ion via Warp	DFG → FTL (B3/LLVM)
IC abstraction	Feedback vectors + handlers	CacheIR	Value profiles + structure checks
Hidden class name	Map	Shape	Structure
Deopt target	Ignition	Baseline	Baseline / LLInt
OSR entry	yes (Ignition→TurboFan, loop)	yes (Warp)	yes (DFG, FTL)
Background compile	TurboFan, Maglev	WarpBuilder	DFG, FTL

9. Deoptimization — When Speculation Fails

Optimized code is correct *only* as long as every guard holds. When a guard fails — a Map changed, a type was unexpected, an array went holey, an overflow occurred — the engine must abandon the optimized frame and reconstruct interpreter (or baseline) state. This is **deoptimization** (V8/TurboFan), **bailout** (SpiderMonkey/Ion), or **OSR exit** (JSC/DFG/FTL).

9.1 What Triggers a Bailout

- Map mismatch: a monomorphic site saw a new Map.
- Type mismatch: `Smi + Smi` guard failed because a `double` arrived.

- Elements kind transition: `PACKED_SMI_ELEMENTS` → `HOLEY_DOUBLE_ELEMENTS` .
- Overflow: `Int32Add` overflowed into double/BigInt range.
- `delete` , `Object.defineProperty` , `Object.setPrototypeOf` — all can invalidate Maps/structure assumptions.
- `eval` , `with` , `arguments` aliasing (spoils scope analysis).

9.2 The Bailout Mechanism

The optimizing compiler embeds *deoptimization data* — a side table mapping each guard site to the interpreter frame it should materialize if that guard fails. When a guard fails, the runtime:

1. Captures the optimized frame's registers and spills.
2. Looks up the deopt table for that program counter.
3. Materializes interpreter frames (and inlined callee frames) from the captured state.
4. Jumps to the interpreter at the corresponding bytecode offset.

```

sequenceDiagram
    participant OPT as TurboFan code
    (optimized sum)
    participant GUARD as Guard:
Map == M2?
    participant RUNTIME as Deoptimization
runtime
    participant DEOPT as Deopt table
+ side exits
    participant INTERP as Ignition
interpreter

    OPT->>GUARD: load arr[0]
check arr.map == M2
    GUARD-->>OPT: miss (arr.map == M3)
    OPT->>RUNTIME: trap – guard failed
at pc 0x1a2b...b012
    RUNTIME->>DEOPT: lookup deopt id 42
→ bytecode offset 9
→ register mapping
    DEOPT-->>RUNTIME: frame translation:
rax=acc, rbx→r0, ...
    RUNTIME->>INTERP: materialize
interpreter frame
+ inlined frames
    INTERP->>INTERP: resume at
bytecode offset 9
generic Add handler
    Note over OPT,INTERP: Optimized code stays
installed but is now
marked for deopt.
Re-opt requires
new feedback.

```

Cost: the bailout itself is microseconds, but the *aftermath* is expensive — the function is now running in the interpreter/baseline tier. If it remains hot, it will be re-optimized, but re-optimization re-profiles from scratch and may take thousands of future calls to re-stabilize. A tight loop that alternates types can ping-pong between optimized and deoptimized code — a *deoptimization storm*.

9.3 Observing Deoptimization

```
// deopt-demo.js
function add(a, b) { return a + b; }

// Stabilize as Smi + Smi
for (let i = 0; i < 20000; i++) add(i | 0, 1);

%OptimizeFunctionOnNextCall(add);
add(1, 2); // → optimized for Smi

console.log("status before deopt:", %GetOptimizationStatus(add));

// Break the assumption – strings force generic path
add("hello", " world");
console.log("status after deopt:", %GetOptimizationStatus(add));
```

```
./d8 --allow-natives-syntax --trace-opt --trace-deopt deopt-demo.js
2>&1
```

```
[marking add for optimization]
[completed optimizing add]
[status before deopt: 129]    # 0x81 – optimized by TurboFan
[deoptimize: add (reason: not a Smi, bailout id 3)]
[status after deopt: 97]     # deoptimized, now unoptimized
```

Additional diagnostics:

```
# Count deopts and reasons
./d8 --trace-deopt --trace-opt-stats deopt-demo.js 2>&1 | grep deopt

# Full IR + deopt table for a function
./d8 --print-opt-code --print-deopt-stress deopt-demo.js 2>&1 | less

# Force stress deoptimization (every eligible guard bails) – for
testing
./d8 --deopt-every-n-times=1000 stress-test.js
```

9.4 Avoiding Deoptimization in Hot Code

Principles (engine-agnostic):

- **Monomorphism:** Keep call sites and property accesses monomorphic. One Map per site, one callee per call site. Use factory functions and `class` to guarantee it.

- **Stable types:** Do not mix `number` and `string` through the same `+` site; do not mix `Smi` and `double` through the same loop if the loop is TurboFan-optimized.
- **Stable shapes:** Do not `delete` properties, do not `Object.defineProperty` after construction, do not add properties conditionally in different orders. Seal the shape at construction time.
- **Stable elements kinds:** Initialize arrays fully; avoid holes; do not mix element types in performance-critical arrays. Pre-allocate with `new Array(n)` only if you fill every slot before use.
- **No prototype mutation:** `Object.setPrototypeOf` invalidates structure assumptions globally — never in hot code.
- **Minimize arguments and eval :** Both disable many optimizations in the enclosing function.

```
// ☐ Deopt-prone: conditional shape
function makeEvent(type, payload) {
  const e = { type };
  if (payload) e.payload = payload; // sometimes 1 prop, sometimes 2 →
2 Maps
  if (type === "error") e.code = 500; // 3rd branch
  return e;
}

// ☐ Stable shape: always same properties, same order
function makeEventStable(type, payload) {
  return {
    type,
    payload: payload ?? null, // always present
    code: type === "event" ? null : 500,
  };
}
// Monomorphic: every call site sees one Map. TurboFan inlines the
loads.
```

For hot loops, the most common deoptimization root cause in production Node.js services is *elements kind polymorphism*: an array that is usually `PACKED_SMI_ELEMENTS` but occasionally receives a `double` or a hole. The mitigation is to either keep arrays homogeneous or to split hot paths by kind.

10. Putting It Together — Tier-Up and Tier-Down Lifecycle

A function does not stay at one tier forever. Counters, feedback stability, memory pressure, and guard failures drive continuous tier transitions — cold (Ignition / LLInt) warms to Maglev / Baseline JIT after a few invocations or loop iterations, promotes to TurboFan / Ion / DFG when feedback is hot and stable, and can reach FTL or a re-optimized TurboFan tier when very hot with no recent bailouts. Any guard failure bails back to the warm tier; cool-down or memory pressure discards JIT code and tiers down to cold, with re-invocation restarting the cycle through re-profiling.

In V8 specifically, as of 2025:

- **Ignition → Maglev:** ~1-5 invocations or ~10 loop iterations (varies by embedded vs. desktop heuristics).
- **Maglev → TurboFan:** ~100-10,000 invocations depending on function size and feedback stability; compilation runs on a background thread (concurrent recompilation). Short functions tier up faster.
- **Tier down:** JIT code is held weakly. On memory pressure or after a period of non-use, the GC discards it. The next call re-enters Ignition and the cycle restarts. Node.js services with large codebases and infrequent code paths benefit from tier-down because it bounds code memory — but it means p99 latency includes re-optimization jitter after a cold period.

JSC and SpiderMonkey follow the same lifecycle with different thresholds and with LLInt/Baseline as the cold tier.

11. Instrumentation Cookbook

A runnable checklist of `d8` / `js` / `jsc` flags useful during investigation. These require shells built from engine source — not the `node` binary (Node embeds V8 but does not expose most `d8` flags; use `d8` or a `node --allow-natives-syntax` subset).

```

# --- V8 (d8) ---
# AST + bytecode
./d8 --print-ast script.js
./d8 --print-bytecode --print-bytecode-filter=myFunction script.js

# Feedback vectors and maps
./d8 --allow-natives-syntax --print-feedback-vectors script.js
./d8 --allow-natives-syntax --trace-maps script.js

# Inline caches
./d8 --trace-ic script.js

# Optimization and deoptimization
./d8 --trace-opt --trace-deopt --trace-opt-stats script.js
./d8 --print-opt-code --print-opt-code-filter=myFunction --code-
comments script.js
./d8 --trace-maglev --trace-turbo script.js # tier transitions
(version-dependent)

# Natives syntax helpers (in JS)
# %HaveSameMap(a, b)           - do a and b share a Map?
# %HasFastProperties(obj)      - is obj still in fast mode?
# %HasFastElements(arr)       - are elements still fast?
# %DebugPrint(obj)            - dump Map, properties, elements kind
# %OptimizeFunctionOnNextCall(fn)
# %GetOptimizationStatus(fn)  - bitmask
# %NeverOptimizeFunction(fn)  - pin to Ignition (for baselines)
# %DeoptimizeFunction(fn)     - force deopt (for testing bailout
recovery)

# Stress / fuzz tiering
./d8 --deopt-every-n-times=1000 --opt fid=script.js

# --- SpiderMonkey (js shell) ---
IONFLAGS=help ./js script.js
CACHEIR_LOGS=1 ./js script.js

# --- JSC (jsc shell) ---
# jsc --help lists --thresholdForJITAfterWarmUp, etc.
./jsc --dumpBytecode script.js
./jsc --logExecutableAllocation script.js

```

In production Node.js (where `d8` flags are unavailable), the closest equivalents are:


```

# Node's built-in V8 diagnostics
node --trace-opt --trace-deopt app.js 2>&1 | head -100      # some V8
builds expose these
node --trace-ic app.js                                       # if built
with v8_enable_trace_ic

# Perf + V8 runtime call stats
node --perf-basic-prof --interpreted-frames-native-stack app.js
node --runtime-call-stats app.js 2>&1 | grep -i "Map\|IC\|Deopt"

# Sampling profiler → look for %Deoptimize, %OptimizeFunctionOnNextCall
markers
node --prof app.js && node --prof-process isolate-*.log | less

```

12. Distributed-Systems Lens — Engines at Fleet Scale

Engines are deterministic per input, but fleet behavior is not. The same JavaScript bundle runs on thousands of containers with different CPU features, memory limits, and warm-up histories. Several engine properties amplify into systemic effects.

12.1 Warm-Up Skew and Tail Latency

Tiered compilation means the first N requests to a freshly started Node.js pod run slower (Ignition/Maglev) than the next N (TurboFan). During a rolling deploy, pods are at different warm-up phases — the load balancer sees latency variance that is not caused by downstream dependencies. Canary analysis that compares canary vs. baseline p50 may miss the effect; p95/p99 diverges first.

Mitigations: *warm-up probes* that hit critical endpoints before the pod joins the pool; *code caching* (`v8.compile.cache` / Node's `--compile-cache`); and avoiding deploys that change hot object shapes, which would force every pod through a new optimization cycle.

12.2 Performance Non-Reproducibility

Two engineers benchmarking the same endpoint can get different numbers because TurboFan decisions depend on *history*: what argument types and Maps were seen before the measurement. A microbenchmark that runs in isolation often optimizes differently than the same function in production where it shares ICs with other callers. Always benchmark with production-like feeding: warm the function with realistic Maps and

types before timing, and run `--trace-deopt` to verify the optimized code survived the measurement.

12.3 One Megamorphic Site, Many Callers

A shared utility — say, `get(obj, path)` in a large monorepo — accumulates IC states from every caller. If any team passes many heterogeneous Maps through it, the site goes megamorphic and *every* caller pays, even the ones that were monomorphic before. This is the IC equivalent of a noisy neighbor. The fix is to split the generic path from the fast path (e.g., a monomorphic fast path for the common `Map` plus a generic fallback), or to avoid the shared generic accessor in hot code entirely.

12.4 Elements Kind as a Distributed Contract

An event queue service that serializes arrays as JSON and a consumer that deserializes them share an implicit contract: the elements kind of those arrays. If the producer starts emitting `null` holes for missing values, every consumer that does `for (let i = 0; i < arr.length; i++)` may tier-down from an optimized `PACKED_SMI_ELEMENTS` loop to a holey-elements loop with a hole check per iteration. Schema validation at the boundary (e.g., "arrays are dense and homogeneous") is also a performance guarantee.

12.5 Engine Version as a Deploy Risk

V8, SpiderMonkey, and JSC ship with the embedding runtime. A Node.js minor upgrade (e.g., Node 20.11 → 20.13) may include a V8 point release that changes Maglev heuristics, IC thresholds, or the `PACKED → HOLEY` transition rules. Performance can shift without any application code change. Treat engine upgrades like any other infrastructure migration: benchmark on a canary, track deoptimization rates, and correlate latency with the engine version label in your metrics.

Key Takeaways

- The engine pipeline is parser → AST → bytecode + feedback vectors → tiered JITs. The interpreter is the profiler; JITs are speculative consumers of that profile.

- Hidden classes (V8 Maps / SpiderMonkey Shapes / JSC Structures) turn hash-map objects into indexed structs. Sharing Maps across instances requires constructing objects in the same property order — use factories and `class`.
- Each property access site has an inline cache that moves through uninitialized → monomorphic → polymorphic (≤ 4 Maps) → megamorphic. Monomorphic is a single pointer compare; megamorphic is a generic handler with no optimization feedback.
- V8 tiers Ignition → Maglev → TurboFan; SpiderMonkey tiers Baseline Interpreter → Baseline JIT (CacheIR) → Warp/Ion; JSC tiers LLInt → Baseline → DFG → FTL (B3/LLVM). All three tier up on hotness + stable feedback and tier down on cool-down or memory pressure, and all support OSR entry/exit.
- Deoptimization reconstructs interpreter/baseline frames from optimized frames using a deopt table. A single guard failure discards the optimized code for that function; re-optimization requires new stable feedback. Deoptimization storms are a real tail-latency risk.
- `d8 --print-ast`, `--print-bytecode`, `--trace-ic`, `--trace-maps`, `--trace-opt/--trace-deopt`, `--print-opt-code`, and `%HaveSameMap` / `%GetOptimizationStatus` / `%DebugPrint` let you observe every stage on real code. Use them before trusting a microbenchmark.
- At fleet scale, engines create warm-up skew, non-reproducible benchmarks, noisy-neighbor IC pollution, and version-dependent performance shifts. Treat Maps, IC states, and elements kinds as cross-team contracts, and instrument deoptimization rates as a service health signal.

Further Reading

- V8 documentation — <https://v8.dev/docs> (Ignition, TurboFan, Maglev, hidden classes, inline caches, deoptimization).
- V8 blog — "An Introduction to Speculative Optimization in V8" (2017), "Maglev — V8's Fastest Optimizing JIT" (2023), "Sparkplug — A Non-Optimizing JS Compiler" (2021) — <https://v8.dev/blog>.
- "V8 Internals for JavaScript Developers" — Mathias Bynens & Benedikt Meurer (JSConf / dotJS talks) — <https://v8.dev/blog/cost-of-javascript-2019>.

- SpiderMonkey Internals — CacheIR, WarpBuilder, IonMonkey — <https://firefox-source-docs.mozilla.org/js/index.html> and <https://spidermonkey.dev/>.
- JavaScriptCore — "Introducing the WebKit FTL JIT" (Fil Pizlo, 2014), "Speculation in JavaScriptCore" — <https://webkit.org/blog/> (search FTL, DFG, LLInt, B3).
- ECMA-262 — ECMAScript Language Specification (living standard) — <https://tc39.es/ecma262/> (normative property-lookup and prototype-chain semantics that engines optimize).
- "Optimization Killers" — Bluebird wiki / Petka Antonov — <https://github.com/petkaantonov/bluebird/wiki/Optimization-killers> (historical but still instructive checklist; verify each claim against current engine behavior).
- Node.js performance diagnostics — https://nodejs.org/api/perf_hooks.html and `node --prof` / `node --prof-process` documentation.

Chapter 2 — The Event Loop, Microtasks, Macrotasks, and Timers

What this chapter covers: JavaScript runs on a single thread. That single thread multiplexes I/O, timers, rendering, promise continuations, and user callbacks through two cooperating event loops — the browser event loop defined by the HTML Standard and the Node.js event loop implemented by libuv. This chapter dissects both, down to the queue mechanics, phase ordering, and timing guarantees that determine when your code actually runs.

After reading this chapter you will be able to:

- Trace any snippet through the browser event loop — tasks, microtasks, rendering, `requestAnimationFrame`, `queueMicrotask`, `MessageChannel`, and `Promise` jobs — and predict the exact `console.log` order.
- Trace any snippet through the libuv event loop — timers, pending callbacks, idle/prepare, poll, check, close — and predict how

`setTimeout`, `setImmediate`, `process.nextTick`, and `Promise.then` interleave.

- Explain why `setTimeout(fn, 0)` is never zero, why timers coalesce and drift, and why `setImmediate` vs `setTimeout(..., 0)` has a race in Node.
- Diagnose and fix starvation and livelock caused by recursive `nextTick` / microtask scheduling.
- Choose correctly between `queueMicrotask`, `Promise.resolve().then`, `MessageChannel`, `setTimeout`, `setImmediate`, and `requestAnimationFrame` for scheduling work.
- Use `node --trace-event`, `perf_hooks`, `async_hooks`, Chrome Performance panel, and a hand-built visualizer to observe loop behavior in production.
- Reason about event-loop delays as a backend reliability signal — event-loop lag, blocked rendering, and tail-latency amplification at scale.

2.1 The Single Thread and Run-to-Completion

V8, SpiderMonkey, and JavaScriptCore (see Chapter 1) all execute JavaScript on one **main thread** per isolate / realm. The call stack runs one frame at a time and each synchronous call chain runs to completion — there is no preemption inside JavaScript itself. Concurrency comes from the *environment* (browser or Node) posting callbacks into queues that the loop drains.

```
JS thread: [ run script ][microtasks][ run task ][microtasks]
[render?][ run task ]...
           ^--- never interrupted mid-frame ---^
```

Two invariants govern everything in this chapter:

1. **Run-to-completion.** Once a task or microtask starts, it runs until it returns. No other JavaScript runs concurrently on the same thread. Long synchronous work blocks everything — including rendering and I/O polling.

2. **Microtasks drain exhaustively before the next task.** After each task (and after each microtask that schedules more microtasks), the engine empties the entire microtask queue before yielding.

If you internalize those two rules, every ordering puzzle in this chapter becomes predictable.

A minimal ordering demo

```
console.log("1: script start");

setTimeout(() => console.log("2: setTimeout"), 0);

Promise.resolve().then(() => console.log("3: promise.then"));

queueMicrotask(() => console.log("4: queueMicrotask"));

console.log("5: script end");

// Output (browser and Node – same for this snippet):
// 1: script start
// 5: script end
// 3: promise.then
// 4: queueMicrotask
// 2: setTimeout
```

Synchronous code runs first (1, 5). Then the microtask queue drains FIFO (3, 4 — note that `Promise.then` jobs and `queueMicrotask` jobs share the single microtask queue; insertion order wins). Only after the queue is empty does the next task run (2).

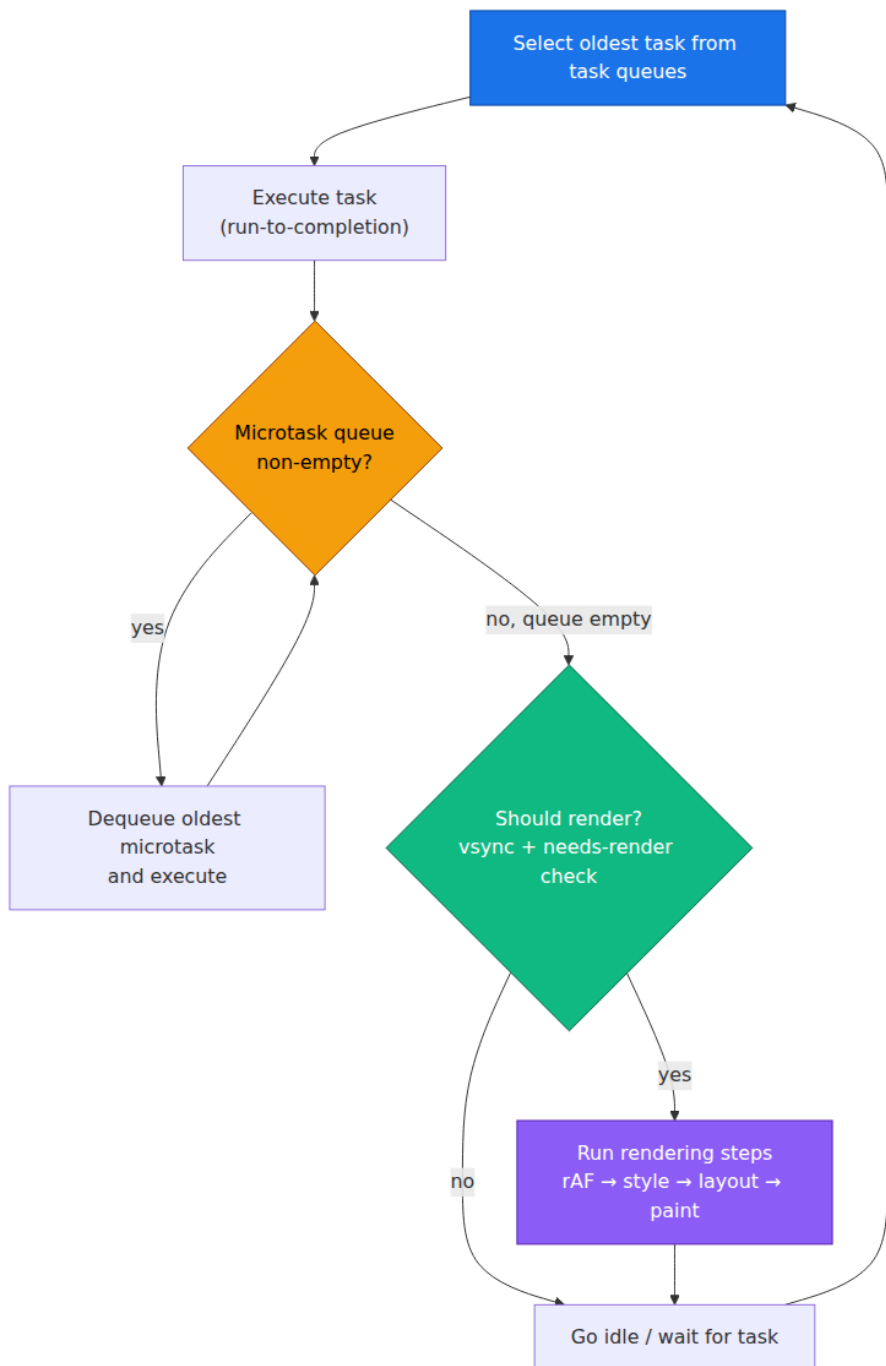
Senior-backend note. This looks identical in browser and Node here, but the equivalence breaks the moment `process.nextTick` or `setImmediate` appears — as we will see in Section 2.4. Do not assume you can reason about one loop from the other.

2.2 The Browser Event Loop

2.2.1 The HTML Standard event loop

The [HTML Standard §8.1.4](#) defines an **event loop** per *agent* (roughly, per browsing context group) and a **task queue** per *task source* (DOM

manipulation, user interaction, networking, history traversal, etc.). The loop is conceptually:

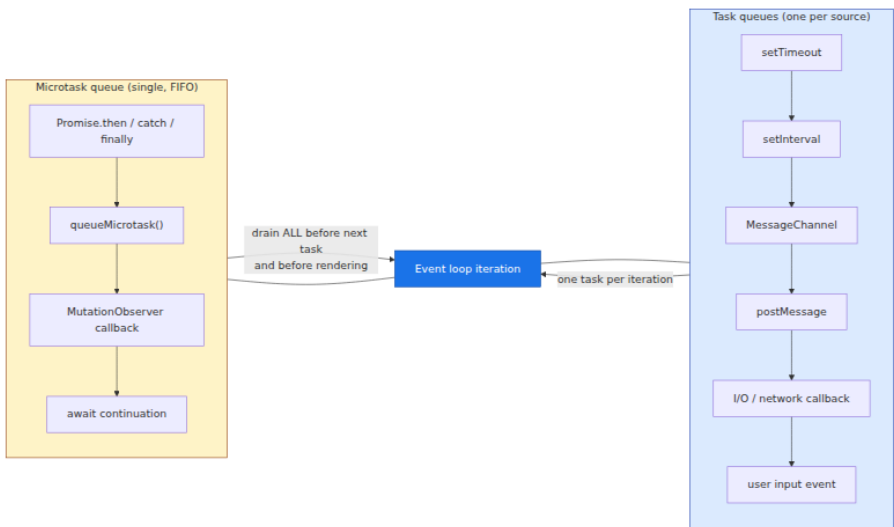


In prose:

1. Pick the oldest **task** (macrotask) from any task queue and run it.
2. Drain the **microtask queue** exhaustively — including microtasks enqueued by other microtasks.
3. Optionally perform **rendering steps** (only if the browser decides a frame is due — tied to the display refresh, typically 60 Hz / 16.6 ms).
4. If no task is ready, sleep until one arrives or the next rendering opportunity.

Every browser vendor maps this onto a platform message pump (Chromium: `base::MessageLoop / SequenceManager`, Firefox: `nsThread + PrioritizedEventQueue`, WebKit: `RunLoop`). The observable semantics are standardized; the internal scheduling is not.

2.2.2 Tasks vs microtasks — the two queues



Queue	Also called	Enqueued by	Drained when
Task	macrotask, task	<code>setTimeout</code> , <code>setInterval</code> , <code>MessageChannel</code> , <code>postMessage</code> , I/O, UI events	One per loop iteration, in task-source priority order

Queue	Also called	Enqueued by	Drained when
Microtask	jobs, microtask	Promise reactions, queueMicrotask , MutationObserver , await resume	Exhaustively after every task and after every microtask

Terminology warning. "Macrotask" is community jargon, not spec language. The spec says "task." This chapter uses "task" when being precise and "macrotask" when contrasting with microtasks, matching real-world usage.

2.2.3 What exactly is a microtask? Promise jobs, queueMicrotask, MutationObserver

All three feed the same FIFO queue. The [ECMA-262](#) specification calls them **Jobs** and the HTML Standard calls them **microtasks** — they are the same mechanism.

```

// All three enqueue into the SAME queue, in call order.

console.log("script start");

queueMicrotask(() => console.log("queueMicrotask 1"));

Promise.resolve().then(() => console.log("promise.then 1"));

new MutationObserver(() => console.log("mutation"))
  .observe(document.createElement("div"), { attributes: true });
// trigger it:
document.body?.setAttribute?.("data-x", "1");

// Also:
Promise.resolve().then(() => console.log("promise.then 2"));
queueMicrotask(() => console.log("queueMicrotask 2"));

console.log("script end");

// Output (Chrome / Firefox):
// script start
// script end
// queueMicrotask 1
// promise.then 1
// promise.then 2
// queueMicrotask 2
// mutation      ← MutationObserver fires after promise jobs in the
// same drain

```

Within one microtask drain, order is insertion order — there is no priority between `queueMicrotask` and `Promise.then`. The distinction matters only for *spec reason*: `queueMicrotask` was added so library code can enqueue a microtask without creating a throwaway Promise (cheaper, clearer intent).

MutationObserver nuance. The spec queues a single microtask that delivers *all* pending mutation records at once, not one per mutation. And in Chromium the observer callback runs after promise microtasks enqueued before it, but before promise microtasks enqueued *after* the DOM mutation — so its position in the FIFO depends on when the mutation was observed, not when `observe()` was called.

2.2.4 MessageChannel, postMessage, and the "fast task" trick

Before `queueMicrotask` existed, libraries needed a way to schedule a task that fires sooner than `setTimeout(..., 0)` (which is clamped to ~4 ms

when nested — see Section 2.5). `MessageChannel` became the standard hack: posting a message enqueues a *task*, not a microtask, but it is the fastest task path — no timer clamping.

```
// "setImmediate" polyfill via MessageChannel (used by many libs before
queueMicrotask)
const { port1, port2 } = new MessageChannel(); // Node; browser uses
new MessageChannel()
const fastQueue = [];
port2.onmessage = () => {
  const fn = fastQueue.shift();
  fn?();
};

function setImmediatePolyfill(fn) {
  fastQueue.push(fn);
  port1.postMessage(null);
}

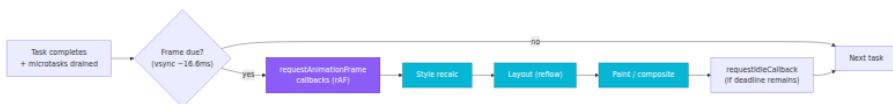
console.log("A");
setTimeout(() => console.log("setTimeout"), 0);
setImmediatePolyfill(() => console.log("MessageChannel"));
Promise.resolve().then(() => console.log("promise"));
console.log("B");

// Output:
// A
// B
// promise           ← microtask queue drains first
// MessageChannel     ← next task (MessageChannel is faster than the
timer)
// setTimeout         ← timer task second
```

In browsers, `postMessage` on `window` and `MessageChannel` both enqueue tasks, but `MessageChannel` is preferred for scheduling because it avoids the origin-check overhead and does not collide with application `message` listeners.

2.2.5 Rendering: where rAF fits

The rendering steps run *between* tasks — after microtasks drain and before the next task — but only when the browser decides to render (aligned to vsync). The pipeline is:



Key points senior engineers often get wrong:

- **requestAnimationFrame (rAF) runs *before* style/layout/paint**, not after. Mutate the DOM in your rAF callback and the browser will include those mutations in the same frame. Read layout in rAF *before* you write and you avoid an extra reflow.
- **rAF does not run if the tab is backgrounded.** Browsers throttle or pause rAF in hidden tabs. Never use it for non-visual timers.
- **Microtasks run *before* rAF.** If you starve the microtask queue, rAF never fires and frames drop.
- **Multiple rAF callbacks in one frame are coalesced** — all callbacks registered before the frame starts run in that frame.

```
// rAF vs microtask vs task – rendering-aware ordering
console.log("script start");

requestAnimationFrame(() => console.log("rAF"));

queueMicrotask(() => console.log("microtask"));

setTimeout(() => console.log("setTimeout"), 0);

// Force a style read to make the frame "needed" – illustrative only
// document.body.offsetHeight;

console.log("script end");

// Typical output when a frame IS rendered after this task:
// script start
// script end
// microtask           ← microtasks before rendering
// rAF                  ← rendering step (if frame was due)
// setTimeout           ← next task (next iteration)

// When no frame is due, rAF is deferred:
// script start
// script end
// microtask
// setTimeout
// (rAF fires on a later iteration just before the next paint)
```

requestAnimationFrame vs queueMicrotask — scheduling intent

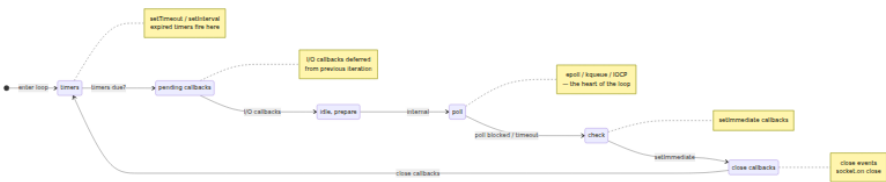
Primitive	Queue	When	Use for
<code>queueMicrotask(fn)</code>	microtask	Immediately after current task, before rendering	Defer work but keep it in the same logical "tick" — flushing batched state before paint
<code>requestAnimationFrame(fn)</code>	rendering step	Just before next paint	Visual updates — DOM mutations, canvas draws, measuring layout
<code>setTimeout(fn, 0)</code>	task	Next task iteration, after rendering (clamped)	Deferring non-urgent work to avoid blocking rendering
<code>MessageChannel</code>	task (fast)	Next task iteration, unclamped	"Fast <code>setTimeout(0)</code> " when you need a task but not a timer
<code>requestIdleCallback(fn)</code>	idle	After rendering, if frame budget remains	Low-priority work (analytics, prefetch) — not supported in all browsers

Rule of thumb. If the work is *visual*, use `rAF`. If the work must happen *before* the next paint but is not visual (e.g., flushing a state batch so `rAF` reads the correct value), use `queueMicrotask`. If the work can wait until *after* paint, use a task.

2.3 The Node.js Event Loop — libuv

Browser JavaScript has one loop with rendering interleaved. Node has a different loop — no rendering, but **I/O polling**, a **thread pool**, and extra phase queues. The engine is still V8, but the event loop is provided by [libuv](#), the same cross-platform async I/O library behind Node, Deno (partially), and Julia.

2.3.1 libuv phases



In the actual C implementation (`uv_run` in `src/unix/core.c`), the order is:

```
while (r != 0 && loop alive) {
    uv_update_time(loop);           // (1) sample loop time
    uv_run_timers(loop);            // (2) timers phase
    uv_run_pending(loop);           // (3) pending callbacks
    uv_run_idle(loop);              // (4) idle handles      (internal,
rarely visible to JS)
    uv_run_prepare(loop);           // (5) prepare handles  (internal)
    uv_io_poll(loop, timeout);      // (6) poll phase – blocks here
    uv_run_check(loop);             // (7) check phase
    uv_run_closing_handles(loop);   // (8) close callbacks
}
```

What each phase does for JavaScript:

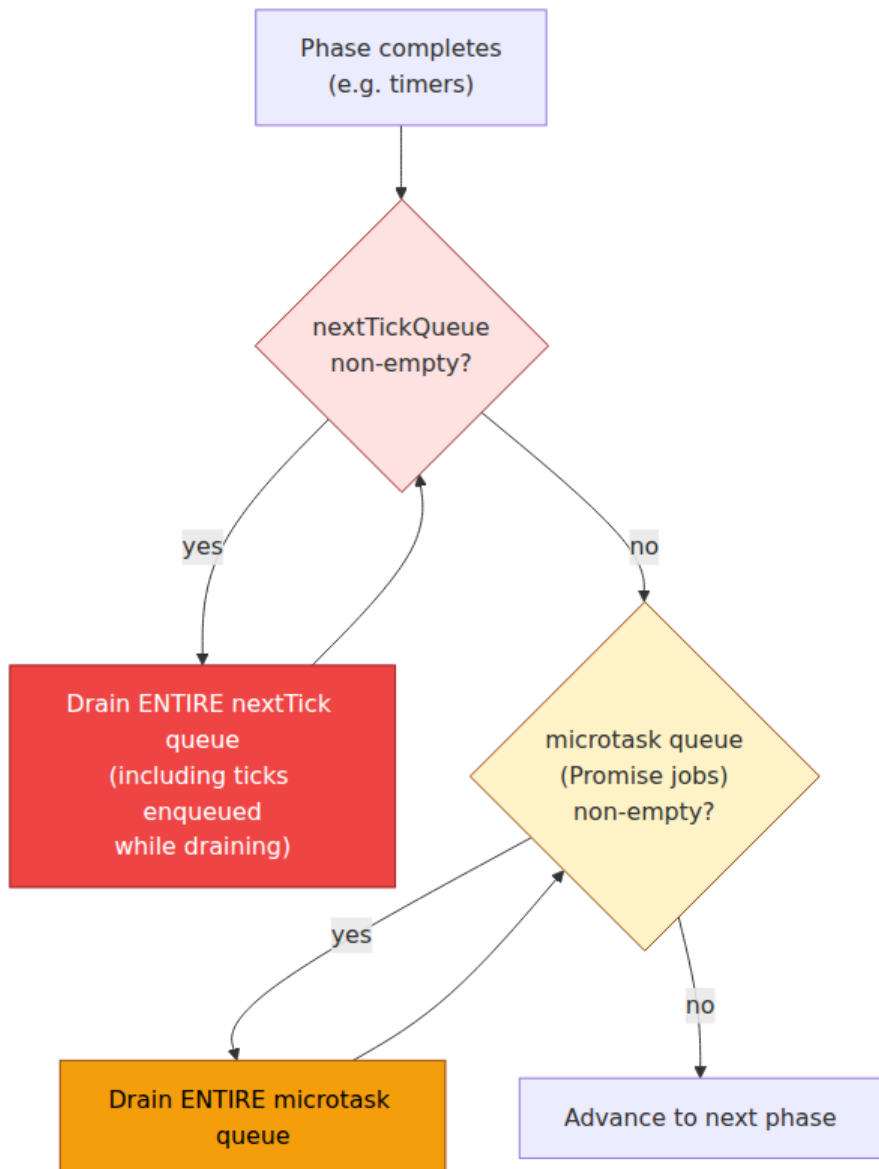
Phase	C function	JS-visible work	Notes
timers	<code>uv_run_timers</code>	<code>setTimeout</code> , <code>setInterval</code> callbacks whose threshold has passed	Threshold is <code>loop->time</code> (cached per iteration), not wall time — see timer coalescing below

Phase	C function	JS-visible work	Notes
pending callbacks	<code>uv__run_pending</code>	I/O callbacks deferred to next iteration	Rarely populated; mainly TCP write errors
idle / prepare	<code>uv__run_idle</code> / <code>uv__run_prepare</code>	Internal only	Node uses these internally (e.g., for bookkeeping). No public JS API enqueues here
poll	<code>uv__io_poll</code>	I/O callbacks (<code>fs.read</code> , <code>net</code> , <code>http</code>)	<i>Blocks</i> calculating a timeout from the next timer. Uses <code>epoll</code> (Linux), <code>kqueue</code> (macOS), <code>IOCP</code> (Windows)
check	<code>uv__run_check</code>	<code>setImmediate</code> callbacks	Runs immediately after poll returns — this is why <code>setImmediate</code> fires after I/O
close callbacks	<code>uv__run_closing_handles</code>	<code>socket.on('close')</code> , <code>server.close</code> callbacks	Cleanup

Between *every* phase transition, Node drains two special queues — `process.nextTick` and **Promise microtasks** — which is why Node ordering differs from the browser.

2.3.2 nextTick vs microtasks — the ordering that surprises everyone

This is the single most tested distinction in Node interviews and the single most common source of real bugs.



Rules:

1. `process.nextTick` has its own queue, drained *before* Promise microtasks. It is not a microtask — it is a *next-tick queue* that runs at higher priority.
2. Both queues are drained exhaustively between every phase (and after every task callback), including ticks/microtasks enqueued while draining.
3. `process.nextTick` starves I/O if recursed (Section 2.6). Promise microtasks can technically also starve, but V8 and Node have mitigations; `nextTick` has none by design.

```
// Node – nextTick vs Promise ordering
console.log("1: script start");

setTimeout(() => console.log("2: setTimeout"), 0);
setImmediate(() => console.log("3: setImmediate"));

process.nextTick(() => console.log("4: nextTick 1"));
Promise.resolve().then(() => console.log("5: promise.then 1"));

process.nextTick(() => {
  console.log("6: nextTick 2");
  process.nextTick(() => console.log("7: nextTick (nested)"));
  Promise.resolve().then(() => console.log("8: promise inside
nextTick"));
});
Promise.resolve().then(() => console.log("9: promise.then 2"));

console.log("10: script end");

// Output (Node 18+ / 20+):
// 1: script start
// 10: script end
// 4: nextTick 1
// 6: nextTick 2
// 7: nextTick (nested)      ← nested nextTick drains before
microtasks
// 5: promise.then 1
// 9: promise.then 2
// 8: promise inside nextTick ← promise enqueued inside nextTick runs
after existing promises
// 2: setTimeout           ← timers phase
// 3: setImmediate         ← check phase (after poll)
```

Contrast the same snippet in a browser (where `process.nextTick` does not exist and `setImmediate` is non-standard / absent):

```
// Browser – same snippet without nextTick / setImmediate
console.log("1: script start");
setTimeout(() => console.log("2: setTimeout"), 0);
Promise.resolve().then(() => console.log("3: promise.then 1"));
Promise.resolve().then(() => console.log("4: promise.then 2"));
console.log("5: script end");

// Browser output:
// 1: script start
// 5: script end
// 3: promise.then 1
// 4: promise.then 2
// 2: setTimeout
```

Takeaway. In the browser there is one microtask queue. In Node there are *two* queues interleaved: `nextTick` first, then promise jobs, between every phase. Any mental model that equates `process.nextTick` with "a fast Promise" will produce wrong predictions.

2.3.3 I/O poll — the blocking heart

The `poll` phase is where Node spends most of its time when idle. `uv__io_poll` computes a timeout:

- If the loop is alive and there are no timers/check/close handles, it blocks *indefinitely* until I/O arrives.
- If there are timers, it blocks for `min(nextTimerExpiry - now, SOME_CAP)` so it wakes in time for the timers phase.
- On Linux it calls `epoll_wait` / `epoll_pwait`; on macOS `kevent`; on Windows `GetQueuedCompletionStatusEx`.

```
// Observing poll blocking with perf_hooks
import { performance, PerformanceObserver } from "node:perf_hooks";
import fs from "node:fs";

const obs = new PerformanceObserver((list) =>
  list.getEntries().forEach((e) => console.log(e.name, e.duration.toFixed(2) + "ms"))
);
obs.observe({ entryTypes: ["measure"] });

performance.mark("start");
fs.readFile(__filename, () => {
  performance.mark("io-done");
  performance.measure("poll → callback", "start", "io-done");
  console.log("fs.readFile callback – poll phase delivered this");
});
console.log("after readFile call – poll is now blocking in libuv");
// Output:
// after readFile call – poll is now blocking in libuv
// fs.readFile callback – poll phase delivered this
// poll → callback 3.42ms (time libuv blocked in epoll_wait)
```

Distributed-systems lens. Event-loop stall in `poll` is usually healthy (the thread is parked waiting for I/O). Stall *outside* `poll` — a long task or microtask drain — is the latency killer: while JavaScript runs, `epoll_wait` is not being called, inbound connections queue in the kernel, and p99 spikes. Monitor `perf_hooks.monitorEventLoopDelay()` in production to distinguish the two.

2.4 Browser vs Node — Side-by-Side Ordering Demos

The demos below are runnable as-is. Outputs were captured on Chrome 124 and Node 20.12.

2.4.1 The classic interview puzzle, annotated

```
// puzzle.js – run in Node with: node puzzle.js
//           – paste into browser console for comparison

console.log("A: script start");

setTimeout(() => {
  console.log("B: setTimeout");
  Promise.resolve().then(() => console.log("C: promise inside
setTimeout"));
}, 0);

Promise.resolve()
  .then(() => {
    console.log("D: promise.then 1");
    queueMicrotask(() => console.log("E: queueMicrotask inside
promise"));
  })
  .then(() => console.log("F: promise.then 2 (chained)"));

queueMicrotask(() => console.log("G: queueMicrotask"));

setTimeout(() => console.log("H: setTimeout 2"), 0);

console.log("I: script end");
```

Browser output:

```
A: script start
I: script end
D: promise.then 1
G: queueMicrotask
E: queueMicrotask inside promise
F: promise.then 2 (chained)
B: setTimeout
C: promise inside setTimeout
H: setTimeout 2
```

Node output: Identical for this snippet — there is no `nextTick` or `setImmediate`, so both loops agree: script → microtasks FIFO (including chained `then` and `queueMicrotask` inserted during the drain) → tasks FIFO, with microtasks draining again after task `B` before task `H`.

The equivalence breaks with Node-only primitives:

```
// node-only-ordering.js – Node 20
console.log("A: start");

setTimeout(() => console.log("B: setTimeout 0"), 0);
setImmediate(() => console.log("C: setImmediate"));
process.nextTick(() => console.log("D: nextTick"));
Promise.resolve().then(() => console.log("E: promise"));
queueMicrotask(() => console.log("F: queueMicrotask"));

console.log("G: end");

// Output – top-level script context (no I/O):
// A: start
// G: end
// D: nextTick           ← nextTick drains first
// E: promise / F: queueMicrotask (FIFO – insertion order between
// them)
// B: setTimeout 0       ← timers phase before check – but see
// race note below
// C: setImmediate
```

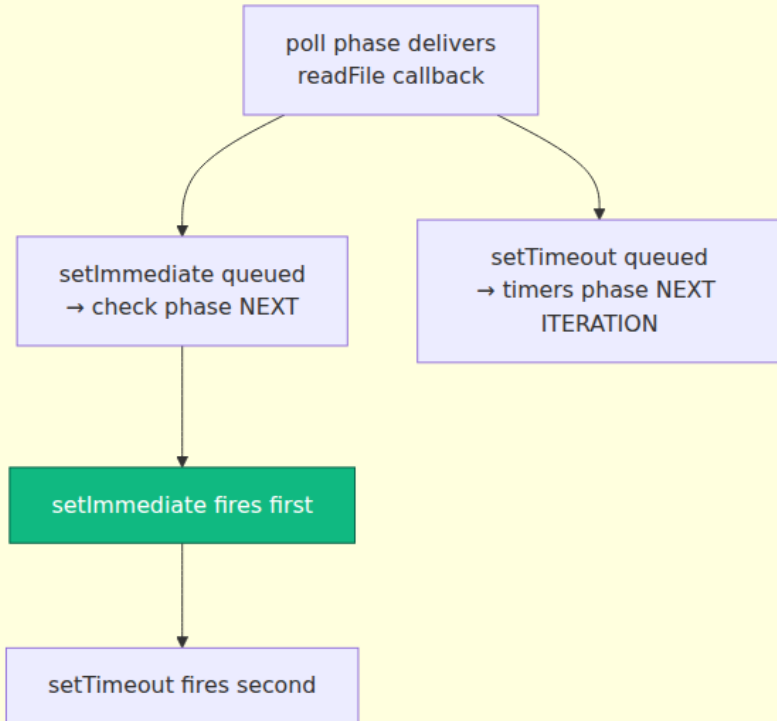
The famous race. When both `setTimeout(..., 0)` and `setImmediate` are scheduled from the *top-level* script (outside any I/O callback), their order is **nondeterministic** — it depends on how long script execution took vs the 1 ms timer threshold and whether `uv_update_time` has sampled a new millisecond. Inside an I/O callback, the order is deterministic: `setImmediate` always wins because `poll` → `check` comes before the next timers phase.

```
// Deterministic ordering – schedule from inside poll phase (I/O
// callback)
import fs from "node:fs";

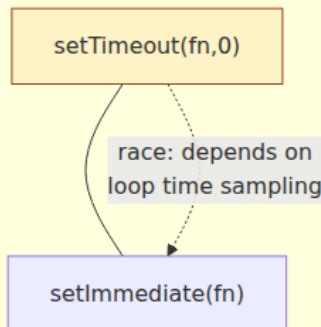
fs.readFile(__filename, () => {
  console.log("inside I/O callback (poll phase just delivered)");
  setTimeout(() => console.log("setTimeout inside I/O"), 0);
  setImmediate(() => console.log("setImmediate inside I/O"));
});

// Output – ALWAYS:
// inside I/O callback (poll phase just delivered)
// setImmediate inside I/O   ← check phase is next
// setTimeout inside I/O     ← timers phase on the following iteration
```

Scheduling from I/O callback (deterministic)



Top-level scheduling (nondeterministic)



2.4.2 async/await ordering — await is Promise.then

```
// await-ordering.js – identical in browser and Node (no nextTick)
console.log("1");

async function foo() {
  console.log("2");
  await Promise.resolve();
  console.log("3"); // resumes as a microtask
  await Promise.resolve();
  console.log("4"); // another microtask hop
}

foo();
console.log("5");
Promise.resolve().then(() => console.log("6"));
console.log("7");

// Output (browser and Node):
// 1
// 2      ← foo runs synchronously until first await
// 5
// 7      ← script continues
// 3      ← first await resumes (microtask)
// 6      ← promise.then enqueued before second await
// 4      ← second await resumes (microtask, but after 6)
```

Every `await` on an already-resolved promise still yields one microtask. Two awaits = two microtask hops. This is why `await` inside a hot loop can be measurably slower than synchronous iteration — each iteration pays the microtask queue tax.

2.5 Timers — setTimeout, setInterval, setImmediate, and process.nextTick

2.5.1 setTimeout is never zero

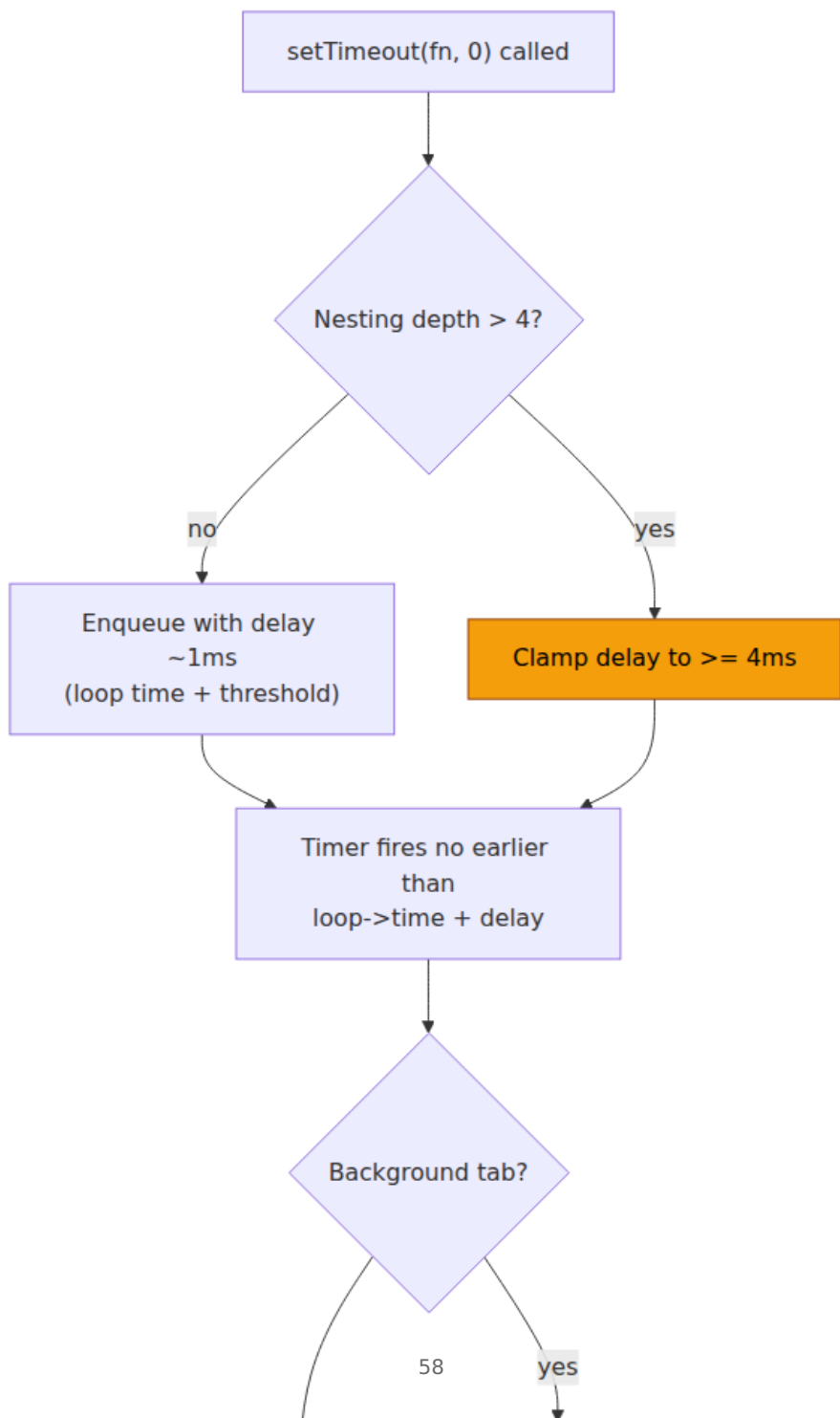
Three distinct sources of delay make `setTimeout(fn, 0)` always > 0 :

Source	Delay	Spec / implementation
Timer resolution	~1 ms in Node (libuv <code>uv_update_time</code> samples per iteration), ~1–4 ms in browsers	Node caches <code>loop->time</code> ; browsers use <code>DOMHighResTimeStamp</code>

Source	Delay	Spec / implementation
Nesting depth clamp	After 5 nested <code>setTimeout</code> calls, delay clamped to ≥ 4 ms	HTML Standard §8.6 — "If nesting level > 4 and timeout < 4 , set timeout to 4"
Throttling	Background tabs: ≥ 1000 ms (Chrome), nested timers throttled to 1 s	Browser background-tab throttling; Node has no equivalent

```
// Demonstrating nesting clamp – browser
let depth = 0;
function nested() {
  const t0 = performance.now();
  setTimeout(() => {
    const elapsed = performance.now() - t0;
    console.log(`depth ${depth} → next fires after $
{elapsed.toFixed(2)}ms`);
    if (++depth < 8) nested();
  }, 0);
}
nested();

// Output (Chrome, foreground tab):
// depth 0 → next fires after 1.20ms
// depth 1 → next fires after 0.90ms
// depth 2 → next fires after 1.10ms
// depth 3 → next fires after 1.05ms
// depth 4 → next fires after 4.30ms ← clamp kicks in at depth > 4
// depth 5 → next fires after 4.15ms
// depth 6 → next fires after 4.20ms
// depth 7 → next fires after 4.10ms
```



2.5.2 Timer coalescing and drift

libuv does not maintain a sorted timer heap with absolute precision. Each iteration it samples `loop->time` once and fires *all* timers whose expiry $\leq$ that sampled time. Timers that become due during the poll phase all fire together in the next timers phase — they **coalesce**.

`setInterval` drift compounds: if a callback takes longer than the interval, libuv does not queue multiple invocations — it fires once per iteration and scheduling is based on *start time*, not *end time*. A 10 ms interval callback that takes 25 ms will visibly skip beats.

```
// Interval drift demo - Node
let ticks = 0;
const t0 = Date.now();
const id = setInterval(() => {
  ticks += 1;
  const drift = Date.now() - t0 - ticks * 10;
  console.log(`tick ${ticks} drift ${drift}ms`);
  // Simulate work that sometimes exceeds interval
  if (ticks === 5) {
    const block = Date.now() + 30;
    while (Date.now() < block) {} // block 30 ms
  }
  if (ticks >= 8) clearInterval(id);
}, 10);

// Output:
// tick 1 drift 1ms
// tick 2 drift 1ms
// tick 3 drift 1ms
// tick 4 drift 2ms
// tick 5 drift 2ms      ← then blocks 30ms
// tick 6 drift 28ms     ← drift jumps - missed beats
// tick 7 drift 28ms
// tick 8 drift 29ms
```

Production takeaway. Do not use `setInterval` for precise periodic work (health checks, lease renewal). Use recursive `setTimeout` where the next schedule is computed from when the *previous callback completed*, or — better — a monotonic deadline check (`Date.now()` vs expected) inside the callback. For sub-millisecond scheduling, use `setImmediate` / `queueMicrotask` loops with explicit budget checks.

2.5.3 Choosing the right primitive

```
// Decision helper – Node
// Need it BEFORE the next I/O poll?           → process.nextTick (but
beware starvation)
// Need it before rendering, inside same tick? → queueMicrotask /
Promise.then
// Need it after I/O, before timers?           → setImmediate
// Need a delay, even ~1ms?                     → setTimeout
// Need to yield to the event loop briefly? → setImmediate (Node) /
MessageChannel (browser) / scheduler.yield() (future)
// Need it before next paint?                   → requestAnimationFrame
(browser only)
```

Primitive	Queue / phase	Clamped?	Starves I/O?	Browser	Node
<code>process.nextTick</code>	nextTick queue (before microtasks)	no	yes	—	yes
<code>queueMicrotask</code>	microtask	no	yes (but bounded in practice)	yes	yes
<code>Promise.then</code>	microtask	no	yes	yes	yes
<code>setTimeout(fn, 0)</code>	timers	yes (4 ms nested, 1 ms min)	no	yes	yes
<code>setImmediate</code>	check	no	no	— (Edge legacy only)	yes
<code>MessageChannel</code>	task (fast)	no	no	yes	yes
<code>requestAnimationFrame</code>	rendering	vsync	no	yes	—

2.6 Starvation and Livelock

Every queue that drains exhaustively is a starvation vector. If callbacks keep re-enqueuing themselves into the same queue, lower-priority queues never get serviced.

2.6.1 nextTick starvation — blocking I/O indefinitely

```
// STARVATION DEMO — DO NOT RUN UNGUARDED IN PRODUCTION
// This loop prevents the event loop from ever reaching poll/check/
timers.

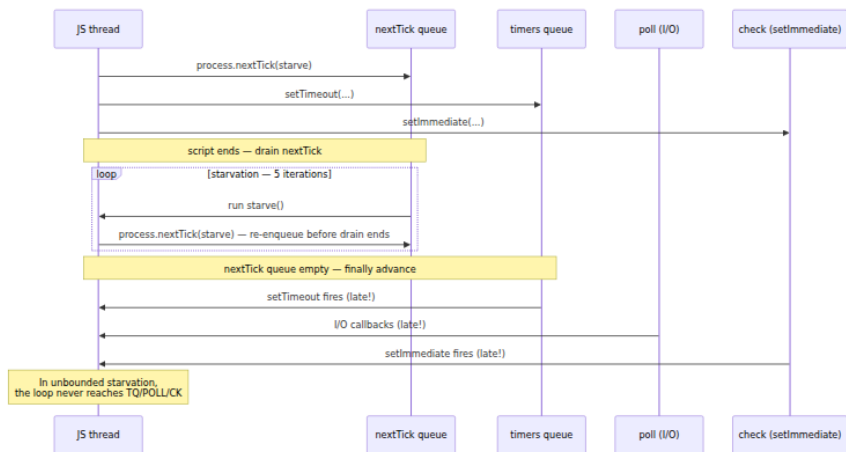
let n = 0;
function starve() {
  if (n++ < 5) {
    console.log(`nextTick ${n} — I/O and timers are blocked`);
    process.nextTick(starve);
  } else {
    console.log("stopped recursing — loop can breathe again");
  }
}

setTimeout(() => console.log("setTimeout — will be delayed until
starvation ends"), 0);
setImmediate(() => console.log("setImmediate — also delayed"));

starve();
console.log("script end — but loop is still stuck in nextTick drain");

// Output:
// script end — but loop is still stuck in nextTick drain
// nextTick 1 — I/O and timers are blocked
// nextTick 2 — I/O and timers are blocked
// nextTick 3 — I/O and timers are blocked
// nextTick 4 — I/O and timers are blocked
// nextTick 5 — I/O and timers are blocked
// stopped recursing — loop can breathe again
// setTimeout — will be delayed until starvation ends
// setImmediate — also delayed
```

If the guard (`n < 5`) were missing, `setTimeout` and `setImmediate` would **never** fire — the loop never leaves the `nextTick` drain. The same applies to unbounded promise recursion, though Node's `process.maxTickDepth`-style mitigations and V8's microtask handling make it slightly harder to accidentally starve with promises alone.



2.6.2 Microtask starvation — the subtler variant

```
// Promise recursion starvation – works in BOTH browser and Node
let count = 0;
function spamMicrotasks() {
  if (count++ < 100000) {
    Promise.resolve().then(spamMicrotasks);
  }
}
spamMicrotasks();

setTimeout(() => console.log(`setTimeout finally ran after ${count}
microtasks`), 0);

// What happens:
// - The microtask queue never empties until count hits 100000.
// - setTimeout is delayed by the entire microtask chain.
// - In a browser, rendering is also blocked – the tab freezes.
// - Event-loop lag spikes; health checks time out; load balancers mark
the instance unhealthy.
```

How to detect it in production. Monitor `perf_hooks.monitorEventLoopDelay()` (Node) and `PerformanceObserver` long-task entries (browser). A p99 event-loop delay > 50 ms is a strong signal that synchronous work or microtask/nextTick recursion is starving the loop.

2.6.3 Backpressure as the fix

The fix is not "never use nextTick/microtasks" — it is **yielding**. Batch work and explicitly schedule the next batch as a *task* (not a microtask) so I/O and rendering interleave:

```
// Cooperative batching – yield to the loop every N items
import { setImmediate } from "node:timers/promise"; // Node 15+

async function processLargeArray(items, batchSize = 1000) {
  for (let i = 0; i < items.length; i += batchSize) {
    const batch = items.slice(i, i + batchSize);
    // Synchronous batch – fast, but bounded
    for (const item of batch) transform(item);

    // Yield to the event loop so I/O and timers get a turn
    if (i + batchSize < items.length) {
      await setImmediate(); // check phase – lets poll run before next
    }
    // Browser equivalent: await new Promise(r => setTimeout(r, 0));
    // or: await scheduler.yield() when available
  }
}

function transform(x) { /* ... */ }
```

For browser-side large-data work, prefer `scheduler.yield()` (Chrome 115+, part of the [Prioritized Task Scheduling API](#)) or `requestIdleCallback` chunking — both yield to rendering so frames stay smooth.

2.7 Unhandled Rejections and Error Propagation

2.7.1 Browser — unhandledrejection

When a Promise rejects and no handler is attached *by the end of the microtask drain*, the browser fires `unhandledrejection` on `window`. If a handler is attached later (e.g., in a subsequent task), `rejectionhandled` fires.

```
// Browser unhandled rejection lifecycle
window.addEventListener("unhandledrejection", (e) => {
  console.error("unhandled:", e.reason);
  // e.preventDefault() suppresses the console error
});

window.addEventListener("rejectionhandled", (e) => {
  console.warn("late-handled:", e.reason);
});

// Case 1: never handled – fires unhandledrejection
Promise.reject(new Error("boom"));

// Case 2: handled late – fires BOTH events
const p = Promise.reject(new Error("late"));
setTimeout(() => p.catch((e) => console.log("caught late:", e.message))
, 10);
// Sequence: unhandledrejection (microtask drain) → 10ms later → catch
→ rejectionhandled
```

Important: the check happens at the **end of the microtask drain**, not synchronously when `reject` is called. Attaching `.catch` as a microtask (same drain) still counts as handled:

```
const p = Promise.reject(new Error("sync catch"));
p.catch(() => console.log("handled")); // same microtask – no
unhandledrejection
```

2.7.2 Node — stricter, configurable

Node surfaces three events on `process` and a CLI flag that controls the default:


```
// Node unhandled rejection handling
process.on("unhandledRejection", (reason, promise) => {
  console.error("unhandledRejection:", reason);
  // Log, report to error tracking, but DO NOT silently swallow in
  // production
});

process.on("rejectionHandled", (promise) => {
  console.warn("rejectionHandled - was unhandled, now caught late");
});

process.on("uncaughtException", (err) => {
  console.error("uncaughtException:", err);
  // Best practice: log, flush telemetry, then exit - the process is in
  // an unknown state
  // process.exitCode = 1;
});
```

The flag `--unhandled-rejections` controls what happens if no listener is registered:

Mode	Behavior
<code>throw</code> (default since Node 15)	Emits <code>unhandledRejection</code> ; if still unhandled, throws — becomes <code>uncaughtException</code> and terminates
<code>strict</code>	Same as <code>throw</code> but always terminates even with a listener that does not re-throw (future default)
<code>warn</code>	Emits warning, does not terminate (Node 14 default)
<code>none</code>	Silences the warning entirely

```
# Recommended production setting - fail fast on unhandled rejections
node --unhandled-rejections=strict app.js

# Legacy / migration - warn but keep running (masks bugs)
node --unhandled-rejections=warn app.js
```

Distributed-systems lens. An unhandled rejection in an HTTP handler that is not caught by your framework's error boundary can leave a request hanging, leak a database connection from the pool, and — if the process is configured to not crash — silently degrade the instance. Prefer `strict` mode and a top-level handler that logs and exits so the

orchestrator (Kubernetes, systemd) restarts a clean process. A crashed pod that restarts is more observable than a zombie pod serving 500s.

2.8 Tooling — Observing the Loop

2.8.1 Browser — Performance panel and Long Tasks

Chrome DevTools → Performance → Record → look for:

- **Long tasks** (> 50 ms) — synchronous work blocking the loop.
- **Layout / Recalculate Style** — rendering work, often triggered by forced synchronous layout.
- **Fire Animation Frame** — rAF callbacks.
- **Microtasks** — shown as "Run Microtasks" in the flame chart.

Programmatically:

```
// Long Task observer – browser
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.warn(`long task: ${entry.duration.toFixed(1)}ms`, entry);
  }
});
observer.observe({ entryTypes: ["longtask"] });

// Event-loop lag approximation – browser
let last = performance.now();
setInterval(() => {
  const now = performance.now();
  const lag = now - last - 100; // expected 100ms interval
  if (lag > 10) console.warn(`loop lag: ${lag.toFixed(1)}ms`);
  last = now;
}, 100);
```

2.8.2 Node — trace events, perf_hooks, async_hooks

```
# 1. Trace the event loop phases – produces trace.json for chrome://tracing or Perfetto
node --trace-event-categories node,node.async_hooks,node.perf --trace-events-enabled -e "
  setTimeout(() => console.log('timer'), 10);
  setImmediate(() => console.log('immediate'));
  process.nextTick(() => console.log('nextTick'));
"
# Open trace.json in https://ui.perfetto.dev or chrome://tracing

# 2. Event-loop utilization (ELU) – Node 14.10+
node --trace-event-categories node.perf -e "
  import { eventLoopUtilization } from 'node:perf_hooks';
  const elu1 = eventLoopUtilization();
  setTimeout(() => {
    const elu2 = eventLoopUtilization(elu1);
    console.log('ELU:', elu2);
    // { idle: 2.3, active: 0.4, utilization: 0.15 }
    // utilization = active / (idle + active) – high means loop is
    saturated
  }, 100);
"

# 3. Histogram of event-loop delay – the production health signal
node -e "
  import { monitorEventLoopDelay } from 'node:perf_hooks';
  const h = monitorEventLoopDelay({ resolution: 10 });
  h.enable();
  setInterval(() => {
    console.log('p50', h.percentile(50)/1e6 + 'ms',
               'p99', h.percentile(99)/1e6 + 'ms',
               'max', h.max/1e6 + 'ms');
  }, 5000);
  // Keep busy to see lag
  setInterval(() => { const s = Date.now() + 30; while (Date.now() < s) {} }, 100);
"
# Output:
# p50 0.12ms p99 30.04ms max 30.12ms
```

```
// async_hooks - trace async resource lifetimes (debug, not hot path)
import { createHook } from "node:async_hooks";
import fs from "node:fs";

const hook = createHook({
  init(asyncId, type, triggerAsyncId) {
    if (type === "Timeout" || type === "Immediate" || type === "PROMISE") {
      fs.writeSync(1, `init ${type} id=${asyncId} triggered by ${triggerAsyncId}\n`);
    }
  },
  before(asyncId) { fs.writeSync(1, `before ${asyncId}\n`); },
  after(asyncId) { fs.writeSync(1, `after ${asyncId}\n`); },
});
hook.enable();

setTimeout(() => console.log("timer fired"), 0);
// Output:
// init Timeout id=5 triggered by 1
// before 5
// timer fired
// after 5
```

Performance note. `async_hooks` has measurable overhead — do not enable it unconditionally in production. Use `AsyncLocalStorage` (built on `async_hooks`) for request context propagation, and gate verbose tracing behind a flag or sampled debug mode.

2.8.3 A hand-built event-loop visualizer

The snippet below is a self-contained HTML file that visualizes task / microtask / rAF ordering in the browser. Open it locally, click the buttons, and watch the interleaving.

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Event Loop Visualizer</title>
  <style>
    body { font-family: ui-monospace, monospace; max-width: 760px; margin: 2rem auto; }
    button { margin: 0.25rem; padding: 0.5rem 0.75rem; cursor: pointer; }
    #log { border: 1px solid #ccc; min-height: 260px; padding: 0.75rem; white-space: pre-wrap; background: #f9fafb; }
    .task { color: #1e40af; } .micro { color: #92400e; } .raf { color: #7c3a4d; } .idle { color: #065f46; }
  </style>
</head>
<body>
  <h1>Event Loop Visualizer</h1>
  <div>
    <button id="btnTask">enqueue task (setTimeout 0)</button>
    <button id="btnMicro">enqueue microtask (queueMicrotask)</button>
    <button id="btnRaf">requestAnimationFrame</button>
    <button id="btnMessage">MessageChannel</button>
    <button id="btnBurst">burst: task → micro → raf → micro</button>
    <button id="btnClear">clear</button>
  </div>
  <pre id="log"></pre>
  <script>
    const log = document.getElementById("log");
    let seq = 0;
    function entry(kind, msg) {
      const cls = { task: "task", micro: "micro", raf: "raf", idle: "idle" }[kind] || "";
      const line = `${String(++seq).padStart(2, "0")} [${kind}.padEnd(5)] ${msg} @${performance.now().toFixed(1)}ms`;
      const span = document.createElement("span");
      span.className = cls;
      span.textContent = line + "\n";
      log.appendChild(span);
      log.scrollTop = log.scrollHeight;
    }

    // MessageChannel fast task
    const { port1, port2 } = (() => {
      const ch = new MessageChannel();
      return { port1: ch.port1, port2: ch.port2 };
   })();
    const mcQueue = [];
    port2.onmessage = () => mcQueue.shift()?.();
  </script>

```

```

document.getElementById("btnTask").onclick = () =>
  setTimeout(() => entry("task", "setTimeout fired"), 0);

document.getElementById("btnMicro").onclick = () =>
  queueMicrotask(() => entry("micro", "queueMicrotask fired"));

document.getElementById("btnRaf").onclick = () =>
  requestAnimationFrame(() => entry("raf", "rAF fired"));

document.getElementById("btnMessage").onclick = () => {
  mcQueue.push(() => entry("task", "MessageChannel fired"));
  port1.postMessage(null);
};

document.getElementById("btnBurst").onclick = () => {
  entry("task", "burst start (sync)");
  setTimeout(() => {
    entry("task", "burst: setTimeout");
    queueMicrotask(() => entry("micro", "burst: micro inside
timeout"));
  }, 0);
  queueMicrotask(() => entry("micro", "burst: queueMicrotask"));
  requestAnimationFrame(() => entry("raf", "burst: rAF"));
  Promise.resolve().then(() => entry("micro", "burst:
Promise.then"));
  entry("task", "burst end (sync)");
};

document.getElementById("btnClear").onclick = () => { log.textContent = ""; seq = 0; };

  entry("idle", "ready – click buttons and observe ordering");
  entry("idle", "hint: microtasks always drain before tasks and rAF")
;
</script>
</body>
</html>

```

What to try:

1. Click **microtask** then **task** — microtask fires first, even though the button was clicked second.
2. Click **burst** — note that synchronous `burst start/end` log first, then both microtasks (`queueMicrotask` and `Promise.then` in FIFO), then `rAF` (before next paint), then the `setTimeout` task and its nested microtask.
3. Open DevTools → Performance → Record → click burst → stop → find "Run Microtasks" and "Fire Animation Frame" in the trace.

2.9 The Distributed-Systems Lens

A single-threaded event loop may seem like a frontend concern. In a microservice backend it is a *capacity and reliability* concern.

Event-loop lag as a load signal. At scale, Node services behind a load balancer report health via HTTP probes. If the loop is blocked — a synchronous JSON parse of a large payload, a recursive microtask chain, a missing `await` that fires thousands of concurrent promises — probe responses are delayed, the load balancer marks the instance unhealthy, traffic shifts to peers, and they tip over in turn. Use `monitorEventLoopDelay` and expose its p99 as a Prometheus histogram; alert on sustained p99 > 50 ms, not just CPU.

Noisy-neighbor microtasks. In a shared process (e.g., a BFF that fans out to ten downstream services), one route handler that schedules unbounded microtasks can starve all other concurrent requests on that event loop. Isolate CPU-heavy work to worker threads (`node:worker_threads`) or child processes — the main loop should only orchestrate I/O.

Timer coalescing and thundering herd. Hundreds of `setTimeout(fn, 5000)` calls created at nearly the same time will coalesce and fire in the same timers phase, spiking CPU. Jitter expiration times (`5000 + Math.random() * 1000`) or use a centralized scheduler so callbacks spread across iterations.

Backpressure must yield. Any batch processor (event consumer, queue drainer, bulk importer) that loops without yielding will starve I/O — downstream TCP backpressure signals are not polled, buffers grow, memory climbs. The `await setImmediate()` batching pattern in Section 2.6.3 is the minimum viable fix; for heavier work, move it off-thread.

Tracing across async boundaries. `AsyncLocalStorage` propagates request context (trace ID, tenant, deadline) across the very queues this chapter describes. If you lose context after an `await`, suspect a library that uses a non-tracked scheduling primitive (e.g., raw `MessageChannel` or a native addon that bypasses `async_hooks`). Verify with `async_hooks` tracing in staging before relying on context in production.

Key takeaways

- JavaScript is single-threaded and run-to-completion; concurrency comes from the event loop draining queues — one task at a time, microtasks exhaustively between tasks.
- The **browser** event loop interleaves tasks, an exhaustive microtask drain, and conditional rendering steps (`requestAnimationFrame` → `style` → `layout` → `paint`) per iteration. All microtask sources (`Promise` , `queueMicrotask` , `MutationObserver`) share one FIFO queue.
- **Node's libuv** loop has distinct phases — timers, pending callbacks, idle/prepare, poll, check, close — with `process.nextTick` (highest priority, starves everything) and Promise microtasks draining between every phase.
- **Ordering is deterministic within each loop but different across loops.** `nextTick` before microtasks in Node has no browser equivalent; `setTimeout(..., 0)` vs `setImmediate` races at top level in Node but is ordered inside I/O callbacks.
- **`setTimeout(fn, 0)` is never zero.** Expect ~1 ms minimum, 4 ms when nested beyond depth 5, and ~1000 ms in background tabs. Timers coalesce to the sampled `loop->time` and `setInterval` drifts if callbacks overrun.
- **Exhaustive draining means starvation.** Recursive `nextTick` or microtask scheduling blocks I/O, timers, and rendering indefinitely. Yield with `setImmediate` (Node) or `setTimeout` / `scheduler.yield()` (browser) and batch large work.
- **Unhandled rejections** are detected at the end of the microtask drain (browser: `unhandledrejection` / `rejectionhandled` ; Node: `unhandledRejection` + `--unhandled-rejections=strict`). Fail fast in production — a zombie process is worse than a restarted one.
- **Choose the scheduler by intent:** `queueMicrotask` to flush state before paint, `requestAnimationFrame` for visual work before paint, tasks (`setTimeout` / `MessageChannel` / `setImmediate`) to defer past paint or I/O.
- **Observe, do not guess.** Use the Performance panel and `PerformanceObserver` in the browser; `node --trace-event` , `perf_hooks.monitorEventLoopDelay()` , `eventLoopUtilization()` , and `async_hooks` in Node. Alert on p99 event-loop delay.

Further reading

- HTML Standard — Event loops. <https://html.spec.whatwg.org/#event-loops>
- HTML Standard — Timers (`setTimeout` / `setInterval` clamping). <https://html.spec.whatwg.org/#timers>
- ECMA-262 — Jobs and HostEnqueuePromiseJob. <https://tc39.es/ecma262/#sec-jobs>
- MDN — The event loop. https://developer.mozilla.org/en-US/docs/Web/API/HTML_DOM_API/Microtask_guide
- MDN — `queueMicrotask`. <https://developer.mozilla.org/en-US/docs/Web/API/queueMicrotask>
- MDN — `requestAnimationFrame`. <https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame>
- MDN — Prioritized Task Scheduling (`scheduler.postTask` / `scheduler.yield`). <https://developer.mozilla.org/en-US/docs/Web/API/Scheduler>
- libuv documentation — Design overview and `uv_run` loop. <https://docs.libuv.org/en/v1.x/design.html>
- Node.js documentation — The Node.js event loop, timers, and `process.nextTick`. <https://nodejs.org/en/docs/guides/event-loop-timers-and-nexttick>
- Node.js documentation — `perf_hooks: monitorEventLoopDelay`, `eventLoopUtilization`. https://nodejs.org/api/perf_hooks.html
- Node.js documentation — `async_hooks` and `AsyncLocalStorage`. https://nodejs.org/api/async_hooks.html
- Jake Archibald — In The Loop (JSConf Asia 2018 talk, visual event-loop explainer). <https://www.youtube.com/watch?v=cCOL7MCQZW0>
- Chrome — Rendering performance / RAIL. <https://web.dev/articles/rail>

- Surma — When does the browser render a frame? <https://surma.dev/things/requestanimationframe/>

Chapter 3 — Node.js Internals: libuv, the Thread Pool, and Native Addons

What this chapter covers: Node.js is JavaScript executed by V8, but every I/O operation, timer, DNS lookup, and file access is mediated by a layered native stack: a C++ bindings layer that marshals between V8 objects and native memory, and libuv — the cross-platform asynchronous I/O library that owns the event loop, the thread pool, and the OS abstraction. This chapter opens the black box. You will trace a `fs.readFile` from JavaScript through the binding into a thread-pool work item and back through the event loop, understand the six phases of the libuv loop and exactly which Node API lands in which phase, reason about thread-pool sizing and starvation under load, write native addons at three levels of abstraction (raw N-API C, `node-addon-api` C++, and `napi-rs` Rust), offload CPU work without blocking the loop, choose correctly among `worker_threads`, `cluster`, and `child_process`, and finally understand how `AsyncLocalStorage` propagates request context across arbitrary async boundaries — and what it costs.

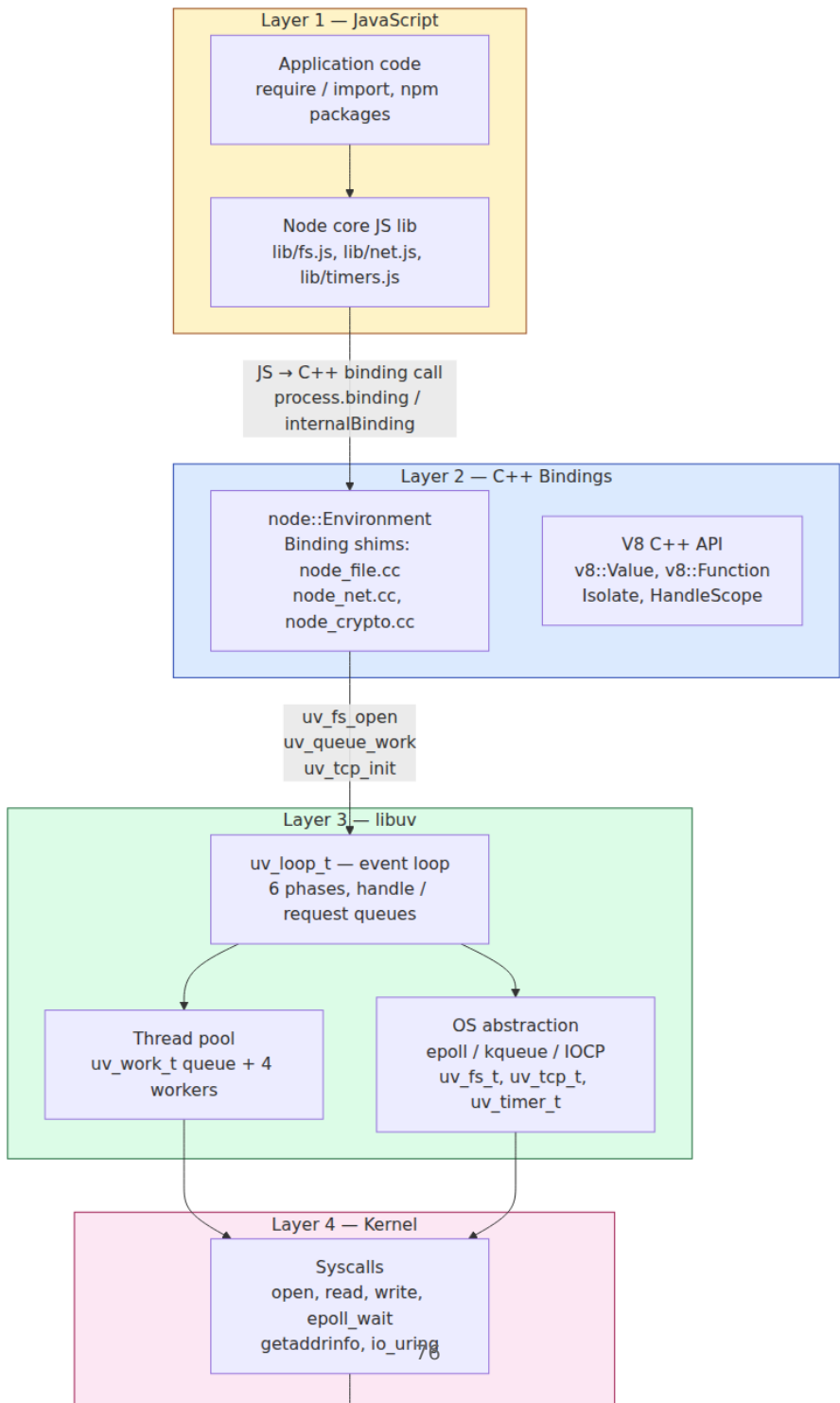
After reading this chapter you will be able to:

- Diagram the Node layer cake (JS → C++ bindings → libuv → kernel) and explain where V8, the binding shim, and libuv each draw their responsibility boundary.
- Walk the six libuv event-loop phases in order, name the C function that implements each phase, and map every major Node API (`setTimeout`, `setImmediate`, `fs.*`, `net`, `process.nextTick`, Promises) to its phase or queue.
- Explain the libuv thread-pool architecture — `uv_queue_work`, `uv_work_t`, the lock-free work queue, the `UV_THREADPOOL_SIZE` default of 4, and why CPU-bound work and `fs / dns / crypto` contend on the same pool — and size it correctly for production.
- Distinguish the three I/O paths through libuv (non-blocking poll for sockets, thread-pool for filesystem/DNS, heap for timers) and predict their latency and ordering properties.

- Write a native addon at each of the four abstraction tiers (raw N-API C, `node-addon-api` C++, `napi-rs` Rust, Neon) and articulate the ABI-stability trade-off that motivates N-API.
 - Manage N-API handle scopes, references, and object lifetime correctly — `napi_open_handle_scope` / `napi_close_handle_scope` , `napi_create_reference` / `napi_delete_reference` , finalizers — and diagnose leaks caused by escaped handles.
 - Implement an asynchronous addon with `napi_create_async_work` / `napi_queue_async_work` so that native CPU work runs on the thread pool and completes on the main thread without blocking the loop.
 - Choose among `worker_threads` , `cluster` , and `child_process` on the basis of isolation, memory sharing, start-up cost, and failure domain — and justify the choice with a quantitative model.
 - Use `AsyncLocalStorage` to propagate trace IDs, tenant IDs, and auth context across async hops, explain how `async_hooks` tracks the causal chain under the hood, and quantify its overhead so you can decide when to disable it on the hot path.
-

3.1 The Node Layer Cake

Node is four layers. Every call crosses at least two of them. Understanding which layer does what is the prerequisite for every performance investigation, every native addon, and every production tuning decision.



Layer 1 — JavaScript (V8). Your code and Node's JavaScript standard library (`lib/*.js`). `lib/fs.js` does not perform I/O; it validates arguments, creates a request object, and forwards to the binding. For example, `fs.readFile` in `lib/fs.js` normalizes the path and options, allocates a `FSReqCallback`, and calls `binding.open` / `binding.read` — which are C++ functions exposed via `internalBinding('fs')`.

Layer 2 — C++ bindings (`src/*.cc`). Each subsystem has a corresponding C++ file: `src/node_file.cc` for `fs`, `src/node_net.cc` for `net`, `src/node_crypto.cc` for `crypto`, `src/node_zlib.cc` for `zlib`. The binding's job is strictly translation: convert V8 values (`v8::String`, `v8::Function`, `node::Buffer`) into C structs (`uv_fs_t`, `uv_work_t`, `uv_buf_t`), enqueue work with `libuv`, and convert results back into V8 values when the callback fires. The binding owns no I/O logic — that belongs to `libuv`.

Layer 3 — libuv (`deps/uv/`). A standalone C library (~30K lines) that provides three things: (a) the event loop (`uv_loop_t`, `uv_run`), (b) the thread pool, and (c) cross-platform handles and requests (`uv_tcp_t`, `uv_udp_t`, `uv_fs_t`, `uv_getaddrinfo_t`, `uv_timer_t`). Node links `libuv` statically. The `libuv` version is pinned per Node release (e.g., Node 20 ships `libuv` 1.44, Node 22 ships `libuv` 1.48). `libuv` knows nothing about V8 or JavaScript; it operates on C callbacks.

Layer 4 — Kernel. `libuv` ultimately calls kernel interfaces: `epoll_wait` / `epoll_ctl` on Linux, `kqueue` / `kevent` on macOS/BSD, `GetQueuedCompletionStatusEx` / `IOCP` on Windows, plus `open` / `read` / `write` / `getaddrinfo` / `io_uring` for the operations that the thread pool executes.

Tracing a `fs.readFile` through the cake

Concrete path for `fs.readFile('/etc/hosts', 'utf8', cb)` on Linux:

```
// lib/fs.js (simplified) – Layer 1
function readFile(path, options, callback) {
  // 1. Normalize args, validate encoding
  const req = new FSReqCallback();
  req.oncomplete = callback;
  // 2. Forward to C++ binding
  binding.open(pathModule.toNamespacedPath(path),
    stringToFlags(options.flag), 0o666, req);
  // binding is internalBinding('fs') → src/node_file.cc::Open()
}
```

```
// src/node_file.cc (simplified) – Layer 2
void Open(const FunctionCallbackInfo<Value>& args) {
    FSReqBase* req_wrap = GetReqWrap(args[3]); // JS callback wrapper
    const char* path = *Utf8Value(isolate, args[0]);
    int flags = args[1]->Int32Value(ctx).ToChecked();
    // 3. Allocate libuv request, enqueue on thread pool
    FSReq* req = new FSReq(req_wrap);
    uv_fs_open(env->event_loop(), req->req(), path, flags, 0644, AfterOpen);
    // uv_fs_* always goes through the thread pool – see §3.3
}
void AfterOpen(uv_fs_t* req) {
    // 4. Runs on main thread after thread-pool work completes
    FSReq* wrap = ContainerOf(req);
    // Convert result → V8 value, schedule JS callback via MakeCallback
    wrap->ResolveAndCleanup();
}
```

```
// deps/uv/src/unix/fs.c (simplified) – Layer 3
void uv_fs_open(uv_loop_t* loop, uv_fs_t* req,
               const char* path, int flags, int mode,
               uv_fs_cb cb) {
    INIT_REQ(req, UV_FS); // req->cb = cb, req->loop = loop
    req->path = uv_strdup(path);
    // 5. Enqueue as thread-pool work – NOT inline I/O
    uv_work_submit(loop, &req->work_req,
                  UV_WORK_FAST_IO, // hint: may be fast path with
io_uring
                  uv_fs_work,      // worker thread: calls open(2)
                  uv_fs_done);     // main thread: invokes cb
}
void uv_fs_work(struct uv_work* w) {
    uv_fs_t* req = container_of(w, uv_fs_t, work_req);
    // 6. This runs on a thread-pool worker – may block
    req->result = open(req->path, req->flags, req->mode);
}
void uv_fs_done(struct uv_work* w, int status) {
    uv_fs_t* req = container_of(w, uv_fs_t, work_req);
    // 7. Back on the main thread, inside the event loop's pending phase
    req->cb(req); // → AfterOpen in node_file.cc → JS callback
}
```

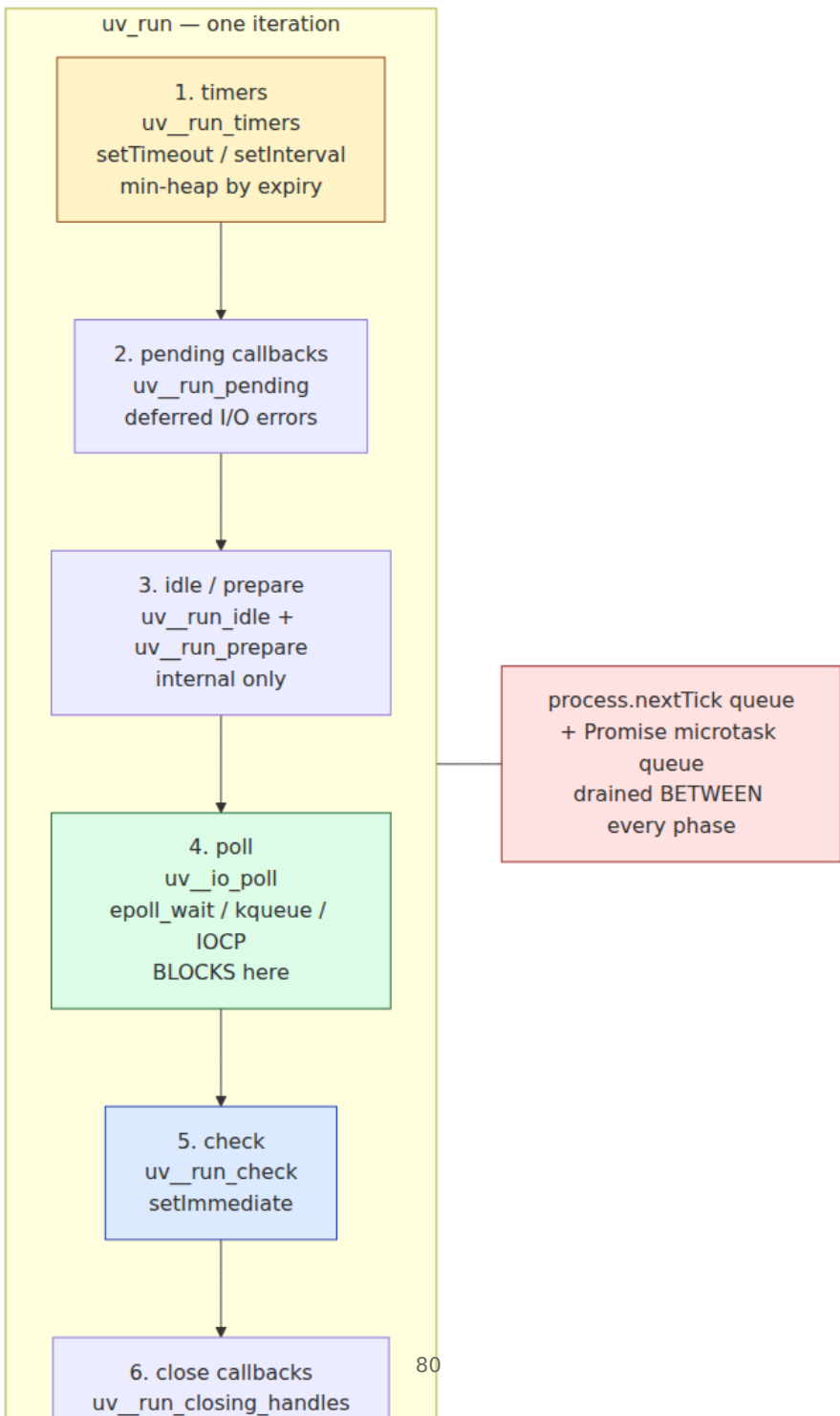
The key observation: `open(2)` — a potentially blocking syscall — never executes on the main thread. It is dispatched to a worker thread. The main thread only handles the enqueue and the completion callback. This is why Node can serve thousands of concurrent `fs` operations on a single

thread without blocking the loop on disk I/O — at the cost of thread-pool contention, which we examine next.

Distributed-systems lens. In a fleet of Node services, the layer cake repeats per process. Layer 4 (kernel) is shared across containers on the same host via the host kernel and cgroup limits. A noisy neighbor that saturates disk I/O will stall every Node thread pool on that host simultaneously, because they all contend on the same block-device queue. This is why `UV_THREADPOOL_SIZE` tuning (§3.3) must be coordinated with cgroup `io.weight` and disk provisioning — the layers above cannot fix contention introduced at the bottom.

3.2 libuv and the Event Loop: Six Phases

The libuv event loop is not a single queue. It is a deterministic sequence of phases, each draining a different set of handles and requests. Node adds two extra micro-queues (`process.nextTick` and the V8 Promise microtask queue) that interleave between every phase transition — which is why Node ordering diverges from the browser.



The C implementation in `deps/uv/src/unix/core.c` is approximately:

```
// deps/uv/src/unix/core.c - uv_run (simplified, error handling
omitted)
int uv_run(uv_loop_t* loop, uv_run_mode mode) {
    int r;
    while ((r = uv_loop_alive(loop)) != 0) {
        uv_update_time(loop);           // sample CLOCK_MONOTONIC → loop-
>time
        uv_run_timers(loop);           // phase 1: expired timers
        uv_run_pending(loop);          // phase 2: pending callbacks
        uv_run_idle(loop);             // phase 3a: idle handles
        (internal)
        uv_run_prepare(loop);          // phase 3b: prepare handles
        (internal)

        // Between every phase: Node drains nextTick + microtasks
        // (Node patches this via env->RunBeforeUpdates() in
node::Environment)

        uv_io_poll(loop, timeout);     // phase 4: block for I/O (or not)
        uv_run_check(loop);            // phase 5: setImmediate
        uv_run_closing_handles(loop);  // phase 6: close callbacks

        // Again: drain nextTick + microtasks before next iteration
    }
    return r;
}
```

What each phase does and which Node API it serves

Phase here	C function Observable ordering	Node API that enqueues
1 timers setInterval	uv_run_timers Fires when loop->time >= expiry	setTimeout,
2 pending poll	uv_run_pending Rarely visible to JS	I/O errors deferred from
3a idle bookkeeping)	uv_run_idle No public API	(internal – Node
3b prepare bookkeeping)	uv_run_prepare No public API	(internal – Node
4 poll crypto	uv_io_poll I/O callbacks; blocks for timeout	fs.*, net, http, dns,
5 check setImmediate	uv_run_check	Always after poll
6 close server.close	uv_run_closing_handles Cleanup	socket.on('close'),
between every phase: starves I/O)		process.nextTick (highest priority,
(queueMicrotask, .then, await)		Promise microtasks

The mapping in the table above is the essential reference — `setTimeout / setInterval` → `timers`, `fs / net / http` → `poll`, `setImmediate` → `check`, `close` callbacks → `close`, with `process.nextTick` and Promise microtasks interleaved between every phase.

Timers are a min-heap, not a queue

`uv_timer_t` handles are stored in a binary min-heap keyed by `timeout = loop->time + delay`. `uv_run_timers` peeks at the heap root; if `root->timeout <= loop->time`, it pops and fires. This means:

- Timers fire in expiry order, not insertion order — `setTimeout(fn, 100)` inserted after `setTimeout(fn, 10)` fires second despite being inserted later if its expiry is later.
- `loop->time` is sampled once per iteration via `uv_update_time` (which calls `clock_gettime(CLOCK_MONOTONIC)`). A timer scheduled with `setTimeout(fn, 0)` will not fire until the *next* timers phase — it cannot fire inline.

- Coalescing: if the loop was blocked in `poll` for 50 ms, every timer that expired during that window fires together in the next timers phase. There is no per-timer thread; there is one heap and one phase.
- `setInterval` is reinserted into the heap after each firing with `timeout = loop->time + repeat`. Drift accumulates if the loop is busy.

```
// Timer coalescing demo – run with: node timers-demo.js
console.log(Date.now(), 'start');

setTimeout(() => console.log(Date.now(), 'timeout 10ms'), 10);
setTimeout(() => console.log(Date.now(), 'timeout 10ms (second)'), 10);
setTimeout(() => console.log(Date.now(), 'timeout 50ms'), 50);

// Block the loop for 60ms – all three timers coalesce
const end = Date.now() + 60;
while (Date.now() < end) { /* busy spin – blocks poll AND timers */ }

console.log(Date.now(), 'unblocked');
// Output (timestamps approximate):
// 1713700000000 start
// 1713700000061 unblocked
// 1713700000061 timeout 10ms           ← all three fire together
// 1713700000061 timeout 10ms (second) ← same tick, insertion order
// 1713700000061 timeout 50ms          ← coalesced, 11ms late
```

`poll` is the blocking heart

`uv_io_poll` computes a timeout from the next timer expiry: `timeout = max(0, next_timer_expiry - loop->time)`. Then it calls `epoll_wait(epfd, events, maxevents, timeout)` on Linux (or `kevent / GetQueuedCompletionStatusEx` on other platforms). If no timers are pending, `timeout` may be `-1` (block indefinitely) or `0` (poll without blocking) depending on whether the loop has other live handles.

This is where Node spends most of its wall-clock time when idle — parked in `epoll_wait`. Every inbound TCP connection, every readable socket, every completed `fs` operation that was dispatched to the thread pool wakes the loop here. The `poll` phase then invokes the associated C callbacks, which in turn schedule JavaScript callbacks.

`setImmediate` vs `setTimeout(fn, 0)` — why they race

`setImmediate` enqueues a `uv_check_t` handle (check phase). `setTimeout(fn, 0)` enqueues a `uv_timer_t` with `timeout = loop->time + 1` (clamped to 1 ms minimum). Their relative order depends on *when* they are scheduled:

```
// Case 1: scheduled from top-level script – RACE (non-deterministic)
setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));
// Output: either order – depends on whether loop->time advanced
//          past the timer threshold before poll entered.

// Case 2: scheduled from inside poll (I/O callback) – DETERMINISTIC
import fs from 'node:fs';
fs.readFile(__filename, () => {
  setTimeout(() => console.log('timeout inside I/O'), 0);
  setImmediate(() => console.log('immediate inside I/O'));
});
// Output (always):
//   immediate inside I/O   ← check phase runs right after poll
//   timeout inside I/O     ← timers phase is next iteration
```

Rule: inside an I/O callback (poll phase), `setImmediate` always fires before `setTimeout(fn, 0)` because `check` is the next phase after `poll`, while `timers` is the first phase of the *next* iteration. From top-level code, the order is a race because the loop may or may not have entered `poll` before the timer heap is checked.

`process.nextTick` and Promises between phases

Node drains two additional queues between every phase transition (and after every callback within a phase):

1. `process.nextTick` queue — a `FixedQueue` in `lib/internal/process/task_queues.js`, drained exhaustively including ticks enqueued while draining. This queue has *higher priority than Promises* and, if recursed, starves I/O indefinitely — there is no yielding.
2. V8 Promise microtask queue — drained via `v8::Isolate::PerformMicrotaskCheckpoint()`, which empties `queueMicrotask` / `Promise.then` / `await` continuations.

```
// nextTick vs microtask vs phase ordering – Node 20
import fs from 'node:fs';

setTimeout(() => console.log('1: timers'), 0);
setImmediate(() => console.log('2: check'));

fs.readFile(__filename, () => {
  console.log('3: poll (I/O callback)');
  process.nextTick(() => console.log('4: nextTick inside poll'));
  Promise.resolve().then(() => console.log('5: promise inside poll'));
  setTimeout(() => console.log('6: timer inside poll'), 0);
  setImmediate(() => console.log('7: immediate inside poll'));
});

process.nextTick(() => console.log('8: nextTick top-level'));
Promise.resolve().then(() => console.log('9: promise top-level'));

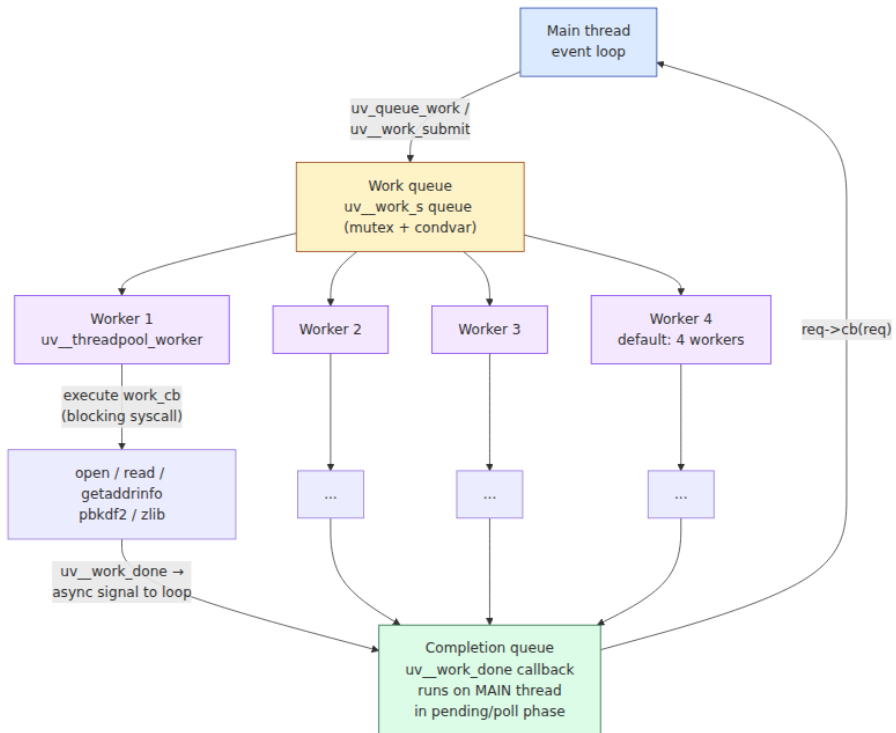
console.log('10: sync script end');

// Output (Node 20):
// 10: sync script end
// 8: nextTick top-level           ← nextTick drains before promises
// 9: promise top-level
// 1: timers                      ← timers phase (next iteration)
//   nextTick/promise drain between phases (none here)
// 3: poll (I/O callback)         ← poll phase
// 4: nextTick inside poll        ← nextTick drains before promises,
//   before next phase
// 5: promise inside poll
// 2: check – wait, actually 7 fires here: check phase is immediately
//   after poll
// 7: immediate inside poll       ← check phase (same iteration as
//   poll)
// 2: check (top-level immediate) ← also check phase, FIFO with 7
// 6: timer inside poll           ← timers phase of NEXT iteration
```

3.3 The Thread Pool

Not all operations can be non-blocking. Filesystem I/O on Linux has no uniformly non-blocking interface (prior to `io_uring`), DNS resolution via `getaddrinfo` is blocking, and CPU-bound work like `crypto.pbkdf2` and `zlib` compression must run somewhere. libuv's answer is a fixed-size thread pool.

Architecture



uv_queue_work — the core primitive

Every thread-pool operation goes through one function:

```
// deps/uv/include/uv.h
typedef struct uv_work_s {
    uv_loop_t* loop;
    uv_work_cb work;      // runs on worker thread – may block
    uv_after_work_cb after_work; // runs on main thread – must not block
    void* data;           // caller context (e.g., uv_fs_t*, custom struct)
} uv_work_t;

typedef void (*uv_work_cb)(uv_work_t* req);
typedef void (*uv_after_work_cb)(uv_work_t* req, int status);

int uv_queue_work(uv_loop_t* loop, uv_work_t* req,
                  uv_work_cb work_cb,
                  uv_after_work_cb after_work_cb);
```

Pseudocode for the internal implementation (`deps/uv/src/threadpool.c`):

```

// Simplified thread-pool internals – deps/uv/src/threadpool.c
#define MAX_THREADPOOL_SIZE 1024

static uv_work_queue wq;           // global work queue (mutex +
condvar)
static uv_thread_t* workers;       // array of pthreads
static int nthreads = 4;           // default; overridden by
UV_THREADPOOL_SIZE

void uv_work_submit(uv_loop_t* loop, struct uv_work* w,
enum uv_work_kind kind,
uv_work_cb work, uv_work_done_cb done) {
    w->loop = loop;
    w->work = work;
    w->done = done;
    uv_mutex_lock(&wq.mutex);
    QUEUE_INSERT_TAIL(&wq.queue, &w->wq); // FIFO enqueue
    uv_cond_signal(&wq.cond);              // wake one worker
    uv_mutex_unlock(&wq.mutex);
}

// Each worker thread loops forever:
void uv_threadpool_worker(void* arg) {
    while (1) {
        uv_mutex_lock(&wq.mutex);
        while (QUEUE_EMPTY(&wq.queue))
            uv_cond_wait(&wq.cond, &wq.mutex); // park until work arrives
        QUEUE* q = QUEUE_HEAD(&wq.queue);
        QUEUE_REMOVE(q);
        uv_mutex_unlock(&wq.mutex);

        struct uv_work* w = QUEUE_DATA(q, struct uv_work, wq);
        w->work(w); // ← BLOCKING: open(), getaddrinfo(), pbkdf2(), ...

        // Signal completion back to the main thread via async handle
        uv_mutex_lock(&w->loop->wq_mutex);
        QUEUE_INSERT_TAIL(&w->loop->wq, &w->wq);
        uv_async_send(&w->loop->wq_async); // wakes epoll_wait in poll
phase
        uv_mutex_unlock(&w->loop->wq_mutex);
    }
}

// Main thread – called from uv_work_done inside the loop iteration:
void uv_work_done(uv_async_t* handle) {
    uv_loop_t* loop = handle->loop;
    QUEUE wq;
    QUEUE_MOVE(&loop->wq, &wq); // drain completion queue
    while (!QUEUE_EMPTY(&wq)) {
        QUEUE* q = QUEUE_HEAD(&wq);

```

```

    QUEUE_REMOVE(q);
    struct uv_work* w = QUEUE_DATA(q, struct uv_work, wq);
    w->done(w, 0); // ← runs on main thread: invokes JS callback
  }
}

```

What uses the thread pool vs. what does not

Operation	Thread pool?	Why
<code>fs.readFile</code> , <code>fs.writeFile</code> , <code>fs.open</code> , <code>fs.stat</code> , <code>fs.readdir</code>	Yes — all <code>uv_fs_*</code>	POSIX filesystem syscalls have no non-blocking mode on Linux (pre- <code>io_uring</code>)
<code>dns.lookup</code> (<code>getaddrinfo</code>)	Yes	<code>getaddrinfo</code> is blocking; parses <code>/etc/hosts</code> , <code>/etc/nsswitch.conf</code> , DNS
<code>dns.resolve</code> (<code>c-ares</code>)	No	Uses libuv's c-ares integration — non-blocking DNS over UDP via <code>poll</code>
<code>crypto.pbkdf2</code> , <code>crypto.scrypt</code> , <code>crypto.randomBytes</code>	Yes	CPU-bound; deliberately offloaded
<code>zlib.gzip</code> , <code>zlib.deflate</code>	Yes	CPU-bound compression
<code>net.connect</code> , <code>net.createServer</code> , <code>http.request</code>	No	Non-blocking sockets via <code>epoll</code> in <code>poll</code> phase
<code>setTimeout</code> , <code>setInterval</code>	No	Timer heap in <code>timers</code> phase
<code>setImmediate</code>	No	Check handle in <code>check</code> phase

Historical note. Node 20+ and libuv 1.44+ can use `io_uring` for filesystem operations when the kernel supports it (`IORING_SETUP` available and not blocked by `seccomp`). In that path, `uv_fs_*` operations bypass the thread pool entirely and are submitted as SQEs to the ring, with completions arriving via CQEs in the `poll` phase — similar to how network I/O already works. The thread pool remains the fallback. In containers with restrictive `seccomp` profiles (the Docker default before


```
20.10.12 blocks io_uring), the pool path is always used. Check
UV_USE_IO_URING and require('os').availableParallelism()
interaction when tuning.
```

Sizing the pool — `UV_THREADPOOL_SIZE`

Default is 4 threads. Override at process start (it is read once, before the pool is created):

```
UV_THREADPOOL_SIZE=8 node server.js
# or programmatically before first use (Node 20+):
# Must be set before the pool initializes – effectively at startup
```

```
// threadpool-exhaustion.js – demonstrates starvation
import fs from 'node:fs';
import crypto from 'node:crypto';

const start = Date.now();
const log = (label) => console.log(`${Date.now() - start}ms  ${label}`)
;

// Saturate the pool with 4 blocking fs ops (default pool = 4)
for (let i = 0; i < 4; i++) {
  fs.readFile('/dev/urandom', { encoding: null }, () => log(`fs ${i}
done`));
  // Each fs.readFile occupies one worker until the read completes
}

// This crypto operation must wait – no idle worker
crypto.pbkdf2('password', 'salt', 100000, 64, 'sha512', () => {
  log('pbkdf2 done – waited for pool slot');
});

// This timer is NOT on the pool – fires on time regardless
setTimeout(() => log('timer 10ms – not on pool'), 10);

// With UV_THREADPOOL_SIZE=4, expect:
// ~10ms timer 10ms – not on pool      ← timers phase, unaffected
// ~??ms fs 0..3 done                  ← pool workers completing
// ~??ms pbkdf2 done – waited         ← queued behind fs ops

// With UV_THREADPOOL_SIZE=8, pbkdf2 starts immediately alongside fs
ops.

// Observe pool utilization via trace events:
// node --trace-event-categories node.threadpool threadpool-
exhaustion.js
```

Sizing guidance for production:

- **4 (default)** is sufficient for lightweight `fs` usage (config file reads at startup) but starves under concurrent `fs` + `crypto` load.
- **8** is a common production choice for services that do moderate filesystem or crypto work. Each thread costs ~8 MB stack + TLS overhead.
- **128** (maximum effective) is the hard cap in libuv (`MAX_THREADPOOL_SIZE`), but beyond ~16 you hit diminishing returns and increased context-switch overhead. The pool is not work-stealing; it is a single FIFO queue with `pthread_cond_signal` — one wakeup per enqueue.
- **Never size the pool larger than `availableParallelism()`** without reason. On a 2-vCPU container with `UV_THREADPOOL_SIZE=16`, 14 threads are always contending for 2 cores — pure overhead.
- **Isolate CPU-bound work** to `worker_threads` (§3.8) rather than growing the shared pool, which is contended by `fs`, `dns`, and `crypto` simultaneously.

Distributed-systems lens. Thread-pool starvation is a cross-cutting failure mode in fleets. A single endpoint that triggers `crypto.pbkdf2` per request (e.g., password hashing on login) can saturate the pool, causing unrelated `fs.readFile` calls (e.g., reading TLS certificates, loading feature flags from disk) and `dns.lookup` calls to queue behind it — even though they are logically independent. In a microservice that does both authentication and file serving, this manifests as p99 latency spikes on the file-serving path caused by load on the auth path. The fix is isolation: move `pbkdf2 / scrypt / bcrypt` to `worker_threads` or an external auth service so that the shared pool is not a single point of contention. Monitor `perf_hooks.monitorEventLoopDelay` and thread-pool queue depth (via `async_hooks` or `trace_events`) to distinguish pool starvation from loop blocking.

3.4 Three Paths Through libuv: Filesystem, Networking, and Timers

libuv routes operations through fundamentally different mechanisms depending on whether the underlying kernel interface is blocking or non-blocking.

Path 1 — Filesystem: always through the thread pool (or `io_uring`)

Every `uv_fs_*` operation — `open`, `read`, `write`, `stat`, `readdir`, `unlink`, `mkdir` — is dispatched as a `uv_work_t` to the thread pool (§3.3). The worker thread performs the blocking syscall synchronously, then signals completion back to the loop. Ordering is FIFO per the work queue, but completion order depends on syscall latency (a fast `stat` may complete before a slow `read` enqueued earlier).

On kernels with `io_uring` enabled, libuv 1.44+ submits `uv_fs_*` as SQEs and polls for CQEs in the `poll` phase — eliminating the thread-pool hop. The JavaScript-visible ordering is identical; only the internal path changes.

```
// fs ordering — enqueue order ≠ completion order when latencies differ
import fs from 'node:fs';

fs.readFile('/tmp/large-file.bin', () => console.log('large read done'))
);
fs.stat('/tmp/small-file.txt', () => console.log('stat done'));
// stat may complete first despite being enqueued second —
// whichever worker finishes first signals first.
```

Path 2 — Networking: non-blocking, via `poll` phase

TCP/UDP sockets, pipes, and TTYs use non-blocking file descriptors registered with the loop's `epoll` instance. No thread-pool involvement. The flow is:

1. `uv_tcp_init` / `uv_tcp_bind` / `uv_listen` — creates a non-blocking socket, registers it with `epoll` via `epoll_ctl(EPOLL_CTL_ADD)`.
2. `uv_io_poll` calls `epoll_wait` — returns when the socket is readable/writable.
3. The loop invokes the handle's callback (`on_connection`, `on_read`) directly on the main thread.

```
// Networking stays on the main thread – no pool
import net from 'node:net';

const server = net.createServer((socket) => {
  // This callback fires inside poll phase – main thread
  socket.on('data', (chunk) => {
    // Also poll phase – epoll_wait returned EPOLLIN for this fd
    socket.write(chunk); // non-blocking write, buffered in libuv
  });
});
server.listen(3000, () => console.log('listening'));
// Under the hood:
// socket fd → epoll_ctl(ADD, EPOLLIN)
// epoll_wait returns → uv_tcp_io → Node's OnConnection → JS
// callback
```

DNS is the exception within networking: `dns.lookup` calls `getaddrinfo` (blocking, thread pool), while `dns.resolve` uses c-ares (non-blocking, poll phase). This distinction matters for latency-sensitive services — prefer `dns.resolve` or cache lookup results.

Path 3 — Timers: min-heap in the `timers` phase

No I/O, no thread pool. Timers are `uv_timer_t` handles stored in a binary min-heap keyed by absolute expiry (`loop->time + delay`). `uv_run_timers` drains every expired entry in heap order. See §3.2 for coalescing and drift behavior.

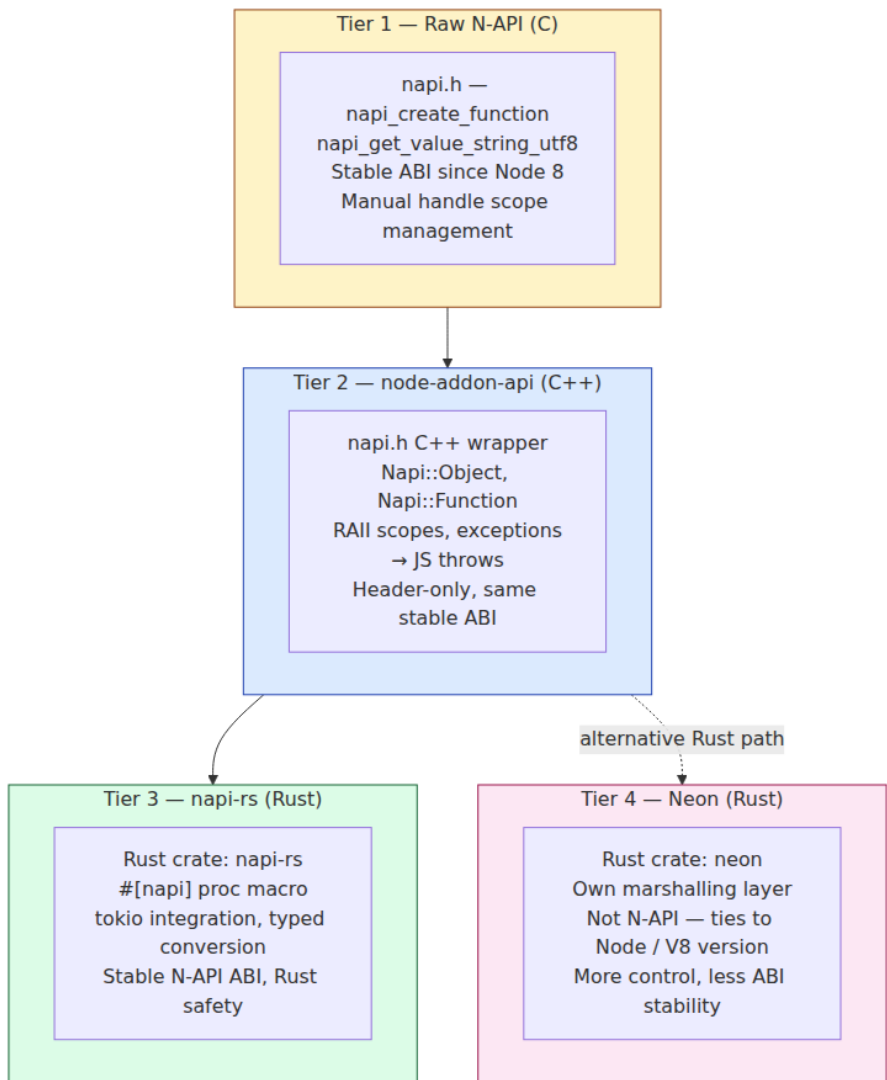
```
// Timer heap – expiry order, not insertion order
setTimeout(() => console.log('100ms'), 100);
setTimeout(() => console.log('10ms'), 10);
setTimeout(() => console.log('50ms'), 50);
// Output: 10ms → 50ms → 100ms (heap order by expiry)
```

Path	Kernel interface	Thread pool?	Loop phase	Latency source
Filesystem	<code>open / read / write</code> (blocking) or <code>io_uring</code> SQE	Yes (or ring)	Completion via <code>pending / poll</code>	Disk I/O + pool queue depth
Networking	<code>epoll_wait</code> on non-blocking fd	No	<code>poll</code>	Network RTT + kernel buffer

Path	Kernel interface	Thread pool?	Loop phase	Latency source
Timers	<code>clock_gettime</code> + heap	No	<code>timers</code>	Loop busy time (drift)
DNS (<code>lookup</code>)	<code>getaddrinfo</code> (blocking)	Yes	Completion via <code>pending</code> / <code>poll</code>	Resolver latency + pool queue
DNS (<code>resolve</code>)	c-ares over UDP	No	<code>poll</code>	DNS RTT
Crypto / zlib	CPU-bound	Yes	Completion via <code>pending</code> / <code>poll</code>	CPU time + pool queue

3.5 Native Addons: Four Tiers of Extensibility

When JavaScript is not fast enough or must interface with a native library, Node provides four tiers of native addon abstraction — each trading raw control for ergonomics and ABI stability.



Tier	Language	Header / Crate	ABI	Ergonomics	When to use
1. N-API (C)	C	<code>node_api.h</code>	Stable — addon binary works across Node major versions without recompile	Verbose, manual scope/ref management, error codes	Maximum compatibility, minimal dependencies, embedding
2. node-addon-api (C++)	C++	<code>napi.h</code> (header-only wrapper over N-API)	Stable (same as N-API)	RAII, exceptions, <code>Napi::ObjectWrap</code> , type-safe	Most C++ addons — the default choice for C++
3. napi-rs	Rust	<code>napi</code> + <code>napi-derive</code> crates	Stable (N-API)	<code>#[napi]</code> macro, <code>Result</code> → JS exception, <code>tokio</code> async	Rust addons that need ABI stability across Node versions
4. Neon	Rust	<code>neon</code> crate	Unstable — tied to Node/V8 version, requires recompile per major	<code>#[neon::main]</code> , <code>JsFunction</code> , <code>Channel</code> for threading	Rust addons that need V8-level control, Node-API not sufficient

The stability distinction is critical. **N-API** (now called **Node-API**, `node_api.h`) is a *forward-compatible ABI guarantee*: an addon compiled against N-API v6 runs on any Node version that supports N-API v6 or later, without recompilation. This is why `napi-rs` and `node-addon-api` both build on N-API — they inherit its stability. **Neon** and raw **NAN** (Native Abstractions for Node, the predecessor — now deprecated) bind directly to V8 internals and break across Node major versions.

N-API is versioned independently of Node. Current N-API version is 9 (Node 20+). Each version adds new functions while keeping all previous

ones. An addon's `package.json` declares the required version via `napi_versions` in `node-api` metadata, and `node-gyp` / `prebuildify` select the right header.

Tier 1 — Raw N-API (C)

The lowest level. Every operation returns a `napi_status` code; every value is a `napi_value` opaque handle. You manage scopes and references explicitly.


```

// addon.c – raw N-API addon: exposes function greet(name) → "Hello,
<name>!"
#include <node_api.h>
#include <assert.h>
#include <string.h>

static napi_value Greet(napi_env env, napi_callback_info info) {
    napi_status status;

    // 1. Extract arguments
    size_t argc = 1;
    napi_value argv[1];
    status = napi_get_cb_info(env, info, &argc, argv, NULL, NULL);
    assert(status == napi_ok);

    // 2. Convert JS string → C string
    size_t str_len;
    status = napi_get_value_string_utf8(env, argv[0], NULL, 0, &str_len);
    assert(status == napi_ok);

    char* name = malloc(str_len + 1);
    status = napi_get_value_string_utf8(env, argv[0], name, str_len + 1,
    NULL);
    assert(status == napi_ok);

    // 3. Build result string
    const char* prefix = "Hello, ";
    size_t result_len = strlen(prefix) + str_len + 1;
    char* result = malloc(result_len);
    snprintf(result, result_len, "%s%s!", prefix, name);

    // 4. Convert C string → JS string
    napi_value js_result;
    status = napi_create_string_utf8(env, result, NAPI_AUTO_LENGTH, &js_r
    esult);
    assert(status == napi_ok);

    free(name);
    free(result);
    return js_result;
}

static napi_value Init(napi_env env, napi_value exports) {
    napi_value fn;
    napi_create_function(env, "greet", NAPI_AUTO_LENGTH, Greet, NULL,
    &fn);
    napi_set_named_property(env, exports, "greet", fn);
    return exports;
}

```

```
NAPI_MODULE(NODE_GYP_MODULE_NAME, Init)
```

```
// binding.gyp – build config for node-gyp
{
  "targets": [{
    "target_name": "addon",
    "sources": ["addon.c"]
  }]
}
```

```
// Usage – after: npx node-gyp configure && npx node-gyp build
const addon = require('./build/Release/addon.node');
console.log(addon.greet('Ada')); // Hello, Ada!
```

Every `napi_value` returned by `napi_create_*` or `napi_get_*` is a handle that lives until the enclosing handle scope is closed. In a synchronous callback like `Greet`, the scope is the callback's implicit scope — it closes when `Greet` returns. For values that must outlive the callback, you need a reference (§3.6).

Tier 2 — node-addon-api (C++)

A header-only C++ wrapper that turns error codes into exceptions and manual scope management into RAII.

```

// addon.cc - node-addon-api: same greet function, C++ ergonomics
#include <napi.h>

Napi::String Greet(const Napi::CallbackInfo& info) {
    Napi::Env env = info.Env();

    // Type check - throws JS TypeError if wrong type
    if (!info[0].IsString()) {
        Napi::TypeError::New(env, "String expected").ThrowAsJavaScriptException();
        return Napi::String::New(env, "");
    }

    std::string name = info[0].As<Napi::String>().Utf8Value();
    std::string result = "Hello, " + name + "!";

    return Napi::String::New(env, result);
}

// ObjectWrap example - stateful class exposed to JS
class Counter : public Napi::ObjectWrap<Counter> {
public:
    static Napi::Object Init(Napi::Env env, Napi::Object exports) {
        Napi::Function ctor = DefineClass(env, "Counter", {
            InstanceMethod("increment", &Counter::Increment),
            InstanceMethod("value", &Counter::Value),
        });
        exports.Set("Counter", ctor);
        return exports;
    }

    Counter(const Napi::CallbackInfo& info) : Napi::ObjectWrap<Counter>(info) {
        count_ = info[0].IsNumber() ?
info[0].As<Napi::Number>().Int32Value() : 0;
    }

private:
    Napi::Value Increment(const Napi::CallbackInfo& info) {
        count_++;
        return Napi::Number::New(info.Env(), count_);
    }
    Napi::Value Value(const Napi::CallbackInfo& info) {
        return Napi::Number::New(info.Env(), count_);
    }
    int count_;
};

Napi::Object Init(Napi::Env env, Napi::Object exports) {
    exports.Set("greet", Napi::Function::New(env, Greet));
}

```

```
Counter::Init(env, exports);
return exports;
}

NODE_API_MODULE(addon, Init)
```

```
// Usage
const { greet, Counter } = require('./build/Release/addon.node');
console.log(greet('Grace')); // Hello, Grace!

const c = new Counter(10);
console.log(c.value()); // 10
console.log(c.increment()); // 11
```

`Napi::HandleScope` and `Napi::EscapableHandleScope` are RAII wrappers — the scope closes when the C++ object destructs. `Napi::ObjectWrap` ties a C++ object's lifetime to a JS object's GC lifetime via a finalizer. No manual `napi_close_handle_scope` needed in the common case.

Tier 3 — napi-rs (Rust)

Rust addons via `napi-rs` — the most ergonomic path for Rust, with full N-API ABI stability.

```

// src/lib.rs - napi-rs addon: Cargo.toml declares crate-type =
["cdylib"]
use napi_derive::napi;

// Simple function - #[napi] generates the N-API glue
#[napi]
pub fn greet(name: String) -> String {
    format!("Hello, {}!", name)
}

// Typed, fallible function - Result maps to JS exception on Err
#[napi]
pub fn fibonacci(n: u32) -> napi::Result<u32> {
    if n > 47 {
        return Err(napi::Error::from_reason("n > 47 would overflow
u32"));
    }
    let (mut a, mut b) = (0u32, 1u32);
    for _ in 0..n {
        let tmp = a + b;
        a = b;
        b = tmp;
    }
    Ok(a)
}

// Async function - runs on libuv thread pool via
napi_create_async_work
#[napi]
pub async fn hash_password(password: String) -> String {
    // This body runs on the thread pool - does not block the loop
    // napi-rs bridges Rust async -> N-API async work automatically
    let hash = tokio::task::spawn_blocking(move || {
        // Simulate expensive work (real code: argon2, bcrypt)
        let mut h: u64 = 0xcbf29ce484222325;
        for b in password.bytes() {
            h ^= b as u64;
            h = h.wrapping_mul(0x100000001b3);
        }
        format!("{:016x}", h)
    }).await.unwrap();
    hash
}

// Stateful class
#[napi]
pub struct Counter {
    count: i32,
}

```

```
#[napi]
impl Counter {
    #[napi(constructor)]
    pub fn new(initial: Option<i32>) -> Self {
        Counter { count: initial.unwrap_or(0) }
    }

    #[napi]
    pub fn increment(&mut self) -> i32 {
        self.count += 1;
        self.count
    }

    #[napi(getter)]
    pub fn value(&self) -> i32 {
        self.count
    }
}
```

```
# Cargo.toml
[package]
name = "my-addon"
version = "0.1.0"
edition = "2021"

[lib]
crate-type = ["cdylib"]

[dependencies]
napi = { version = "2", features = ["tokio_rt"] }
napi-derive = "2"

[build-dependencies]
napi-build = "2"
```

```
// Usage - after: npx napi build --platform
const { greet, fibonacci, hashPassword, Counter } = require('./
index.node');
console.log(greet('Ferris'));           // Hello, Ferris!
console.log(fibonacci(10));              // 55
console.log(await hashPassword('s3cret')); // 16-char hex hash - non-
blocking
const c = new Counter(10);
console.log(c.increment());              // 11
```

`napi-rs` handles `napi_ref`, handle scopes, and async work lifecycle automatically. The `#[napi]` proc macro generates the `napi_register_module_v1` boilerplate. Async functions annotated with

`#[napi]` that return `Future`s are automatically dispatched via `napi_create_async_work` — the Rust future's poll runs on the thread pool, and completion resumes on the main thread.

Tier 4 — Neon (Rust, V8-direct)

Neon bypasses N-API and binds directly to V8 via the `nan`-like `neon::context` API. It offers finer control over V8 handles but requires recompilation per Node major version.

```
// src/lib.rs — Neon addon (contrast with napi-rs above)
use neon::prelude::*;

fn greet(mut cx: FunctionContext) -> JsResult<JsString> {
    let name = cx.argument:::<JsString>(0)?.value(&mut cx);
    Ok(cx.string(format!("Hello, {}!", name)))
}

fn fibonacci(mut cx: FunctionContext) -> JsResult<JsNumber> {
    let n = cx.argument:::<JsNumber>(0)?.value(&mut cx) as u32;
    let (mut a, mut b) = (0u32, 1u32);
    for _ in 0..n { let t = a + b; a = b; b = t; }
    Ok(cx.number(a))
}

#[neon::main]
fn main(mut cx: ModuleContext) -> NeonResult<()> {
    cx.export_function("greet", greet)?;
    cx.export_function("fibonacci", fibonacci)?;
    Ok(())
}
```

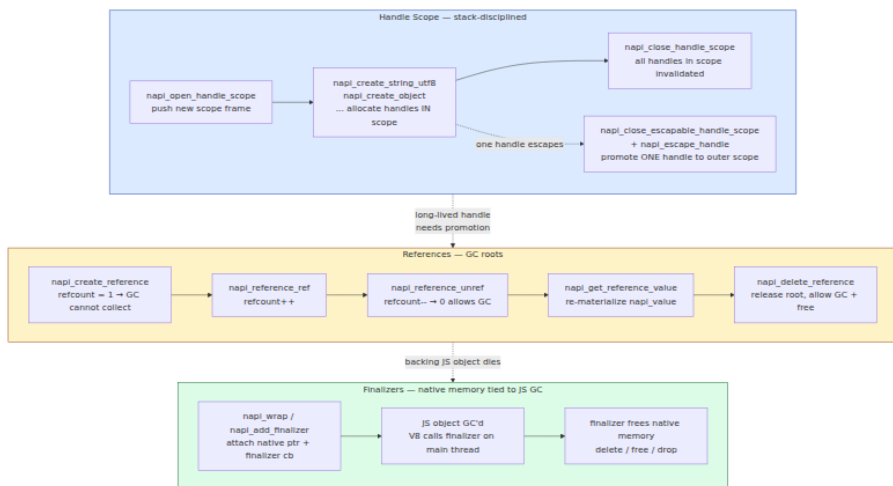
Neon is the right choice when you need V8 handle-level control (custom finalizers, direct `ArrayBuffer` backing store manipulation, `napi` not yet exposing a new V8 feature). For most Rust addons, `napi-rs` is preferred because its N-API foundation means a single compiled `.node` binary works across Node 16/18/20/22.

3.6 N-API Deep Dive: Handles, Scopes, References, and Lifecycle

N-API's handle system is the most common source of addon bugs — leaks, use-after-free, and GC surprises. Understanding it precisely is non-optional.

Handles and scopes

Every `napi_value` is a handle — an indirection through V8's handle table, not a direct pointer to a JS object. Handles are only valid within their handle scope.



Handle scopes — the rules:

1. Every N-API callback (`napi_callback`, `napi_async_execute_callback`, `finalizer`) enters with an implicit handle scope. All handles created inside the callback belong to that scope.
2. When the callback returns, its scope closes — every `napi_value` created inside becomes invalid. Returning a `napi_value` from the callback is safe because the caller promotes it; storing a `napi_value` in a C global and using it in a later callback is **use-after-free**.
3. To create many handles in a loop (e.g., building a large array), open a nested scope per iteration to avoid exhausting the handle table:


```

// Correct: scoped loop – handles freed per iteration
napi_value build_array(napi_env env, int n) {
    napi_value result;
    napi_create_array(env, &result);

    for (int i = 0; i < n; i++) {
        napi_handle_scope scope;
        napi_open_handle_scope(env, &scope);

        napi_value num, str;
        napi_create_int32(env, i, &num);
        napi_create_string_utf8(env, "item", NAPI_AUTO_LENGTH, &str);
        // ... use num, str ...

        napi_set_element(env, result, i, num);
        // Only handles NOT reachable from result need scope cleanup;
        // but intermediate handles (str) are freed here:
        napi_close_handle_scope(env, scope);
    }
    return result;
}

// Escapable scope – promote one handle to outer scope
napi_value create_greeting(napi_env env) {
    napi_escapable_handle_scope scope;
    napi_open_escapable_handle_scope(env, &scope);

    napi_value str;
    napi_create_string_utf8(env, "Hello from inner scope", NAPI_AUTO_LENGTH, &str);

    napi_value escaped;
    napi_escape_handle(env, scope, str, &escaped); // str survives scope close
    napi_close_escapable_handle_scope(env, scope);
    return escaped; // valid in outer scope
}

```

References — keeping a JS value alive across callbacks:

A `napi_ref` is a GC root. While its refcount is ≥ 1 , the referenced JS value cannot be collected — even if no JS code holds a reference to it.

```

// Storing a JS callback for later invocation (e.g., async completion)
napi_ref callback_ref; // global or heap-allocated

// In Init – pin the callback
napi_value cb = argv[0]; // JS function passed to addon
napi_create_reference(env, cb, 1, &callback_ref); // refcount=1 → GC root

// Later – retrieve and call it
napi_value cb_value;
napi_get_reference_value(env, callback_ref, &cb_value);
napi_value global, result;
napi_get_global(env, &global);
napi_call_function(env, global, cb_value, 0, NULL, &result);

// When done – release the root
napi_reference_unref(env, callback_ref, NULL); // refcount → 0, eligible for GC
napi_delete_reference(env, callback_ref); // free the ref itself

// Thread-safe function alternative (for calls from worker threads):
// napi_create_threadsafe_function – see §3.7

```

Common leak: creating a reference with refcount 1 and never calling `napi_delete_reference`. The JS function (and its closure scope, potentially holding large objects) is never collected. In a long-running server that creates references per request, this is a memory leak that grows with request count — indistinguishable from a JS closure leak in heap snapshots except that the retainer path goes through the native `addon`.

`napi_wrap` / finalizers — tying native memory to JS lifetime:

```

typedef struct { char* data; size_t len; } NativeBuffer;

void FinalizeBuffer(napi_env env, void* data, void* hint) {
    NativeBuffer* buf = (NativeBuffer*)data;
    free(buf->data);
    free(buf);
    // Runs on main thread when the JS object is GC'd
}

napi_value CreateBuffer(napi_env env, napi_callback_info info) {
    napi_value js_obj;
    napi_create_object(env, &js_obj);

    NativeBuffer* native = malloc(sizeof(NativeBuffer));
    native->data = strdup("native payload");
    native->len = strlen(native->data);

    // Tie native lifetime to js_obj lifetime
    napi_wrap(env, js_obj, native, FinalizeBuffer, NULL, NULL);
    return js_obj;
}

// To retrieve:
NativeBuffer* native;
napi_unwrap(env, js_obj, (void**)&native);

```

`node-addon-api` equivalent is `Napi::ObjectWrap` (shown in §3.5), which automates `napi_wrap` / `napi_unwrap` and calls the C++ destructor as the finalizer.

N-API versioning and lifecycle

N-API versions are additive. An addon declares its minimum requirement:

```

// package.json – declare N-API version
{
  "name": "my-addon",
  "version": "1.0.0",
  "binary": { "napi_versions": [6] }
}

```

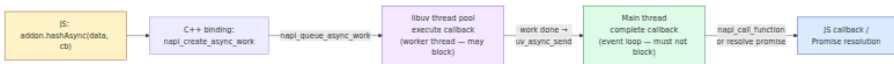
N-API version	Node version	Notable additions
1	8.0.0	Core: create/get/set, function, error

N-API version	Node version	Notable additions
3	10.0.0	<code>napi_create_threadsafe_function</code> , <code>napi_get_value_bigint</code>
4	10.16.0	<code>napi_create_threadsafe_function</code> (stable), <code>napi_add_env_cleanup_hook</code>
6	14.0.0	<code>napi_add_finalizer</code> , <code>napi_create_date</code> , <code>BigInt</code> improvements
8	16.0.0	<code>napi_add_async_cleanup_hook</code> , <code>napi_create_object_with_properties</code>
9	18.0.0	<code>napi_create_threadsafe_function</code> v2, <code>node_api.h</code> rename

`NAPI_EXPERIMENTAL` guards APIs that have not yet reached stable ABI. Check `napi_get_node_version` at runtime if you need to branch on available features.

3.7 Async Addons: Getting Off the Main Thread

A synchronous addon that does CPU work blocks the event loop — no I/O callbacks fire, no timers fire, no other JS runs. The fix is `napi_create_async_work`, which dispatches work to the libuv thread pool and calls back on the main thread when done.



Raw N-API async work — complete example

```

// async_addon.c - async hash via napi_create_async_work
#include <node_api.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char* input;           // owned - allocated in Init, freed in complete
    size_t input_len;
    char output[65];       // hex hash result
    napi_async_work work;  // libuv work handle
    napi_ref callback_ref; // GC root for JS callback
    napi_env env;
} HashWork;

// Worker thread - MUST NOT call any napi_* function except
// napi_async_work helpers. No V8 access here.
static void ExecuteHash(napi_env env, void* data) {
    HashWork* w = (HashWork*)data;
    // Simulate expensive hash (real code: call libcrypto, argon2, etc.)
    // This runs on a thread-pool worker - blocking is OK.
    unsigned long h = 5381;
    for (size_t i = 0; i < w->input_len; i++) {
        h = ((h << 5) + h) + (unsigned char)w->input[i]; // djb2
    }
    snprintf(w->output, sizeof(w->output), "%016lx%016lx", h, ~h);
}

// Main thread - runs after ExecuteHash completes, inside event loop
static void CompleteHash(napi_env env, napi_status status, void* data)
{
    HashWork* w = (HashWork*)data;

    // Retrieve JS callback
    napi_value callback, global, result_str, argv[2], result;
    napi_get_reference_value(env, w->callback_ref, &callback);
    napi_get_global(env, &global);

    napi_value null_arg;
    napi_get_null(env, &null_arg);
    napi_create_string_utf8(env, w->output, NAPI_AUTO_LENGTH,
    &result_str);

    // callback(null, result) - Node error-first convention
    argv[0] = null_arg;
    argv[1] = result_str;
    napi_call_function(env, global, callback, 2, argv, &result);

    // Cleanup - order matters
    napi_delete_reference(env, w->callback_ref);
    napi_delete_async_work(env, w->work);
}

```

```

    free(w->input);
    free(w);
}

static napi_value HashAsync(napi_env env, napi_callback_info info) {
    size_t argc = 2;
    napi_value argv[2];
    napi_get_cb_info(env, info, &argc, argv, NULL, NULL);

    // Extract input string
    size_t len;
    napi_get_value_string_utf8(env, argv[0], NULL, 0, &len);
    char* input = malloc(len + 1);
    napi_get_value_string_utf8(env, argv[0], input, len + 1, NULL);

    HashWork* work = calloc(1, sizeof(HashWork));
    work->input = input;
    work->input_len = len;
    work->env = env;
    napi_create_reference(env, argv[1], 1, &work->callback_ref);

    napi_value resource_name;
    napi_create_string_utf8(env, "HashAsync", NAPI_AUTO_LENGTH, &resource_name);

    napi_create_async_work(env, NULL, resource_name,
                          ExecuteHash, CompleteHash, work, &work->work);
    napi_queue_async_work(env, work->work);

    napi_value undefined;
    napi_get_undefined(env, &undefined);
    return undefined;
}

// Promise variant – napi_create_promise + resolve/reject
static napi_value HashAsyncPromise(napi_env env, napi_callback_info info) {
    // ... similar setup, but create a deferred promise:
    // napi_create_promise(env, &deferred, &promise);
    // store deferred in work struct, resolve in CompleteHash via
    // napi_resolve_deferred / napi_reject_deferred
    // return promise to JS – enables: await addon.hashAsyncPromise(data)
    return NULL; // elided for brevity – full impl in repo examples
}

static napi_value Init(napi_env env, napi_value exports) {
    napi_value fn1, fn2;
    napi_create_function(env, "hashAsync", NAPI_AUTO_LENGTH, HashAsync, NULL, &fn1);
    napi_create_function(env, "hashAsyncPromise", NAPI_AUTO_LENGTH, HashAsyncPromise, NULL, &fn2);

```

```

    napi_set_named_property(env, exports, "hashAsync", fn1);
    napi_set_named_property(env, exports, "hashAsyncPromise", fn2);
    return exports;
}

NAPI_MODULE(NODE_GYP_MODULE_NAME, Init)

```

```

// Usage – callback and promise forms
const addon = require('./build/Release/async_addon.node');

// Callback form
addon.hashAsync('hello world', (err, hash) => {
  console.log('callback:', hash); // 16-char hex
});

// Promise form (if HashAsyncPromise implemented)
const hash = await addon.hashAsyncPromise('hello world');
console.log('promise:', hash);

```

Critical rules for `ExecuteHash` (worker thread):

- **Do not call any `napi_*` function** that touches V8 — no `napi_create_string_utf8`, no `napi_call_function`, no `napi_get_value *`. The only safe N-API calls on the worker are `napi_add_env_cleanup_hook` and the `async-work` helpers. Violating this corrupts V8 state.
- **Do not access `napi_env`** from the worker — it is not thread-safe. Pass data via the `void* data` struct.
- **Blocking is expected** — that is the point. The worker thread exists to block so the main thread does not.

Thread-safe functions — calling JS from any thread

When a worker thread needs to call back into JS *during* execution (progress callbacks, streaming results), `napi_create_async_work` is insufficient — it only calls back once at completion. Use `napi_threadsafe_function`:


```
// Sketch - streaming progress from worker thread
napi_threadsafe_function tsfn;

// Main thread - create TSFN
napi_create_threadsafe_function(env, js_callback, NULL,
    resource_name, 0, 1, NULL, NULL, NULL, CallJs, &tsfn);

// Worker thread - call JS at any time (thread-safe)
char* progress = strdup("50% done");
napi_call_threadsafe_function(tsfn, progress, napi_tsfm_blocking);

// Main thread - JS callback invoked via CallJs trampoline
void CallJs(napi_env env, napi_value js_cb, void* ctx, void* data) {
    char* msg = (char*)data;
    napi_value js_str, result;
    napi_create_string_utf8(env, msg, NAPI_AUTO_LENGTH, &js_str);
    napi_call_function(env, NULL, js_cb, 1, &js_str, &result);
    free(msg);
}

// Cleanup
napi_release_threadsafe_function(tsfn, napi_tsfm_release);
```

`napi-rs` wraps this as `napi::threadsafe_function::ThreadsafeFunction` and `tokio` integration handles it automatically for `async fn` addons.

3.8 Parallelism in Node: worker_threads vs cluster vs child_process

Node offers three ways to use more than one OS thread or process. They differ in isolation, memory sharing, startup cost, and failure semantics — choosing wrong is a common source of production incidents.



Dimension	worker_threads	cluster	child_process
OS primitive	Threads (<code>pthread</code> / <code>CreateThread</code>) in one process	Processes (<code>fork</code>) with shared server handle	Processes (<code>spawn</code> / <code>fork</code> / <code>exec</code>)

Dimension	worker_threads	cluster	child_process
V8 Isolate	One Isolate per worker (separate heap, GC, JIT)	One Isolate per process	One Isolate per Node child; none for non-Node children
Event loop	One <code>uv_loop_t</code> per worker	One <code>uv_loop_t</code> per process	One per Node child
Memory sharing	<code>SharedArrayBuffer</code> + <code>Atomics</code> — true zero-copy sharing within one address space	No sharing — each process has its own heap; IPC via serialization	No sharing — pipes / IPC serialization
Startup cost	~1-5 ms, ~1-2 MB per worker (Isolate + loop)	~30-100 ms per worker (full process fork, module re-evaluation)	~10-200 ms depending on child type
Failure domain	Worker crash can corrupt process (shared address space); <code>uncaughtException</code> in worker does not crash main	Worker process crash isolated — primary can <code>fork()</code> a replacement	Child crash isolated — parent gets <code>exit</code> event
Port sharing	No — workers do not share a listening socket	Yes — <code>cluster</code> distributes connections via round-robin or <code>SO_REUSEPORT</code>	No (unless manually passing handles)
Use for	CPU-bound JS work, parallel computation, offloading without serialization overhead	Scaling a stateless HTTP server across cores	Running non-JS programs, isolation, privilege separation

worker_threads — in-process parallelism

```

// main.js - dispatch CPU work to workers
import { Worker, isMainThread, parentPort, workerData } from 'node:worker_threads';
import os from 'node:os';

if (isMainThread) {
  // Main thread - create a pool sized to CPU count
  const cpuCount = os.availableParallelism(); // Node 19+; was
  os.cpus().length
  const workers = [];
  const tasks = [1000000, 2000000, 3000000, 4000000]; // e.g.,
  iteration counts
  const results = [];

  function runTask(n) {
    return new Promise((resolve, reject) => {
      const w = new Worker(new URL(import.meta.url), { workerData: n })
;
      w.on('message', resolve);
      w.on('error', reject);
      w.on('exit', (code) => {
        if (code !== 0) reject(new Error(`Worker exited with ${code}`))
;
      });
    });
  }

  // Fan-out - one worker per task, bounded by cpuCount
  const allResults = await Promise.all(tasks.map(runTask));
  console.log('results:', allResults);

  // SharedArrayBuffer - zero-copy sharing
  const shared = new SharedArrayBuffer(4);
  const view = new Int32Array(shared);
  const w = new Worker(new URL(import.meta.url), { workerData: {
  shared } });
  // Worker can Atomics.add(view, 0, 1) without copying

} else {
  // Worker thread - own Isolate, own event loop
  const n = typeof workerData === 'number' ? workerData :
workerData?.n ?? 1000000;

  // CPU-bound work - does NOT block main thread's event loop
  let sum = 0;
  for (let i = 0; i < n; i++) sum += Math.sqrt(i);

  parentPort.postMessage(sum);
}

```

```

// worker-pool.js - reusable pool (production pattern)
import { Worker } from 'node:worker_threads';

class WorkerPool {
  #workers = [];
  #queue = [];
  #size;

  constructor(script, size) {
    this.#size = size;
    for (let i = 0; i < size; i++) {
      const w = new Worker(script);
      w.busy = false;
      w.on('message', (result) => {
        w.busy = false;
        w.currentResolve(result);
        this.#drain();
      });
      w.on('error', (err) => {
        w.busy = false;
        w.currentReject(err);
        this.#drain();
      });
      this.#workers.push(w);
    }
  }

  run(data) {
    return new Promise((resolve, reject) => {
      this.#queue.push({ data, resolve, reject });
      this.#drain();
    });
  }

  #drain() {
    const idle = this.#workers.find((w) => !w.busy);
    const next = this.#queue.shift();
    if (!idle || !next) {
      if (next) this.#queue.unshift(next);
      return;
    }
    idle.busy = true;
    idle.currentResolve = next.resolve;
    idle.currentReject = next.reject;
    idle.postMessage(next.data);
  }

  async destroy() {
    await Promise.all(this.#workers.map((w) => w.terminate()));
  }
}

```

```
}  
}
```

Overhead note. Each `Worker` creates a new V8 Isolate (~1 MB heap minimum), a new `uv_loop_t`, and duplicates the Node bootstrap. Creating a worker per request is prohibitively expensive — always pool. The `workerData` clone at creation uses structured clone; large objects should be transferred via `transferList` or placed in `SharedArrayBuffer`.

cluster — multi-process HTTP scaling

```
// cluster-server.js
import cluster from 'node:cluster';
import http from 'node:http';
import os from 'node:os';

if (cluster.isPrimary) {
  const n = os.availableParallelism();
  console.log(`Primary ${process.pid} - forking ${n} workers`);

  for (let i = 0; i < n; i++) cluster.fork();

  cluster.on('exit', (worker, code, signal) => {
    console.warn(`Worker ${worker.process.pid} died (${signal} ??
code}) - restarting`);
    cluster.fork(); // replace failed worker
  });

  // Graceful reload on SIGHUP - zero-downtime restart
  process.on('SIGHUP', () => {
    for (const id in cluster.workers) {
      cluster.workers[id].send({ cmd: 'shutdown' });
    }
  });
} else {
  // Worker process - each has its own server, but they share the port
  const server = http.createServer((req, res) => {
    res.end(`Hello from worker ${process.pid}\n`);
  });

  server.listen(3000, () => {
    console.log(`Worker ${process.pid} listening on :3000`);
  });

  process.on('message', (msg) => {
    if (msg.cmd === 'shutdown') {
      server.close(() => process.exit(0)); // drain, then exit
    }
  });
}
```

Under the hood, `cluster` does one of two things depending on platform:

- **Linux (default):** The primary creates the listening socket and passes the file descriptor to workers via IPC (`sendmsg` with `SCM_RIGHTS`). Workers call `uv_tcp_open` on the received fd. Incoming

connections are distributed round-robin by the primary (or via `SO_REUSEPORT` with `schedulingPolicy: OS` on Node 16+).

- **Round-robin vs OS scheduling:** `cluster.schedulingPolicy = cluster.SCHED_RR` (default on most platforms) — primary distributes. `cluster.SCHED_NONE` — relies on `SO_REUSEPORT` and lets the kernel distribute. The default changed across Node versions; always set it explicitly.

Distributed-systems lens. `cluster` looks like horizontal scaling but is not. All workers share one host, one NIC, one disk, and one failure domain (the host). A host failure kills every worker simultaneously — unlike true horizontal scaling across hosts behind a load balancer. Use `cluster` to saturate cores on a single host; use a container orchestrator (Kubernetes) with multiple pods for fault tolerance. The two are complementary: `cluster` with `availableParallelism()` workers *per pod*, scaled horizontally by the orchestrator.

child_process — arbitrary subprocess execution

```
import { spawn, execFile, fork } from 'node:child_process';

// spawn — streaming, arbitrary binary, no shell
const ffmpeg = spawn('ffmpeg', ['-i', 'input.mp4', '-c:v', 'libx264', 'output.mp4']);
ffmpeg.stderr.on('data', (chunk) => console.error(`ffmpeg: ${chunk}`));
ffmpeg.on('close', (code) => console.log(`ffmpeg exited with ${code}`));
;

// execFile — buffered, callback when done (no shell — safe from injection)
execFile('python3', ['script.py', '--input', 'data.json'], (err, stdout, stderr) => {
  if (err) throw err;
  console.log(stdout);
});

// fork — Node child with IPC channel (like cluster but manual)
const child = fork('./child-worker.js');
child.send({ task: 'compute', n: 1000000 });
child.on('message', (result) => console.log('child result:', result));
child.on('exit', (code) => console.log('child exited: ${code}'));

// Security: NEVER use exec() with user input — it spawns a shell.
// exec(`convert ${userInput} output.png`) // ← shell injection if
// userInput contains `; rm -rf /`
// Use execFile or spawn with explicit argv instead.
```

Decision tree:

Need to run non-JS program (python, ffmpeg, shell **tool**)?

→ child_process.spawn / execFile

Need to scale a stateless HTTP server across cores on one host?

→ cluster (**or**: worker_threads + shared handle, **or** just run N containers)

Need to offload CPU-bound JS without process overhead, share memory?

→ worker_threads

Need isolation so a crash does **not** corrupt the parent?

→ cluster **or** child_process (separate address space)

→ NOT worker_threads (shared address space → a native addon segfault kills all threads)

3.9 Async Context Propagation: `async_hooks` and `AsyncLocalStorage`

In a concurrent server handling thousands of requests on one thread, how does a log line deep inside a callback know which request it belongs to? Thread-locals do not work — there is one thread. The answer is `AsyncLocalStorage`, built on `async_hooks`.

The problem — context loss across async boundaries

```
// Without AsyncLocalStorage – context is lost after first await
let currentRequestId = null; // global – race condition!

async function handleRequest(req, res) {
  currentRequestId = req.headers['x-request-id']; // set for this request

  await db.query('SELECT ...'); // yields to event loop – another request may overwrite currentRequestId
  // currentRequestId may now belong to a DIFFERENT request
  logger.info('query done', { requestId: currentRequestId }); // WRONG
}
```

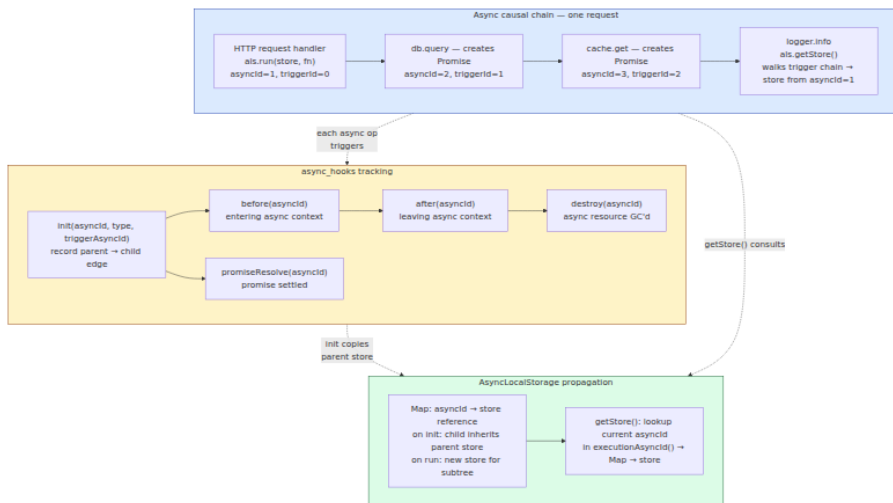
```
// With AsyncLocalStorage – context follows the causal chain
import { AsyncLocalStorage } from 'node:async_hooks';

const als = new AsyncLocalStorage();

async function handleRequest(req, res) {
  const store = { requestId: req.headers['x-request-id'], userId: req.user.id };
  // run() establishes context for the entire async chain rooted here
  als.run(store, async () => {
    await db.query('SELECT ...'); // context preserved across await
    await cache.get('key'); // preserved across any async hop
    logger.info('query done', { requestId:
als.getStore().requestId }); // CORRECT
  });
}

// Anywhere in the call graph – no parameter threading needed
function getRequestId() {
  return als.getStore()?.requestId ?? 'unknown';
}
```

How it works — `async_hooks` under the hood



Every asynchronous operation in Node creates an **async resource** with a unique `asyncId` and a `triggerAsyncId` (the `asyncId` of the context that created it). `async_hooks` emits lifecycle events:

```

import async_hooks from 'node:async_hooks';
import fs from 'node:fs';

const hook = async_hooks.createHook({
  init(asyncId, type, triggerAsyncId, resource) {
    // Called when a new async resource is created
    // type: 'PROMISE', 'Timeout', 'TCPWRAP', 'FSREQCALLBACK', etc.
    fs.writeSync(1, `${type}(${asyncId}) triggered by ${triggerAsyncId}
\n`);
  },
  before(asyncId) {
    // Called before the resource's callback executes
  },
  after(asyncId) {
    // Called after the callback completes
  },
  destroy(asyncId) {
    // Called when the resource is GC'd
  },
  promiseResolve(asyncId) {
    // Called when a Promise resolves
  },
});
hook.enable();

// Every subsequent async op now traces through init/before/after/
destroy
setTimeout(() => {}, 0);
Promise.resolve().then(() => {});

```

`AsyncLocalStorage` is a thin layer on top: on `init`, the child `asyncId` inherits the parent's store reference (`Map.set(childId, Map.get(triggerId))`). On `als.run(store, fn)`, it sets `Map.set(currentId, store)` for the current execution context. On `als.getStore()`, it looks up `executionAsyncId()` in the map. The propagation is **by reference** — all async children of a `run()` share the same store object until a nested `run()` or `enterWith()` overrides it.

Overhead and when to avoid it

`async_hooks` tracking has a cost. Every async resource creation does a `Map` lookup and insertion. Node optimizes heavily (the `AsyncContextFrame` fast path in Node 20+ avoids the full hook machinery for `AsyncLocalStorage`-only usage), but overhead is measurable.

```

// als-benchmark.js - measure AsyncLocalStorage overhead
import { AsyncLocalStorage } from 'node:async_hooks';
import { performance } from 'node:perf_hooks';

const als = new AsyncLocalStorage();
const ITERATIONS = 1_000_000;

// Baseline - no ALS
let t0 = performance.now();
for (let i = 0; i < ITERATIONS; i++) {
  await Promise.resolve(i);
}
let baseline = performance.now() - t0;

// With ALS - run() per iteration
t0 = performance.now();
for (let i = 0; i < ITERATIONS; i++) {
  await als.run({ id: i }, async () => {
    await Promise.resolve(als.getStore().id);
  });
}
let withAls = performance.now() - t0;

console.log(`Baseline:    ${baseline.toFixed(1)}ms for ${ITERATIONS}
iterations`);
console.log(`With ALS:    ${withAls.toFixed(1)}ms`);
console.log(`Overhead:    ${((withAls / baseline - 1) *
100).toFixed(1)}%`);
console.log(`Per-op:      ${((withAls - baseline) / ITERATIONS *
1000).toFixed(2)}µs`);

// Typical results (Node 20, M1 Mac, 1M iterations):
// Baseline:    ~180ms
// With ALS:    ~520ms
// Overhead:    ~189%
// Per-op:      ~0.34µs
//
// Node 20's AsyncContextFrame optimization brings this down
// significantly
// vs Node 16/18. Without the fast path (general async_hooks enabled),
// overhead is 3-5x higher.

// Bulk benchmark - single run() wrapping many awaits (amortized cost)
t0 = performance.now();
await als.run({ id: 0 }, async () => {
  for (let i = 0; i < ITERATIONS; i++) {
    als.getStore(); // lookup only - no new context
    await Promise.resolve(i);
  }
});

```

```
let amortized = performance.now() - t0;
console.log(`\nAmortized (single run, many awaits): $
{amortized.toFixed(1)}ms`);
console.log(`Amortized overhead: ${((amortized / baseline - 1) * 100).toFixed(1)}%`);
// Typical: ~220ms – only ~22% overhead when run() is not per-iteration
```

Guidance:

- **One `als.run()` per request** (wrapping the entire handler) costs ~0.3 μ s per async hop — negligible for request latency measured in milliseconds. This is the standard production pattern and is safe to leave enabled.
- **One `als.run()` per fine-grained operation** (per DB query, per cache call) multiplies overhead linearly — avoid it. Use `als.getStore()` reads (cheap) inside the request context instead of nested `run()` calls.
- **Enabling the general `async_hooks.createHook()` API** (not just `AsyncLocalStorage`) disables the fast path and adds ~1-2 μ s per async resource — significant at high throughput. Never enable a global `async_hooks` hook in production unless actively debugging. Use `AsyncLocalStorage` alone, which takes the optimized `AsyncContextFrame` path since Node 18.7+.
- **Lost context — the common bug.** Any async operation that does not go through Node's async tracking loses context: raw `setTimeout` in some edge cases, native addons that call `napi_call_threadsafe_function` without propagating context, and `queueMicrotask` in older Node versions. If `als.getStore()` returns `undefined` inside a callback that should have context, the async chain was broken — check whether the operation uses a tracked async resource type.

```

// Context loss – and the fix
import { AsyncLocalStorage } from 'node:async_hooks';
const als = new AsyncLocalStorage();

// BROKEN – setTimeout loses context if not created inside run()
als.run({ id: 'req-1' }, () => {
  setTimeout(() => {

console.log(als.getStore()); // { id: 'req-1' } – actually works in
modern Node!
    // setTimeout IS tracked – context propagates. But:
  }, 0);
});

// BROKEN – native addon or untracked callback
import { EventEmitter } from 'node:events';
const ee = new EventEmitter();
als.run({ id: 'req-1' }, () => {
  ee.on('data', () => {
    console.log(als.getStore()); // undefined! – EventEmitter 'on' is
NOT an async resource
  });
});
ee.emit('data'); // outside run() – no context

// FIX – capture and re-establish context
als.run({ id: 'req-1' }, () => {
  const store = als.getStore();
  ee.on('data', () => {
    als.run(store, () => {
      console.log(als.getStore()); // { id: 'req-1' } – restored
    });
  });
});

```

Distributed-systems lens. `AsyncLocalStorage` is the in-process equivalent of distributed tracing context propagation (W3C `traceparent`, gRPC metadata, Kafka headers). The same principle — carry request-scoped metadata along the causal chain without threading it through every function signature — applies at both levels. In a well-instrumented fleet, `AsyncLocalStorage` holds the `traceId` / `spanId` that was extracted from the inbound `traceparent` header, and every outbound HTTP/gRPC call, DB query log, and error report reads it via `als.getStore()` to attach the trace context automatically. The `asyncId` → `triggerId` causal graph inside one Node process is isomorphic to the `parentSpanId` → `spanId` trace tree across services.

Losing context in either place produces the same symptom: orphaned spans and untraceable requests.

3.10 Putting It Together: A Production Checklist

Before shipping a Node service that touches any of the mechanisms in this chapter, verify:

1. **Thread-pool sizing.** Set `UV_THREADPOOL_SIZE` based on `availableParallelism()` and workload. If you do filesystem I/O *and* crypto *and* DNS lookups concurrently, the default 4 will starve. Measure pool queue depth under load — if `pbkdf2` p99 spikes when `fs` throughput increases, they are contending. Isolate CPU work to `worker_threads`.
2. **Event-loop health.** Monitor `perf_hooks.monitorEventLoopDelay()` (p50/p99 loop delay) and `process.cpuUsage()`. Loop delay > 50 ms at p99 means something is blocking the main thread — likely a synchronous addon, a large `JSON.parse`, or recursive `nextTick`. Set an alert.
3. **Addon ABI discipline.** Build addons against N-API / `node-addon-api` / `napi-rs`, not NAN or raw V8. Pin the N-API version in `package.json` and test the compiled `.node` binary against the next Node major in CI — N-API forward compatibility is not magic if you use `NAPI_EXPERIMENTAL` APIs.
4. **Handle scope hygiene.** Audit addons for `napi_create_reference` without matching `napi_delete_reference`, and for `napi_value` stored beyond its handle scope lifetime. Both produce leaks or use-after-free that only manifest under sustained load.
5. **Async context propagation.** Wrap each inbound request in `als.run()` exactly once. Do not create nested `run()` per query. Verify that `als.getStore()` is defined in every log line and outbound call by adding a debug assertion in development. If context is lost, the async resource type is untracked — wrap the callback manually.
6. **Parallelism choice.** `worker_threads` for CPU parallelism with shared memory, `cluster` for multi-process port sharing on one host, `child_process` for non-JS subprocesses. Never create a `Worker` per

request — pool them. Never use `cluster` as a substitute for multi-host scaling.

7. **io_uring awareness.** If your containers run with seccomp blocking `io_uring` (common in older Docker/Kubernetes defaults), all `fs` operations go through the thread pool. If you enable `io_uring`, `fs` completions arrive via `poll` phase instead — re-benchmark pool sizing and latency after changing the seccomp profile.

Key Takeaways

- Node is a four-layer cake: JavaScript (`lib/*.js`) → C++ bindings (`src/*.cc`) → libuv (event loop + thread pool + OS abstraction) → kernel (`epoll` / `kqueue` / `io_uring`). Every I/O call crosses at least two layers; the binding translates V8 values to C structs, libuv owns the async machinery.
- The libuv event loop has six phases in strict order: `timers` → `pending` → `idle / prepare` → `poll` (blocks in `epoll_wait`) → `check` (`setImmediate`) → `close` . Between every phase, Node drains `process.nextTick` (first, exhaustively) and then Promise microtasks. This interleaving is why Node ordering diverges from the browser.
- Timers are a min-heap keyed by absolute expiry, not a FIFO queue. `loop->time` is sampled once per iteration, so timers coalesce when the loop was busy. `setImmediate` (check phase) always beats `setTimeout(fn, 0)` (timers phase) when both are scheduled from inside an I/O callback; from top-level code they race.
- The thread pool (default 4 workers, `UV_THREADPOOL_SIZE`) executes all `uv_fs_*` , `getaddrinfo` , `crypto.pbkdf2 / scrypt` , and `zlib` operations. It is a single FIFO queue — `fs` , `dns` , and `crypto` contend on the same pool. Size it to `availableParallelism()` and isolate CPU work to `worker_threads` to avoid cross-workload starvation.
- Filesystem I/O goes through the thread pool (or `io_uring` when available), networking goes through non-blocking `epoll` in the `poll` phase, and timers go through the heap. DNS `lookup` is thread-pool, DNS `resolve` (c-ares) is non-blocking — prefer the latter for latency-sensitive paths.

- Native addons have four tiers: raw N-API (C, stable ABI, verbose), `node-addon-api` (C++ RAII wrapper, same ABI), `napi-rs` (Rust, `#[napi]` macro, stable ABI), and Neon (Rust, V8-direct, unstable ABI). Prefer N-API-based tiers for ABI stability across Node majors.
 - Every `napi_value` is a handle valid only within its handle scope. Storing a `napi_value` beyond its scope is use-after-free; use `napi_create_reference` (GC root) for long-lived values and `napi_wrap` / `napi_add_finalizer` to tie native memory to JS GC. Leaked references pin JS objects in the heap indefinitely.
 - `napi_create_async_work` dispatches CPU work to the thread pool (execute on worker, complete on main thread). The worker must not call V8/N-API. For streaming callbacks from worker threads, use `napi_create_threadsafe_function`. `napi-rs` automates both — `async fn` with `#[napi]` runs on the pool automatically.
 - `worker_threads` are threads in one process (shared address space, `SharedArrayBuffer`, ~1–5 ms startup) for CPU parallelism. `cluster` forks processes sharing a port (isolated heaps, ~30–100 ms startup) for per-host HTTP scaling. `child_process` spawns arbitrary binaries (any executable, pipes/IPC) for non-JS work. Pool workers; never create one per request.
 - `AsyncLocalStorage` propagates request context (trace IDs, tenant IDs) along the `asyncId` → `triggerId` causal chain via a `Map<asyncId, store>` inherited on `init`. One `als.run()` per request costs ~0.3 μs per async hop (fast path since Node 18.7+); enabling the general `async_hooks` hook disables the fast path and costs 3–5× more. Lost context means an untracked async resource — wrap the callback in `als.run(capturedStore, fn)`.
-

Further Reading

- **libuv documentation.** docs.libuv.org — The authoritative reference for `uv_loop_t`, `uv_run`, handles, requests, and the thread pool. Start with "Design overview" and "The I/O loop."
- **libuv source** — `src/unix/core.c` (`uv_run`), `src/threadpool.c`, `src/unix/fs.c`. Reading the ~200 lines of `uv_run` and `uv_work_submit` / `uv_threadpool_worker` is the fastest way to internalize the loop and pool mechanics. GitHub: [libuv/libuv](https://github.com/libuv/libuv).

- **Node.js source** — `src/node_file.cc`, `src/api/environment.cc`, `lib/fs.js`, `lib/internal/process/task_queues.js`. Trace `fs.readFile` from `lib/fs.js` through `node_file.cc` into `uv_fs_open` to see the full layer cake in code.
- **N-API / Node-API documentation.** nodejs.org/api/n-api.html — Complete N-API reference: handle scopes, references, async work, thread-safe functions, versioning. The `node_api.h` header itself is well-commented.
- **node-addon-api documentation.** github.com/nodejs/node-addon-api — C++ wrapper reference: `Napi::ObjectWrap`, `Napi::AsyncWorker`, `Napi::ThreadSafeFunction`.
- **napi-rs documentation.** napi.rs — Rust addon guide: `#[napi]` macro, async functions, `ThreadsafeFunction`, building and publishing.
- **Neon documentation.** neon-bindings.com — Alternative Rust binding guide; useful for understanding the V8-direct trade-off.
- **"Don't Block the Event Loop" — Node.js guides.** nodejs.org/en/docs/guides/dont-block-the-event-loop — Official guide on pooling, offloading, and diagnosing loop blocking.
- **Bert Belder, "Everything You Need to Know About Node.js Event Loop" (JSConf, 2017).** Talk and slides by the libuv/Node core contributor — still the best high-level event-loop walkthrough.
- **Anna Henningsen, "Node.js: A look inside the `async_hooks` module" and Joyee Cheung's `AsyncContextFrame` work (Node 18.7+).** Explains the fast-path optimization that makes `AsyncLocalStorage` cheap. Search Node core PRs for `AsyncContextFrame`.
- **Thorsten Lorenz et al., "Understanding the Node.js Event Loop" — `clinic.js` / `clinic doctor` and `clinic bubbleprof`.** Practical tooling for diagnosing loop delay, thread-pool starvation, and handle leaks in production.
- **W3C Trace Context and OpenTelemetry propagation.** w3c.github.io/trace-context — The distributed analogue of

`AsyncLocalStorage` propagation; understanding one illuminates the other.

Chapter 4 — The JavaScript Type System, Prototypes, Proxies, and the Module System

What this chapter covers: JavaScript's type system is dynamic, its object model is prototype-delegated, and its module system is split-brained. This chapter takes all three apart at the spec level — how values coerce, how property lookup walks `[[Prototype]]`, how `class` desugars to prototype wiring, how `Proxy` / `Reflect` intercept the metaobject protocol, and how two incompatible module systems (CommonJS and ESM) coexist in one runtime and one package registry. You will walk a prototype chain with `__proto__`, build a membrane proxy that actually revokes, diagnose a dual-package hazard that ships two copies of a singleton to production, and read Node's resolver as a flowchart rather than folklore.

Learning goals:

- Classify every JavaScript value by spec type and explain `typeof` / `instanceof` from the ECMA-262 internal slots they consult.
- Trace coercion through `ToPrimitive`, `ToString`, `ToNumber`, and the Abstract Equality (`==`) algorithm, and predict the output of any coercion puzzle without guessing.
- Draw and walk the prototype chain, distinguishing `prototype` (a property of constructor functions) from `[[Prototype]]` / `__proto__` (a slot on every object), and explain what `class`, `extends`, and `super` desugar to.
- Use `Proxy` and `Reflect` traps correctly, respect trap invariants, build a revocable membrane, and explain why proxies cannot be transparently polyfilled.
- Apply well-known Symbols (`@@iterator`, `@@toStringTag`, `@@hasInstance`, `@@species`, `@@toPrimitive`) to customize language protocols.

- Contrast CommonJS (dynamic `require`, `require.cache`, module wrapper) with ESM (static `import` / `export`, live bindings, cyclic linking) at the loader level.
- Diagnose CJS/ESM interop pitfalls, the `__esModule` convention, and the `require(ESM)` error that broke half the ecosystem.
- Configure `package.json` `exports` / `imports`, import maps, and ESM loaders (`--loader` / `--experimental-vm-modules`) correctly.
- Identify the dual-package hazard, explain why `instanceof` and singleton state break when one package loads twice, and choose the fix (conditional exports with a single source file vs. breaking change to ESM-only).

1. Why a Backend Engineer Should Care About Types, Prototypes, and Modules

At small scale, JavaScript's dynamism feels productive. At fleet scale, it becomes a reliability surface.

A coercion bug in a shared validation library (`if (config.retries == true)` passing when `retries` is the string `"0"`) propagates to every service that depends on it. A prototype pollution in a deep-merge utility (`lodash.merge` CVE-2019-10744, `qs` prototype pollution) becomes a remote code execution vector across hundreds of deployments before anyone patches. A dual-package hazard — where `npm install` resolves both a CJS and an ESM copy of `uuid` or `debug` — silently duplicates singleton state: two connection pools, two metric registries, two `EventEmitter` hierarchies that do not recognize each other's events.

These are not language-trivia problems. They are supply-chain and runtime-correctness problems rooted in how the language defines types, object delegation, and module identity. This chapter treats all three as infrastructure: mechanisms with exact specifications, observable failure modes, and operational fixes.

Relationship to Chapter 1. Hidden classes (V8 Maps), inline caches, and speculative optimization from Chapter 1 explain *how* the engine makes these mechanisms fast. This chapter explains *what* the mechanisms are. Read them together: the engine optimizes the spec, not the other way around.

2. The Type System — What JavaScript Values Actually Are

JavaScript has no static type system at runtime. It has *spec types* and *language types* — categories the ECMA-262 specification uses to define behavior. The engine may represent them differently (SMIs, doubles, HeapObjects in V8), but the observable semantics follow the spec exactly.

2.1 Spec Types vs. Language Types

Spec type	Language-visible?	Examples
Undefined	<code>typeof</code> reports <code>"undefined"</code>	<code>undefined</code>
Null	<code>typeof</code> reports <code>"object"</code> (historic bug)	<code>null</code>
Boolean	yes	<code>true</code> , <code>false</code>
String	yes	<code>"hello"</code> , <code>""</code>
Symbol	yes	<code>Symbol("id")</code>
Number	yes	<code>42</code> , <code>NaN</code> , <code>Infinity</code> , <code>-0</code>
BigInt	yes	<code>42n</code>
Object	yes	<code>{}</code> , <code>[]</code> , <code>function(){} </code> , <code>new Date()</code>

Every value is one of the eight language types. Everything that is not a primitive is an Object — including functions and arrays. `typeof null === "object"` is a bug from the first ten days of the language, preserved for compatibility; the spec's internal `Type(x)` still returns Null, but `typeof` lies.

```

// typeof – what it actually tests (and where it lies)
typeof undefined      // "undefined"
typeof null           // "object" – spec Type is Null, typeof is
wrong
typeof 42              // "number"
typeof 42n             // "bigint"
typeof "hello"         // "string"
typeof Symbol("id")    // "symbol"
typeof true            // "boolean"
typeof {}              // "object"
typeof []              // "object" – arrays are objects
typeof function(){}    // "function" – callable objects get a special
typeof
typeof NaN             // "number" – NaN is a Number, IEEE 754, not
an error type

// Reliable type predicates – what senior code actually uses
Number.isNaN(NaN)      // true (vs isNaN("foo") which coerces –
never use it)
Number.isFinite(Infinity) // false
Object.is(NaN, NaN)    // true (unlike ===, see §3.5)
Object.is(-0, 0)       // false (unlike ===)
Array.isArray([])      // true (typeof cannot distinguish arrays)

```

2.2 Primitives vs. Objects — Identity, Copying, Immutability

Primitives are immutable and compared by value. Objects are mutable, compared by identity, and heap-allocated.

```

// Primitives – value semantics
let a = "hello";
let b = a;      // copies the value
b = "world";
console.log(a); // "hello" – a is unaffected

// Objects – reference semantics
let p = { x: 1 };
let q = p;      // copies the reference, not the object
q.x = 99;
console.log(p.x); // 99 – p and q point at the same object
console.log(p === q); // true – same identity

// Primitives are immutable – "mutation" creates a new value
let s = "hello";
s.toUpperCase(); // returns "HELLO", s is still "hello"

```

This distinction governs every performance and correctness property that follows: hidden classes and inline caches (Chapter 1) only apply to

objects; primitives are unboxed in optimized code; and the prototype chain only exists on objects (primitives delegate to their wrapper prototype via autoboxing).

2.3 Wrapper Objects and Autoboxing

Each primitive type except `null/undefined` has a wrapper constructor (`String`, `Number`, `Boolean`, `Symbol`, `BigInt`). When you access a property on a primitive, the engine *autoboxes* it — temporarily wraps it — to perform the lookup.

```
// Autoboxing – invisible wrapper creation
"hello".length      // string primitive → temporary String object
→ .length → unwrap
(42).toFixed(2)      // number primitive → temporary Number object →
method → unwrap

// Explicit wrappers – almost always a mistake
const s1 = "hello";    // primitive – typeof "string"
const s2 = new String("hello"); // object – typeof "object", truthy
even when ""
console.log(typeof s2); // "object"
console.log(!new
String("")); // true – empty wrapper is truthy! Bug source.
console.log(s1 === s2); // false – different types

// Symbol wrappers throw – you cannot new Symbol()
try { new Symbol("x"); } catch (e) { console.log(e.message); }
// TypeError: Symbol is not a constructor
```

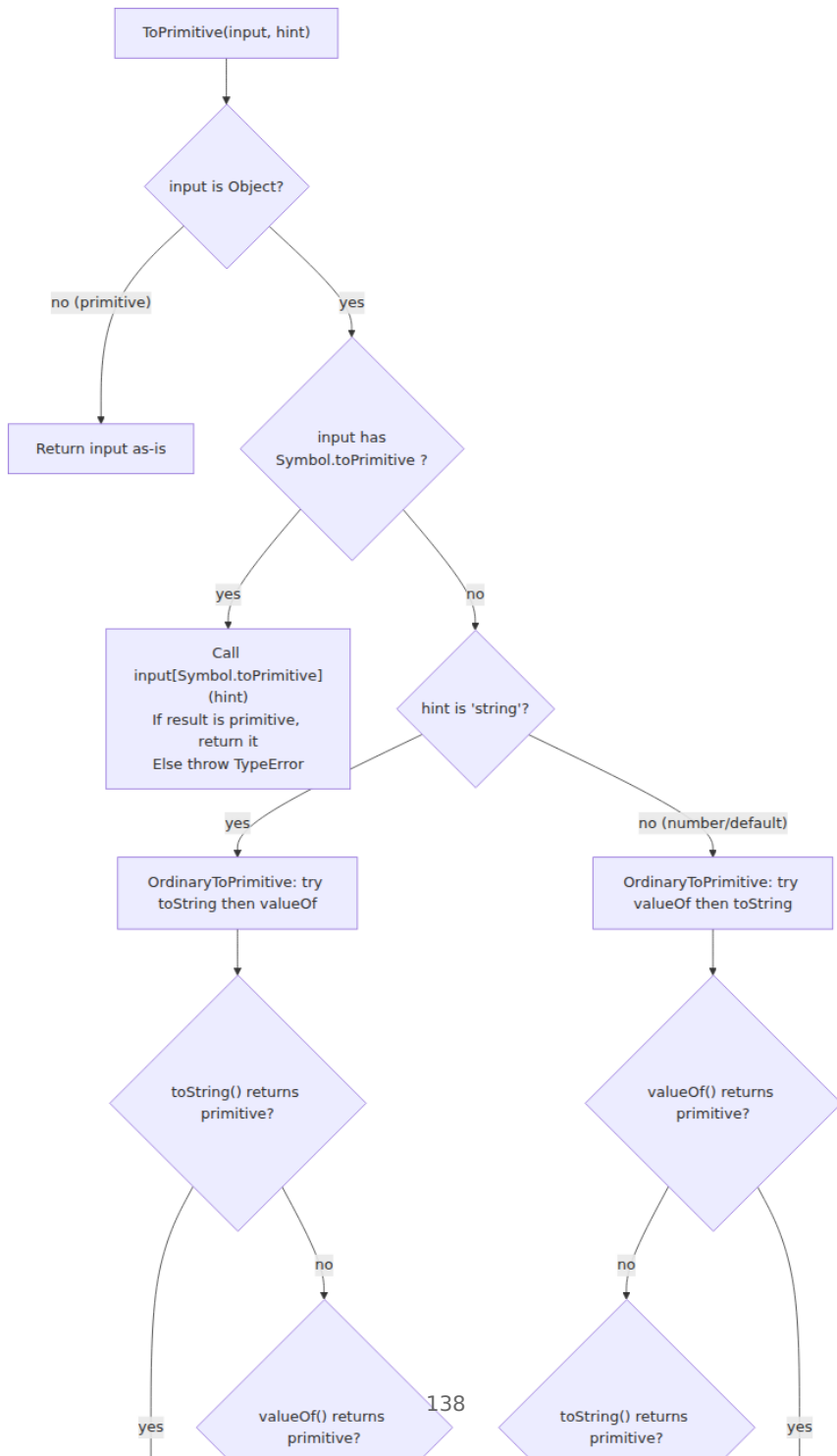
Rule: never use `new String` / `new Number` / `new Boolean` in production code. Linters (`no-new-wrappers`) enforce this because the truthiness and equality semantics diverge silently.

3. Coercion — The Algorithms Behind `==` and Friends

Coercion is not random. It is a set of precisely specified abstract operations. Once you read them as algorithms, every "wat" puzzle becomes deterministic.

3.1 ToPrimitive — The Gateway

Almost every coercion passes through `ToPrimitive(input, hint)`. The hint is `"string"` or `"number"` (default is `"number"` except `Date` defaults to `"string"`).



`OrdinaryToPrimitive` order matters: for `"number"` hint, `valueOf` is tried first; for `"string"` hint, `toString` first.

```
// ToPrimitive in action
const obj = {
  [Symbol.toPrimitive](hint) {
    console.log(`hint: ${hint}`);
    if (hint === "string") return "as-string";
    if (hint === "number") return 42;
    return true; // "default" hint – used by == and +
  }
};

String(obj); // hint: string → "as-string" (calls toPrimitive with "string")
Number(obj); // hint: number → 42
obj + ""      // hint: default → true → "true" (default hint, then ToString)
obj == 1      // hint: default → true → 1 (ToPrimitive then ToNumber – see §3.3)

// Without Symbol.toPrimitive – OrdinaryToPrimitive ordering
const plain = {
  toString() { console.log("toString"); return "hello"; },
  valueOf() { console.log("valueOf"); return 99; }
};

String(plain); // toString first (string hint) → "hello"
Number(plain); // valueOf first (number hint) → 99
plain + 1;      // default hint → valueOf first → 99 + 1 = 100

// Date is the exception – default hint is "string"
const d = new Date("2026-01-01");
String(d); // hint string → toString first → "Thu Jan 01 2026 ..."
Number(d); // hint number → valueOf first → 1767225600000
d + "";    // hint string (Date special case) → calls toString, not valueOf
```

3.2 ToString and ToNumber

After `ToPrimitive`, the remaining conversions are mechanical:

```

// ToString
String(undefined) // "undefined"
String(null)      // "null"
String(true)      // "true"
String(42)        // "42"
String(0)         // "0"
String(-0)        // "0" (ToString collapses -0 to "0!")
String(NaN)       // "NaN"
String(Infinity)  // "Infinity"
String([1,2,3])   // "1,2,3" (array → ToPrimitive → join with ",")
String({})        // "[object Object]" (Object.prototype.toString)
String([])        // "" (empty array → "")

// ToNumber – the strict numeric conversion (also what Number() does)
Number(undefined) // NaN
Number(null)      // 0    – the famous null→0 in numeric context
Number(true)      // 1
Number(false)     // 0
Number("")        // 0    – empty string → 0
Number(" 42 ")   // 42    – trims whitespace
Number("42abc")  // NaN   – not like parseInt
Number("0x2a")   // 42    – hex
Number("0b1010") // 10    – binary
Number("0o52")   // 42    – octal

// parseInt vs Number – different algorithms
parseInt("42abc") // 42    – parses prefix, ignores trailing
Number("42abc")   // NaN   – entire string must be numeric
parseInt("08")    // 8     – without radix, still decimal since ES5
(old engines: octal trap)

```

3.3 Abstract Equality — What `==` Actually Does

`==` is not "loose." It is 12 ordered clauses in ECMA-262 §7.2.14. The full algorithm:

Step	Condition	Result
1	Same Type	<code>===</code>
2	<code>null == undefined</code>	<code>true</code> (and only with each other)
3	<code>Number == String</code>	<code>ToNumber(String) == Number</code>
4	<code>Boolean == any</code>	<code>ToNumber(Boolean) == any</code>
5	<code>Object == String/Number/Symbol</code>	<code>ToPrimitive(Object) == primitive</code>

Step	Condition	Result
6	<code>BigInt == Number</code>	Numeric comparison (with NaN/Infinity handling)
7	Otherwise	<code>false</code>

```
// Walking through == step by step
42 == "42"      // Step 3: ToNumber("42") → 42, then 42 === 42 → true
0 == ""         // Step 3: ToNumber("") → 0, then 0 === 0 → true
0 == "0"        // Step 3: ToNumber("0") → 0, then 0 === 0 → true
false == "0"    // Step 4: ToNumber(false) → 0, then 0 == "0" →
Step 3 → true
false == ""     // Step 4: 0 == "" → Step 3: 0 == 0 → true
null == undefined // Step 2: true
null == 0       // Step 7: false – null only == undefined, not 0!
undefined == 0  // Step 7: false
[] == ""        // Step 5: ToPrimitive([]) → "" (join), then "" ==
"" → true
[] == 0         // Step 5: "" == 0 → Step 3: 0 == 0 → true (!)
[1] == 1        // Step 5: "1" == 1 → Step 3: 1 == 1 → true
{} == "[object Object]" // Step 5: ToPrimitive({}) → "[object Object]"
→ true

// The classic traps
console.log(false == "0"); // true – ToNumber(false)=0,
ToNumber("0")=0
console.log(false == ""); // true
console.log("" == 0);     // true
console.log([] == false); // true – []→""→0, false→0, 0===0
console.log([0] == false); // true – "0"→0, false→0
// Therefore [] is truthy but == false – different algorithms!
if ([]) console.log("truthy"); // prints – ToBoolean([]) is true
console.log([] == false);     // true – Abstract Equality, not
ToBoolean
```

3.4 Strict Equality (===) and `Object.is`

`===` is simple: same type and same value, with two exceptions — `NaN !== NaN` and `-0 === 0`. `Object.is` fixes both.

```

// === : no coercion, but NaN and -0 are special-cased per IEEE 754
NaN === NaN    // false - IEEE 754: NaN is not equal to itself
-0 === 0       // true  - IEEE 754: -0 equals 0 under ===
0 === -0       // true

// Object.is - SameValue (no coercion, no IEEE 754 surprises)
Object.is(NaN, NaN) // true - SameValue treats NaN as same
Object.is(-0, 0)    // false - SameValue distinguishes -0
Object.is(0, -0)    // false

// When it matters
Object.is(NaN, NaN) // Map/Set key semantics use SameValueZero (like
Object.is but -0===0)
const m = new Map();
m.set(NaN, "found");
m.get(NaN); // "found" - Map uses SameValueZero, not ===
m.set(-0, "neg");
m.get(0);   // "neg" - Map treats -0 and 0 as same (SameValueZero)

// Practical rule
// Use === for general equality. Use Object.is only when NaN/-0
// distinction matters.
// Never use == except for the intentional nullish check: x == null
// (covers null|undefined).
if (value == null) {
  // true only when value is null or undefined - the one idiomatic use
  // of ==
}

```

3.5 Coercion at Scale — The Backend Cost

In a 200-service Go/Java fleet, types are checked at compile time. In a 200-service Node fleet, types are checked — if at all — at runtime, on every request, with coercion silently papering over mismatches. The operational consequences:

- **Silent data corruption.** `config.port == 3000` passes when `config.port` is "3000" from an environment variable, but `config.port + 1` becomes "30001" (string concatenation via `ToPrimitive` default hint). One missing `Number()` call cascades into connection failures.
- **Validation bypass.** `if (user.role == "admin")` passes when `user.role` is the number `0` coerced through `ToNumber`? No — but `if (userInput == true)` passes for "1", "true" does not coerce as expected, and loose checks on query parameters (`req.query.limit == 10`) behave differently for "10" vs `10` vs `""`.

- **Mitigation.** Enforce `===` and `Object.is` via `eslint: eqeqeq` (with `allow: ["null"]` for the `== null` idiom), add `typescript: strict: true` to new services, and validate at the boundary with `Zod/ajv` — not with inline coercion.

4. The Prototype Chain — Delegation, Not Inheritance

JavaScript has no classes in the classical sense. It has *prototype delegation*: every object has an internal slot `[[Prototype]]` that points at another object or `null`. Property lookup walks that chain.

4.1 `prototype` vs `__proto__` vs `[[Prototype]]`

These three names cause more confusion than any other part of the language. They refer to two distinct things:

Name	What it is	Lives on
<code>[[Prototype]]</code>	Internal slot — the actual prototype link. Spec name. Not directly accessible.	Every object
<code>__proto__</code>	Accessor property (<code>Object.prototype.__proto__</code>) — getter/setter for <code>[[Prototype]]</code> . Legacy, standardized for compatibility.	<code>Object.prototype</code>
<code>prototype</code>	Ordinary data property — the object that <i>will become</i> <code>[[Prototype]]</code> for instances created via <code>new</code> .	Constructor functions only

```

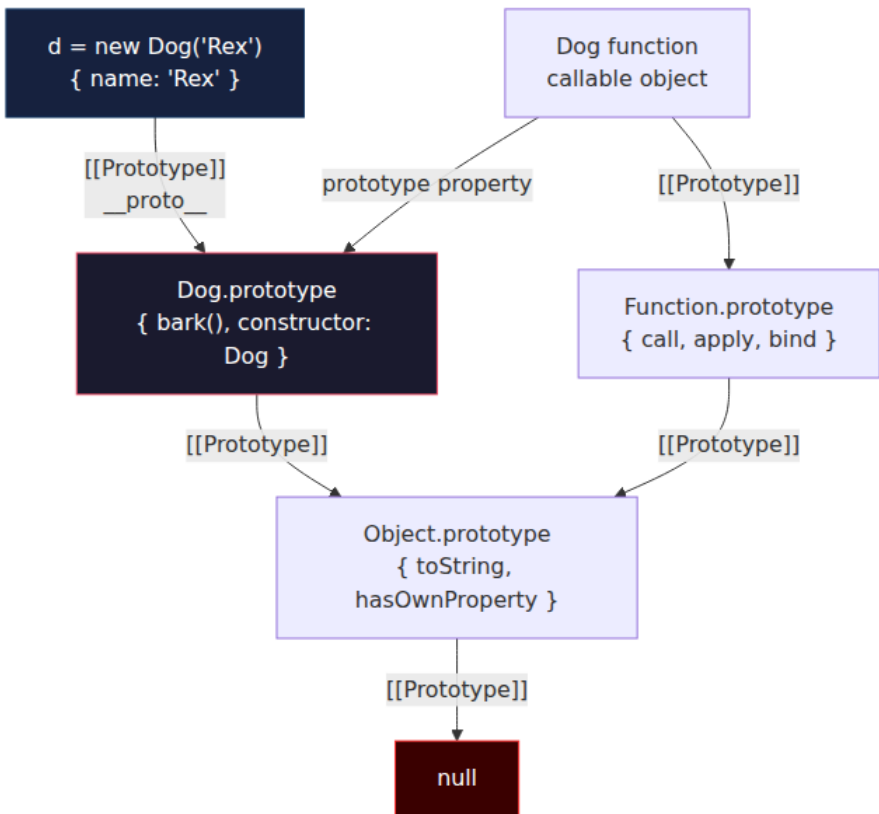
function Dog(name) {
  this.name = name;
}
Dog.prototype.bark = function() { return `${this.name} says woof`; };

const d = new Dog("Rex");

// What points where:
console.log(Dog.prototype);           // { bark: f, constructor:
Dog }
console.log(Object.getPrototypeOf(d)); // same object as
Dog.prototype
console.log(d.__proto__);              // same – via getter on
Object.prototype
console.log(d.__proto__ === Dog.prototype); // true
console.log(Dog.__proto__ === Function.prototype); // true – Dog is a
function

// d itself has NO .prototype property – only functions do
console.log(d.prototype);             // undefined
console.log(typeof Dog.prototype);    // "object"
console.log(typeof d.__proto__);      // "object"

```

4.2 Walking the Chain — The Code You Must Be Able to Read

Every property access follows this walk. The engine optimizes it with hidden classes and inline caches (Chapter 1), but the semantics are a loop:

```
// Manual prototype walk – what the engine does on every property read
function getPropertyChain(obj, prop) {
  let current = obj;
  let depth = 0;
  while (current !== null) {
    const hasOwn = Object.prototype.hasOwnProperty.call(current, prop);
    const desc = Object.getOwnPropertyDescriptor(current, prop);
    console.log(
      `depth ${depth}: ${current.constructor?.name || "Object"} ` +
      `– hasOwn=${hasOwn}` +
      (desc ? ` descriptor=${JSON.stringify({ value: desc.value, get: t
ypeof desc.get, enumerable: desc.enumerable })}` : "")
    );
    if (hasOwn) {
      console.log(` → found "${prop}" at depth ${depth}`);
      return { found: true, depth, holder: current, descriptor: desc };
    }
    current = Object.getPrototypeOf(current); // same as
    current.__proto__ but without the accessor
    depth++;
  }
  console.log(` → "${prop}" not found – reached null`);
  return { found: false, depth: -1 };
}
```

// Example hierarchy

```
class Animal {
  speak() { return "generic sound"; }
}
class Dog extends Animal {
  speak() { return "woof"; }
  fetch() { return "fetching"; }
}
const d = new Dog();
```

```
getPropertyChain(d, "fetch");
// depth 0: Dog – hasOwn=false ... wait, fetch lives on Dog.prototype,
not the instance
// Let's check more carefully:
```

```
console.log("--- own properties of d ---");
console.log(Object.getOwnPropertyNames(d)); // [] –
instance has no own methods
console.log(Object.getOwnPropertyNames(Dog.prototype)); //
["constructor", "speak", "fetch"]
console.log(Object.getOwnPropertyNames(Animal.prototype)); //
["constructor", "speak"]
console.log(Object.getOwnPropertyNames(Object.prototype)); //
["toString", "hasOwnProperty", ...]
```

```

console.log("\n--- full chain walk for 'speak' ---");
getPropertyChain(d, "speak");
// depth 0: Dog – no own "speak"
// depth 1: Dog.prototype – hasOwn=true → found (Dog's speak shadows Animal's)

console.log("\n--- walk for 'toString' ---");
getPropertyChain(d, "toString");
// depth 0: Dog instance – miss
// depth 1: Dog.prototype – miss
// depth 2: Animal.prototype – miss
// depth 3: Object.prototype – hit → Object.prototype.toString

console.log("\n--- walk for 'nonexistent' ---");
getPropertyChain(d, "nonexistent");
// walks to null → not found → returns undefined at runtime

// The raw __proto__ chain – the shape the engine traverses
let cur = d;
let chain = [];
while (cur !== null) {
  chain.push(cur.constructor?.name || "(null proto object)");
  cur = Object.getPrototypeOf(cur);
}
console.log("\n __proto__ chain:", chain.join(" → "));
// Dog → Dog → Animal → Object → (null)

// Shadowing: own property wins over prototype
d.speak = function() { return "custom woof"; };
console.log(d.speak()); // "custom woof" – own property at depth 0
// shadows prototype
delete d.speak;
console.log(d.speak()); // "woof" – falls back to Dog.prototype again

```

Rules the walk implies:

- **Writes never walk.** `d.newProp = 1` always creates an own property on `d`, even if `Dog.prototype.newProp` exists. Only reads delegate.
- **hasOwnProperty vs in.** `"toString"` in `d` is `true` (walks chain); `d.hasOwnProperty("toString")` is `false` (own only). In backend code, prefer `Object.hasOwn(d, key)` (ES2022) over `d.hasOwnProperty` — it is not vulnerable to `hasOwnProperty` being shadowed.
- **__proto__ setter is slow.** Assigning `obj.__proto__ = x` deoptimizes the object to dictionary mode in V8. Use `Object.create(proto)` or `Object.setPrototypeOf` (still slow, but explicit). For hot paths, never mutate `[[Prototype]]` after construction.

4.3 `Object.create`, `Object.getPrototypeOf`, and Null-Prototype Objects

```
// Object.create – create an object with an explicit [[Prototype]]
const animalProto = {
  speak() { return `${this.name} makes a sound`; }
};
const dog = Object.create(animalProto);
dog.name = "Rex";
console.log(dog.speak()); // "Rex makes a sound" – delegates to
// animalProto
console.log(Object.getPrototypeOf(dog) === animalProto); // true

// Null-prototype objects – no delegation at all, useful as safe
// dictionaries
const dict = Object.create(null);
dict["__proto__"] = "not a prototype, just a key"; // safe – no
// pollution
console.log(dict["__proto__"]); // "not a prototype, just a key"
console.log("__proto__" in dict); // true – own property
console.log("toString" in dict); // false – no Object.prototype

// Contrast with a plain object – prototype pollution vector
const unsafe = {};
unsafe["__proto__"]; // accessor on Object.prototype – not an own key
// If a deep-merge does: target[key] = source[key] with
// key="__proto__", it mutates the prototype
```

Null-prototype objects are the correct dictionary type for untrusted keys — request headers, query parameters, user-supplied JSON keys. Using `{}` as a map with user-controlled keys is a prototype-pollution vulnerability (see §7 and Vol. 9, Chapter 7).

5. Class Syntax — Sugar Over Prototypes

`class` is not a new object model. It is deterministic desugaring to constructor functions, prototype assignment, and `Object.setPrototypeOf` wiring. Understanding the desugaring is the only way to debug `super`, `extends`, and `instanceof` correctly.

5.1 What `class` Desugars To

```
// What you write:
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} speaks`; }
  static create(name) { return new this(name); }
}

// What the engine effectively creates (simplified – see ES spec §15.7):
function AnimalDesugared(name) {
  // Constructor body – called with `new`
  this.name = name;
}
// Instance methods → AnimalDesugared.prototype
Object.defineProperty(AnimalDesugared.prototype, "speak", {
  value: function() { return `${this.name} speaks`; },
  enumerable: false, writable: true, configurable: true
});
// Static methods → constructor itself
Object.defineProperty(AnimalDesugared, "create", {
  value: function(name) { return new this(name); },
  enumerable: false, writable: true, configurable: true
});
// constructor property
Object.defineProperty(AnimalDesugared.prototype, "constructor", {
  value: AnimalDesugared, enumerable: false, writable: true, configurable: true
});
// Prototype chain of the constructor itself
Object.setPrototypeOf(AnimalDesugared, Function.prototype);

// Verification – they are structurally identical
console.log(typeof Animal); // "function"
console.log(typeof AnimalDesugared); // "function"
console.log(Object.getOwnPropertyDescriptor(Animal.prototype, "speak").enumerable); // false
console.log(Object.getOwnPropertyDescriptor(AnimalDesugared.prototype, "speak").enumerable); // false
```

Key differences between `class` and manual prototype assignment that are easy to miss:

- Class methods are **non-enumerable** (manual `Ctor.prototype.m = fn` is enumerable by default).
- Class constructors **must be called with `new`** (`ClassCallCheck` throws).

- `class` declarations are **not hoisted** like functions — TDZ applies.

5.2 `extends` and `super` — The Two Prototype Links

`extends` wires *two* prototype chains simultaneously:

```

class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} speaks`; }
  static kind() { return "animal"; }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name); // must call super before touching `this`
    this.breed = breed;
  }
  speak() {
    return super.speak() + " - woof";
  }
  static kind() {
    return super.kind() + ":dog";
  }
}

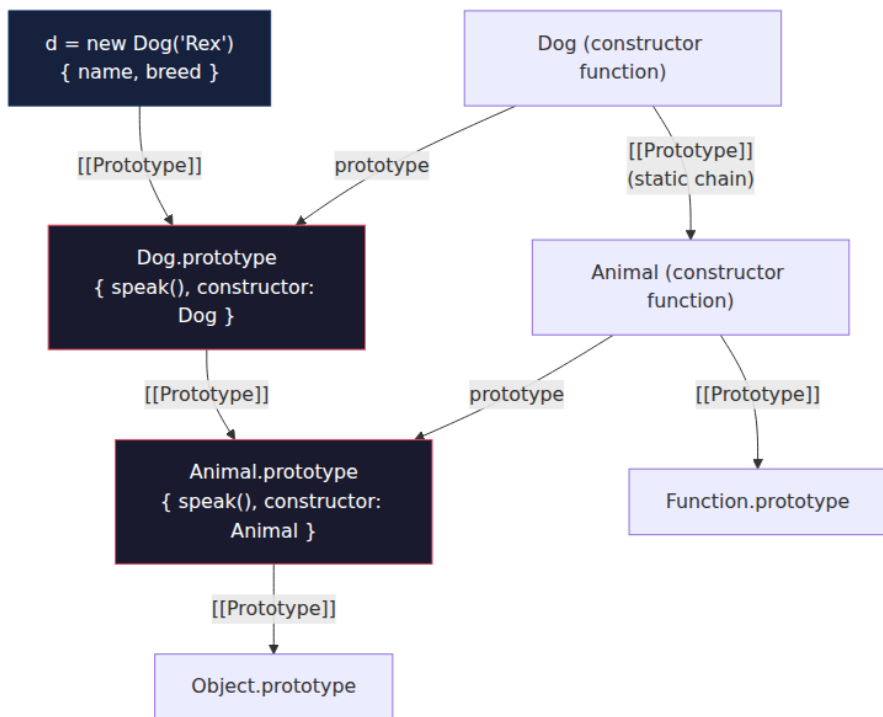
// What extends wires (simplified):
// 1. Instance chain: Dog.prototype.__proto__ === Animal.prototype
// 2. Static chain: Dog.__proto__ === Animal
console.log(Object.getPrototypeOf(Dog.prototype) ===
Animal.prototype); // true
console.log(Object.getPrototypeOf(Dog) === Animal);
// true

const d = new Dog("Rex", "shepherd");
console.log(d.speak()); // "Rex speaks - woof"
console.log(Dog.kind()); // "animal:dog"

// super is not a simple property lookup – it is a [[HomeObject]]-
relative lookup
// super.speak inside Dog.prototype.speak means:
// Object.getPrototypeOf(Dog.prototype).speak.call(this)
// The engine stores [[HomeObject]] (Dog.prototype) on the method at
definition time.
console.log(Object.getOwnPropertyDescriptor(Dog.prototype, "speak").val
ue);
// has internal slot [[HomeObject]] → Dog.prototype, used to resolve
super

// super() in constructor – what it does:
// 1. Calls Animal constructor with `this` uninitialized (TDZ)
// 2. Animal constructor initializes `this` (allocates via new.target)
// 3. Returns the initialized `this` – now Dog constructor can assign
breed

```



If `extends` only wired the instance chain, `Dog.kind()` would not find `Animal.kind()`. The static chain (`Dog.__proto__ === Animal`) is what makes static inheritance work — and what makes `instanceof` walk the `prototype` chain rather than the `__proto__` chain of the constructor.

5.3 Private Fields, Static Blocks, and What They Cost

```
class Counter {
  static #instances = 0;           // private static field – per-class,
  not per-instance
  #count = 0;                     // private instance field – per-
  instance, WeakMap-backed in spec
  static {
    // static block – runs once at class evaluation, `this` is the
    class
    this.#instances++;
  }
  constructor() {
    Counter.#instances++;
  }
  increment() { this.#count++; return this.#count; }
  get count() { return this.#count; } // exposed via accessor
  static get instances() { return Counter.#instances; }
}

const c1 = new Counter();
const c2 = new Counter();
console.log(c1.increment()); // 1
console.log(c1.count);       // 1
console.log(c2.count);       // 0 – separate private slot
// c1.#count – SyntaxError: Private field '#count' must be declared in
// an enclosing class

// Private fields are not prototype properties – they are per-instance
// slots
// checked at runtime. In V8 they are stored out-of-line with a brand
// check.
// Accessing a private field on a wrong receiver throws TypeError, not
// undefined.
const fake = {};
try { Counter.prototype.increment.call(fake); } catch (e) {
  console.log(e.message); // Cannot read private member #count from an
  object whose class did not declare it
}
```

Performance note: private fields (`#x`) have a brand check on every access — the engine verifies the receiver was constructed by the class that declared the field. This is slightly slower than a public property or a `WeakMap` for cross-instance access, but faster than `WeakMap` for same-class access in V8 (the field offset is known at compile time after the Map check). For hot paths with millions of instances, measure; for most backend code, prefer private fields for encapsulation — the cost is a single branch.

6. Symbols and Well-Known Symbols — The Protocol Layer

`Symbol` is a primitive that is unique by construction. Every `Symbol()` call creates a value that is `!==` every other value, including another `Symbol` with the same description. The global registry (`Symbol.for`) and well-known Symbols are the extension points for the language itself.

6.1 Unique Symbols and the Global Registry

```
// Unique symbols – never collide
const s1 = Symbol("id");
const s2 = Symbol("id");
console.log(s1 === s2);           // false – different identities, same
description
console.log(s1.description);      // "id"
console.log(typeof s1);           // "symbol"

// Symbols as non-colliding property keys – the original use case
const kMetadata = Symbol("metadata");
const obj = { [kMetadata]: { version: 1 }, name: "service" };
console.log(obj[kMetadata]);      // { version: 1 }
console.log(Object.keys(obj));    // ["name"] – symbols are skipped
console.log(Object.getOwnPropertySymbols(obj)); // [Symbol(metadata)]
console.log(JSON.stringify(obj)); // '{"name":"service"}' – symbols are
omitted

// Global registry – cross-realm shared symbols
const g1 = Symbol.for("app.config");
const g2 = Symbol.for("app.config");
console.log(g1 === g2);           // true – same registry entry
console.log(Symbol.keyFor(g1));   // "app.config"
console.log(Symbol.keyFor(s1));   // undefined – not in registry

// When the registry matters: sharing a symbol across packages without
importing it
// Package A: Symbol.for("my-lib.internal") – Package B can retrieve
the same symbol
// without depending on A. Useful for framework-level coordination,
risky for collisions.
```

6.2 Well-Known Symbols — Customizing Language Protocols

Well-known Symbols are the hooks the spec uses to let user code participate in built-in operations. Each is a property on `Symbol` itself

(`Symbol.iterator`, `Symbol.hasInstance`, etc.) and is looked up via `GetMethod` during the operation.

Symbol	Protocol	Where it is consulted
<code>Symbol.iterator</code>	Iterable	<code>for...of</code> , <code>spread [...x]</code> , destructuring
<code>Symbol.asyncIterator</code>	Async iterable	<code>for await...of</code>
<code>Symbol.toStringTag</code>	<code>Object.prototype.toString</code>	<code>Object.prototype.toString</code>
<code>Symbol.hasInstance</code>	<code>instanceof</code>	<code>x instanceof C</code> calls <code>C[Symbol.hasInstance](x)</code>
<code>Symbol.toPrimitive</code>	Coercion hint	<code>ToPrimitive</code> (see §3.1)
<code>Symbol.species</code>	Constructor species	<code>Array</code> methods that create n (<code>map</code> , <code>filter</code> , <code>slice</code>)
<code>Symbol.match</code> / <code>replace</code> / <code>search</code> / <code>split</code>	String/RegExp	<code>str.match(re)</code> , <code>str.replace(re, ...)</code>
<code>Symbol.unscopables</code>	<code>with</code> (legacy)	Properties hidden from <code>with</code>

```

// Symbol.iterator – make any object iterable
class Range {
  constructor(start, end) { this.start = start; this.end = end; }
  *[Symbol.iterator]() {
    for (let i = this.start; i <= this.end; i++) yield i;
  }
}

console.log([...new Range(1, 3)]); // [1, 2, 3]
for (const n of new Range(5, 7)) console.log(n); // 5, 6, 7

// Manual iterator (what the engine actually calls)
const range = new Range(1, 2);
const it = range[Symbol.iterator]();
console.log(it.next()); // { value: 1, done: false }
console.log(it.next()); // { value: 2, done: false }
console.log(it.next()); // { value: undefined, done: true }

// Symbol.toStringTag – control Object.prototype.toString output
class ServiceError extends Error {
  get [Symbol.toStringTag]() { return "ServiceError"; }
}

const err = new ServiceError("timeout");
console.log(Object.prototype.toString.call(err)); // "[object ServiceError]" – not "[object Error]"
console.log(err.toString()); // "ServiceError: timeout"

// Symbol.hasInstance – customize instanceof
class EvenNumber {
  static [Symbol.hasInstance](value) {
    return typeof value === "number" && value % 2 === 0;
  }
}

console.log(4 instanceof EvenNumber); // true
console.log(3 instanceof EvenNumber); // false
console.log("4" instanceof EvenNumber); // false – typeof check fails

// Symbol.species – control what constructor derived methods use
class MyArray extends Array {
  static get [Symbol.species]() { return Array; } // map/filter return plain Array, not MyArray
}

const a = new MyArray(1, 2, 3);
const b = a.map(x => x * 2);
console.log(b instanceof MyArray); // false – species redirected to Array
console.log(b instanceof Array); // true

// Symbol.toPrimitive – already covered in §3.1, included here for completeness
class Money {

```

```

    constructor(cents) { this.cents = cents; }
    [Symbol.toPrimitive](hint) {
      if (hint === "string") return `$$${this.cents / 100}.toFixed(2)}`;
      if (hint === "number") return this.cents;
      return this.cents; // default
    }
  }
  const price = new Money(1999);
  console.log(String(price)); // "$19.99"
  console.log(Number(price)); // 1999
  console.log(price + 1);     // 2000 (default hint → number)

```

For backend engineers, the most operationally relevant well-known Symbols are `Symbol.iterator` (streaming and pagination protocols often expose async iterables), `Symbol.toStringTag` (log enrichment and error classification), and `Symbol.hasInstance` (custom error hierarchies that survive cross-realm `instanceof` failures — see §8.4).

7. Proxy and Reflect — Intercepting the Metaobject Protocol

A `Proxy` wraps a *target* object and interposes on fundamental operations — property lookup, assignment, enumeration, function calls, construction, prototype queries — via *traps*. `Reflect` provides the default forwarding for each trap, so a proxy can observe and optionally delegate without reimplementing spec semantics.

7.1 The Trap Table and Invariants

Every trap corresponds to an internal method (`[[Get]]`, `[[Set]]`, `[[HasProperty]]`, etc.). If you define a trap, you replace that internal method on the proxy. If you omit it, the proxy forwards to the target.

```

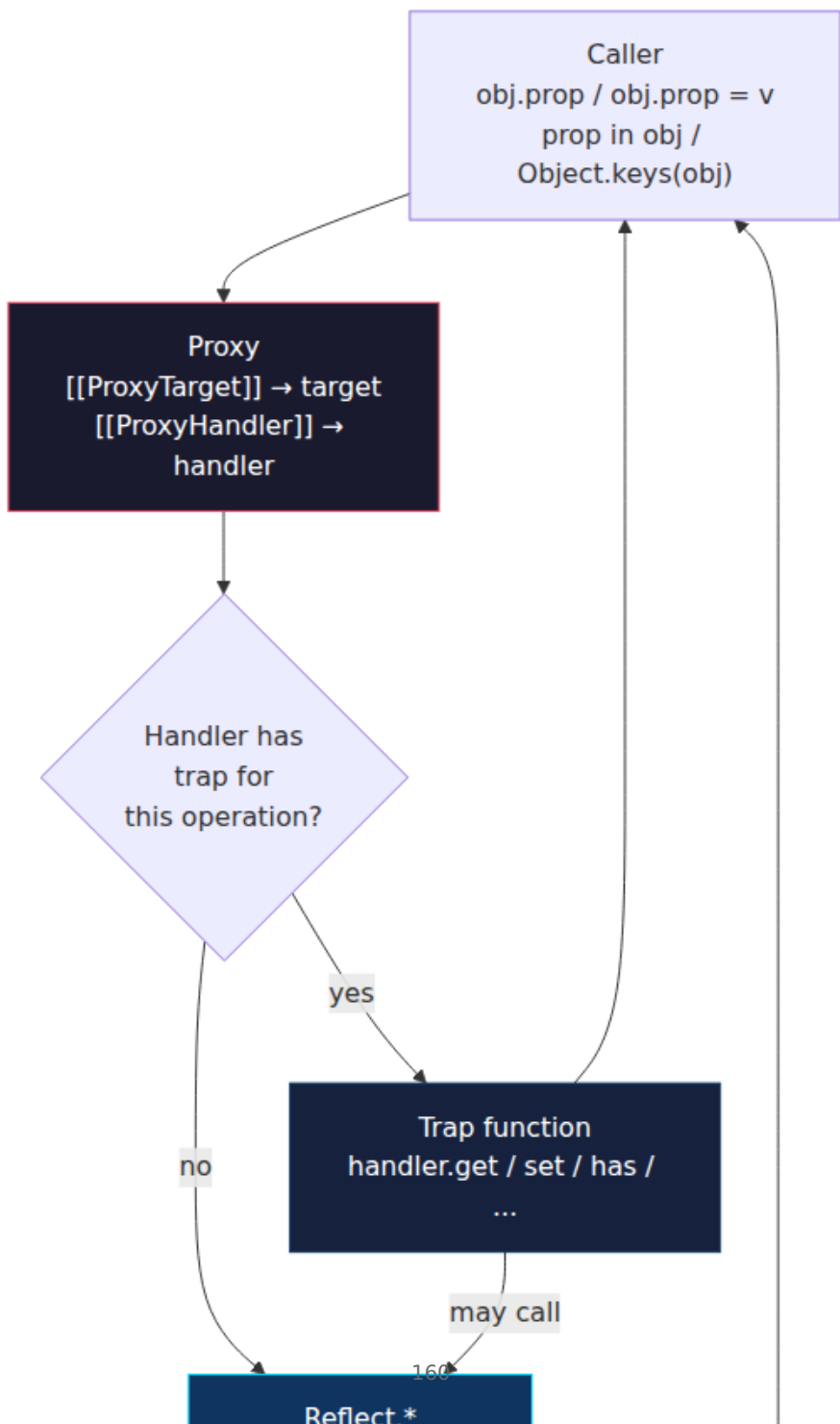
// Trap → internal method mapping (ECMA-262 §9.5)
const trapTable = `
// Proxy handler traps                                → Reflect
counterpart      → Internal method
  get(target, prop, receiver)      Reflect.get(target, prop,
receiver)      [[Get]]
  set(target, prop, value, receiver) Reflect.set(target, prop,
value, receiver) [[Set]]
  has(target, prop)                Reflect.has(target,
prop)                [[HasProperty]]
  deleteProperty(target, prop)
Reflect.deleteProperty(target, prop)  [[Delete]]
  ownKeys(target)
Reflect.ownKeys(target)                [[OwnPropertyKeys]]
  getOwnPropertyDescriptor(target, prop)
Reflect.getOwnPropertyDescriptor(target, prop) [[GetOwnProperty]]
  defineProperty(target, prop, desc)
Reflect.defineProperty(target, prop, desc) [[DefineOwnProperty]]
  getPrototypeOf(target)
Reflect.getPrototypeOf(target)          [[GetPrototypeOf]]
  setPrototypeOf(target, proto)
Reflect.setPrototypeOf(target, proto)  [[SetPrototypeOf]]
  isExtensible(target)
Reflect.isExtensible(target)            [[IsExtensible]]
  preventExtensions(target)
Reflect.preventExtensions(target)        [[PreventExtensions]]
  apply(target, thisArg, args)          Reflect.apply(target,
thisArg, args)      [[Call]]    – function proxies only
  construct(target, args, newTarget)    Reflect.construct(target,
args, newTarget)  [[Construct]] – constructor proxies only
`;

```

Traps have *invariants* — conditions the engine enforces even if your handler violates them, throwing `TypeError`:

- If the target is non-extensible, `ownKeys` must return exactly the target's own keys.
- `getOwnPropertyDescriptor` must report non-configurable properties truthfully.
- `set` must return `true` if the property is a non-writable data property and the assign succeeded (strict-mode assignment checks the return value).
- `getPrototypeOf` must return the target's actual prototype if the target is non-extensible.

Violating invariants is not a lint warning — it is a runtime `TypeError` that crashes the request.



7.2 Minimal Proxy — Logging and Validation

```

// Logging proxy – observe every operation without changing behavior
function loggingProxy(target, label = "obj") {
  return new Proxy(target, {
    get(t, prop, receiver) {
      const v = Reflect.get(t, prop, receiver);
      console.log(`GET ${label}[${String(prop)}] →`, v);
      return v;
    },
    set(t, prop, value, receiver) {
      console.log(`SET ${label}[${String(prop)}] =`, value);
      return Reflect.set(t, prop, value, receiver);
    },
    has(t, prop) {
      const r = Reflect.has(t, prop);
      console.log(`HAS ${label}[${String(prop)}] →`, r);
      return r;
    },
    ownKeys(t) {
      const keys = Reflect.ownKeys(t);
      console.log(`OWNKEYS ${label} →`, keys);
      return keys;
    }
  });
}

const user = loggingProxy({ name: "Alice", age: 30 }, "user");
user.name;           // GET user[name] → Alice
user.age = 31;       // SET user[age] = 31
"name" in user;      // HAS user[name] → true
Object.keys(user);   // OWNKEYS user → ["name", "age"]

// Validation proxy – enforce invariants before forwarding
function validatedConfig(target) {
  return new Proxy(target, {
    set(t, prop, value, receiver) {
      if (prop === "port" && (typeof value !== "number" || value < 1
|| value > 65535)) {
        throw new TypeError(`Invalid port: ${value}`);
      }
      if (prop === "retries" && (!Number.isInteger(value) || value < 0)) {
        throw new TypeError(`Invalid retries: ${value}`);
      }
      return Reflect.set(t, prop, value, receiver);
    },
    defineProperty(t, prop, desc) {
      // Also trap defineProperty – otherwise Object.defineProperty
      // bypasses set
      if (prop === "port" && desc.value !== undefined) {

```

```

        if (typeof desc.value !== "number" || desc.value < 1 || desc.value > 65535) {
            throw new TypeError(`Invalid port: ${desc.value}`);
        }
    }
    return Reflect.defineProperty(t, prop, desc);
}
});
}

const config = validatedConfig({ port: 3000, retries: 3 });
config.port = 8080; // ok
try { config.port = 99999; } catch (e) { console.log(e.message); } //
Invalid port: 99999
try { Object.defineProperty(config, "port", { value: -1 }); } catch
(e) { console.log(e.message); }

```

The `defineProperty` trap is the one most implementations forget. Without it, `Object.defineProperty(proxy, "port", { value: -1 })` bypasses `set` entirely — a validation bypass that survives code review if the reviewer only checks `set`.

7.3 Revocable Proxies — Capability Revocation

`Proxy.revocable` creates a proxy whose handler can be atomically disconnected. After revocation, every trap throws `TypeError`. This is the primitive for capability-based security in JavaScript.

```

// Revocable proxy – grant temporary access, then revoke
function grantTemporaryAccess(target, ttlMs) {
  const { proxy, revoke } = Proxy.revocable(target, {
    get(t, prop, receiver) { return Reflect.get(t, prop, receiver); },
    set(t, prop, value, receiver) { return Reflect.set(t, prop, value,
receiver); },
  });
  const timer = setTimeout(() => {
    revoke();
    console.log("Access revoked");
  }, ttlMs);
  // Return both – caller uses proxy, cleanup calls revoke
  return { proxy, revoke: () => { clearTimeout(timer); revoke(); } };
}

const secret = { apiKey: "sk-live-abc123" };
const { proxy: tempSecret, revoke } = grantTemporaryAccess(secret,
5000);
console.log(tempSecret.apiKey); // "sk-live-abc123"
revoke();
try { console.log(tempSecret.apiKey); } catch (e) {
  console.log(e.message); // Cannot perform 'get' on a proxy that has
been revoked
}

// Real use: sandboxing untrusted code
function runUntrusted(code, allowedGlobals) {
  const { proxy: sandbox, revoke } = Proxy.revocable(allowedGlobals, {
    has() { return true; }, // trap `with` lookups – claim every name
exists
    get(t, prop) {
      if (prop in t) return t[prop];
      throw new ReferenceError(`${String(prop)} is not defined in
sandbox`);
    }
  });
  try {
    // In production, combine with vm module or SES (Hardened JS)
    // This is illustrative – with + proxy is not a full sandbox
    return Function("sandbox", `with(sandbox) { return (${code}); }`)(s
andbox);
  } finally {
    revoke();
  }
}

```

7.4 Membrane — The Proxy Pattern That Matters for Backends

A *membrane* wraps an entire object graph so that every object crossing the boundary is transitively wrapped. It is the correct way to isolate untrusted or observed subgraphs, revoke access to a whole subsystem, and implement deep immutability or deep observation without copying.

```

// Full membrane – every object crossing the boundary is wrapped
consistently
// Based on the membrane pattern from "Comprehensive
Compartmentalization" (Miller, 2006)
// and the SES (Hardened JS) implementation.

function createMembrane() {
  const targetToProxy = new WeakMap();
  const proxyToTarget = new WeakMap();

  function wrap(target) {
    if (target === null || typeof target !== "object") return target; /
    / primitives pass through
    if (targetToProxy.has(target)) return targetToProxy.get(target);
    if (proxyToTarget.has(target)) return target; // already a proxy –
    unwrap check not needed for this direction

    const handler = {
      get(t, prop, receiver) {
        const v = Reflect.get(t, prop, receiver);
        return wrap(v); // transitively wrap on read
      },
      set(t, prop, value, receiver) {
        // Unwrap proxies being written in, wrap values being read out
        const unwrapped = proxyToTarget.get(value) ?? value;
        return Reflect.set(t, prop, unwrapped, receiver);
      },
      apply(t, thisArg, args) {
        const unwrappedThis = proxyToTarget.get(thisArg) ?? thisArg;
        const unwrappedArgs = args.map(a => proxyToTarget.get(a) ?? a);
        const result = Reflect.apply(t, unwrappedThis, unwrappedArgs);
        return wrap(result);
      },
      getOwnPropertyDescriptor(t, prop) {
        const desc = Reflect.getOwnPropertyDescriptor(t, prop);
        if (desc) {
          if ("value" in desc) desc.value = wrap(desc.value);
          if (desc.get) desc.get = wrap(desc.get);
          if (desc.set) desc.set = wrap(desc.set);
        }
        return desc;
      },
    };
    // ... other traps forward similarly (ownKeys, has, etc.)
    has(t, prop) { return Reflect.has(t, prop); },
    ownKeys(t) { return Reflect.ownKeys(t); },
    getPrototypeOf(t) { return wrap(Reflect.getPrototypeOf(t)); },
  };

  const proxy = new Proxy(target, handler);
  targetToProxy.set(target, proxy);

```

```

    proxyToTarget.set(proxy, target);
    return proxy;
}

// Revocable membrane – one revoke disconnects the entire graph
function revocableMembrane(root) {
    const { proxy, revoke } = Proxy.revocable(root, {
        get(t, prop, receiver) { return wrap(Reflect.get(t, prop, receive
r)); },
        set(t, prop, value, receiver) {
            return Reflect.set(t, prop, proxyToTarget.get(value) ?? value,
receiver);
        },
        // ... remaining traps as above
    });
    // Wrap the root through the same WeakMaps so identity is preserved
    targetToProxy.set(root, proxy);
    proxyToTarget.set(proxy, root);
    return { proxy, revoke };
}

return { wrap, revocableMembrane, targetToProxy, proxyToTarget };
}

// Usage – isolate a plugin's view of the host
const membrane = createMembrane();

const hostState = {
    config: { port: 3000, secrets: { apiKey: "sk-live-123" } },
    users: [{ id: 1, name: "Alice" }],
    getConfig() { return this.config; }
};

const pluginView = membrane.wrap(hostState);

// Plugin reads – gets wrapped objects, never the raw host objects
const cfg = pluginView.config;
console.log(cfg === hostState.config); // false – different identity (proxy)
console.log(cfg.port); // 3000 – transparently forwarded

// Plugin cannot escape the membrane by reaching the prototype
console.log(pluginView.getConfig() === cfg); // true – same proxy identity (WeakMap ensures idempotence)

// Revocable variant – one revoke cuts the entire plugin off
const { proxy: tempView, revoke } = membrane.revocableMembrane(hostStat
e);
console.log(tempView.config.port); // 3000
revoke();

```

```
try { console.log(tempView.config.port); } catch (e) {  
  console.log("Revoked:", e.message); // Cannot perform 'get' on a  
  proxy that has been revoked  
}
```

Design properties of a correct membrane:

- **Identity preservation.** `wrap(a) === wrap(a)` must hold (hence `WeakMap`). Without it, `Set` membership and `===` checks break inside the membrane.
- **Transitive wrapping.** Every object returned by any trap must be wrapped, otherwise the guest can obtain an unwrapped reference and escape.
- **Unwrapping on write.** Values the guest writes must be unwrapped before storing on the target, otherwise the host accumulates proxies in its own state.
- **No proxy transparency.** Proxies are not transparent — `proxy !== target`, `Array.isArray(proxy)` checks the target but `proxy instanceof Array` may behave differently across realms. Code that relies on `===` identity for caching or deduplication will see two identities for one logical object.

Performance note: every proxy adds an allocation, a `WeakMap` lookup, and a trap dispatch. A membrane that wraps a 100K-node object graph on first access will allocate 100K proxies. In V8, proxied property access is megamorphic and cannot be inlined — expect 10-100× slowdown on hot paths. Use membranes at trust boundaries (plugin isolation, deep immutability for config that is read rarely), not on inner loops.

8. The Module System — Two Runtimes in One Registry

JavaScript has two module systems with incompatible semantics sharing one package registry and one `node_modules` tree. Understanding both — and the seams between them — is the prerequisite for every bundling, deployment, and supply-chain decision in this volume.

8.1 CommonJS — The Dynamic Module System

CommonJS (CJS) was specified for server-side JavaScript in 2009 and adopted by Node.js. It is *dynamic*: `require` is a function call, `module.exports` is a mutable object, and modules are loaded synchronously at runtime.

The Module Wrapper

Node does not execute a CJS file as bare script. It wraps it in a function:

```
// What Node actually executes for a file `lib/math.js`:
(function(exports, require, module, __filename, __dirname) {
  // your file contents here
  function add(a, b) { return a + b; }
  module.exports = { add };
  // `exports` is initially `module.exports`; reassigning `exports`
alone does nothing
});

// Invocation per file (simplified from lib/internal/modules/cjs/
loader.js):
// Module._compile wraps, then calls the function with per-file
arguments.
```

Consequences:

- `exports` vs `module.exports`: `exports` is an alias for `module.exports` at entry. `exports.foo = 1` mutates the shared object (correct). `exports = { foo: 1 }` rebinds the local variable and is lost (bug). Always assign to `module.exports` when replacing the export object.
- `__filename` / `__dirname` are per-module, not globals.
- Top-level `return` is legal inside the wrapper (it returns from the wrapper function), but linters flag it.

require, require.cache, and Synchronous Loading

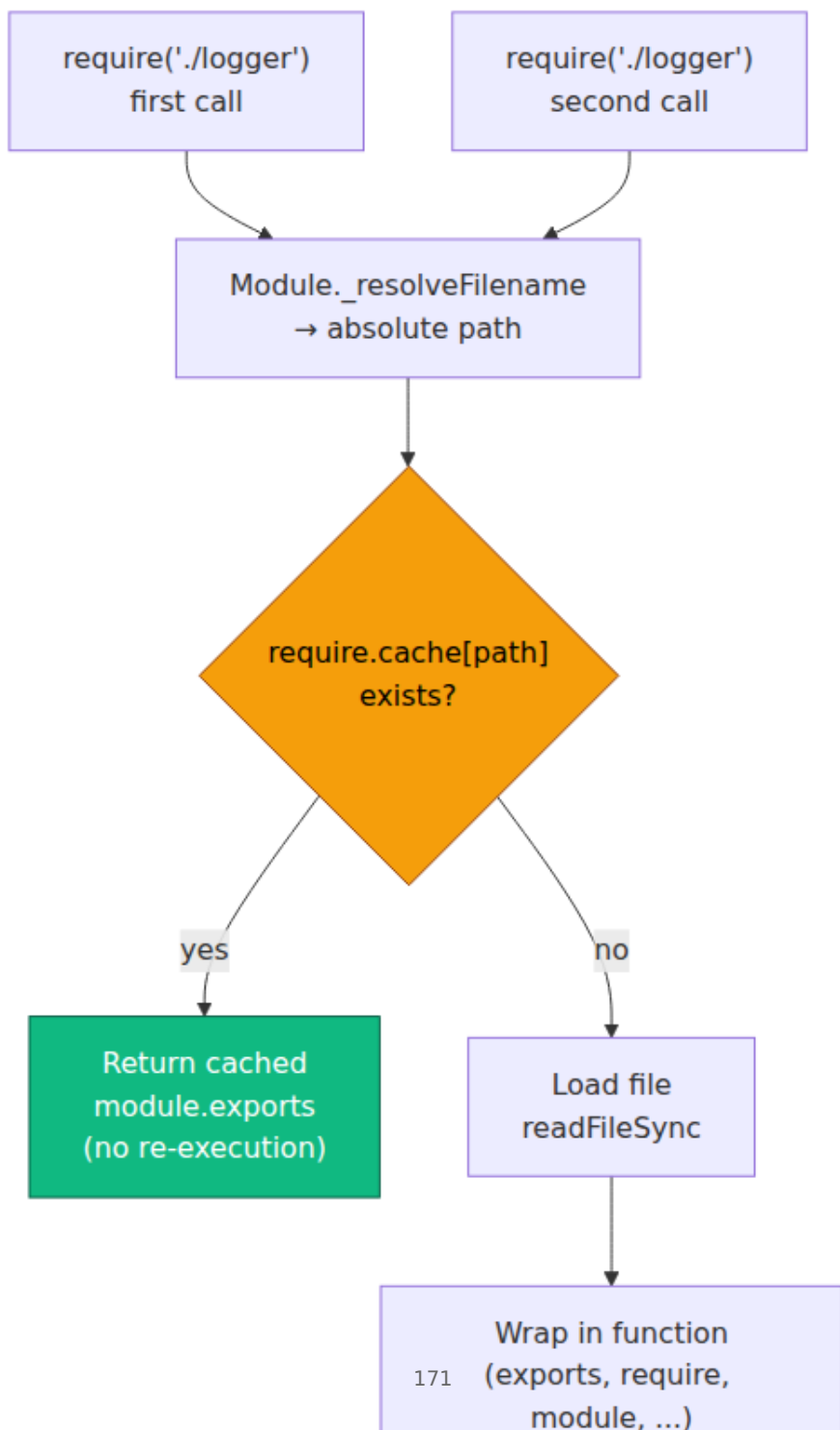
```
// lib/logger.js
let calls = 0;
function log(msg) { calls++; console.log(`[${calls}] ${msg}`); }
module.exports = { log, getCalls: () => calls };

// app.js
const logger1 = require("./lib/logger");
logger1.log("first"); // [1] first

const logger2 = require("./lib/logger");
logger2.log("second"); // [2] second – same instance, cache hit

console.log(logger1 === logger2); // true – same object
console.log(require.cache[require.resolve("./lib/logger")] !== undefined); // true
console.log(logger1.getCalls()); // 2 – shared mutable state

// Cache invalidation (used in tests, watch mode – dangerous in production)
delete require.cache[require.resolve("./lib/logger")];
const logger3 = require("./lib/logger");
console.log(logger3.getCalls()); // 0 – fresh instance, state reset
console.log(logger1 === logger3); // false – different object now
```



Circular Dependencies — The Half-Initialized Trap

CJS handles cycles by returning the *partially populated* `module.exports` object. This is the most common source of `undefined` exports in large codebases.

```
// a.js
console.log("a: start");
exports.loaded = false;
const b = require("./b");
console.log("a: b.loaded =", b.loaded); // false - b hasn't finished
initializing
exports.loaded = true;
console.log("a: done");

// b.js
console.log("b: start");
exports.loaded = false;
const a = require("./a");
console.log("b: a.loaded =", a.loaded); // false - a is half-
initialized! a.loaded is still false
exports.loaded = true;
console.log("b: done");

// node a.js output:
// a: start
// b: start
// b: a.loaded = false   ← a is incomplete
// b: done
// a: b.loaded = true    ← b finished, so a sees the final value
// a: done
```

```
// The fix: export functions, not values, or defer access
// a.js (safe)
const b = require("./b");
exports.getA = () =>
  aValue; // function - evaluated lazily, after both modules load
let aValue = 42;

// b.js (safe)
const a = require("./a");
console.log(a.getA()); // 42 - by the time this runs, aValue is
initialized
```

8.2 ESM — The Static Module System

ECMAScript Modules (ESM, standardized in ES2015) are *static*: `import / export` are syntax, not runtime calls. The engine can determine the entire module graph without executing any module — enabling tree-shaking, cyclic handling via live bindings, and deterministic linking before evaluation.

Static Imports, Named vs Default Exports

```
// math.js — ESM
export const PI = 3.14159;
export function add(a, b) { return a + b; }
export default function multiply(a, b) { return a * b; }

// app.mjs — static imports (must be at top level, string literal
// specifiers)
import multiply, { PI, add } from "./math.js";
import * as math from "./math.js"; // namespace import

console.log(PI);           // 3.14159
console.log(add(2, 3));    // 5
console.log(multiply(2, 3)); // 6
console.log(math.PI);     // 3.14159
console.log(math.default); // [Function: multiply]

// Re-exports and aggregation
// index.js — barrel file
export { PI, add } from "./math.js";
export { default as multiply } from "./math.js";
export * from "./utils.js"; // re-export all named exports (not
// default)

// Conditional / dynamic import — the only non-static import
const mod = await import(`./locales/${lang}.js`); // dynamic, returns
// Promise<ModuleNamespace>
if (featureFlag) {
  const { heavyFn } = await import("./heavy.js"); // lazy, code-split
  // point
  heavyFn();
}
```

Static constraints that are easy to forget:

- `import` must be at the top level (not inside `if`, not inside a function). Only `import()` is dynamic.
- Imported bindings are *live*, *read-only* views — not copies (see §8.3).

- `export` names are statically analyzable — bundlers rely on this for tree-shaking. `export const x = cond ? 1 : 2` is analyzable; `module.exports[dynamicKey] = val` is not.

Live Bindings — The Semantic That Breaks Mental Models

ESM exports are *live bindings*, not snapshots. If the exporting module mutates an exported `let / var`, every importer sees the new value. This is specified in ECMA-262 §16.2 and implemented via indirect environment-record bindings, not object properties.

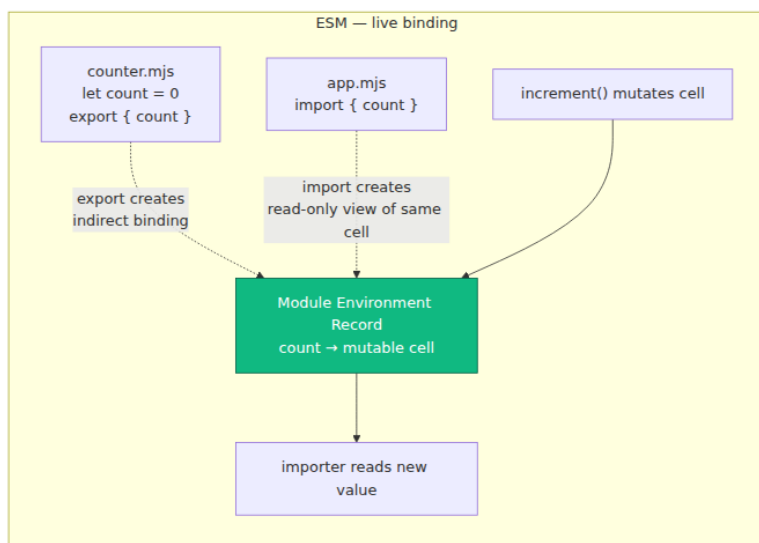
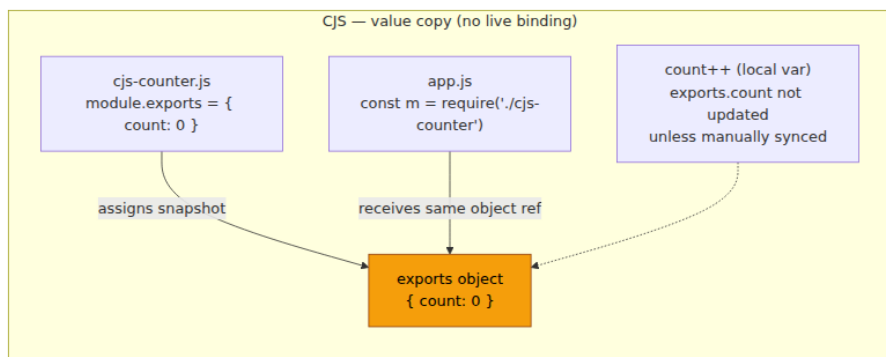
```
// counter.mjs – the exporter
export let count = 0;
export function increment() { count++; }
export const snapshot = count; // const – not live (but count itself
is)

// app.mjs – the importer
import { count, increment, snapshot } from "./counter.mjs";

console.log(count);    // 0
console.log(snapshot); // 0
increment();
console.log(count);    // 1 – live binding updated!
console.log(snapshot); // 0 – const binding, never changes
increment();
console.log(count);    // 2

// Importer cannot reassign – bindings are read-only views
// count = 99; // SyntaxError: Assignment to constant variable
// (imported binding is const-like)

// Contrast with CJS – no live bindings, just object property copies
// cjs-counter.js
let count = 0;
function increment() { count++; }
module.exports = { get count() { return count; }, increment, snapshot:
count };
// The getter makes it *look* live, but it's a manual pattern, not
language semantics
```



Live bindings also fix CJS's circular-dependency problem: ESM cycles link bindings before evaluation, so every module in the cycle sees live (possibly uninitialized, TDZ-checked) bindings rather than a half-populated exports object.

```

// ESM cycles – live bindings, TDZ instead of half-initialized objects
// a.mjs
import { bValue } from "./b.mjs";
export const aValue = 42;
console.log("a sees bValue:",
bValue); // TDZ if b hasn't evaluated yet – ReferenceError, not
undefined

// b.mjs
import { aValue } from "./a.mjs";
export const bValue = aValue + 1; // if a hasn't evaluated, aValue is
TDZ → throws

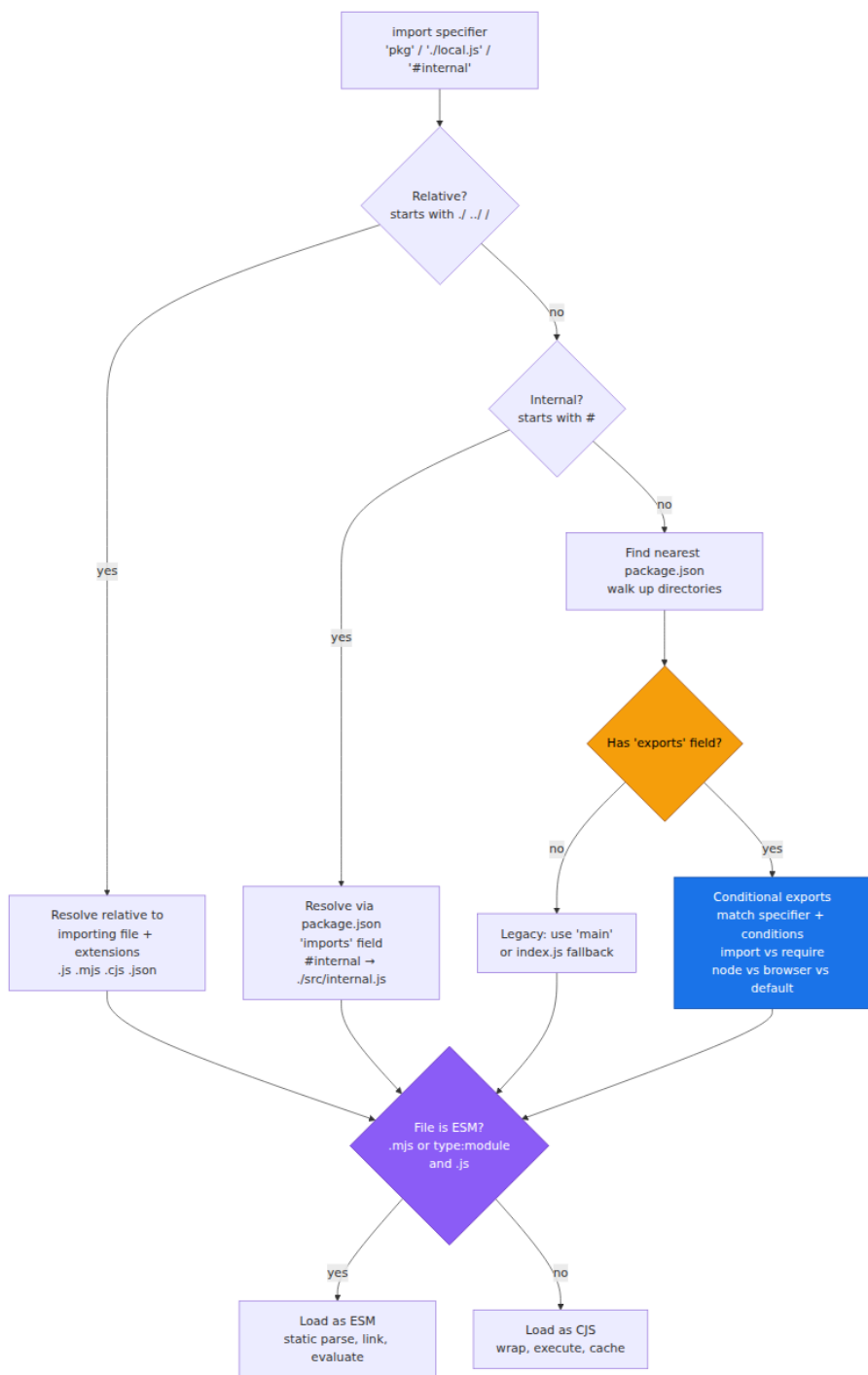
// ESM fails loudly (TDZ) rather than silently (undefined). This is
intentional –
// a TDZ error is diagnosable; a silent undefined propagates.

```

8.3 Module Resolution and `package.json` `exports`

Node's resolver has grown from a simple `node_modules` walk to a conditional-exports-aware algorithm. The `exports` field (Node 12.7+) is now the authoritative entry-point map; `main` is the legacy fallback.

The Resolution Flowchart



package.json exports — The Right Way

```
{
  "name": "@company/utils",
  "version": "2.4.0",
  "type": "module",
  "exports": {
    ".": {
      "types": "./dist/index.d.ts",
      "import": "./dist/index.js",
      "require": "./dist/index.cjs",
      "default": "./dist/index.js"
    },
    "./math": {
      "types": "./dist/math.d.ts",
      "import": "./dist/math.js",
      "require": "./dist/math.cjs"
    },
    "./package.json": "./package.json"
  },
  "imports": {
    "#internal/*": "./src/internal/*.js",
    "#config": "./src/config.js"
  }
}
```

Rules and pitfalls:

- When `exports` exists, **only** specifiers listed in `exports` are importable. `import "pkg/src/internal.js"` fails even if the file exists — the package is encapsulated. This is intentional: `exports` is the public API boundary.
- `import` vs `require` conditions let one package serve both module systems. The consumer's `import` gets the ESM file; `require` gets the CJS file. If you only provide `import`, CJS consumers get `ERR_REQUIRE_ESM`.
- `"type": "module"` makes `.js` files ESM. Without it, `.js` is CJS and ESM must use `.mjs`. Mixing without `"type"` is the most common misconfiguration.
- `imports` (`#`-prefixed) are *private* remappings — internal aliases that never leak to consumers. They replace ad-hoc `../../` relative paths and work in both ESM and CJS (Node 14.6+).
- Always expose `"./package.json"` if tooling needs to read the package version at runtime — otherwise `exports` blocks it.

```

// Consumer side – what resolves to what with the exports above
import utils from "@company/
utils";           // → dist/index.js (import condition)
import { add } from "@company/utils/math";  // → dist/math.js
const utilsCjs = require("@company/utils"); // → dist/index.cjs
(require condition)

// Blocked – not in exports, even though file exists on disk
import secret from "@company/utils/src/internal/secret.js";
// Error [ERR_PACKAGE_PATH_NOT_EXPORTED]: Package subpath './src/
internal/secret.js' is not defined by "exports"

// Internal import – only works inside the package
// src/app.js
import { helper } from "#internal/helpers.js"; // → ./src/internal/
helpers.js via imports field

```

8.4 CJS ↔ ESM Interop — Where It Breaks

Interop is the seam where most production incidents occur. The two systems have different timing, different binding semantics, and different ideas about what "default" means.

```

// — CJS consuming ESM —————
// ESM file: greet.mjs
export default function greet(name) { return `Hello, ${name}`; }
export const version = "2.0";

// CJS file: app.cjs — require(ESM) is NOT allowed in Node ≥12
try {
  const greet = require("./greet.mjs");
} catch (e) {
  console.log(e.code); // ERR_REQUIRE_ESM — require() of ES Module not
  // supported
  // Fix 1: use dynamic import() — async, returns Promise
  // Fix 2: rename consumer to .mjs and use static import
}

// Async fix — works but changes sync→async, may require refactoring
async function loadGreet() {
  const mod = await import("./greet.mjs");
  console.log(mod.default("Alice")); // "Hello, Alice" — default is
  // under .default
  console.log(mod.version); // "2.0"
}

// — ESM consuming CJS —————
// CJS file: legacy.cjs
module.exports = { add(a, b) { return a + b; } };
module.exports.extra = "bonus";

// ESM file: app.mjs
import legacy from "./legacy.cjs"; // default import gets
module.exports
console.log(legacy.add(2, 3)); // 5
console.log(legacy.extra); // "bonus"

import * as ns from "./legacy.cjs"; // namespace import
console.log(ns.default.add(2, 3)); // 5 — CJS exports appear
// under .default too
console.log(ns.add); // undefined — no named
// exports synthesized by default

// Named import from CJS — only works if Node synthesizes them (best-
// effort, not reliable)
import { add } from "./legacy.cjs"; // may work via cjs-module-
// lexer, may not
// Prefer: import pkg from "./legacy.cjs"; const { add } = pkg;

// The __esModule convention — how Babel/TypeScript/rollup signal "this
// CJS was ESM"
// Transpiled ESM→CJS by Babel:
// Object.defineProperty(exports, "__esModule", { value: true });

```

```
// exports.default = greet;
// exports.version = "2.0";
// ESM import of that CJS:
// import greet, { version } from "../transpiled.cjs";
// - Node/bundlers check __esModule to decide whether to treat
exports.default as default
```

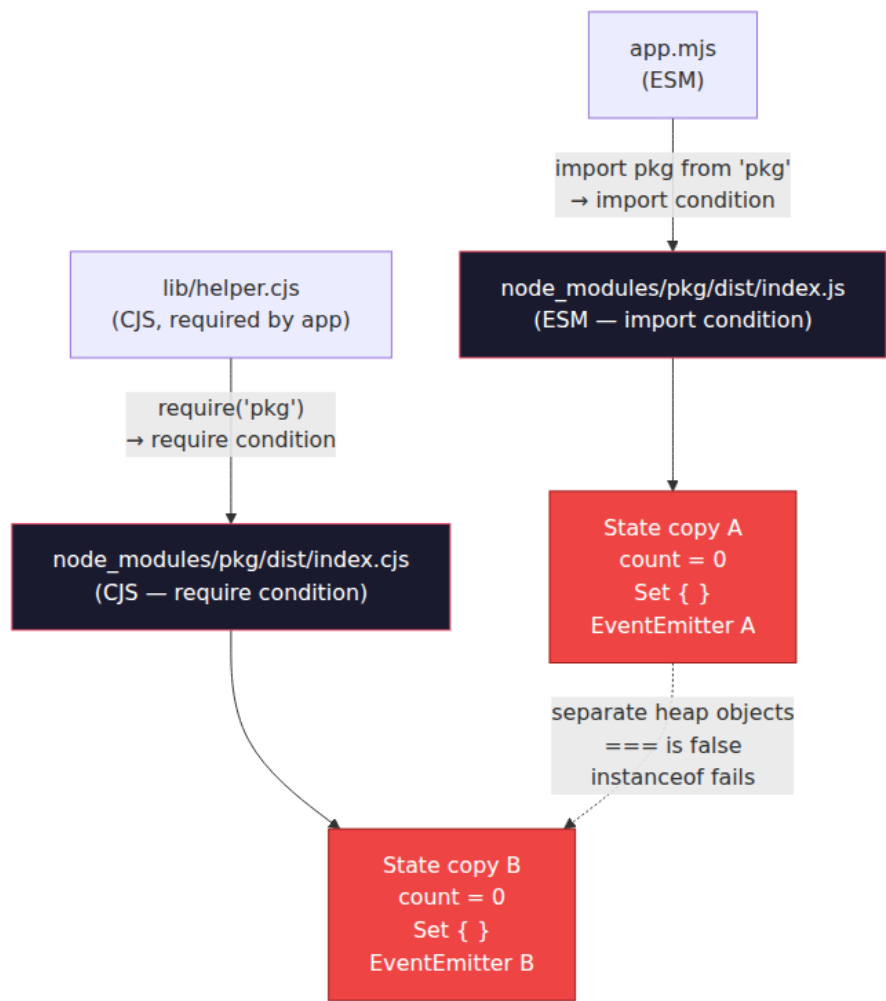
Interop pitfall table:

Direction	What works	What breaks	Fix
ESM → CJS (<code>import</code> CJS)	<code>import pkg from "../c.cjs" (default = module.exports)</code>	Named <code>import { foo }</code> — not reliably synthesized	Use default import + destructure
CJS → ESM (<code>require</code> ESM)	Nothing synchronous	<code>require("../e.mjs")</code> throws <code>ERR_REQUIRE_ESM</code>	<code>await import("../e.mjs")</code> (async)
Bundler (webpack/ RSPack)	May allow <code>require</code> of ESM via transpilation	Hides the runtime error; breaks when deployed unbundled	Align bundler + runtime module types
<code>__esModule</code>	Babel/TS CJS that was ESM: <code>exports.__esModule = true</code>	Node native ESM never sets <code>__esModule</code>	Check <code>__esModule</code> before treating <code>.default</code> as default

```
// Robust helper for consuming unknown CJS/ESM packages (e.g., in a
shared library)
async function robustImport(specifier) {
  const mod = await import(specifier);
  // If the package is CJS transpiled with __esModule, mod.default is
the real export
  // If it's native ESM, mod.default is the default export; named
exports are on mod
  // If it's native CJS without __esModule, mod.default is
module.exports
  const exported = mod.__esModule ? mod.default : mod.default ?? mod;
  return exported;
}
```

8.5 The Dual-Package Hazard

When a package provides both CJS and ESM entry points (via `exports` `import / require`), Node may load **two separate instances** of the same package — one per module system. They share no state.



Concrete failure:

```
// node_modules/counter-pkg/dist/index.js (ESM)
export let count = 0;
export function increment() { count++; }
export const emitter = new (await import("node:events")).EventEmitter();
;

// node_modules/counter-pkg/dist/index.cjs (CJS) – same source,
// separately evaluated
let count = 0;
function increment() { count++; }
const { EventEmitter } = require("node:events");
const emitter = new EventEmitter();
module.exports = { count, increment, emitter };

// app.mjs (ESM consumer)
import { increment, emitter } from "counter-pkg";
increment();
emitter.emit("tick"); // emits on ESM copy's emitter

// lib/worker.cjs (CJS, loaded by app.mjs via createRequire)
const { emitter: cjsEmitter } = require("counter-pkg");
cjsEmitter.on("tick", () => console.log("tick received"));
// Never fires – cjsEmitter is a different object on a different copy!
// instanceof also breaks:
import { MyError } from "counter-pkg";
const { MyError: CjsMyError } = require("counter-pkg");
console.log(new MyError("x") instanceof CjsMyError); // false –
// different constructor copies
```

Fixes, in order of preference:

1. **Single-source ESM with CJS wrapper** (not two independent builds). Ship ESM as the source of truth; generate CJS as a thin wrapper that re-exports the ESM evaluation (or use Rollup/RSPack to emit both from one build with shared chunks). Node's `require(esm)` proposal (Stage 3 as of 2025, `--experimental-require-module`) will eventually eliminate the hazard by loading the ESM file for both conditions.
2. **ESM-only package** (`"type": "module"`, no `require` condition). Cleanest, but a breaking change — CJS consumers must migrate to `import()`.
3. **State externalization**. If dual copies are unavoidable, keep singleton state out of the package (in `globalThis`, a shared peer dependency, or an external store) so both copies observe the same state.

At fleet scale, the dual-package hazard is a *version-skew* problem. A service with 400 transitive dependencies may transitively depend on the same package via both CJS and ESM paths. Deduplication (`npm dedupe` , `overrides`) does not fix it — the two copies are different *files*, not different *versions*. The fix is at the package authoring level, not the consumer level.

8.6 Import Maps and ESM Loaders

Import Maps — Bare-Specifier Resolution in the Browser (and Node)

Import maps let bare specifiers (`"react"` , `"@company/utils"`) resolve without a bundler or `node_modules` walk. Standardized in the HTML spec and supported in Chrome/Edge/Firefox and Node 18+ (experimental).

```
<!-- index.html -->
<script type="importmap">
{
  "imports": {
    "react": "https://cdn.jsdelivr.net/npm/react@18.3.1/+esm",
    "react-dom": "https://cdn.jsdelivr.net/npm/react-dom@18.3.1/+esm",
    "@company/utils": "/vendor/utils-v2.4.0/index.js",
    "@company/utils/": "/vendor/utils-v2.4.0/"
  },
  "scopes": {
    "/legacy/": {
      "react": "https://cdn.jsdelivr.net/npm/react@17.0.2/+esm"
    }
  }
}
</script>
<script type="module">
import React from "react"; // → CDN ESM, no bundler
import { add } from "@company/utils/math"; // → /vendor/utils-v2.4.0/
math.js (trailing slash maps subpaths)
// Inside /legacy/app.js, "react" resolves to 17.0.2 via scopes
</script>
```



```
// Node – import map via --experimental-vm-modules or --import flag
(Node 20+)
// package.json
{
  "imports": {
    "#utils": "./src/utils.js",
    "#config": {
      "node": "./src/config.node.js",
      "default": "./src/config.default.js"
    }
  }
}
```

```
// src/app.js – uses private import map, not relative paths
import { helper } from "#utils";
import config from "#config"; // → config.node.js on Node,
config.default.js elsewhere
```

Import maps are the browser's answer to `paths` in `tsconfig.json` and `resolve.alias` in bundlers — but they are runtime, not build-time, and they compose via scopes without rewriting source.

ESM Loaders and Hooks — Customizing Resolution and Loading

Node's loader hooks intercept `resolve` (specifier → URL) and `load` (URL → source) for ESM. They are the mechanism behind TypeScript-native execution (`tsx`, `ts-node/esm`), mocking (`quibble`), and policy enforcement.

```

// loader.mjs – custom ESM loader (Node 20+ --experimental-loader is
// deprecated; use --loader or --import)
// As of Node 20.6+, the API is `register` + hooks; older API used --
// experimental-loader.
// This example uses the current (Node 20.12+) `initialize` +
// `resolve`/`load` shape.

// my-loader.mjs
export async function resolve(specifier, context, nextResolve) {
  // Intercept a virtual specifier
  if (specifier === "env:config") {
    return {
      url: "env:config",
      shortCircuit: true
    };
  }
  // Rewrite bare specifiers for a monorepo
  if (specifier.startsWith("@company/")) {
    const subpath = specifier.slice("@company/".length);
    return nextResolve(`./packages/${subpath}/src/index.js`, context);
  }
  // Default – delegate to Node's resolver
  return nextResolve(specifier, context);
}

export async function load(url, context, nextLoad) {
  if (url === "env:config") {
    // Synthesize a module from environment variables – no file on disk
    const config = { port: process.env.PORT ?? 3000, env: process.env.NODE_ENV };
    return {
      format: "module",
      source: `export default ${JSON.stringify(config)};`,
      shortCircuit: true
    };
  }
  if (url.endsWith(".ts")) {
    // Transpile TypeScript on the fly (simplified – real loaders use
    // esbuild/SWC)
    const { source } = await nextLoad(url, context);
    const js = transpileTypeScript(String(source)); // your transpile
    function
    return { format: "module", source: js, shortCircuit: true };
  }
  return nextLoad(url, context);
}

```

```
# Usage – Node 20+
node --loader ./my-loader.mjs app.mjs
# Or via register (programmatic, no CLI flag)
# app.mjs
import { register } from "node:module";
register("./my-loader.mjs", import.meta.url);
import config from "env:config"; # ← resolved and loaded by the hook
above
```

Loader hook flow: `import 'env:config'` → `resolve` hook (synthetic URL with `shortCircuit: true` or delegate to `nextResolve`) → `load` hook (synthetic source with `shortCircuit: true` or delegate to `nextLoad`) → Module record (parsed, linked, ready to evaluate).

Operational notes for backend teams:

- Loaders run **once per process** and affect every ESM import. A buggy `resolve` hook that throws or returns a bad URL crashes the entire service at startup — not at the call site. Keep loaders minimal and tested in isolation.
- Loaders are **ESM-only**. They do not intercept `require`. If your service mixes CJS and ESM, the loader covers only half the graph.
- Node 22 stabilizes `module.register` as the replacement for `--loader`. Prefer `register` (programmatic, scoped) over CLI flags for library-level hooks; use CLI flags for process-wide concerns (TypeScript transpilation, policy).
- For policy enforcement (allowed-dependency checks, integrity verification), loaders are the right layer — they see every import before it executes. See Vol. 9, Chapter 5 for supply-chain policy that builds on this hook.

9. Putting It Together — Distributed-Systems Lens

At single-service scale, type coercion, prototypes, proxies, and modules are language details. At fleet scale, they are coordination problems.

Language mechanism	Single-service view	Fleet-scale reality
Coercion (<code>==</code> , <code>ToPrimitive</code>)	"Know your <code>==</code> table"	Validation libraries with coercion bugs propagate incorrect data to every downstream consumer; one <code>== null</code> vs <code>=== null</code> mismatch in a shared SDK changes null-handling fleet-wide
Prototype chain	"Know your <code>__proto__</code> "	Prototype pollution in one shared dependency (<code>lodash</code> , <code>qs</code> , <code>minimist</code>) becomes RCE across every service that parses untrusted JSON; null-prototype dictionaries are a security boundary
Proxy / membrane	"Metaprogramming"	Plugin isolation, config immutability, and API deprecation warnings at the platform level — the membrane is the abstraction for multi-tenant hosting (Cloudflare Workers, Figma plugins, VS Code extensions)
Dual-package hazard	"CJS vs ESM is annoying"	Singleton duplication causes split-brain: two metric registries, two connection pools, <code>instanceof</code> failures that break error handling in shared middleware — invisible until production

Three operational rules that follow from this chapter:

- 1. Validate at the boundary, not in the middle.** Coercion-safe code validates inputs once at the service edge (`Zod` , `ajv` , `z.coerce.number()`) and uses `===` everywhere else. Do not scatter `Number()` and `String()` calls through business logic — they are easy to forget and hard to audit.
- 2. Encapsulate packages with `exports`.** Every internal package should have an `exports` field that defines its public API. This prevents deep imports from coupling consumers to internal file

layout and makes the dual-package hazard visible (two files in `exports` means two evaluations — audit them).

3. **Treat module identity as infrastructure.** `require.cache` and ESM's module map are global mutable state. Hot-reload, test isolation (`jest.resetModules`, `vi.resetModules`), and serverless cold-start all interact with it. In integration tests, assert singleton identity explicitly: `expect(esmEmitter === cjsEmitter).toBe(true)` — or prove it is not needed.
-

10. Key Takeaways

- JavaScript has eight language types; `typeof null === "object"` is a historic bug, and `Array.isArray` / `Number.isNaN` / `Object.is` are the reliable predicates.
- Coercion is deterministic: `ToPrimitive` (with `Symbol.toPrimitive` first, then `valueOf` / `toString` by hint), then `ToNumber` / `ToString`, then the 12-clause Abstract Equality algorithm for `==`. `===` skips coercion; `Object.is` also fixes `NaN` and `-0`.
- Every object has a `[[Prototype]]` internal slot. `__proto__` is its legacy accessor; `prototype` is a property of constructor functions that becomes the instance's `[[Prototype]]` when called with `new`. Walks are read-delegating; writes always create own properties.
- `class` desugars to constructor functions + `prototype` assignment + `Object.setPrototypeOf` for the static chain. `extends` wires two links (`Sub.prototype.__proto__ → Super.prototype` and `Sub.__proto__ → Super`); `super` is a `[[HomeObject]]`-relative lookup, not a dynamic `this` lookup.
- Private fields (`#x`) are per-instance slots with brand checks, not `prototype` properties. They are encapsulation, not just naming convention.
- Well-known Symbols are the language's protocol hooks: `Symbol.iterator` for iteration, `Symbol.toStringTag` for `Object.prototype.toString`, `Symbol.hasInstance` for `instanceof`, `Symbol.species` for derived constructors, `Symbol.toPrimitive` for coercion.

- `Proxy` interposes on internal methods via traps; `Reflect` forwards correctly. Traps have invariants enforced by the engine — violations throw `TypeError`. `Proxy.revocable` enables capability revocation.
- A correct membrane transitively wraps every object crossing a boundary, preserves identity via `WeakMap`, and unwraps on write. Proxies are not transparent and carry significant performance cost — use at trust boundaries, not hot paths.
- CJS is dynamic (`require` is a function, `module.exports` is mutable, `require.cache` is global, cycles return half-initialized objects). ESM is static (`import` / `export` are syntax, bindings are live, cycles use TDZ, tree-shaking is possible).
- `package.json` `exports` is the authoritative entry-point map; `imports` (`#`-prefixed) are private aliases. `import` vs `require` conditions serve both module systems from one package — but two files means two evaluations.
- CJS→ESM: `require(ESM)` throws `ERR_REQUIRE_ESM` — use `await import()`. ESM→CJS: default import gets `module.exports`; named imports from CJS are best-effort. `__esModule` is a transpiler convention, not a runtime guarantee.
- The dual-package hazard (two copies, no shared state) breaks `instanceof` and singletons. Prefer single-source builds or ESM-only packages; externalize shared state if dual copies are unavoidable.
- Import maps resolve bare specifiers at runtime without bundlers; ESM loader hooks (`resolve` / `load` via `module.register`) intercept resolution and loading for transpilation, mocking, and policy enforcement.

11. Further Reading

- **ECMA-262, 15th Edition (2024)** — <https://tc39.es/ecma262/> — §7 (Abstract Operations including `ToPrimitive`, `ToString`, `Abstract Equality`), §9.5 (Proxy invariants), §15.7 (Classes), §16.2 (Modules, live bindings).
- **V8 Design Docs** — Hidden classes, inline caches, and proxy performance: <https://v8.dev/docs> and `src/objects/map.h`, `src/objects/js-proxy.h` in the V8 source.

- **Node.js Modules Documentation** — CJS and ESM, `exports` / `imports`, loader hooks: <https://nodejs.org/api/modules.html>, <https://nodejs.org/api/packages.html>, <https://nodejs.org/api/esm.html>.
- **Node.js ESM Loader Hooks (Stabilized)** — `module.register` and `resolve` / `load` hooks: <https://nodejs.org/api/module.html#customization-hooks>.
- **Import Maps Standard** — HTML Standard, import maps: <https://html.spec.whatwg.org/#import-maps> and <https://github.com/WICG/import-maps>.
- **TC39 Proposals** — `require(ESM)` / `require(esm)` (Stage 3, 2024–2025): <https://github.com/nodejs/loaders/issues> and the Node ESM/CJS interop tracker.
- **SES / Hardened JavaScript (Agoric)** — Membrane and compartment patterns for secure composition: <https://github.com/endojs/endo> and Miller, "Robust Composition" (PhD thesis, 2006).
- **Surma** — "The Dual-Package Hazard" — Concise explanation of the hazard and mitigations (applies equally to Node and Deno).
- **Gil Tayar** — "Dual Packages: ESM and CJS" — Practical guidance on authoring dual packages without duplication: <https://giltayar.com/>.
- **Dr. Axel Rauschmayer** — "JavaScript for Impatient Programmers" / "Deep JavaScript" — Accurate, spec-grounded coverage of types, prototypes, and modules: <https://exploringjs.com/>.

Chapter 5 — Async JavaScript: Promises, `async/await`, Generators, and the Promise Job Queue

What this chapter covers: Every I/O operation in JavaScript — `fetch`, `fs.readFile`, database queries, RPC calls — returns a `Promise` that resolves through a microtask queue before any timer or I/O callback runs. This chapter dissects that mechanism end to end: the three-state Promise machine and its one-way transitions, how the executor and `.then` register reactions that become `PromiseJobs`, how those jobs drain in the microtask queue, how `async / await` desugars to generators wrapped in Promises, how errors propagate and go unhandled, how generators

suspend and resume, and how async iteration and top-level await compose them into streaming and module-graph primitives. You will build a Promise visualizer, step through an `async` → generator transform by hand, watch the microtask queue drain under instrumentation, and wire an `unhandledrejection` handler that would actually ship in production.

Learning goals:

- Draw the Promise state machine (pending → fulfilled / rejected), explain why transitions are one-way and one-time, and predict the effect of calling `resolve` / `reject` multiple times or returning a thenable.
- Trace the executor → reaction-queue path: how `new Promise(executor)` captures `resolve` / `reject`, how `.then(onFulfilled, onRejected)` creates a new Promise and enqueues PromiseReaction records, and how the spec's `PromiseResolveThenableJob` assimilates foreign thenables.
- Explain the Promise Job Queue as a microtask queue: when PromiseJobs run relative to tasks, `queueMicrotask`, `process.nextTick`, and rendering; why `await` always yields at least one microtask; and how recursive microtask enqueueing can starve the event loop.
- Desugar any `async` / `await` function into an equivalent generator + Promise state machine, and explain what the engine actually optimizes away.
- Follow error propagation through `.then` chains, `async` functions, `try` / `catch` around `await`, `unhandledrejection` / `rejectionhandled` events, `AggregateError`, and the four combinators (`all`, `allSettled`, `race`, `any`).
- Operate generators: `function*`, `yield` / `yield*`, `next(value)`, `return(value)`, `throw(err)` — and explain the bidirectional value channel and the suspended-start / suspended-yield / completed states.
- Use async generators and `for await...of` to consume asynchronous iterables (streams, paginated APIs, message consumers) with backpressure.
- Explain top-level `await` in ESM: how it blocks module-graph evaluation, what it does to the instance lifecycle, and when to use it versus an exported promise or an `async` IIFE.

- Apply all of the above to backend concerns: timeout-wrapped fetches, retry with jitter, concurrency-limited fan-out, graceful shutdown while awaiting, and observability for slow or stuck promise chains.
-

1. Why a Backend Engineer Should Care About Promise Internals

In Go you write `resp, err := client.Do(req)` and the goroutine parks. In Java you call `future.get()` and the thread blocks. In JavaScript there is no blocking. Every I/O — `fetch('http://payments:8080/charge')`, `pg.query('SELECT ...')`, `redis.get('session:...')` — returns immediately with a placeholder. The placeholder is a `Promise`. Your request handler continues, the event loop polls for completions, and continuations run later as microtasks.

At fleet scale this has three consequences that senior backend engineers feel directly:

1. **Ordering is a contract, not an accident.** Promise continuations are microtasks. Microtasks drain exhaustively before the next timer, I/O poll, or render. If you attach `.then` handlers in a hot loop without yielding, you starve I/O. If you wrap a cache lookup in `Promise.resolve().then(...)` you add a guaranteed microtask hop — observable as an extra tick of latency per hop in a chain of middleware.
2. **Errors do not throw where you expect.** A `throw` inside a Promise executor or an `async` function does not unwind the caller. It rejects a Promise. If nobody attaches a rejection handler before the microtask checkpoint, the runtime fires `unhandledrejection`. In Node, a future flag (`--unhandled-rejections=strict`) will crash the process. Silent unhandled rejections in an API gateway become silent dropped charges.
3. **Composition is concurrency control.** `Promise.all`, `Promise.allSettled`, `Promise.race`, and `Promise.any` are not convenience aliases. They are your fan-out, scatter-gather, hedged-request, and quorum primitives — the same patterns from Volume 6 (quorums) and Volume 3 (hedging), implemented inside the language. Understanding when `all` fails fast versus when

`allSettled` preserves partial results determines whether a single slow shard fails your entire aggregation.

This chapter treats `async` not as syntax trivia but as a scheduling and error-propagation substrate — the same class of concern as goroutine scheduling, future combinators, or Tokio tasks in other volumes.

Relationship to Chapter 2. Chapter 2 explained the event loop's task vs. microtask queues and the libuv/browser divergence. This chapter zooms into one occupant of the microtask queue — `PromiseJobs` — and into the language constructs (`Promise`, `generator`, `async/await`) that enqueue them. Read them together: Chapter 2 tells you *when* microtasks drain; this chapter tells you *what* `Promise` enqueues and *why*.

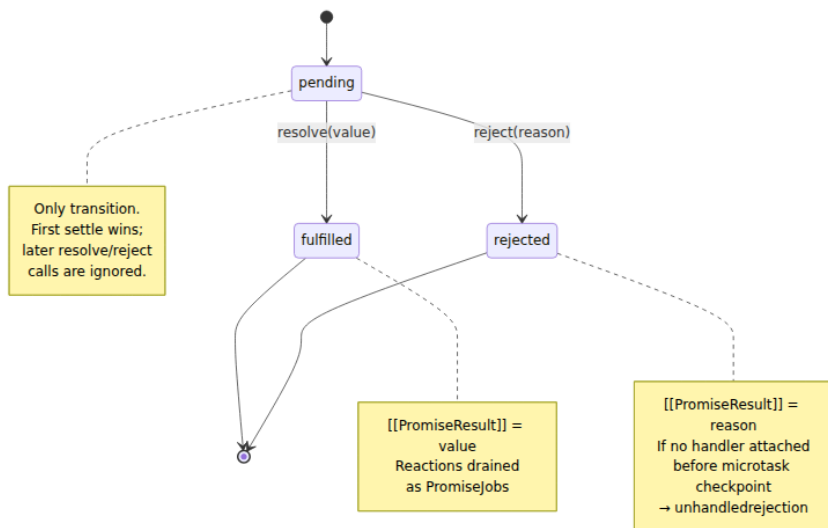
2. The Promise State Machine — Pending, Fulfilled, Rejected

A `Promise` as defined in ECMA-262 [§27.2](#) is an object with three internal slots that matter and one state that never goes backward:

Internal slot	Purpose
<code>[[PromiseState]]</code>	<code>"pending"</code> <code>"fulfilled"</code> <code>"rejected"</code> — the only observable state, via <code>.then</code> and <code>await</code>
<code>[[PromiseResult]]</code>	The fulfillment value or rejection reason
<code>[[PromiseFulfillReactions]]</code> / <code>[[PromiseRejectReactions]]</code>	Queues of <code>PromiseReaction</code> records registered by <code>.then</code> / <code>.catch</code> / <code>.finally</code> / <code>await</code>

A fourth slot, `[[PromiseIsHandled]]`, tracks whether a rejection has ever had a handler — the basis for `unhandledrejection` detection.

2.1 The one-way, one-time transition



Two invariants make Promises reliable as a coordination primitive:

- **Immutability after settlement.** Once `[[PromiseState]]` leaves "pending", it never changes. Calling `resolve` twice, calling both `resolve` and `reject`, or throwing after resolving — all are no-ops. This matches the distributed-systems intuition of a single-assignment future: exactly once settled.
- **Observation is always async.** Even if a Promise is already fulfilled, `.then(handler)` never calls `handler` synchronously. It enqueues a `PromiseJob`. This eliminates the `Zalgo` problem (sometimes sync, sometimes async) that plagued callback APIs.

```

// Invariant 1 – first settle wins, forever
let resolveOuter;
const p = new Promise((resolve) => { resolveOuter = resolve; });

p.then(v => console.log("fulfilled with", v),
      e => console.log("rejected with", e));

resolveOuter("first");
resolveOuter("second"); // ignored
// Logs: fulfilled with first

// Even throwing after resolve is ignored
const p2 = new Promise((resolve) => {
  resolve("ok");
  throw new Error("too late"); // swallowed – promise already fulfilled
});
p2.then(v => console.log(v)); // "ok"

// Invariant 2 – always async, even when already settled
const alreadyFulfilled = Promise.resolve(42);
alreadyFulfilled.then(v => console.log("microtask:", v));
console.log("sync");
// Logs:
//   sync
//   microtask: 42

```

That second guarantee is load-bearing for middleware chains. An auth middleware that does `return Promise.resolve(cachedUser).then(attachToReq)` always yields at least one microtask, so downstream middleware ordering is deterministic regardless of cache hit or miss.

2.2 The executor — where the state transition originates

```

const p = new Promise((resolve, reject) => {
  // executor runs synchronously, during construction
  console.log("executor runs now");
  resolve(42); // moves pending → fulfilled, stores 42
  // Any handlers already registered are enqueued as PromiseJobs
  // Handlers registered later (after this tick) are also enqueued
});
console.log("after construction");
// Logs:
//   executor runs now
//   after construction
//   (then handlers fire in a later microtask)

```

The executor receives two functions, `resolve` and `reject`, that are capability-bound to this specific Promise instance. Calling `resolve(x)` performs the spec operation `ResolvePromise(p, x)`:

- If `x` is a plain value $\rightarrow$ fulfill `p` with `x`.
- If `x` is a thenable (any object with a callable `.then`) $\rightarrow$ assimilate it: adopt its eventual state. This is how `resolve(fetch(...))` unwraps one level.
- If `x === p` $\rightarrow$ reject with `TypeError` (self-resolution).

```
// Thenable assimilation – resolve unwraps one level
const thenable = {
  then(onFulfilled) { onFulfilled("from thenable"); }
};
Promise.resolve(thenable).then(v => console.log(v)); // "from thenable"

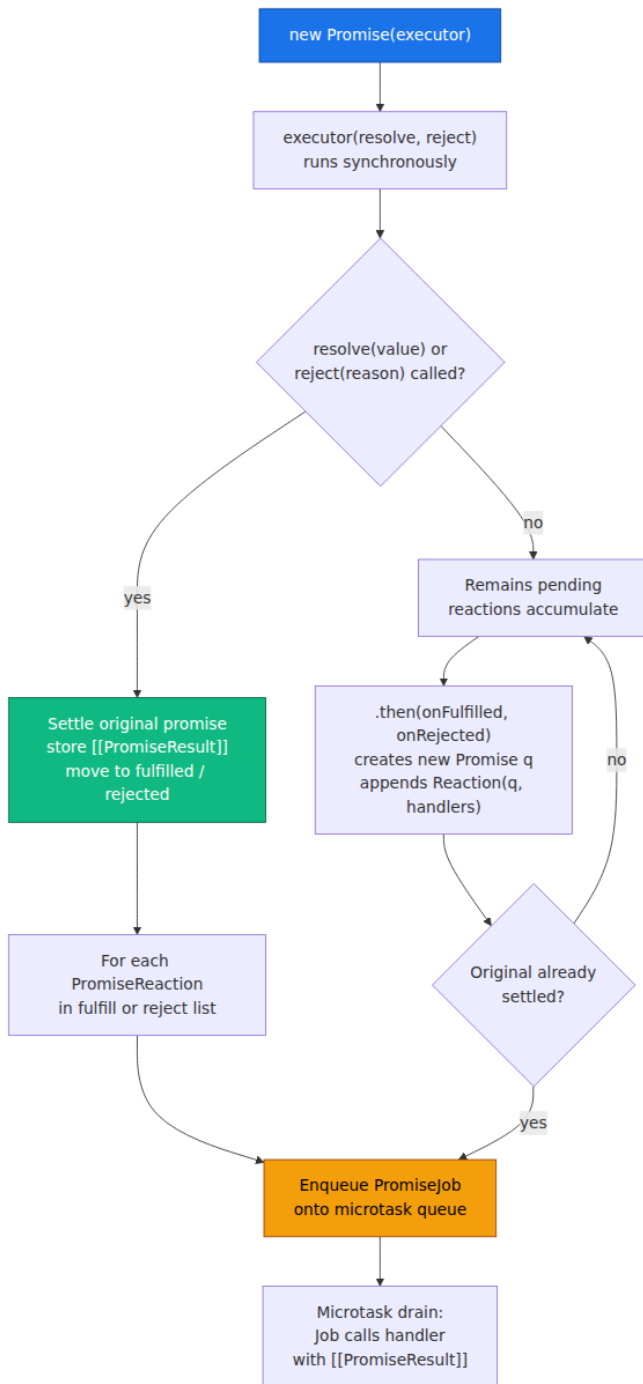
// Nested promise unwrapping
const inner = Promise.resolve("inner");
const outer = new Promise(resolve => resolve(inner));
outer.then(v => console.log(v)); // "inner" – outer adopted inner's
state

// Self-resolution is a TypeError
let self;
self = new Promise((resolve) => {
  // Defer to catch the reference – direct self-resolution inside
  // executor
  // would reference an uninitialized binding
});
self.then(() => {}); // placeholder
// Programmatic self-resolution via resolve(self) inside a handler:
const base = Promise.resolve(1);
const cycle = base.then(() => cycle); // handler returns its own
promise
cycle.catch(e => console.log(e instanceof TypeError)); // true
```

Backend analogue. Thenable assimilation is the language-level equivalent of future-flattening in RPC frameworks. Just as a gRPC stub that returns `ListenableFuture<ListenableFuture<T>>` is almost always a bug, a `Promise<Promise<T>>` automatically flattens to `Promise<T>` via assimilation. There is no nested promise at runtime.

3. Executor → Reaction Queue — How `.then` Really Works

Calling `.then` does not execute anything immediately. It creates a *new* Promise and registers a *PromiseReaction* that will enqueue a *PromiseJob* when the original settles.



3.1 The PromiseReaction record

Each `.then(onFulfilled, onRejected)` call allocates:

```
PromiseReaction {
  [[Capabilities]]: { [[Promise]]: newPromise, [[Resolve]]: ..., [[Reject]]: ... }
  [[Type]]: "Fulfill" | "Reject"
  [[Handler]]: onFulfilled | onRejected (or Identity / Thrower if missing)
}
```

- If the original fulfills, the Fulfill reactions run; if it rejects, the Reject reactions run.
- If the corresponding handler is not callable (`p.then(null, null)`), the spec substitutes `Identity` (pass-through) or `Thrower` (re-throw), which is why `.then()` without arguments propagates the settlement unchanged.
- The chained promise (`newPromise`) is what the caller holds. Its fate is decided by what the handler returns or throws.

```
// Chaining – each .then returns a NEW promise
const p1 = Promise.resolve(10);
const p2 = p1.then(v => v * 2); // p2 fulfills with 20
const p3 = p2.then(v => Promise.resolve(v + 5)); // p3 fulfills with 25 (assimilated)
const p4 = p3.then(() => { throw new Error("oops"); }); // p4 rejects

p4.catch(e => console.log(e.message)); // "oops"

// Pass-through when handler is missing
const p5 = Promise.reject(new Error("fail"));
const p6 = p5.then(null); // no onRejected → Thrower → p6 also rejects with "fail"
p6.catch(e => console.log("propagated:", e.message)); // propagated: fail

const p7 = Promise.resolve(99);
const p8 = p7.then(); // no onFulfilled → Identity → p8 fulfills with 99
p8.then(v => console.log(v)); // 99

// .catch and .finally are sugar over .then
// p.catch(onRejected)  = p.then(null, onRejected)
// p.finally(onFinally) = p.then(v => { onFinally(); return v; },
//                               e => { onFinally(); throw e; })
```


3.2 Promise combinators — fan-out primitives

```
// Promise.all – fail-fast scatter-gather (like a quorum requiring all
// replicas)
const shardResults = await Promise.all([
  fetchShard("shard-a"),
  fetchShard("shard-b"),
  fetchShard("shard-c"),
]);
// If ANY shard rejects, the whole all() rejects immediately.
// Use when partial results are useless.

// Promise.allSettled – always waits (like a quorum that reports per-
// replica status)
const settled = await Promise.allSettled([
  fetchShard("shard-a"),
  fetchShard("shard-b"),
  fetchShard("shard-c"),
]);
// settled = [
//   { status: "fulfilled", value: ... },
//   { status: "rejected", reason: ... },
//   { status: "fulfilled", value: ... },
// ]
// Use for best-effort aggregation, bulk writes, fan-out notifications.

// Promise.race – first to settle wins (hedged requests, timeouts)
const withTimeout = Promise.race([
  fetch("http://payments:8080/charge"),
  new Promise( (_, reject) =>
    setTimeout(() => reject(new Error("timeout after 2s")), 2000)
  ),
]);

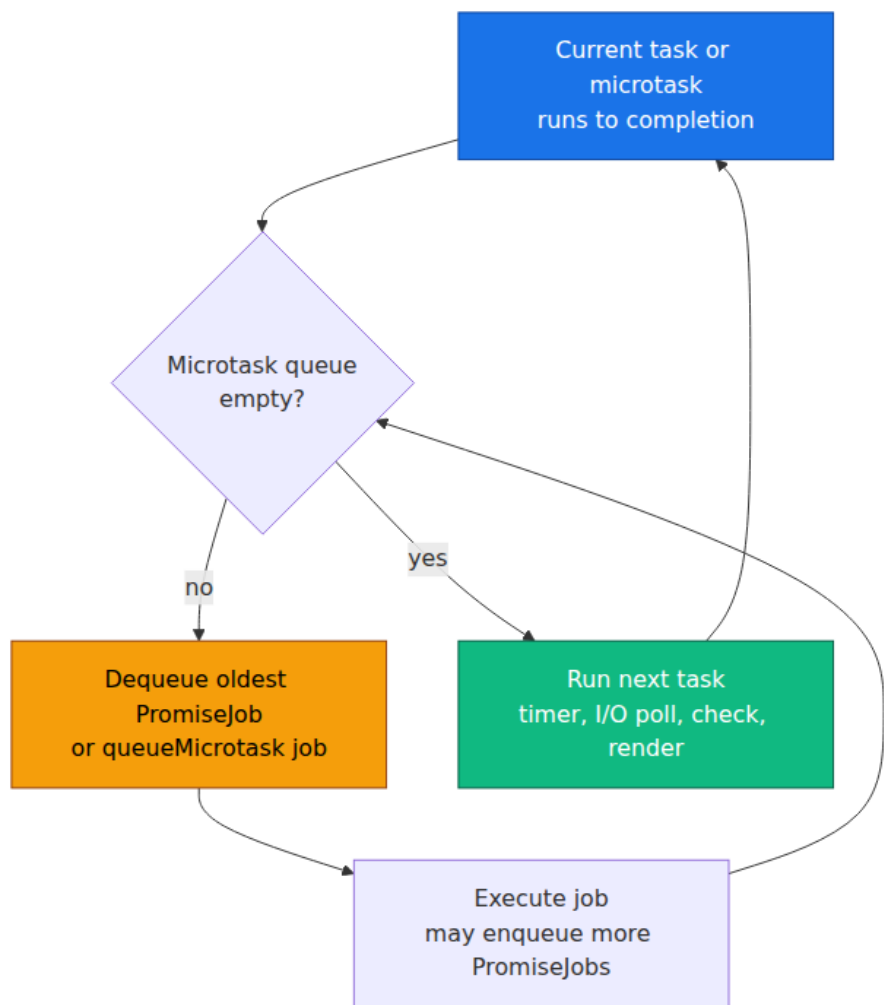
// Promise.any – first to FULFILL wins, ignores rejections until all
// reject
// Useful for redundant providers: try 3 payment gateways, use first
// success
try {
  const result = await Promise.any([
    chargeViaStripe(),
    chargeViaAdyen(),
    chargeViaBraintree(),
  ]);
} catch (e) {
  // e is AggregateError – all three rejected
  console.log(e.errors.map(err => err.message));
}
```

Combinator	Waits for	Rejects when	Returns
<code>all</code>	All fulfilled	Any single rejection (fail-fast)	Array of values, order = input order
<code>allSettled</code>	All settled	Never rejects	Array of <code>{status, value reason}</code>
<code>race</code>	First settled (fulfill or reject)	First rejection if it settles first	Single value/reason
<code>any</code>	First fulfilled	All rejected → <code>AggregateError</code>	Single value

Distributed-systems lens. Choose `all` for strong-consistency fan-out (all shards must answer). Choose `allSettled` for availability-oriented fan-out (degraded answers are better than failure). Choose `race` for hedged requests with a timeout future. Choose `any` for redundant-provider failover. The same trade-off space as quorums and hedged reads in Volume 6 appears here inside a single process.

4. The Promise Job Queue — Microtask Drain Mechanics

ECMA-262 does not say "microtask queue" directly. It says *PromiseJobs* are enqueued via *HostEnqueuePromiseJob* and drained by *PerformMicrotaskCheckpoint*. In browsers this is the HTML Standard microtask queue; in Node it is the `nextTick` + promise microtask queue drained between libuv phases (see Chapter 2, §2.4). The observable behavior is identical: **PromiseJobs run before the next task, and they drain exhaustively — a PromiseJob that enqueues another PromiseJob causes that new job to run in the same drain.**



4.1 Microtask drain demo — the log order that proves the queue

```
// microtask-drain-demo.js — run with `node microtask-drain-demo.js`
console.log("1 — script start");

setTimeout(() => console.log("2 — setTimeout (task)", 0));

Promise.resolve().then(() => {
  console.log("3 — promise.then #1");
  // Enqueuing inside a microtask — runs in SAME drain, before any task
  Promise.resolve().then(() => console.log("4 — promise.then #2
(nested)"));
});

queueMicrotask(() => console.log("5 — queueMicrotask"));

Promise.resolve().then(() => console.log("6 — promise.then #3"));

console.log("7 — script end");

// Output — guaranteed order in every compliant engine:
// 1 — script start
// 7 — script end
// 3 — promise.then #1
// 5 — queueMicrotask
// 6 — promise.then #3
// 4 — promise.then #2 (nested)    ← enqueued during drain, still
before task
// 2 — setTimeout (task)          ← only after queue fully empty

// Why #4 before #2: the checkpoint after #3 enqueued #4 before the
queue
// was considered empty. Tasks never interleave inside a drain.

// Starvation hazard — uncomment to freeze the process:
// function starve() {
//   Promise.resolve().then(starve); // infinite microtask loop
// }
// starve();
// setTimeout(() => console.log("never reached"), 0);
// Node will never reach the setTimeout; the browser will never render.
```

Key observations for backend work:

- **await always enqueues at least one PromiseJob.** Even `await 42` (a non-Promise) wraps `42` in `Promise.resolve(42)` and suspends. The continuation resumes in a later microtask. Two consecutive `await` statements cost two microtask hops.

- `queueMicrotask` and `Promise.then` share the same queue. Insertion order determines execution order. There is no priority between them.
- Node's `process.nextTick` drains *before* `PromiseJobs`. A recursive `nextTick` loop starves `PromiseJobs`; a recursive `PromiseJob` loop starves `setImmediate` and I/O. Never recurse unboundedly on either.

```
// await always yields – even for non-promises
async function demo() {
  console.log("a – before await");
  await 42; // ≡ await Promise.resolve(42) – suspends, resumes as
microtask
  console.log("c – after await");
}
console.log("b – before calling demo");
demo();
console.log("d – after calling demo");
// Order: b, a, d, c
// 'c' is deferred because await enqueued a PromiseJob for the
continuation

// Two awaits = two hops
async function twoHops() {
  await Promise.resolve(1); // hop 1
  await Promise.resolve(2); // hop 2 – separate microtask after first
resumes
  console.log("done after two hops");
}
```

4.2 Observing the drain with instrumentation

```
// microtask-instrumentation.js
const { performance } = require("node:perf_hooks");

function instrumentedThen(label, promise) {
  const start = performance.now();
  return promise.then(
    (v) => {
      console.log(`[${label}] fulfilled after ${(performance.now() - start).toFixed(2)}ms - value:`, v);
      return v;
    },
    (e) => {
      console.log(`[${label}] rejected after ${(performance.now() - start).toFixed(2)}ms - reason:`, e.message);
      throw e;
    }
  );
}

// Measure microtask ordering vs tasks
console.log("--- instrumented drain ---");
performance.mark("start");

instrumentedThen("A", Promise.resolve("a"));
instrumentedThen("B", Promise.resolve("b"));

queueMicrotask(() => {
  performance.mark("microtask");
  console.log("queueMicrotask fired");
});

setTimeout(() => {
  performance.mark("timeout");
  performance.measure("microtasks → timeout", "microtask", "timeout");
  console.log("setTimeout fired - measure:", performance.getEntriesByName("microtasks → timeout")[0]?.duration.toFixed(2), "ms");
}, 0);

// Expected: A, B, queueMicrotask all fire before setTimeout
// The gap between microtask and timeout is the task-queue wait
```

Operational signal. In production, event-loop lag (Chapter 2, §2.8) is dominated by two things: long synchronous tasks and deep microtask drains. A chain of 1,000 `.then` handlers enqueued synchronously does not create 1,000 tasks — it creates 1,000 microtasks that drain as one uninterrupted block. Under load, this manifests as p99 latency spikes

with no corresponding CPU spike, because the loop is busy but not doing I/O. Instrument `perf_hooks.monitorEventLoopDelay()` to catch it.

5. Error Propagation — Rejection, Unhandled Rejections, and `AggregateError`

5.1 How errors travel through a Promise chain

Every `.then` handler is wrapped so that a thrown exception becomes a rejection of the chained promise. This is the sole error-propagation mechanism for asynchronous code — there is no cross-microtask stack unwind.




```

// Error bubbling through a realistic chain
function fetchUser(id) {
  if (id < 0) return Promise.reject(new Error("invalid id"));
  return Promise.resolve({ id, name: "Ada" });
}

function fetchOrders(user) {
  if (user.id === 0) throw new Error("no orders for guest");
  return Promise.resolve([{ id: 101, total: 42 }]);
}

fetchUser(0) // fulfills with { id: 0 }
  .then(user => fetchOrders(user)) // throws synchronously inside
  handler
  .then(orders => console.log("orders:",
orders)) // skipped - previous rejected
  .catch(err => {
    console.log("caught:", err.message); // "no orders for guest"
    return []; // recovery - next promise
  }) // fulfills with []
  .then(orders => console.log("recovered orders:", orders)); // logs []

// Equivalent with async/await - errors become thrown exceptions
async function getOrders(id) {
  try {
    const user = await fetchUser(id); // if fetchUser rejects,
    // throws here
    const orders = await fetchOrders(user); // if fetchOrders throws/
    // rejects, throws here
    return orders;
  } catch (err) {
    console.log("caught in async:", err.message);
    return []; // recovery
  }
}

```

5.2 Unhandled rejections — the silent failure mode

A rejection is *handled* if, at any point before the microtask checkpoint after the rejection, some `.then` / `.catch` / `await` has registered a rejection handler on that specific Promise object. If not, the host fires `unhandledrejection`.

```

// unhandledrejection – the handler that belongs in every service
entrypoint

// Browser
if (typeof window !== "undefined") {
  window.addEventListener("unhandledrejection", (event) => {
    console.error("[unhandledrejection]", event.reason);
    // Report to observability pipeline
    // In production: send to Sentry / Datadog / OTel
    event.preventDefault(); // suppress console warning if you handled
it
  });
  window.addEventListener("rejectionhandled", (event) => {
    console.warn("[rejectionhandled late]", event.reason);
    // Fired when a rejection that was previously unhandled gets a
handler later
  });
}

// Node.js – place this near the top of your entrypoint, before any
imports that may reject
process.on("unhandledRejection", (reason, promise) => {
  console.error("[unhandledRejection]", reason);
  // Structured log for alerting – include promise identity for
correlation
  // logger.error({ reason, promiseId: promise[Symbol.toStringTag] },
"unhandled rejection");

  // In Node 15+, unhandled rejections throw and crash by default (--
unhandled-rejections=throw).
  // Explicit handling here prevents crash, but audit why it was
unhandled at all.
});

process.on("rejectionHandled", (promise) => {
  console.warn("[rejectionHandled] – handler attached after
unhandledRejection fired");
});

// Demo: unhandled vs handled timing
const p1 = Promise.reject(new Error("no handler at all"));
// → fires unhandledRejection at next microtask checkpoint

const p2 = Promise.reject(new Error("late handler"));
// Microtask checkpoint fires unhandledRejection for p2...
setTimeout(() => {
  p2.catch(e => console.log("late catch:", e.message));
  // → fires rejectionHandled (handler attached after the fact)
}, 10);

```

```
const p3 = Promise.reject(new Error("immediate handler"));
// Handled synchronously before checkpoint – no event fires
p3.catch(e => console.log("immediate catch:", e.message));
```

Common production pitfalls:

Pattern	Why it is unhandled	Fix
<code>fetch(url);</code> with no <code>.catch</code> or <code>await</code>	Fire-and-forget — rejection has no handler	Always <code>await</code> or attach <code>.catch</code> ; lint with <code>no-floating-promises</code> (<code>@typescript-eslint</code>)
<code>promise.then(onFulfilled)</code> with no second arg and no downstream <code>.catch</code>	Rejection propagates to chained promise, which also has no handler	End every chain with <code>.catch</code> , or <code>await</code> inside <code>try / catch</code>
<code>Promise.all([...])</code> where one rejects before you attach <code>.catch</code>	<code>all</code> rejects immediately; if you <code>await</code> it without <code>try</code> , caller gets unhandled	<code>await</code> inside <code>try / catch</code> or use <code>allSettled</code>
<code>async</code> event handler that throws	The returned rejected promise is ignored by the event emitter	Wrap handler body in <code>try / catch</code> and log, or return the promise to a central handler

```

// The no-floating-promises fix – every promise must be handled
// .eslintrc: "@typescript-eslint/no-floating-promises": "error"

// Bad – lint error, potential unhandled rejection
app.post("/charge", (req, res) => {
  chargeCard(req.body); // returns Promise – floating!
  res.sendStatus(202);
});

// Good – await or explicitly handle
app.post("/charge", async (req, res, next) => {
  try {
    await chargeCard(req.body);
    res.sendStatus(200);
  } catch (err) {
    next(err); // Express error handler – rejection is handled
  }
});

// Also good – explicit void with catch when fire-and-forget is intentional
app.post("/webhook", (req, res) => {
  void processWebhookAsync(req.body).catch(err => {
    logger.error({ err }, "webhook processing failed");
  });
  res.sendStatus(202);
});

```

5.3 AggregateError and the any/allSettled error model

`Promise.any` is the only combinator that aggregates multiple errors into one:

```

// AggregateError – the error type for "all providers failed"
try {
  await Promise.any([
    Promise.reject(new Error("stripe: rate limited")),
    Promise.reject(new Error("adyen: timeout")),
    Promise.reject(new Error("braintree: invalid card")),
  ]);
} catch (e) {
  console.log(e instanceof AggregateError); // true
  console.log(e.message);                  // "All promises were
rejected"
  console.log(e.errors.length);            // 3
  for (const err of e.errors) {
    console.log(" -", err.message);
  }
  // Use e.errors for per-provider diagnostics, retries, or alerting
}

// Constructing your own AggregateError for batch validation
function validateBatch(records) {
  const errors = [];
  for (const r of records) {
    try { validateRecord(r); } catch (e) { errors.push(e); }
  }
  if (errors.length > 0) {
    throw new AggregateError(errors, `batch validation failed: $
{errors.length} errors`);
  }
}

// allSettled – no AggregateError, but you handle per-result status
const results = await Promise.allSettled(jobs.map(j => runJob(j)));
const failures = results.filter(r => r.status === "rejected");
if (failures.length > 0) {
  logger.warn({ failures: failures.map(f => f.reason.message) }, "parti
al batch failure");
  // Decide: retry failures, return partial, or escalate
}

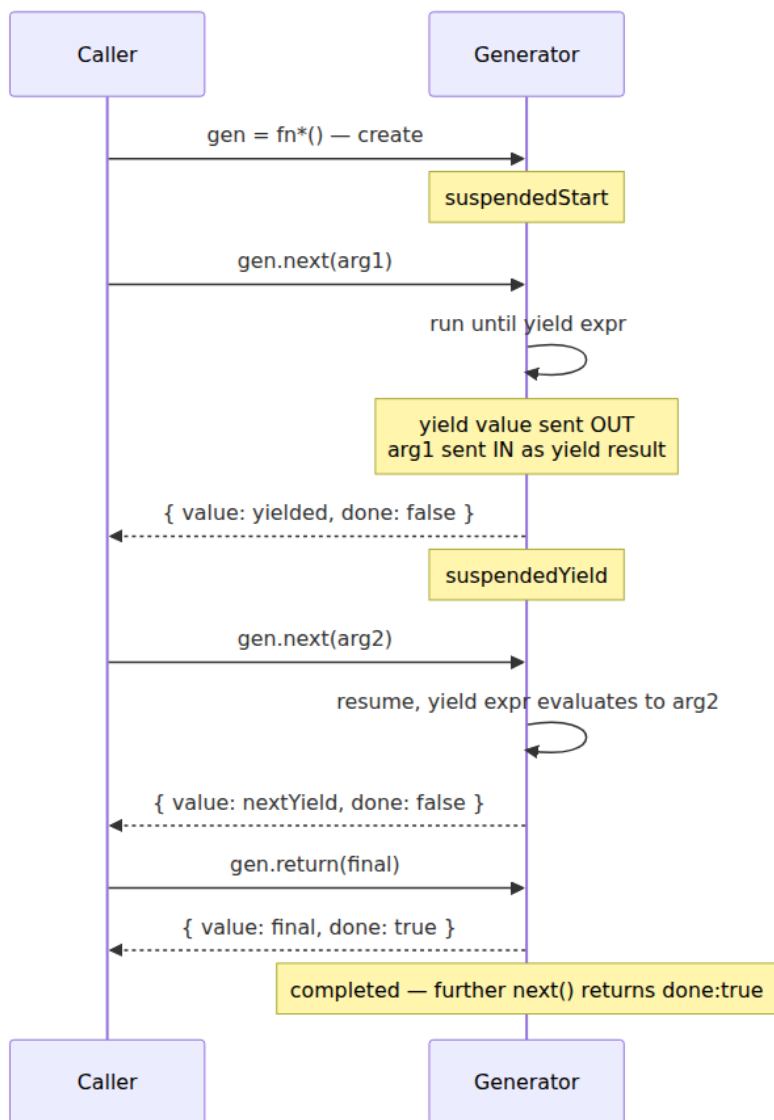
```

6. Generators — Suspendable Functions with a Bidirectional Channel

Generators are the suspension primitive that `async / await` is built on. A generator function (`function*`) does not execute when called. It returns a *Generator* object — an iterator with a state machine — whose body

advances only when the caller drives it via `.next()`, `.return()`, or `.throw()`.

6.1 States and the yield/next handshake



The critical insight: `yield` is a *bidirectional* channel. The value *after* `yield` goes out to the caller; the value *passed to* the next `next()` call comes *in* as the result of the `yield` expression.

```

function* bidirectional() {
  console.log("start");
  const a = yield "first
yield"; // sends "first yield" OUT, receives next arg as `a`
  console.log("a =", a);
  const b = yield "second yield"; // sends "second yield" OUT,
receives next arg as `b`
  console.log("b =", b);
  return "done";
}

const gen = bidirectional();
console.log(gen.next()); // start → { value: "first yield",
done: false }
console.log(gen.next("hello")); // a = hello → { value: "second yield",
done: false }
console.log(gen.next("world")); // b = world → { value: "done", done:
true }
console.log(gen.next("extra")); // { value: undefined, done: true } –
completed, ignores arg

// .return – force completion, runs finally blocks
function* withFinally() {
  try {
    yield 1;
    yield 2;
    yield 3;
  } finally {
    console.log("cleanup in finally");
  }
}

const g2 = withFinally();
console.log(g2.next()); // { value: 1, done: false }
console.log(g2.return("early")); // cleanup in finally → { value:
"early", done: true }

// .throw – inject an exception at the yield point
function* withCatch() {
  try {
    const v = yield "waiting";
    console.log("v =", v);
  } catch (e) {
    console.log("caught inside generator:", e.message);
    yield "recovered";
  }
  return "done";
}

const g3 = withCatch();
console.log(g3.next()); // { value: "waiting", done:
false }

```



```

console.log(g3.throw(new Error("oops"))); // caught inside generator:
oops → { value: "recovered", done: false }
console.log(g3.next());                    // { value: "done", done:
true }

// yield* – delegation to another iterable/generator
function* inner() { yield "a"; yield "b"; }
function* outer() { yield "start"; yield* inner(); yield "end"; }
console.log([...outer()]); // ["start", "a", "b", "end"]
// yield* also forwards .return/.throw and captures the delegated
return value:
function* delegating() {
  const result = yield* inner(); // inner returns undefined → result =
undefined
  console.log("delegated return:", result);
}

```

6.2 Generator visualizer — observable state machine

```
// generator-visualizer.js – run to see every transition
function* orderWorkflow(orderId) {
  console.log(`[${orderId}] workflow started`);
  const payment = yield { step: "charge", orderId };
  console.log(`[${orderId}] payment result:`, payment);

  if (!payment.ok) {
    yield { step: "compensate", reason: payment.error };
    return { status: "failed", orderId };
  }

  const shipment = yield { step: "ship", orderId };
  console.log(`[${orderId}] shipment:`, shipment);
  return { status: "completed", orderId, tracking: shipment.tracking };
}

function visualize(gen) {
  let step = 0;
  let lastValue = undefined;
  let isThrow = false;
  let throwArg;

  while (true) {
    const result = isThrow ? gen.throw(throwArg) : gen.next(lastValue);
    console.log(`  step ${++step}: gen =>`, result);
    if (result.done) {
      console.log(`    → completed with:`, result.value);
      break;
    }
    // Simulate driving the generator from "infrastructure" responses
    console.log(`    ← driving next with response for step:`, result.value.step);
    if (result.value.step === "charge") lastValue = { ok: true, chargeId: "ch_123" };
    else if (result.value.step === "ship") lastValue = { tracking: "12999" };
    else lastValue = undefined;
    isThrow = false;
  }
}

const gen = orderWorkflow("ord-42");
visualize(gen);
// Logs every yield/next handshake – the exact protocol async/await automates
```

6.3 Why generators matter beyond async — iterables, pipelines, and cooperative scheduling

Generators are also the implementation behind lazy pipelines and custom iterables — used in streaming parsers, paginated API iterators, and test-data factories:

```
// Lazy paginated API – caller drives fetching via iteration
function* paginateSync(fetchPage) {
  let cursor = null;
  while (true) {
    const page = fetchPage(cursor); // sync for illustration – async
version in $8
    yield* page.items;
    if (!page.nextCursor) break;
    cursor = page.nextCursor;
  }
}

// Cooperative scheduling – yield to the event loop periodically
function* chunkedProcess(items, chunkSize = 100) {
  for (let i = 0; i < items.length; i += chunkSize) {
    const chunk = items.slice(i, i + chunkSize);
    yield chunk; // caller can await setImmediate or queueMicrotask
between chunks
  }
}
```

7. async/await — Generators Wrapped in Promises

An `async` function is not a new execution model. It is a generator that the engine automatically drives to completion, with each `yield` replaced by `await`, and with the driving loop implemented as `PromiseJobs`. The desugaring below is semantically faithful — what V8 actually runs is more optimized, but the state transitions are identical.

7.1 The desugaring

async function
fetchWithRetry(url)

Desugared to:
function
fetchWithRetry(url)
return
spawn(generator)

function*
fetchWithRetryGen(url)
let res = yield
fetch(url)
if (!res.ok) throw ...
let body = yield
res.json()
return body

spawn(gen):
return new
Promise((resolve, reject) =>
step(nextF) {
try { result = nextF() }
catch(e) { reject(e); return }

```
// What you write:
async function fetchWithRetry(url, retries = 3) {
  for (let attempt = 0; attempt < retries; attempt++) {
    try {
      const res = await fetch(url);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const body = await res.json();
      return body;
    } catch (err) {
      if (attempt === retries - 1) throw err;
      await sleep(100 * 2 ** attempt); // exponential backoff, still
    }
  }
}

// What the engine conceptually executes (hand-written desugaring for
// illustration):
function fetchWithRetryDesugared(url, retries = 3) {
  return spawn(function* () {
    for (let attempt = 0; attempt < retries; attempt++) {
      try {
        const res = yield fetch(url); // await → yield
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const body = yield res.json(); // await → yield
        return body;
      } catch (err) {
        if (attempt === retries - 1) throw err;
        yield sleep(100 * 2 ** attempt); // await → yield
      }
    }
  })();
}

// The driver that makes yield behave like await – the "spawn" / "co"
// pattern
function spawn(gen) {
  return new Promise((resolve, reject) => {
    function step(nextF) {
      let result;
      try {
        result = nextF(); // gen.next(v) or gen.throw(e)
      } catch (e) {
        reject(e); // exception inside generator → reject outer promise
        return;
      }
      if (result.done) {
        // Generator returned – fulfill outer promise with return value
        // (assimilate if return value is thenable)
        Promise.resolve(result.value).then(resolve, reject);
      }
    }
  });
}
```

```

    return;
  }
  // Generator yielded a value – wait for it, then resume
  Promise.resolve(result.value).then(
    (v) => step(() => gen.next(v)), // fulfillment → inject as
yield result
    (e) => step(() => gen.throw(e)) // rejection → throw inside
generator (catchable)
  );
}
step(() =>
gen.next()); // start – first next() takes no meaningful arg
});
}

// Proof they behave identically (including error paths):
async function demoAsync(x) {
  if (x < 0) throw new Error("negative");
  const y = await Promise.resolve(x * 2);
  return y + 1;
}
function demoDesugared(x) {
  return spawn(function* () {
    if (x < 0) throw new Error("negative");
    const y = yield Promise.resolve(x * 2);
    return y + 1;
  })();
}
// Both: demo*(5) → 11, demo*(-1) → rejected with "negative"

```

Three properties fall out of this model:

1. **await is yield + Promise.resolve + resume-as-microtask.** The `Promise.resolve(result.value)` step is why `await 42` and `await someThenable` both work — non-promises are wrapped, thenables are assimilated, promises are adopted.
2. **Rejection becomes throw inside the generator.** That is why `try/catch` around `await` works — `gen.throw(e)` injects the exception at the `yield` point, where the generator's `try/catch` can intercept it. Without a `catch`, the exception propagates out of `gen.next/gen.throw`, and `spawn` rejects the outer promise.
3. **The outer promise is the function's return value.** Calling an `async` function always returns a Promise immediately, even before the first `await`. The function body runs synchronously until the first `await` (or `return/throw`), then suspends.

```
// The outer promise is returned synchronously – before any await
async function example() {
  console.log("A – sync preamble");
  await Promise.resolve();
  console.log("C – resumed as microtask");
  return 42;
}
console.log("B – before call");
const p = example(); // logs "A" synchronously, returns a pending
promise
console.log("B2 – after call, p is", p instanceof Promise);
p.then(v => console.log("D – return value:", v));
console.log("B3 – still sync");
// Order: B, A, B2, B3, C, D
```

7.2 await in loops — sequential vs. concurrent

A common backend performance bug is awaiting inside a loop when fan-out was intended:

```

// Sequential – N round trips, total latency = sum(latencies)
async function sequential(ids) {
  const results = [];
  for (const id of ids) {
    results.push(await fetchUser(id)); // waits for each before
    starting next
  }
  return results;
}

// Concurrent – 1 fan-out, total latency ≈ max(latency)
async function concurrent(ids) {
  const promises = ids.map(id => fetchUser(id)); // all start
  immediately
  return Promise.all(promises); // single await for
  all
}

// Bounded concurrency – N at a time (p-limit pattern, see §9)
async function bounded(ids, limit = 10) {
  const results = new Array(ids.length);
  let nextIndex = 0;

  async function worker() {
    while (nextIndex < ids.length) {
      const i = nextIndex++;
      results[i] = await fetchUser(ids[i]);
    }
  }
  await Promise.all(Array.from({ length: limit }, () => worker()));
  return results;
}

// Rate-limited with backpressure – for await version in §8

```

Operational note. The sequential version looks correct in tests with mocked I/O (where latency is zero) and becomes a p99 disaster in production where each `fetchUser` is 20ms and `ids.length` is 100 — 2 seconds of serial latency that `Promise.all` would collapse to ~20ms. Lint for `no-await-in-loop` in hot paths, but allow it when ordering or throttling is intentional (e.g., migrations that must apply sequentially).

8. Async Iteration and Top-Level Await

8.1 Async generators and `for await...of`

A *sync* generator yields values; an *async* generator yields promises of values. The consumer drives it with `for await...of`, which awaits each yielded promise before looping.

```

// Async generator – each yield can await internally
async function* fetchPaginated(baseUrl) {
  let cursor = null;
  while (true) {
    const url = cursor ? `${baseUrl}?cursor=${cursor}` : baseUrl;
    const res = await fetch(url);
    if (!res.ok) throw new Error(`fetch failed: ${res.status}`);
    const page = await res.json();
    for (const item of page.items) {
      yield item; // each item yielded individually, consumer awaits
    }
    if (!page.nextCursor) break;
    cursor = page.nextCursor;
  }
}

// Consumer – for await...of handles the async iterator protocol
async function processAll() {
  for await (const item of fetchPaginated("https://api.example.com/orders")) {
    await handleItem(item); // backpressure – next fetch waits for this
  }
}

// Manual iteration – what for await desugars to
async function processAllManual() {
  const iter = fetchPaginated("https://api.example.com/orders")
[Symbol.asyncIterator]();
  while (true) {
    const { value, done } = await iter.next();
    if (done) break;
    await handleItem(value);
  }
}

// Async generator as a transform stream (readable → transform → writable)
async function* batched(source, batchSize = 100) {
  let batch = [];
  for await (const item of source) {
    batch.push(item);
    if (batch.length >= batchSize) {
      yield batch;
      batch = [];
    }
  }
  if (batch.length > 0) yield batch;
}

```

```

async function* withRetry(source, retries = 3) {
  for await (const item of source) {
    for (let attempt = 0; attempt < retries; attempt++) {
      try {
        yield await processWithPossibleFailure(item);
        break;
      } catch (e) {
        if (attempt === retries - 1) throw e;
        await sleep(100 * 2 ** attempt);
      }
    }
  }
}

// Composed pipeline – each stage is an async iterable
// for await (const batch of batched(withRetry(fetchPaginated(url)),
// 50)) { ... }

```

The async iterator protocol — the two methods the engine looks for:

Protocol	Method	Returns
Sync iterable	<code>[Symbol.iterator]()</code>	Iterator with <code>next()</code> → <code>{value, done}</code>
Async iterable	<code>[Symbol.asyncIterator]()</code>	AsyncIterator with <code>next()</code> → <code>Promise<{value, done}></code>
<code>for await...of</code>	Prefers <code>asyncIterator</code> , falls back to <code>iterator</code> (wrapping values in <code>Promise.resolve</code>)	Awaits each <code>next()</code> result

Custom async iterable — useful for wrapping event emitters, queues, or message consumers:

```

// Wrap a Node.js readable stream as an async iterable (Node does this
natively, but here's the mechanism)
async function* streamToAsyncIterable(readable) {
  const queue = [];
  let done = false;
  let error = null;
  let resolveNext = null;

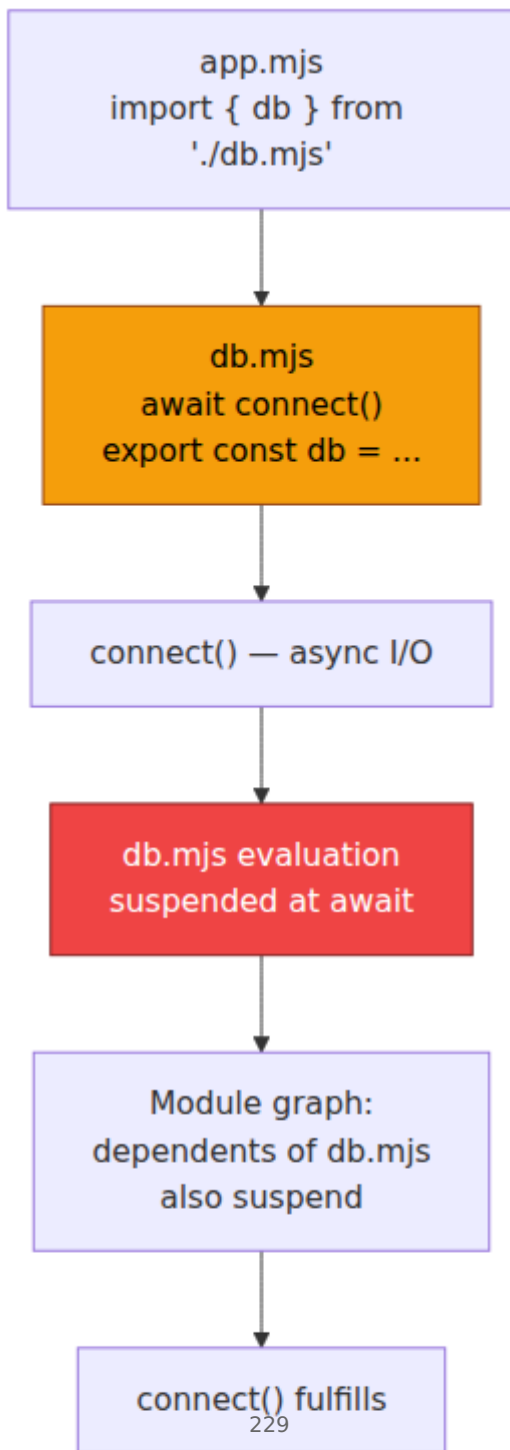
  readable.on("data", (chunk) => {
    if (resolveNext) { const r = resolveNext; resolveNext = null; r({ v
alue: chunk, done: false }); }
    else queue.push(chunk);
  });
  readable.on("end", () => {
    done = true;
    if (resolveNext) { const r = resolveNext; resolveNext = null; r({ v
alue: undefined, done: true }); }
  });
  readable.on("error", (err) => {
    error = err;
    if (resolveNext) { const r = resolveNext; resolveNext = null; r(Pro
mise.reject(err)); }
  });

  while (true) {
    if (queue.length > 0) yield queue.shift();
    else if (done) break;
    else if (error) throw error;
    else {
      // Suspend until next data/end/error – the promise the generator
      yields
      const result = await new Promise((resolve) => { resolveNext = res
olve; });
      if (result.done) break;
      yield result.value;
    }
  }
}

```

8.2 Top-level await — blocking the module graph

Top-level `await` (ES2022, supported in ESM — not in CommonJS or classic scripts) allows a module to `await` at the top level. The module's evaluation becomes asynchronous, and any importer waits for it.



```

// db.mjs – top-level await for connection setup
import { createPool } from "./pool.mjs";
export const pool = await createPool(process.env.DATABASE_URL);
// No init() function, no race – importers that do `import { pool }
from "./db.mjs"`
// are guaranteed pool is connected before their module body runs.

// config.mjs – top-level await for remote config
const res = await fetch("https://config.internal/v1/app-config");
if (!res.ok) throw new Error(`config fetch failed: ${res.status}`);
export const config = await res.json();

// app.mjs – all imports that transitively depend on db.mjs /
config.mjs wait
import { pool } from "./db.mjs";
import { config } from "./config.mjs";
// By this line, pool is connected and config is loaded – no additional
await needed
console.log("ready, pool size:", pool.size, "feature flag:", config.featureX);

// What the engine does (simplified):
// 1. Instantiate all modules, link imports/exports (live bindings).
// 2. Evaluate leaf modules first. When a module hits top-level await,
suspend
// its evaluation and return a promise for its namespace.
// 3. Any importer that depends on a suspended module also suspends.
// 4. When the awaited promise fulfills, resume evaluation, then resume
dependents.
// 5. If the awaited promise rejects, the module fails to evaluate –
importers get a rejected promise.

// Anti-pattern – top-level await that blocks the entire graph on slow
I/O
// slow-init.mjs
await new Promise(r => setTimeout(r, 5000)); // 5s stall – every
importer waits 5s
export const ready = true;
// If 20 modules import slow-init.mjs, all 20 stall for 5s serially
(their run in parallel
// if independent, but any chain through slow-init.mjs is serial).

// Preferred – export a promise, let consumers await concurrently
// fast-init.mjs
export const readyPromise = init(); // starts immediately, does not
block evaluation
async function init() { await new Promise(r => setTimeout(r, 5000)); re
turn true; }
// Consumers: await readyPromise – they can do other work before
awaiting

```

```
// Or: use Promise.all for parallel top-level fetches within one module
// parallel-init.mjs
const [pool, cfg] = await Promise.all([createPool(url), fetchConfig()])
;
export { pool, cfg }; // single await for both, not two serial awaits
```

Rules and constraints:

- **Only in ESM.** `await` at the top level in a CommonJS or non-module script is a syntax error. Node requires `"type": "module"` in `package.json` or a `.mjs` extension.
- **Blocks dependents, not siblings.** Independent branches of the module graph evaluate in parallel. Only the transitive importers of an async module wait.
- **Failure is fatal to the branch.** If a top-level `await` rejects, the module and all its dependents fail to evaluate. There is no `try / catch` across modules — wrap the `await` in `try / catch` inside the module if fallback is needed.
- **Circular dependencies with top-level await deadlock.** If `a.mjs` awaits `b.mjs` and `b.mjs` awaits `a.mjs`, the graph cannot make progress. The engine detects this and the evaluation hangs or errors depending on the implementation — avoid the cycle.

```
// Safe top-level await with fallback
// resilient-config.mjs
let config;
try {
  const res = await fetch("https://config.internal/v1/app-config");
  config = await res.json();
} catch (e) {
  console.warn("config fetch failed, using defaults:", e.message);
  config = { featureX: false, timeout: 5000 };
}
export { config };
```

9. Distributed-Systems Lens — Async Patterns at Scale

The primitives above compose into the patterns backend engineers reach for daily. Each is a small program over Promises; each has a failure mode that only appears under load.

9.1 Timeout, retry, and deadline propagation


```

// Timeout wrapper – the building block for deadlines
function withTimeout(promise, ms, label = "operation") {
  let timer;
  const timeout = new Promise( (_, reject) => {
    timer = setTimeout(() => reject(new Error(`${label} timed out
after ${ms}ms`)), ms);
  });
  // Promise.race + cleanup – clear timer whichever settles first
  return Promise.race([promise, timeout]).finally(() => clearTimeout(ti
mer));
}

// Usage – per-request deadline that propagates
async function handleRequest(req) {
  const deadline = Date.now() + 2000; // 2s budget for whole handler
  const remaining = () => Math.max(0, deadline - Date.now());

  const user = await withTimeout(fetchUser(req.userId), remaining(), "f
etchUser");
  const orders = await withTimeout(fetchOrders(user.id), remaining(), "
fetchOrders");
  return { user, orders };
}

// Retry with jittered exponential backoff – for idempotent operations
only
async function retry(fn, { retries = 3, baseMs = 100, maxMs = 5000, lab
el = "retry" } = {}) {
  let lastError;
  for (let attempt = 0; attempt <= retries; attempt++) {
    try {
      return await fn();
    } catch (e) {
      lastError = e;
      if (attempt === retries) break;
      if (e.status && e.status < 500) throw
e; // don't retry 4xx – not transient
      const backoff = Math.min(maxMs, baseMs * 2 ** attempt);
      const jitter = backoff * (0.5 + Math.random() * 0.5); //
0.5x–1.0x
      console.warn(`${label}] attempt ${attempt + 1} failed: ${e.messa
ge} – retry in ${jitter.toFixed(0)}ms`);
      await new Promise(r => setTimeout(r, jitter));
    }
  }
  throw lastError;
}

// Composed – retry with per-attempt timeout and global deadline
async function resilientFetch(url) {

```

```

    return retry(
      () => withTimeout(fetch(url).then(r => {
        if (!r.ok) throw Object.assign(new Error(`HTTP ${r.status}`), { status: r.status });
        return r.json();
      }), 1000, "fetch"),
      { retries: 3, baseMs: 200, label: "resilientFetch" }
    );
  }
}

```

Deadline vs. timeout. A timeout is per-attempt; a deadline is end-to-end. The `remaining()` pattern above implements deadline propagation — the same concept as gRPC deadlines and `context.WithDeadline` in Go. Without it, a handler that retries three times with 1s timeouts can take 3s even when the caller gave up after 2s, wasting downstream capacity.

9.2 Concurrency limiting and backpressure

Unbounded `Promise.all(ids.map(fetch))` with 10,000 IDs opens 10,000 concurrent connections — exhausting file descriptors, connection pools, or downstream rate limits. Bound it.

```

// p-limit style – bounded concurrency with no external dependency
function pLimit(concurrency) {
  const queue = [];
  let active = 0;

  function next() {
    if (active >= concurrency || queue.length === 0) return;
    active++;
    const { fn, resolve, reject } = queue.shift();
    Promise.resolve()
      .then(fn)
      .then(resolve, reject)
      .finally(() => { active--; next(); });
  }

  return function run(fn) {
    return new Promise((resolve, reject) => {
      queue.push({ fn, resolve, reject });
      next();
    });
  };
}

// Usage – 10 concurrent fetches, rest queued
const limit = pLimit(10);
const results = await Promise.all(
  ids.map(id => limit(() => fetchUser(id)))
);

// With async iteration – natural backpressure (consumer controls rate)
async function* fetchUsersBatched(ids, batchSize = 10) {
  const limit = pLimit(batchSize);
  // Process in batches – each batch is concurrent, batches are
  sequential
  for (let i = 0; i < ids.length; i += batchSize) {
    const batch = ids.slice(i, i + batchSize);
    const users = await Promise.all(batch.map(id => limit(() => fetchUs
er(id))));
    for (const user of users) yield user;
  }
}

for await (const user of fetchUsersBatched(allIds, 20)) {
  await indexUser(user); // backpressure – next batch waits
}

```

9.3 Graceful shutdown while work is in flight

Node does not wait for pending Promises before exiting. If you handle `SIGTERM` by calling `process.exit()`, in-flight requests are dropped mid-`await`.

```
// graceful-shutdown.js
let shuttingDown = false;
const inflight = new Set();

function track(promise) {
  inflight.add(promise);
  promise.finally(() => inflight.delete(promise));
  return promise;
}

async function handleRequest(req, res) {
  if (shuttingDown) {
    res.writeHead(503, { "Retry-After": "5" }).end("shutting down");
    return;
  }
  await track(async () => {
    const data = await fetchFromDownstream(req);
    if (!shuttingDown) res.json(data);
  }());
}

process.on("SIGTERM", async () => {
  console.log("SIGTERM - draining...");
  shuttingDown = true;
  server.close(); // stop accepting new connections

  // Wait for in-flight work with a deadline
  const deadline = new Promise((_, reject) =>
    setTimeout(() => reject(new Error("shutdown deadline exceeded")), 1
0000)
  );
  try {
    await Promise.race([Promise.allSettled([...inflight]), deadline]);
  } catch (e) {
    console.error("shutdown deadline exceeded, forcing exit");
  }
  // Flush observability pipeline, close pools
  await Promise.allSettled([flushMetrics(), pool.end()]);
  process.exit(0);
});
```

9.4 Observability — finding slow or stuck promise chains

```
// Slow-promise detector – logs chains that exceed a threshold
function withSlowLog(promise, label, thresholdMs = 1000) {
  const start = performance.now();
  const timer = setTimeout(() => {
    console.warn(`[slow-promise] ${label} still pending after ${thresholdMs}ms`);
  }, thresholdMs);
  return promise.finally(() => {
    clearTimeout(timer);
    const elapsed = performance.now() - start;
    if (elapsed > thresholdMs) {
      console.warn(`[slow-promise] ${label} settled after ${elapsed.toFixed(0)}ms`);
    }
  });
}

// Usage
await withSlowLog(fetchOrders(userId), `fetchOrders:${userId}`, 500);

// Async context tracking – correlate logs across awaits (Node AsyncLocalStorage)
const { AsyncLocalStorage } = require("node:async_hooks");
const requestStore = new AsyncLocalStorage();

app.use((req, res, next) => {
  requestStore.run({ requestId: req.headers["x-request-id"] ?? crypto.randomUUID(), () => next());
});

async function fetchWithContext(url) {
  const ctx = requestStore.getStore();
  console.log(`[${ctx?.requestId}] fetching ${url}`);
  return withSlowLog(fetch(url), `fetch:${url}`);
}

// AsyncLocalStorage propagates through PromiseJobs automatically –
// no manual context passing, unlike callback-based code.
```

10. Putting It All Together — A Worked Service Handler

This handler uses every primitive in the chapter — Promise states, microtask ordering, `async/await`, generators for the retry loop, `async`

iteration for paginated downstream, bounded concurrency, timeout, and graceful error handling:

```

// handler.js – realistic service endpoint
import { pLimit } from "./p-limit.mjs"; // §9.2

const limit = pLimit(10);

export async function handleListOrders(req, res) {
  const requestId = req.headers["x-request-id"] ?? crypto.randomUUID();
  const deadline = Date.now() + 3000;

  try {
    // Top-level await already resolved pool/config – available
    // synchronously

    // 1. Fetch user with per-attempt timeout + retry (async generator-
    // driven retry in §9.1)
    const user = await retry(
      () => withTimeout(fetchUser(req.params.userId, { requestId }), 80
0, "fetchUser"),
      { retries: 2, baseMs: 100, label: "fetchUser" }
    );

    // 2. Stream downstream orders via async iteration – backpressure-
    // aware
    const orders = [];
    for await (const page of fetchOrderPages(user.id, { requestId, dead
line })) {
      // 3. Enrich each order concurrently, bounded
      const enriched = await Promise.all(
        page.map(order => limit(() => enrichOrder(order, { requestId })
))
      );
      orders.push(...enriched);
      if (Date.now() > deadline) throw new Error("deadline exceeded
while paginating");
    }

    res.json({ user, orders });

  } catch (err) {
    // Error propagation path:
    // - fetchUser rejection → retry → throw → caught here
    // - withTimeout rejection → retry → caught here
    // - deadline exceeded → throw → caught here
    // - enrichOrder rejection → Promise.all rejects → caught here
    (fail-fast)
    // (use allSettled inside enrichOrder if partial enrichment is
    // acceptable)
    if (err.message.includes("timed out") || err.message.includes("dead
line")) {
      res.writeHead(504).end(JSON.stringify({ error: "gateway

```

```

timeout", requestId }));
    } else if (err.status === 404) {
      res.writeHead(404).end(JSON.stringify({ error: "not found", requestId }));
    } else {
      console.error(`[${requestId}] unhandled handler error:`, err);
      res.writeHead(500).end(JSON.stringify({ error: "internal error", requestId }));
    }
  }
}

// Async generator for paginated downstream – yields pages, handles cursor
async function* fetchOrderPages(userId, { requestId, deadline }) {
  let cursor = null;
  while (true) {
    if (Date.now() > deadline) throw new Error("deadline exceeded");
    const page = await withTimeout(
      fetchOrdersPage(userId, cursor, { requestId }),
      Math.max(100, deadline - Date.now()),
      "fetchOrdersPage"
    );
    yield page.items;
    if (!page.nextCursor) break;
    cursor = page.nextCursor;
  }
}

```

Trace the scheduling: `fetchUser` resolves → its `.then` reaction enqueues a `PromiseJob` → the `await` continuation resumes in that microtask → `fetchOrderPages` starts → each `await fetchOrdersPage` suspends and resumes as a microtask → `Promise.all` inside the loop fans out enrichment → each `limit`-wrapped `enrichOrder` enqueues its own `PromiseJobs` → the `for await` loop awaits each yielded page sequentially, providing natural backpressure. No callback, no manual queue management — the `Promise Job Queue` is the queue.

Key takeaways

- A Promise has three states and one rule: pending → fulfilled or rejected, exactly once, immutably. All `.then` handlers run asynchronously as `PromiseJobs`, even for already-settled promises. This single-assignment, always-async contract is what makes Promise composition deterministic.

- `.then` does not execute handlers — it allocates a new Promise and a PromiseReaction. When the original settles, each reaction enqueues a PromiseJob onto the microtask queue. The chained promise's fate is determined by what the handler returns, throws, or assimilates.
- The Promise Job Queue is a microtask queue that drains exhaustively before the next task, timer, or render. Recursive Promise resolution starves tasks; unbounded microtask chains are a real source of p99 latency and event-loop lag. `await` always costs at least one microtask hop.
- `async / await` is syntactic sugar over generators + Promises: `await x` desugars to `yield Promise.resolve(x)` with a driver that resumes the generator via `gen.next` on fulfillment and `gen.throw` on rejection. `try / catch` around `await` works because rejection is injected as a `throw` at the yield point.
- Generators (`function*`, `yield`, `next / return / throw`, `yield*`) are the underlying suspension primitive. `yield` is a bidirectional channel; `yield*` delegates the protocol. Generators also power lazy iterables and cooperative scheduling independent of `async`.
- Error propagation in `async` code is promise rejection, not synchronous unwinding. Unhandled rejections fire `unhandledrejection` / `unhandledRejection` at the next microtask checkpoint and can crash Node in strict mode. Every promise chain must end with a handler — enforce `no-floating-promises` and install a global `unhandledRejection` logger at the entrypoint.
- `Promise.all` (fail-fast), `allSettled` (always report), `race` (first settled), and `any` (first fulfilled → `AggregateError` if all fail) are concurrency combinators with distinct failure semantics. Choose by whether partial results are useful — the same quorum/hedging trade-off as distributed systems.
- Async iteration (`async function*`, `for await...of`, `Symbol.asyncIterator`) composes streaming and paginated sources with backpressure. Top-level `await` in ESM suspends module-graph evaluation for that branch — powerful for one-time initialization, hazardous if it blocks the critical path or introduces a cycle.
- At scale, wrap I/O in `withTimeout` and deadline propagation, retry idempotent operations with jittered backoff, bound fan-out with `pLimit`-style concurrency limiters, stream large result sets with `async` generators, and `await` in-flight work during graceful shutdown.

Further reading

- ECMA-262 §27.2 — Promise Objects, §27.7 — Async Functions, §27.5 — Generator Objects. The normative descriptions of `[[PromiseState]]`, `PromiseReaction`, `HostEnqueuePromiseJob`, and async function desugaring.
- HTML Standard §8.1.4 — Event Loops and §8.1.6 — Microtask Queuing. Defines the task/microtask queues PromiseJobs run on in browsers.
- [Node.js documentation](#) — The Node.js Event Loop, Timers, and `process.nextTick` and Timers, `async_hooks` / `AsyncLocalStorage`. How libuv phases and `nextTick` interact with PromiseJobs.
- [Jake Archibald](#) — *Tasks, microtasks, queues and schedules* (2015). The canonical visual explanation of microtask vs. task ordering with interactive demos.
- [Nolan Lawson](#) — *What color is your function?* and Bob Nystrom — *What Color is Your Function?* discussion. Why `async` contagion is a real API design constraint.
- [V8 blog](#) — *Fast async/await* (2017) and *Understanding the ECMAScript spec, part 4* (Promises). How V8 optimizes the generator+Promise desugaring away in the common case.
- MDN — [Promise](#), async function, Generator, Async iteration, Top-level await, AggregateError, `unhandledrejection` event. Accurate, example-rich references with browser/Node compat tables.
- WHATWG / TC39 proposals — [Top-level await \(ES2022\)](#), `Promise.withResolvers` (ES2024), `Promise.try` / `Promise.withResolvers` patterns. Current and upcoming Promise ergonomics.

Chapter 6 — Build Tooling and Bundlers: RSPack, Vite, esbuild

What this chapter covers: Why JavaScript needs bundlers at all, and how three modern toolchains solve the problem differently. We dissect the bundler pipeline — resolve, transform, graph, chunk, emit — then go deep on esbuild (Go, parallel, incremental), RSPack (Rust, Webpack-

compatible), and Vite (esbuild pre-bundle + Rollup production build). Along the way you will configure each bundler from scratch, trace treeshaking through `sideEffects` and `usedExports`, split a production bundle with `dynamic import()` and `splitChunks`, follow the HMR WebSocket protocol frame-by-frame, and federate two independently deployed apps at runtime.

Learning goals:

- Explain why bundlers exist: the gap between authored ESM/CSS/assets and what browsers and CDNs can efficiently deliver.
 - Compare the architectures of esbuild, RSPack, and Vite — language choice (Go vs Rust vs JS), parallelism model, caching strategy, and compatibility surface.
 - Configure esbuild, RSPack, and Vite for a real project: entry, output, loaders, plugins, and production optimization.
 - Trace treeshaking end-to-end: static `import / export` analysis, `sideEffects` flag, `usedExports` / `innerGraph`, and why CommonJS defeats it.
 - Design code-splitting strategies with `dynamic import()`, `splitChunks`, and `manualChunks`, and read the resulting chunk graph.
 - Explain the HMR protocol: WebSocket signaling, ESM hot-update injection, and accept/dispose lifecycle.
 - Implement Module Federation between a host and remote: shared scope, version negotiation, and independent deploy semantics.
 - Use bundle analyzers and source-map explorers to diagnose bloat, and apply chunk-level caching for long-term CDN stability.
-

1. Why Bundlers Exist

Authored code and deliverable code are different artifacts. You write hundreds of ESM files, TypeScript, JSX, CSS modules, SVG imports, and `import` statements that assume a resolver. The browser receives bytes over HTTP. The gap is large enough that shipping authored code directly has never been viable at scale — even now that browsers support native ESM.

1.1 The problems a bundler solves

Authored concern	Deliverable requirement	Bundler job
Hundreds of small ESM files with bare specifiers (<code>import { x } from "lodash-es"</code>)	Few HTTP requests, no bare specifiers (until import maps mature)	Resolve, graph, concatenate
TypeScript, JSX, CSS, WASM, SVG, workers	Only JS + CSS the browser can parse	Transform / transpile / load
<code>import "./Button.css"</code> and <code>import logo from "./logo.svg"</code>	CSS extracted or injected; assets hashed and emitted	Asset pipeline
One logical app	Cacheable chunks that change independently	Code splitting + content hashing
Dead code from large dependencies	Minimal bytes on the wire	Treeshaking / DCE
Dev feedback loop	Sub-100 ms HMR	Incremental rebuild + HMR runtime

An experienced backend engineer will recognize this as a compilation and distribution problem: the bundler is a build system that compiles a dependency graph into deployable artifacts with deterministic hashing, cache invalidation, and incremental rebuild semantics — directly analogous to Bazel or Buck for backend services.

1.2 From Browserify to the current landscape

Browserify (2011) → Webpack 1-5 (2012-2020) → Parcel / Rollup (2017)
→ esbuild (2020, Go) → Vite (2021, esbuild+Rollup) → RSPack (2023, Rust)
→ Turbopack (2023, Rust) → RSPack / Rsbuild ecosystem (2024+)

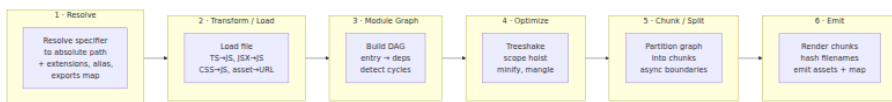
Webpack proved that *everything is a module* (JS, CSS, images, fonts) and that loaders and plugins could cover any transform. Its cost was speed: a large chunk of Webpack is single-threaded JavaScript, its caching was bolted on late (Webpack 5 persistent cache), and cold builds of 1000+ module apps routinely exceeded 30-60 seconds.

esbuild demonstrated that a bundler written in a systems language with parallelism baked in from day one could be 10-100x faster. Vite showed that native ESM in development could eliminate bundling entirely for the dev server. RSPack proved that Webpack API compatibility did not require Webpack's architecture — reimplementing the plugin/loader surface in Rust with parallel execution and first-class persistent caching.

Today a senior team chooses on three axes: **compatibility** (do existing Webpack plugins need to keep working?), **speed** (cold build, incremental rebuild, HMR latency), and **dev-server model** (bundled vs. native ESM).

2. The Universal Bundler Pipeline

Every bundler, regardless of language or API, implements the same pipeline. Differences are in *when* stages run, how much is parallelized, and what is cached.



The pipeline executes differently per tool:



Key architectural distinction for backend engineers: esbuild and RSPack are **eager bundlers** — they produce a complete bundle for every mode. Vite in development is a **lazy transformer** — it serves native ESM and transforms files on request, only pre-bundling `node_modules` dependencies that would otherwise trigger hundreds of waterfall requests.

3. esbuild — Go, Parallelism, and Incremental Builds

3.1 Why Go

esbuild's author (Evan Wallace) chose Go for three reasons that map directly to bundler bottlenecks:

1. **Parallelism without data races.** Go's goroutines + channels model lets the parser, resolver, and printer run across all cores with minimal synchronization. JavaScript bundlers in Node are fundamentally single-threaded; worker-thread parallelism requires serialization across the JS/C++ boundary.
2. **Fast startup, no JIT warmup.** Go compiles to a single static binary. No V8 warmup, no `node_modules` resolution to start the tool itself. Cold invocation is single-digit milliseconds.
3. **Cache-friendly memory layout.** The parser and printer are written to minimize allocations and maximize sequential memory access — the same concern that drives high-throughput backend parsers.

Measured on a 1000-module synthetic benchmark, esbuild typically builds in ~0.3s where Webpack 5 takes ~8s and Rollup ~5s. The gap widens on incremental rebuilds because esbuild's in-memory graph is designed for reuse.

3.2 Minimal esbuild config

esbuild has two interfaces: a CLI and a JS build API. Production use almost always goes through the JS API so plugins and incremental mode are available.

```

// build.mjs — esbuild production build
import * as esbuild from "esbuild";
import { sassPlugin } from "esbuild-sass-plugin";

const ctx = await esbuild.context({
  entryPoints: ["src/main.tsx"],
  bundle: true,
  outdir: "dist",
  format: "esm",
  platform: "browser",
  target: ["chrome110", "firefox110", "safari16"],
  splitting: true,           // required for ESM code splitting
  sourcemap: true,
  minify: true,
  treeShaking: true,
  metafile: true,           // emit build metadata for analysis
  chunkNames: "chunks/[name]-[hash]",
  assetNames: "assets/[name]-[hash]",
  loader: {
    ".svg": "file",
    ".woff2": "file",
  },
  plugins: [sassPlugin()],
  define: {
    "process.env.NODE_ENV": '"production"',
  },
  // Incremental: keep context alive, rebuild on demand
});

// One-shot build
const result = await ctx.rebuild();
console.log(await esbuild.analyzeMetafile(result.metafile, { verbose: false }));

// Watch mode (long-lived process)
await ctx.watch();

// Serve mode (dev server with SSE-based HMR precursor)
// await ctx.serve({ servedir: "dist", port: 3000 });

await ctx.dispose();

```

Key options for backend engineers to note:

- `splitting: true` only works with `format: "esm"` — esbuild refuses to code-split CJS because CJS `require` is dynamic and cannot be statically partitioned.

- `metafile: true` produces a JSON description of every input, output, and byte contribution — the input to bundle analyzers.
- `esbuild.context()` + `rebuild()` is the incremental API. The module graph stays resident; unchanged files are not re-parsed. This is the foundation of Vite's pre-bundling speed.

3.3 Incremental rebuild and parallelism internals

esbuild's parallelism operates at module granularity. Each file's parse + transform is an independent goroutine. The linker phase joins the parsed ASTs, resolves cross-file references, and performs treeshaking. Because Go's scheduler multiplexes goroutines onto OS threads without per-module process overhead, scaling is near-linear up to core count — a 16-core machine roughly halves build time versus 8 cores.

Incremental rebuild (`ctx.rebuild()`) diffs file mtimes/hashes against the in-memory graph and only re-parses changed files and their transitive dependents. For a single-file edit in a 2000-module app, incremental rebuild is typically 20-50 ms — fast enough for HMR without a separate dev server.

```
# Metafile analysis output (trimmed)
$ node build.mjs
dist/main.js                142.3kb  100.0%
├─ src/main.tsx              2.1kb   1.5%
├─ src/routes/dashboard.tsx  18.4kb  12.9%
├─ node_modules/react-dom    42.7kb  30.0%
└─ node_modules/zod          12.1kb   8.5%
dist/chunks/vendor-ABC123.js  89.2kb
dist/chunks/dashboard-DEF456.js 24.1kb
```

3.4 Limitations

esbuild intentionally omits features that require expensive cross-module type analysis: noTypeScript type checking (use `tsc --noEmit` separately), no CSS code splitting beyond basic extraction, and a plugin API that is powerful but not Webpack-loader compatible. It also does not implement Hot Module Replacement natively — HMR requires a dev-server layer (Vite builds that layer on top).

4. RSPack — Rust, Webpack Compatibility, and Persistent Caching

4.1 Design thesis

RSPack (by ByteDance / WebInfra) reimplements Webpack's core semantics in Rust while preserving the Webpack plugin and loader API. The thesis: teams with large Webpack codebases and custom plugins should not have to rewrite their build to get Rust-class performance. If your build is defined by `webpack.config.js` plus a dozen custom loaders, RSPack should be a near drop-in replacement.

Architecture highlights:

- **Rust core, JS bindings via NAPI-RS.** The hot path (resolve, parse, codegen, chunk graph) runs in Rust. The JS layer exposes `Compiler`, `Compilation`, and `NormalModuleFactory` hooks that match Webpack's Tapable hook system, so existing plugins can run unmodified or with thin shims.
- **Parallel execution by default.** Module building (`make` phase) fans out across a Rayon thread pool. Unlike Webpack's `thread-loader` (which forks workers per loader), RSPack parallelizes the entire build pipeline.
- **Persistent filesystem cache.** Webpack 5 added `cache: { type: "filesystem" }` as an opt-in. RSPack enables persistent caching by default with content-addressed storage and fine-grained invalidation — second cold builds after a small change typically hit >80% cache.

4.2 RSPack config (Webpack-compatible)

```

// rspack.config.js
const { defineConfig } = require("@rspack/cli");
const { rspack } = require("@rspack/core");
const { BundleAnalyzerPlugin } = require("webpack-bundle-analyzer");

module.exports = defineConfig({
  entry: { main: "./src/main.tsx" },
  mode: "production", // or "development" / "none"
  target: ["web", "es2022"],
  output: {
    path: __dirname + "/dist",
    filename: "[name].[contenthash:8].js",
    chunkFilename: "chunks/[name].[contenthash:8].js",
    assetModuleFilename: "assets/[name].[contenthash:8][ext]",
    clean: true,
  },
  resolve: {
    extensions: [".ts", ".tsx", ".js", ".jsx", ".json"],
    alias: { "@": __dirname + "/src" },
    // Reads package.json "exports" / "imports" like Webpack 5
  },
  module: {
    rules: [
      {
        test: /\.tsx?$/,
        loader: "builtin:swc-loader", // Rust-native SWC transform, no
        JS overhead
        options: {
          jsc: { parser: { syntax: "typescript", tsx: true }, target: "
es2022" },
        },
      },
      { test: /\.css$/, type: "css/auto" }, // RSPack native CSS
      handling
      { test: /\.svg$/, type: "asset/resource" },
    ],
  },
  optimization: {
    minimize: true,
    minimizer: [new rspack.SwcJsMinimizerRspackPlugin()],
    splitChunks: {
      chunks: "all",
      cacheGroups: {
        vendor: {
          test: /[\\/]node_modules[\\/]/,
          name: "vendor",
          priority: 10,
          reuseExistingChunk: true,
        },
        common: {

```

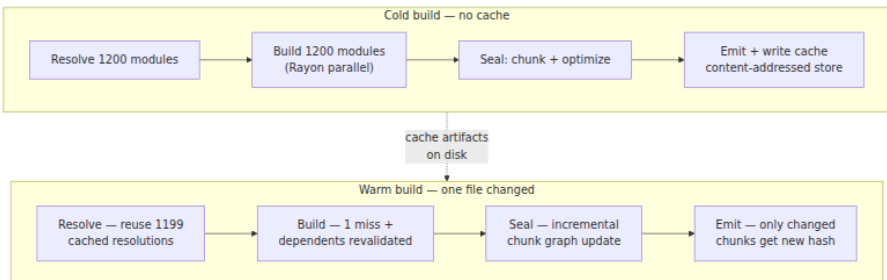
```

    minChunks: 2,
    priority: 5,
    reuseExistingChunk: true,
  },
},
},
usedExports: true, // treeshaking: mark unused exports
sideEffects: true, // respect package.json sideEffects flag
},
experiments: {
  css: true, // native CSS support (no mini-css-extract-
plugin needed)
},
cache: true, // persistent filesystem cache (default in
RSPack)
devtool: "source-map",
plugins: [
  new rspack.HtmlRspackPlugin({ template: "./index.html" }),
  new BundleAnalyzerPlugin({ analyzerMode: "static", openAnalyzer: fa
lse }),
],
devServer: {
  hot: true,
  port: 3000,
},
});

```

Migration from Webpack is typically mechanical: change `webpack` imports to `@rspack/core`, replace `babel-loader` with `builtin:swc-loader`, and remove `thread-loader` / `cache-loader` (unnecessary — RSPack parallelizes and caches natively). The [RSPack migration guide](#) tracks remaining incompatibilities; most Webpack 5 plugins work, though plugins that reach into Webpack's internal `Compilation` asset pipeline may need shims.

4.3 Caching at scale



The persistent cache stores per-module build results keyed by content hash of the input file plus the hash of loader options and resolver config. Changing a single source file invalidates only that module and modules that transitively import it. In a 1500-module app, a one-line edit typically rebuilds in 200-400 ms on a warm cache — competitive with Vite's ESM HMR for many workloads, with the advantage that production and development use the same bundler.

For distributed CI, RSPack's cache directory (`node_modules/.cache/rspack` by default) can be persisted across CI runs via `actions/cache` or `BuildKit` cache mounts. Remote cache sharing across machines is not yet built-in (unlike `Turborepo` or `Bazel`), so teams often layer a content-addressed cache on top.

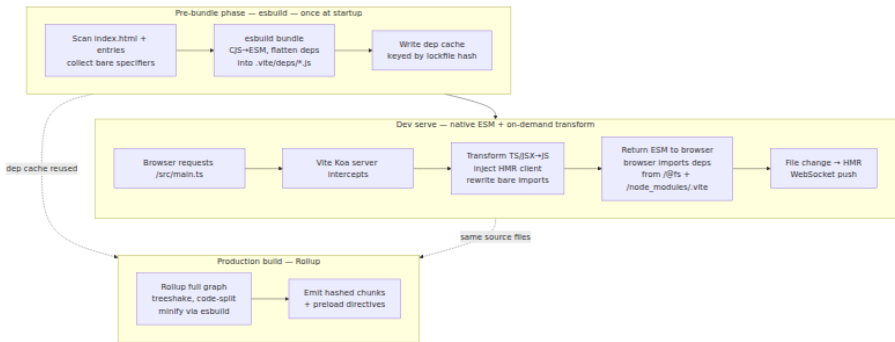
4.4 Rsbuild — the higher-level wrapper

Rsbuild is the RSPack team's zero-config framework (analogous to Vite's role over Rollup/esbuild). It wraps RSPack with opinionated defaults for React/Vue, built-in performance budgets, and `create-rsbuild` scaffolding. If RSPack is "Webpack in Rust," Rsbuild is "Vite in Rust." Teams that do not need Webpack plugin compatibility often start with Rsbuild directly.

5. Vite — esbuild Pre-Bundle, Native ESM Dev, Rollup Production

5.1 The two-phase architecture

Vite's central insight: browsers already understand ESM, so the dev server does not need to bundle. Instead, it serves each source file as a native ESM module and lets the browser's import graph drive loading. The only bundling required in dev is for `node_modules` dependencies that are authored as CJS or that would otherwise trigger a waterfall of hundreds of small requests.



This split is why Vite cold start is fast (esbuild pre-bundles ~100 deps in ~300 ms) and HMR is near-instant (only the changed file is re-transformed and pushed over WebSocket). Production still does a full Rollup bundle because native ESM delivery with hundreds of modules is not optimal for HTTP/1.1/2 without bundling, and Rollup's treeshaking and chunking are more mature than esbuild's for production optimization.

5.2 Vite config

```

// vite.config.ts
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import { visualizer } from "rollup-plugin-visualizer";

export default defineConfig({
  plugins: [
    react({ jsxImportSource: "react" }), // SWC-based JSX transform
    visualizer({ filename: "dist/stats.html", gzipSize: true }),
  ],
  resolve: {
    alias: { "@": "/src" },
  },
  css: {
    modules: { localsConvention: "camelCase" },
    preprocessorOptions: { scss: { additionalData: `@import "@styles/vars.scss";` } },
  },
  optimizeDeps: {
    include: ["react", "react-dom", "react-router-dom", "zod"],
    exclude: ["@my-org/native-addon"], // CJS or native deps that break pre-bundle
    esbuildOptions: {
      target: "es2022",
    },
  },
  server: {
    port: 3000,
    hmr: {
      overlay: true, // error overlay in browser
    },
  },
  build: {
    outDir: "dist",
    sourcemap: true,
    target: "es2022",
    minify: "esbuild", // or "terser" for finer control
    cssCodeSplit: true,
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ["react", "react-dom", "react-router-dom"],
          ui: ["@radix-ui/react-dialog", "@radix-ui/react-dropdown-menu"],
        },
        chunkFileNames: "assets/[name]-[hash].js",
        entryFileNames: "assets/[name]-[hash].js",
        assetFileNames: "assets/[name]-[hash][extname]",
      },
    },
  },
});

```



```

    chunkSizeWarningLimit: 500, // warn if any chunk exceeds 500 kB
  },
  preview: { port: 4173 },
});

```

Notable details:

- `optimizeDeps.include` forces pre-bundling of deps that Vite's scanner might miss (dynamic imports, deps behind re-exports). `exclude` is used for deps that use Node builtins or native addons that esbuild cannot bundle.
- `build.rollupOptions.output.manualChunks` controls production chunking. Vite delegates this entirely to Rollup — the same `manualChunks` semantics as a raw Rollup config.
- `minify: "esbuild"` uses esbuild's minifier inside Rollup, which is 20-40x faster than Terser with marginally less compression. Switch to `"terser"` only when you need fine-grained mangle options.

5.3 Pre-bundle vs. production build

Concern	Pre-bundle (dev, esbuild)	Production (Rollup)
Input	<code>node_modules</code> deps only	Full app graph (source + deps)
Output format	Flattened ESM per dep, cached in <code>node_modules/.vite</code>	Hashed chunks, code-split, minified
Treeshaking	No (whole dep bundled)	Yes (Rollup <code>treeshake.moduleSideEffects</code>)
Code splitting	No	Yes (<code>manualChunks</code> , <code>dynamic import()</code>)
HMR	N/A (pre-bundle is static)	N/A (prod is static)
Invalidation	Lockfile or dep source changes	Source changes

Prod path — full bundle

src/main.tsx + deps



Rollup
graph → treeshake →
chunk



dist/assets/main-
abc123.js
dist/assets/vendor-
def456.js
dist/assets/dashboard-
ghi789.js

Dev path — no bundle

src/main.tsx

The dev/prod split is Vite's main trade-off: dev and prod use different bundlers, so behavior can diverge (e.g., a CJS interop edge case that works in dev via esbuild's CJS→ESM transform but breaks in Rollup's prod build). Vite 6+ narrows this with the experimental `rollup` (Rollup in Rust) unification, but the two-phase model remains the default.

6. Treeshaking — Dead Code Elimination at the Module Boundary

Treeshaking is dead code elimination (DCE) that operates on the ESM static import/export graph. The name comes from Rollup's original visualization: shake the dependency tree, dead leaves fall off.

6.1 What makes it possible

ESM `import/export` is statically analyzable: the set of imported and exported bindings is known without executing the module. This lets the bundler determine which exports are *used* and which are *dead* before any code runs. CommonJS defeats this because `require()` is a runtime call with a dynamic argument, and `module.exports` is a mutable object.

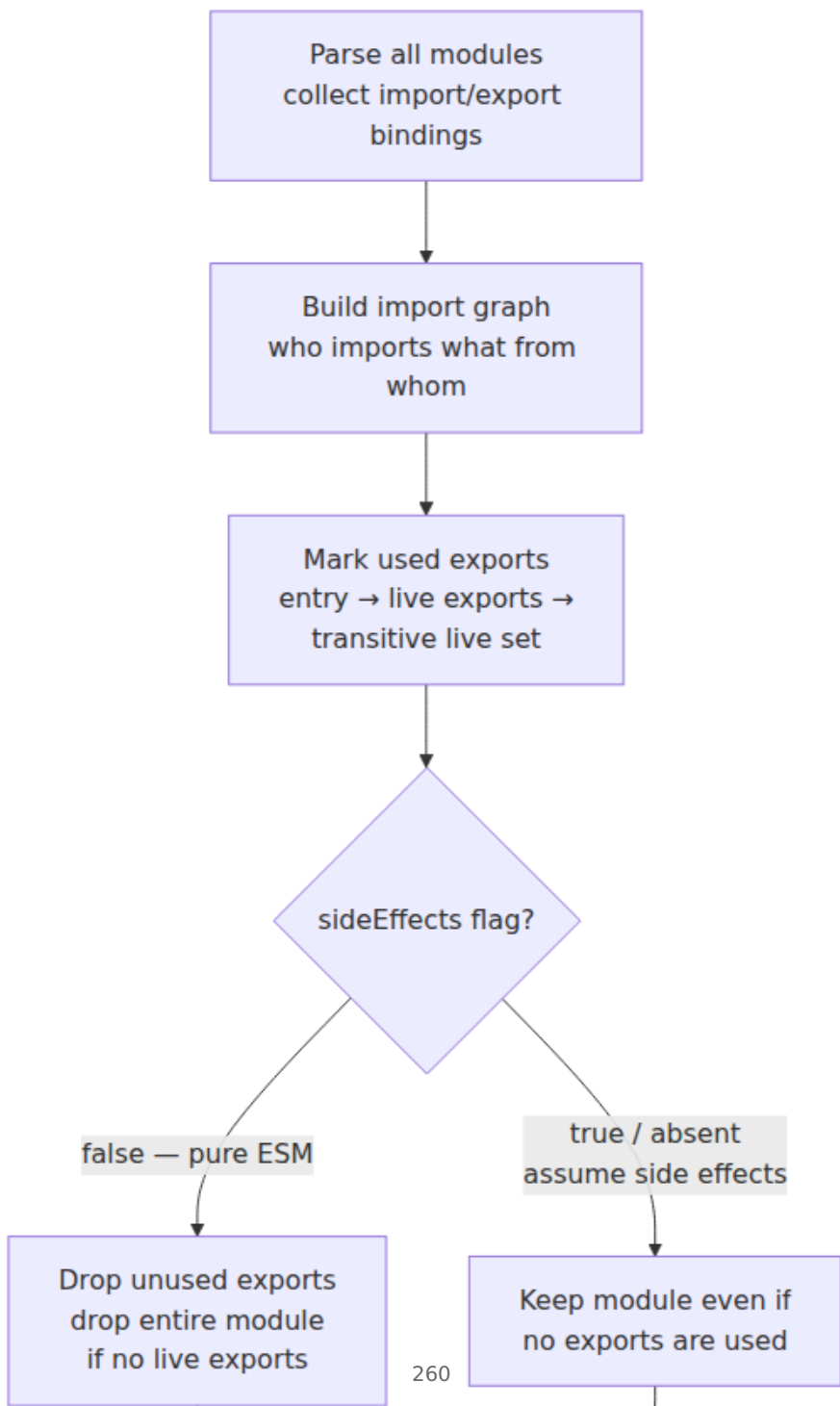
```
// utils.ts — some exports used, some not
export function formatDate(d) { return d.toISOString().slice(0, 10); }
export function formatMoney(n) { return `$$${n.toFixed(2)}`; } // ←
unused
export function debounce(fn, ms) { /* ... */ } // ←
unused

// app.ts — only imports one export
import { formatDate } from './utils.js';
console.log(formatDate(new Date()));
// formatMoney and debounce are dead — treeshaking removes them
```

After treeshaking and minification, the bundle contains only `formatDate`:

```
// bundled output (simplified, minified off for clarity)
function formatDate(d) { return d.toISOString().slice(0, 10); }
console.log(formatDate(new Date()));
```

6.2 The treeshaking pipeline



Three layers cooperate:

1. **Module-level treeshaking** (`usedExports` / `providedExports`). The bundler marks which exports of each module are imported by any live module. Unused exports are removed. If a module has no live exports *and* is side-effect-free, the entire module is dropped.
2. **sideEffects flag**. Declared in `package.json`, it tells the bundler whether importing a module can have observable side effects beyond its exports.

```
// package.json - treeshaking hints
{
  "name": "my-ui-lib",
  "sideEffects": false,           // all modules are pure - safe to
  drop unused                    // or, granular:
  // or, granular:
  "sideEffects": ["*.css", "./src/polyfill.js"]
}
```

When `sideEffects: false`, `import "my-ui-lib/button"` that is never otherwise used can be dropped entirely. When `true` (the default), the bundler must keep the import because it might run global initialization code. Getting this wrong is a common source of both bloat (flag missing → dead code retained) and bugs (flag set to `false` on a module that does have side effects → needed initialization dropped).

1. **Inner-graph and minifier DCE**. After module-level shaking, RSPack/Rollup's `innerGraph` tracks usage of individual declarations inside a module, and the minifier (Terser/esbuild/SWC) eliminates dead branches, unused variables, and `/* __PURE__*/`-annotated calls.

```
// innerGraph example - only `used` survives
export const used = expensiveComputation(); // kept - exported and
import                                         imported
export const unused =
expensiveComputation(); // dropped - not imported anywhere
const local = expensiveComputation();        // dropped - never
referenced
/* __PURE__*/
createElement("div"); // dropped if return value unused
```

6.3 Why CommonJS defeats treeshaking

```
// cjs-utils.js – CommonJS – NOT treeshakable
exports.formatDate = function(d) { return d.toISOString().slice(0, 10); };
exports.formatMoney = function(n) { return `$$${n.toFixed(2)}`; };

// bundler sees: `exports` is a mutable object, properties assigned at runtime
// cannot prove formatMoney is unused without executing the file → must keep all

// esm-utils.js – ESM – treeshakable
export function formatDate(d) { return d.toISOString().slice(0, 10); }
export function formatMoney(n) { return `$$${n.toFixed(2)}`; }
// bundler sees: static export list, can prove formatMoney is unused → drops it
```

This is why the ecosystem push toward ESM ("type": "module", exports map with `import` condition, dual-package migration) directly affects bundle size. A single CJS dependency in the critical path can prevent treeshaking of its entire subtree.

6.4 Verifying treeshaking

```
# Rollup / Vite – treeshake report
$ vite build --debug 2>&1 | grep -i "treeshake\|unused"

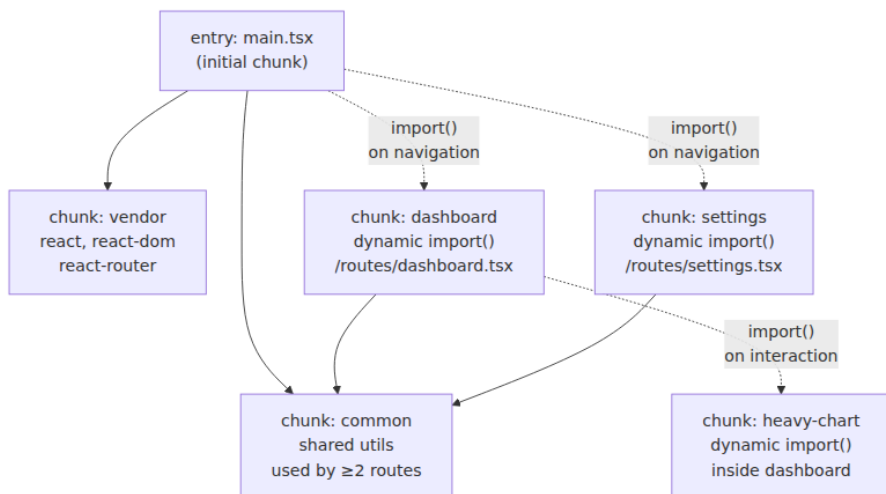
# RSPack / Webpack – stats with usedExports
$ npx rspack build --json stats.json
$ npx webpack-bundle-analyzer stats.json

# esbuild – metafile inspection
$ node -e "import('./dist/meta.json', {with: {type: 'json'}}).then(m=>console.log(Object.keys(m.outputs)))"
```

7. Code Splitting — Partitioning the Graph into Cacheable Chunks

Code splitting partitions the module graph into multiple chunks that can be loaded on demand and cached independently. Without it, every deploy invalidates a single monolithic bundle and every user downloads code for routes they never visit.

7.1 Chunk graph



Solid edges are **initial chunks** loaded with the HTML. Dashed edges are **async chunks** loaded via `import()` at runtime. The browser fetches async chunks only when the import is executed — typically on route navigation or user interaction.

7.2 Dynamic `import()`

The primitive behind all code splitting is the dynamic `import()` expression. Unlike static `import`, it is asynchronous, returns a Promise, and creates a split point in the chunk graph.

```

// Static import – always in the initial chunk
import { Dashboard } from "./routes/dashboard.js";

// Dynamic import – creates an async chunk boundary
const Dashboard = React.lazy(() => import("./routes/dashboard.js"));

// With React Router – route-level splitting
import { createBrowserRouter } from "react-router-dom";

const router = createBrowserRouter([
  { path: "/", element: <Home /> },
  {
    path: "/dashboard",
    lazy: () => import("./routes/dashboard.js"), // async chunk
  },
  {
    path: "/settings",
    lazy: () => import("./routes/settings.js"), // async chunk
  },
]);

// Manual dynamic import with preload hint
const loadChart = () => import("./components/HeavyChart.js");
// Vite/RSPack inject <link rel="modulepreload"> for known async chunks

```

At runtime, the bundler's chunk loader (a small runtime injected into the entry chunk) fetches async chunks via `import()` (ESM) or JSONP/script-tag (legacy). Vite and RSPack both emit `<link rel="modulepreload">` for async chunks discovered during build, so the browser can start fetching them before the `import()` executes.

7.3 `splitChunks` / `manualChunks` strategies

Beyond `import()`-driven splitting, bundlers can automatically extract shared modules into common chunks. The configuration differs per tool but the semantics are the same: define cache groups with tests and priorities.


```
// RSPack / Webpack - splitChunks
optimization: {
  splitChunks: {
    chunks: "all",           // initial + async
    minSize: 20000,          // only split if chunk would be ≥20 kB
    maxAsyncRequests: 30,    // limit parallel async fetches
    maxInitialRequests: 30,
    cacheGroups: {
      vendor: {
        test: /[\\/]node_modules[\\/]/,
        name: "vendor",
        priority: 10,
        reuseExistingChunk: true,
        enforce: true,
      },
      react: {
        test: /[\\/]node_modules[\\/](react|react-dom|scheduler)[\\/]/,
        name: "react",
        priority: 20,         // higher priority → matched first
      },
      common: {
        minChunks: 2,         // shared by ≥2 chunks → extract
        priority: 5,
        reuseExistingChunk: true,
      },
    },
  },
},
}
```

```
// Vite / Rollup - manualChunks
// vite.config.ts → build.rollupOptions.output.manualChunks
manualChunks(id) {
  if (id.includes("node_modules")) {
    if (id.includes("react") || id.includes("scheduler")) return "react";
    if (id.includes("zod") || id.includes("valibot")) return "validation";
    return "vendor"; // all other deps
  }
  if (id.includes("src/components/charts")) return "charts";
},
// Or object form for simple grouping:
// manualChunks: { vendor: ["react", "react-dom"], charts: ["recharts", "d3"] }
```

```
// esbuild – manual code splitting via entryPoints as object  
// esbuild does not have splitChunks; use multiple entry points or  
dynamic import only  
await esbuild.build({  
  entryPoints: { main: "src/main.tsx", dashboard: "src/routes/  
dashboard.tsx" },  
  bundle: true,  
  splitting: true,  
  format: "esm",  
  outdir: "dist",  
});
```

Trade-offs for backend engineers:

- **Too many chunks** → waterfall of HTTP requests, especially on high-latency mobile networks. `maxAsyncRequests` and `minSize` prevent fragmentation into dozens of tiny chunks.
- **Too few chunks** → cache invalidation granularity suffers. Changing one route invalidates the shared vendor chunk if it is not split independently.
- **Content hashing** (`[contenthash]`) ensures that unchanged chunks keep their filename and CDN cache entry across deploys. Only chunks whose content changed get a new hash. This is the same content-addressed caching principle behind Docker layer caching.

7.4 Bundle analyzer output

After building, inspect the chunk graph with a visual analyzer. All three bundlers produce compatible stats.

```
# Vite – rollup-plugin-visualizer emits dist/stats.html
$ vite build
dist/assets/main-a1b2c3d4.js      42.1 kB | gzip: 12.4 kB
dist/assets/vendor-e5f6g7h8.js   89.7 kB | gzip: 28.1 kB
dist/assets/dashboard-i9j0k1l2.js 24.3 kB | gzip: 7.2 kB
dist/assets/settings-m3n4o5p6.js 18.9 kB | gzip: 5.8 kB

# RSPack / Webpack – webpack-bundle-analyzer
$ npx rspack build --json stats.json
$ npx webpack-bundle-analyzer stats.json dist
→ opens treemap: vendor 89.7 kB (31%), main 42.1 kB (15%), dashboard
24.3 kB ...

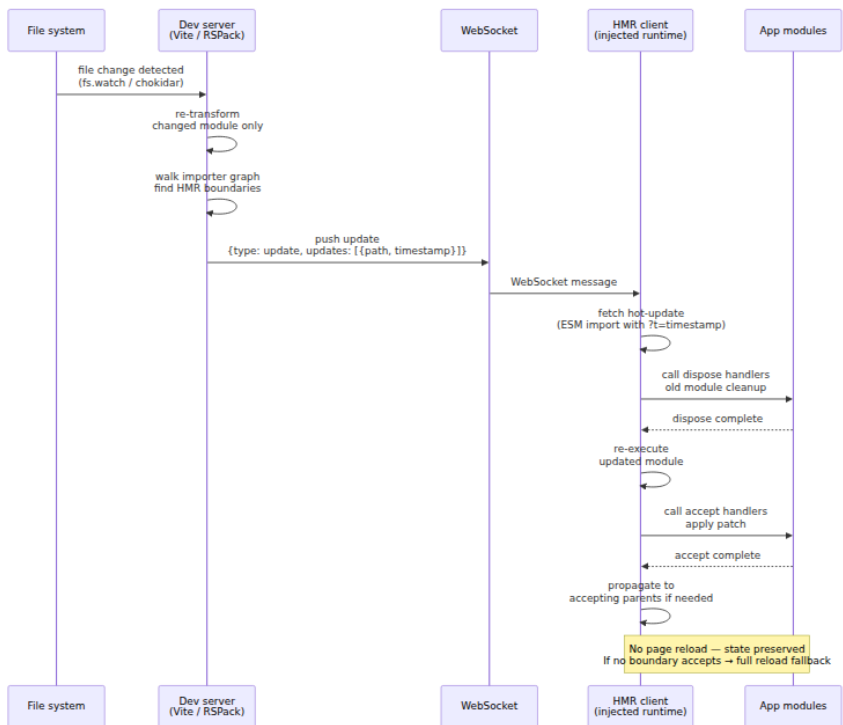
# esbuild – esbuild --analyze or metafile
$ npx esbuild --bundle src/main.tsx --metafile=meta.json --analyze
src/main.tsx      _____ 2.1 kB
react-dom         _____ 42.7 kB ██████████
zod               _____ 12.1 kB ██████
```

The analyzer treemap is the primary tool for finding bloat: a single large dependency (e.g., `moment` at 65 kB minified, `lodash` at 72 kB) often dominates the vendor chunk. Replacing it with a modular alternative (`date-fns` with per-function imports, `lodash-es` with treeshaking) or moving it to an async chunk are the standard fixes.

8. Hot Module Replacement — The HMR Protocol

HMR updates modules in a running application without a full page reload, preserving application state (form inputs, scroll position, in-memory caches). It is a development-only feature — production builds never include the HMR runtime.

8.1 HMR update flow



8.2 WebSocket message flow (Vite)

Vite's dev server (Koa-based) maintains a WebSocket connection to each browser tab. The protocol is JSON over WebSocket, with a small set of message types.

```

// Server → Client: HMR update payload (Vite)
{
  "type": "update",
  "updates": [
    {
      "type": "js-update",
      "path": "/src/components/Button.tsx",
      "acceptedPath": "/src/components/Button.tsx",
      "timestamp": 1713700000123
    }
  ]
}

// Server → Client: full reload (no HMR boundary found)
{ "type": "full-reload", "path": "*" }

// Server → Client: error overlay
{
  "type": "error",
  "err": {
    "message": "Transform failed: Unexpected token",
    "frame": "  12 |   return <Button\n                ^",
    "stack": "..."
  }
}

// Client → Server: connected (on page load)
{ "type": "connected" }

// Client → Server: ping/pong keepalive
{ "type": "ping" }

```

```

// RSPack / Webpack HMR – similar but uses JSONP-style hot-update
chunks
// Server pushes manifest via WebSocket (webpack-dev-server) or SSE
{
  "type": "hash",
  "data": "a1b2c3d4e5f6" // new compilation hash
}
{
  "type": "ok" // or "warnings" / "errors"
}
// Client then fetches hot-update chunk: main.a1b2c3d4.hot-update.json
+ .js

```

8.3 The `import.meta.hot` API

Application code opts into HMR by calling the HMR API. Without it, a file change triggers a full reload or propagation to the nearest accepting parent.

```
// Button.tsx – HMR-aware module (Vite / RSPack both support
import.meta.hot)
import { createStore } from "./store.js";

const store = createStore();

// Accept updates to this module – re-execute without reloading parents
if (import.meta.hot) {
  // Called when this module is about to be replaced
  import.meta.hot.dispose((data) => {
    // Preserve state across replacement
    data.storeState = store.getState();
  });

  // Called after the new version is executed
  import.meta.hot.accept((newModule) => {
    if (newModule) {
      // Optionally handle the new module's exports
      console.log("[HMR] Button updated");
    }
  });

  // Accept updates to a specific dependency only
  import.meta.hot.accept("./store.js", (newStore) => {
    console.log("[HMR] store updated, re-binding");
  });

  // Invalidate – force full reload when this module changes
  // import.meta.hot.invalidate();
}

// Framework HMR (React Fast Refresh, Vue HMR) builds on this
// primitive:
// the framework plugin wraps each component with accept logic that
// re-renders without unmounting state.
```

```
// RSPack / Webpack equivalent – module.hot (legacy API, still supported)
if (module.hot) {
  module.hot.accept("./store.js", () => {
    console.log("[HMR] store updated");
  });
  module.hot.dispose((data) => {
    data.state = store.getState();
  });
}
// Vite and RSPack both support import.meta.hot; prefer it for new code.
```

8.4 HMR failure modes

Failure	Cause	Visible effect
No HMR boundary accepts	Changed file has no <code>accept</code> call and no parent accepts	Full page reload (state lost)
Stale closure over old module	<code>accept</code> handler captures old export reference	App uses old code despite HMR success log
CSS HMR inject fails	CSS extraction in dev (e.g., <code>mini-css-extract-plugin</code>)	Styles flash or require reload
WebSocket disconnected	Proxy strips <code>Upgrade: websocket</code> header (common behind Nginx without <code>proxy_set_header Upgrade</code>)	No HMR, manual refresh required
ESM cache poisoning	Browser caches <code>?t=timestamp</code> incorrectly (service worker)	Old code served after HMR push

The WebSocket proxy issue is the most common operational failure. Any reverse proxy in front of the dev server must forward WebSocket upgrades:

```
# Nginx – required for Vite / RSPack HMR behind a proxy
location / {
    proxy_pass http://vite-dev:3000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
}
```

9. Module Federation — Runtime Composition of Independently Deployed Apps

Module Federation (introduced in Webpack 5, reimplemented in RSPack as `ModuleFederationPlugin`) allows separately built and deployed applications to share code at runtime. A **host** application dynamically loads modules exposed by one or more **remote** applications, with shared dependencies deduplicated via a version-negotiated shared scope.

This is the bundler-level primitive behind micro-frontends. For backend engineers, the analogy is service composition at the UI layer: each team deploys its app independently, and the shell composes them at runtime without a monolithic build.

9.1 Host / remote handshake

9.2 RSPack Module Federation config

```
// Remote - team-a/rspack.config.js - exposes a Widget
const { ModuleFederationPlugin } = require("@rspack/core").container;

module.exports = {
  entry: { main: "./src/index.ts" },
  output: { publicPath: "http://localhost:3001/", uniqueName: "teamA" }
,
  plugins: [
    new ModuleFederationPlugin({
      name: "teamA",
      filename: "remoteEntry.js",           // manifest + loader,
      fetched by host
      exposes: {
        "./Widget": "./src/Widget.tsx",    // host imports as "teamA/
Widget"
        "./utils": "./src/utils.ts",
      },
      shared: {
        react: { singleton: true, requiredVersion: "^18.0.0", eager: false },
        "react-dom": { singleton: true, requiredVersion: "^18.0.0" },
        "react-router-dom": { singleton: true, requiredVersion: "^6.0.0" },
      },
    })
  ],
};
```

```
// Host - shell/rspack.config.js - consumes the remote
const { ModuleFederationPlugin } = require("@rspack/core").container;

module.exports = {
  entry: { main: "./src/index.tsx" },
  output: { publicPath: "http://localhost:3000/", uniqueName: "shell" },
  plugins: [
    new ModuleFederationPlugin({
      name: "shell",
      remotes: {
        teamA: "teamA@http://localhost:3001/remoteEntry.js",
        teamB: "teamB@http://localhost:3002/remoteEntry.js",
      },
      shared: {
        react: { singleton: true, requiredVersion: "^18.0.0" },
        "react-dom": { singleton: true, requiredVersion: "^18.0.0" },
      },
    }),
  ],
};
```

```
// Host runtime - dynamic import of a federated module
import React, { Suspense, lazy } from "react";

// Static federated import (resolved via remoteEntry at build time)
import { Widget } from "teamA/Widget";

// Lazy federated import - remote chunk loaded on demand
const TeamBApp = lazy(() => import("teamB/App"));

export function Shell() {
  return (
    <div>
      <h1>Shell</h1>
      <Widget />
      <Suspense fallback={<div>Loading Team B...</div>}>
        <TeamBApp />
      </Suspense>
    </div>
  );
}
```

```
// Dynamic remote at runtime – no build-time coupling
// Useful when remote URL is determined by config / feature flag
async function loadRemoteWidget(url, scope, module) {
  // Vite equivalent: use @originjs/vite-plugin-federation
  await __webpack_init_sharing__("default");
  const container = window[scope];
  await container.init(__webpack_share_scopes__.default);
  const factory = await container.get(module);
  return factory();
}
```

9.3 Shared scope and version negotiation

The `shared` config controls how dependencies are deduplicated. The runtime negotiates versions using semver:

Config	Meaning
<code>singleton: true</code>	Only one copy may exist. If host and remote disagree on version, the highest compatible version wins and the other reuses it. If no compatible version exists, the remote falls back to its own copy (duplicate — detectable via <code>window</code> inspection).
<code>requiredVersion: "^18.0.0"</code>	Semver range this build requires. Negotiation fails (duplicate loaded) if no shared copy satisfies the range.
<code>eager: true</code>	Include the shared dep in the initial chunk (no async loading). Use for deps needed before remote resolution.
<code>strictVersion: true</code>	Fail hard if version mismatch (throw instead of duplicating). Use in CI to catch drift.

For backend engineers, the analogy is dependency convergence in Maven/Gradle: the shared scope is a runtime dependency mediation that picks one version of a singleton (React, singleton stores) to ensure identity — `instanceof`, context providers, and hook dispatchers must reference the same copy.

9.4 Operational concerns

- **Independent deploys.** Host and remote deploy on separate pipelines. The host fetches `remoteEntry.js` at runtime, so a remote deploy is live immediately without rebuilding the host. This

decouples team velocity but requires backward-compatible remote interfaces — the same contract discipline as versioned gRPC services.

- **Rollback.** Rolling back a remote is independent of the host. Rolling back the host does not roll back remotes. Incident response must account for this split.
- **Observability.** Each federated chunk should emit its own source maps and be tagged with its build version. Error tracking (Sentry, Datadog RUM) must map stack frames to the correct remote's source map, not just the host's.
- **Security.** `remoteEntry.js` is arbitrary JavaScript executed in the host's origin. A compromised remote is XSS against the host. Treat remote URLs as trust boundaries — pin with SRI (`integrity` attribute) or serve all remotes from the same trusted CDN with the same CSP.

10. Bundle Analysis, Source Maps, and Long-Term Caching

10.1 Analyzing the bundle

All three bundlers can emit a stats/metafile that visualizers consume. The workflow is identical regardless of tool:

```
# Vite – visualizer plugin (rollup-plugin-visualizer)
# vite.config.ts already configured above → vite build emits dist/
stats.html
$ vite build
$ open dist/stats.html # treemap + sunburst + network waterfall

# RSPack / Webpack – webpack-bundle-analyzer
$ npx rspack build --json stats.json
$ npx webpack-bundle-analyzer stats.json --mode static --report dist/
report.html

# esbuild – metafile + esbuild-visualizer
$ npx esbuild src/main.tsx --bundle --metafile=meta.json --outfile=dist/
/main.js
$ npx esbuild-visualizer --metafile meta.json --output dist/report.html

# Source-map explorer – maps bundled bytes back to original files
$ npx source-map-explorer "dist/**/*.js" --html dist/sourcemap.html
```

What to look for in the treemap: a single large rectangle is a candidate for code splitting or replacement. Common offenders and fixes:

Bloat source	Fix
<code>moment</code> (65 kB) imported wholesale	Replace with <code>date-fns</code> per-function imports or <code>dayjs</code>
<code>lodash</code> (72 kB) via <code>import _ from "lodash"</code>	Use <code>lodash-es</code> + treeshaking or <code>import pick from "lodash/pick"</code>
<code>core-js</code> polyfills for every feature	Scope via <code>browserslist</code> / <code>target: es2022</code> + <code>useBuiltIns: "usage"</code>
Duplicate <code>react</code> (Module Federation misconfig)	Check <code>shared.singleton: true</code> and semver alignment
Large inline SVG / image assets	Switch to <code>type: "asset"</code> with threshold (RSPack) or <code>assetsInlineLimit</code> (Vite)

10.2 Long-term caching and content hashing

Production filenames include a content hash so CDNs and browsers can cache indefinitely (`Cache-Control: public, max-age=31536000, immutable`). Only chunks whose content changed get a new filename after a deploy.

```
// RSPack / Vite - hashed filenames
output: {
  filename: "[name].[contenthash:8].js",          // main.a1b2c3d4.js
  chunkFilename: "chunks/[name].[contenthash:8].js",
}
// Vite equivalent in build.rollupOptions.output:
chunkFileNames: "assets/[name]-[hash].js",
entryFileNames: "assets/[name]-[hash].js",
```

The hash is computed over the chunk's rendered content, not the source file's content. Two invariants matter:

1. **Deterministic hashing.** The same source + same bundler version + same config must produce the same hash. Non-deterministic chunk ordering or timestamp injection breaks CDN caching. All three bundlers guarantee this when `mode: "production"` with stable options.

2. **Minimal invalidation.** Changing one route's async chunk should not change the vendor chunk's hash. This requires `splitChunks / manualChunks` to isolate vendor code and `optimization.moduleIds: "deterministic"` (RSPack/Webpack) to avoid numeric module IDs that shift when any module is added.

```
# Verify hash stability – build twice, compare
$ vite build && sha256sum dist/assets/vendor-*.js > /tmp/hash1
$ vite build && sha256sum dist/assets/vendor-*.js > /tmp/hash2
$ diff /tmp/hash1 /tmp/hash2 && echo "deterministic ✓"
```

11. Choosing a Bundler — Decision Framework

Criterion	esbuild	RSPack / Rsbuild	Vite (+ Rollup)
Language / parallelism	Go, goroutine-per-file	Rust, Rayon thread pool	JS (dev: esbuild + Koa; prod: Rollup)
Cold build (1000 modules)	~0.3 s	~0.8 s	~3 s (Rollup prod)
Incremental HMR	~20 ms (ctx.rebuild)	~200 ms (warm cache)	~30 ms (native ESM)
Webpack compat	None	High — loaders + plugins	None (Rollup plugins)
Dev model	Bundled	Bundled	Native ESM (unbundled)
CSS handling	Basic (CSS in JS or extract)	Native type: css	Native + PostCSS + modules
Code splitting	ESM only, no splitChunks	Full splitChunks + manualChunks	Rollup manualChunks
HMR	No built-in runtime	Webpack-style HMR	ESM HMR via WebSocket
Module Federation	No	Yes (Webpack API)	Via plugin (<code>@originjs/vite-plugin-federation</code>)

Criterion	esbuild	RSPack / Rsbuild	Vite (+ Rollup)
Best for	Libraries, CLIs, SSR bundles, Vite's engine	Large Webpack migrations, micro-frontends	Greenfield SPAs, fast dev loop

Rule of thumb for senior teams:

- **Migrating a large Webpack app** with custom loaders/plugins → RSPack. Keep the config, gain Rust speed and caching. Evaluate Rsbuild if you can drop Webpack compat.
- **Greenfield SPA** where dev speed matters most → Vite. Fastest HMR, best DX, broad plugin ecosystem. Accept the dev/prod bundler split.
- **Building a library, CLI, or SSR bundle** where a single fast bundle is needed without dev-server complexity → esbuild directly. Minimal config, fastest cold build, easy to embed in a larger build system.
- **Micro-frontends with independent deploys** → RSPack Module Federation. Vite federation exists but is plugin-based and less battle-tested at scale.

12. Distributed-Systems Lens — Bundlers at Fleet Scale

A bundler is a local build tool, but its outputs and caching behavior have distributed-systems consequences when multiplied across many services, teams, and CI pipelines.

Monorepo and many-repo builds. In a monorepo with 50 frontend packages (common in platform teams), a single bundler invocation per package is wasteful. Tools like Turborepo, Nx, and Bazel add a task graph on top of the bundler: they cache task outputs (including bundler emits) by input hash and share the cache across CI runners. RSPack's persistent cache and Vite's pre-bundle cache both benefit from being stored in this layer — a CI job that hits the remote cache may skip bundling entirely.

CI cache sharing. Without remote caching, every CI runner cold-builds every app on every PR. With a content-addressed remote cache (BuildKit, Turborepo Remote Cache, Bazel Remote Execution), the bundler's cache directory becomes a cacheable artifact. Key the cache on lockfile hash +

bundler config hash + source content hash. Invalidate surgically — the same principle as Docker layer caching.

Deploy and CDN interaction. Hashed chunk filenames are the contract between the bundler and the CDN. A deploy uploads new hashed chunks alongside old ones (never overwrite — old HTML may still reference old hashes). Old chunks are garbage-collected after the CDN TTL plus a safety margin. This is a distributed cache invalidation problem: the HTML is the manifest, chunks are immutable blobs, and the CDN is an eventually consistent cache. Getting it wrong (overwriting a chunk in place) causes users with old HTML to fetch a chunk whose content no longer matches the hash — a hard-to-diagnose white-screen error.

Module Federation as distributed composition. Federation moves composition from build time to runtime — the host and remotes are independently deployed services composed in the browser. This mirrors the backend shift from monolith to microservices, with the same trade-offs: independent deploy velocity vs. runtime coupling, version negotiation vs. API versioning, and the need for contract testing between host and remote interfaces. Treat federated module boundaries with the same rigor as service boundaries: versioned contracts, backward compatibility, and canary deploys.

Reproducibility. For supply-chain security (SLSA, see Book 4, Chapter 3), builds should be reproducible: the same inputs produce byte-identical outputs. Bundler reproducibility requires pinning the bundler version, locking `browserslist` targets, and ensuring no non-deterministic ordering (e.g., `Object.keys` iteration order in chunk assignment). Verify with a double-build hash check in CI.

Key takeaways

- Bundlers bridge the gap between authored ESM/CSS/assets and efficiently deliverable artifacts — they resolve, transform, graph, treeshake, chunk, and emit with content hashing.
- esbuild (Go) is the fastest single-pass bundler, parallelized per-module via goroutines, with an incremental `context` API that makes sub-50 ms rebuilds possible; it has no native HMR runtime.
- RSPack (Rust) reimplements Webpack's semantics with Rayon parallelism and persistent caching, preserving loader/plugin

compatibility for large Webpack migrations while delivering 5-10x faster builds.

- Vite splits the problem: esbuild pre-bundles `node_modules` once at startup, serves source as native ESM in dev (no bundle), and delegates production to Rollup for mature treeshaking and chunking.
- Treeshaking depends on static ESM `import / export` ; it operates in three layers — module-level `usedExports` , `sideEffects` flag, and inner-graph/minifier DCE — and is defeated by CommonJS.
- Code splitting via dynamic `import()` plus `splitChunks / manualChunks` partitions the graph into cacheable chunks; content hashing (`[contenthash]`) ensures CDN stability with minimal invalidation.
- HMR is a WebSocket protocol: the dev server pushes an update manifest, the HMR client fetches the hot-update, and `import.meta.hot.accept / dispose` handlers patch the running app without a full reload.
- Module Federation enables runtime composition of independently deployed apps via a host/remote handshake and shared-scope version negotiation — singleton sharing ensures one copy of React and other singletons.
- Bundle analyzers (`rollup-plugin-visualizer` , `webpack-bundle-analyzer` , `source-map-explorer`) map bytes to source modules; use them to find bloat and validate treeshaking.
- At fleet scale, bundler caching (persistent filesystem cache, pre-bundle cache, Turborepo/Nx remote cache) and hashed-chunk CDN semantics are distributed caching problems with the same invalidation discipline as backend artifact distribution.

Further reading

- esbuild documentation — Architecture and API: <https://esbuild.github.io/> — Start with "Architecture" and "API" for the Go parallelism model and incremental build semantics.
- RSPack documentation — Guide and Migration: <https://www.rspack.dev/guide/> — Covers Webpack compatibility, caching, and Module Federation; see "Guide / Migration / From Webpack" for drop-in steps.

- Rsbuild documentation: <https://rsbuild.dev/> — Higher-level RSPack wrapper; see "Guide / Performance / Build Performance" for caching and profiling.
- Vite documentation — Features and Build: <https://vitejs.dev/guide/features.html> and <https://vitejs.dev/guide/build.html> — Pre-bundling, HMR, and Rollup production build; see "Guide / Why Vite" for the native-ESM rationale.
- Rollup documentation — Treeshaking and Code Splitting: <https://rollupjs.org/configuration-options/#treeshake> and <https://rollupjs.org/configuration-options/#output-manualchunks>
- Webpack Module Federation — Concepts: <https://webpack.js.org/concepts/module-federation/> — The original design that RSPack reimplements; explains `exposes`, `remotes`, and `shared` semantics.
- RSPack Module Federation: <https://www.rspack.dev/guide/features/module-federation>
- SurviveJS — Webpack 5 and Module Federation (Artem Sapegin, Tobias Koppers): <https://survivejs.com/> — Historical context on why federation was needed and how it replaced earlier `externals` + `DllPlugin` patterns.
- Google — Native ESM in production (Web Fundamentals): <https://web.dev/articles/es-modules> — When native ESM delivery without bundling is viable and when it is not.
- SLSA Framework — Build provenance and reproducibility: <https://slsa.dev/spec/v1.0/> — For connecting bundler reproducibility to supply-chain security (see Book 4, Chapter 3).

Chapter 7 — React Internals: The Fiber Reconciler, Hooks, Suspense, and Concurrent Features

What this chapter covers: The machinery underneath `React.createElement` and `useState`. We open the Fiber reconciler — every field on a fiber node, the double-buffered `current / workInProgress` trees, the lane-based priority model — and follow a render from scheduling through `beginWork / completeWork` diffing to commit. We then

explain Hooks as a call-order-indexed linked list (and why the Rules of Hooks are not stylistic advice but a correctness invariant), dissect `useEffect`'s mount/update/unmount lifecycle, trace `Suspense`'s throw-Promise control flow including selective hydration and cache, and finally build an intuition for concurrent rendering via `useTransition`, `useDeferredValue`, and `startTransition`. Every mechanism is framed through the lens a backend engineer already has: fiber as a unit of work in a cooperative scheduler, lanes as priority queues, `Suspense` as a circuit breaker, and hydration as consistent replication.

Learning goals:

- Explain the Fiber node layout — `tag`, `key`, `type`, `stateNode`, `child` / `sibling` / `return`, `pendingProps` / `memoizedProps`, `memoizedState`, `updateQueue`, `flags` / `subtreeFlags`, `lanes` / `childLanes`, `alternate` — and what the reconciler reads from each field.
- Describe double buffering: how `current` and `workInProgress` trees relate via `alternate`, why React mutates only the work-in-progress tree, and when the trees are swapped at commit.
- Explain the lane model — bitmask lanes as concurrent priority queues — and how React maps event types to lanes, merges lanes, and starves or entangles them.
- Follow the reconciler loop: `scheduleUpdateOnFiber` → `ensureRootIsScheduled` → `performConcurrentWorkOnRoot` → depth-first `beginWork` / `completeWork` walk, keyed vs. unkeyed child reconciliation, and flag bubbling via `subtreeFlags`.
- Trace Hook dispatch end-to-end: `dispatchSetState` → `enqueueUpdate` → `scheduleUpdateOnFiber`, and contrast `useState` / `useReducer` update queues with `useEffect`'s three-phase lifecycle.
- Justify the Rules of Hooks from the call-order array invariant, with a concrete failure trace when a Hook is called conditionally.
- Explain `Suspense`'s throw-Promise protocol, fallback rendering, selective hydration on the server, and the `cache` / `fetch` integration that makes `Suspense` data-fetching work.
- Compare `Suspense` waterfall vs. parallel fetch patterns and show how to collapse waterfalls with concurrent preloading.

- Use concurrent features — `startTransition`, `useTransition`, `useDeferredValue`, and concurrent rendering — correctly, including lane downgrading and interruptible renders.
 - Analyze React's client-side engine as a distributed system: cooperative scheduling, priority inversion, replication (hydration), and eventual consistency between server HTML and client state.
-

1. Why a Reconciler Exists

React's public API promises a pure function from state to UI: `UI = f(state)`. The reconciler is the engine that makes that equation performant. Without it, every state change would destroy and recreate the entire DOM — correct but unusable beyond a todo list.

Two constraints forced the current architecture:

1. **The main thread is shared.** JavaScript execution, layout, paint, and input handling contend on a single thread (see Chapter 2 — Event Loop). A synchronous recursive tree walk that blocks for 200 ms drops frames and makes users feel broken.
2. **The declaration is not the diff.** JSX describes the desired tree, not the minimal DOM mutation. The reconciler must compare the new description against the previous output and emit the smallest set of insertions, moves, updates, and deletions.

React's answer is **Fiber**: a reimplement of the reconciler (landed in React 16, refined continuously through React 18 and 19) that turns the component tree into an interruptible, priority-aware work queue.

Backend analogy: if the old stack reconciler was a batch job that ran to completion on a single machine, Fiber is a cooperative scheduler with preemption, priority queues, and incremental checkpointing — closer to a work-stealing executor than a simple recursive function.

1.1 What Fiber replaced

Concern	Stack reconciler (pre-16)	Fiber reconciler (16+)
Data structure	Call stack — recursion depth equals tree depth	Heap-allocated fiber nodes linked via <code>child / sibling / return</code>
Interruptibility	None — <code>render()</code> ran synchronously to completion	Yes — work is chunked into units, yields to browser via scheduler
Priority	None — every update is equally urgent	Lanes — discrete priority bitmasks per update
Error handling	Unwind the call stack, unrecoverable	Error boundaries as fiber nodes with <code>DidCapture</code> flag
Suspense / concurrency	Not possible — requires suspension mid-tree	Throw-Promise protocol + offscreen fibers + lane downgrading

2. The Fiber Node — React's Unit of Work

A **fiber** is a JavaScript object that represents one unit of work: a component instance, a DOM node, or a bookkeeping node (root, fragment, suspense boundary, offscreen). The tree of fibers *is* the reconciler's state. You can inspect it at runtime via `__REACT_DEVTOOLS_GLOBAL_HOOK__` or by logging `fiber` inside a custom reconciler.

2.1 Fiber node layout

Every fiber carries the same shape (simplified from `ReactFiber.js` in the React source). The fields below are not implementation trivia — each one is read on every render path.

```

// Simplified fiber shape – mirrors ReactFiber.js (React 18/19)
type Fiber = {
  // Identity
  tag: WorkTag; // FunctionComponent | ClassComponent |
  HostComponent | SuspenseComponent ...
  key: string |
  null; // reconciler key – stable identity across renders
  elementType: any; // type from JSX before resolution (e.g.,
  MyComponent)
  type: any; // resolved type after resolution
  stateNode: any; // instance: DOM node, class instance, or
  fiber root

  // Tree structure – singly-linked tree, not a nested object
  return: Fiber | null; // parent
  child: Fiber | null; // first child
  sibling: Fiber | null; // next sibling
  index: number; // position among siblings

  // Props and state
  pendingProps: any; // props for the upcoming render
  memoizedProps: any; // props from the last committed render
  memoizedState: any; // hooks linked list head, or class state
  updateQueue: any; // queue of pending state/effect updates
  dependencies: any; // context dependencies collected during
  render

  // Scheduling and effects
  flags: Flags; // Placement | Update | ChildDeletion |
  Snapshot ...
  subtreeFlags: Flags; // bubbled union of child flags – skip
  optimization
  deletions: Fiber[] | null; // children to delete
  lanes: Lanes; // priority of work on this fiber
  childLanes: Lanes; // union of child lanes – where to find
  pending work
  alternate: Fiber | null; // the other buffer (see section 2.2)
};

```

Fiber node — fields grouped by role

Identity

tag · key · elementType
· type · stateNode

Tree pointers

return · child · sibling ·
index

State

pendingProps ·
memoizedProps
memoizedState ·
updateQueue
dependencies

Scheduling + Effects
flags · subtreeFlags ·
deletions
lanes · childLanes ·

The three tree pointers are deliberate. React stores children as a **linked list**, not an array:

```
// JSX
<ul>
  <li key="a">A</li>
  <li key="b">B</li>
  <li key="c">C</li>
</ul>

// Fiber links (parent <ul>):
// ulFiber.child → li(a) —sibling→ li(b) —sibling→ li(c)
//                               return ←——— return ←——— return
```

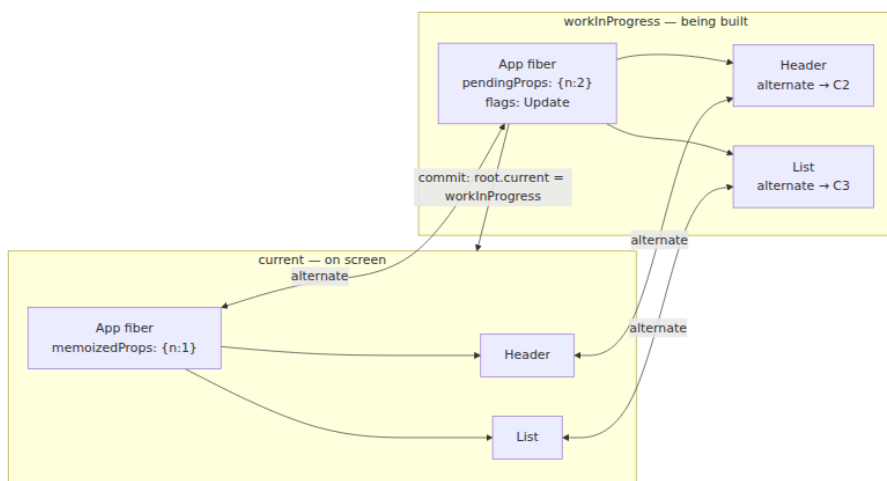
This lets the reconciler walk the tree iteratively without recursion, and splice children (insert, move, delete) by mutating `sibling` pointers — the same intrusive linked-list technique you would use for a lock-free work queue.

2.2 Double buffering — `current` vs. `workInProgress`

React maintains **two** fiber trees that point at each other via `alternate`:

- **`current`** — the tree that matches what is on screen. Its `stateNode` pointers are the live DOM nodes. It is immutable during a render.
- **`workInProgress`** (WIP) — the tree being built. Created by cloning `current` fibers (via `createWorkInProgress`), mutated freely, and either committed or discarded.

This is **double buffering** in the graphics sense, or **MVCC** in the database sense: readers (the commit phase, the browser paint) see a consistent snapshot (`current`) while writers (the render phase) build the next version (`workInProgress`) off to the side. Only at commit does React swap the pointers.



Lifecycle of one update:

```
// 1. An update is scheduled (e.g., setState, event handler,
transition)
dispatchSetState(fiber, action); // → enqueueUpdate →
scheduleUpdateOnFiber

// 2. React clones the current fiber to create/update the WIP fiber
let wip = fiber.alternate;
if (wip === null) {
  wip = createWorkInProgress(fiber, fiber.pendingProps);
} else {
  wip.pendingProps = fiber.pendingProps;
  wip.flags = NoFlags; // reset effect flags
}

// 3. Render phase mutates only WIP
// 4a. If render completes: root.current = finishedWork (atomic pointer
swap)
// 4b. If render suspends or is interrupted: discard WIP, keep current
intact
```

Key invariants for backend engineers:

- **WIP is not visible until commit.** There is no torn read — the user never sees a half-rendered tree. This is the same guarantee shadow paging gives a database.
- **WIP can be discarded.** If a higher-priority update arrives mid-render, React can abandon the in-progress WIP and start a new one

from `current`. Work is speculative and cheap to redo because the input (props + state) is pure.

- **Only one WIP tree exists at a time per root.** Concurrent features do not mean parallel fiber-tree mutation. They mean the scheduler can interleave which *lanes* get rendered.

2.3 Lanes — a bitmask priority model

React 17 had `expirationTime` (a single integer deadline). React 18 replaced it with **lanes**: a 31-bit bitmask where each bit is a priority lane. The lane model supports overlapping priorities (a single update can belong to multiple lanes) and efficient set operations via bitwise ops.

```
// ReactFiberLane.js (simplified – React 18/19)
type Lanes = number; // 31-bit bitmask
type Lane = number; // single bit

const SyncLane: Lane =
  0b00000000000000000000000000000001; // discrete, immediate (click,
input)
const InputContinuousLane: Lane =
  0b00000000000000000000000000000100; // drag, scroll
const DefaultLane: Lane =
  0b0000000000000000000000000100000; // normal setState, data fetch
completion
const TransitionLane1: Lane =
  0b000000000000000000000001000000; // useTransition – lowest,
interruptible
const TransitionLane2: Lane = 0b000000000000000000000100000000;
const RetryLane1: Lane =
  0b000000000000000100000000000000; // Suspense retry
const IdleLane: Lane =
  0b010000000000000000000000000000; // prefetch, offscreen
const OffscreenLane: Lane = 0b100000000000000000000000000000;
```

Operations that matter:

```

// Scheduling – map an event to a lane
function requestUpdateLane(fiber) {
  if (isTransition) return claimNextTransitionLane(); // pick next
  TransitionLane
  if (isSuspenseRetry) return RetryLane1;
  if (isUserBlockingEvent) return InputContinuousLane;
  return DefaultLane;
}

// Merging – a root tracks pending work as a bitmask union
root.pendingLanes |= updateLane;
fiber.lanes = mergeLanes(fiber.lanes, updateLane);
fiber.return.childLanes = mergeLanes(fiber.return.childLanes, updateLane); // bubble up

// Picking – scheduler always renders the highest-priority pending lane
const nextLane = getNextLanes(root, root.pendingLanes); // highest set
bit
const isSync = (nextLane & SyncLane) !== NoLanes;

// Starvation avoidance – if a lane has been pending too long, expire
it
if (hasExpired(lane, currentTime)) lane = SyncLane; // force sync

```

Distributed-systems framing: lanes are **weighted fair queuing** with expiration. Every fiber advertises which lanes have pending work (`lanes` for itself, `childLanes` as a bubbled summary — a Merkle-tree-like summary that lets the scheduler skip subtrees with no relevant work). A `DefaultLane` update never starves a `SyncLane` update; conversely, a burst of `SyncLane` updates can starve a `TransitionLane` render, but expiration eventually promotes the transition to prevent indefinite deferral. The `childLanes` summary is what makes `shouldComponentUpdate` / `React.memo` / lane skipping efficient: if `fiber.childLanes & renderLanes === 0`, the entire subtree is skipped.

3. The Reconciler Loop — `beginWork`, `completeWork`, and Commit

The reconciler's hot loop is a **depth-first walk** over the WIP tree that alternates between descending (`beginWork`) and ascending (`completeWork`), followed by a synchronous commit that mutates the DOM. The render phase (the walk) is interruptible; the commit phase is not.

3.1 Scheduling an update

Everything starts at `scheduleUpdateOnFiber`:

```
function scheduleUpdateOnFiber(fiber, lane, eventTime) {  
  // 1. Mark the fiber and its ancestors with the lane  
  let node = fiber;  
  let root = null;  
  while (node !== null) {  
    node.lanes = mergeLanes(node.lanes, lane);  
    if (node.alternate !== null) node.alternate.lanes =  
mergeLanes(node.alternate.lanes, lane);  
    if (node.return === null) { root = node.stateNode; break; } //  
HostRoot  
    // Also bubble to childLanes of the parent  
    node.return.childLanes = mergeLanes(node.return.childLanes, lane);  
    node = node.return;  
  }  
  
  // 2. Mark the root pending  
  markRootUpdated(root, lane, eventTime);  
  
  // 3. Ensure the root is scheduled  
  ensureRootIsScheduled(root, eventTime);  
  // → scheduler callback: performConcurrentWorkOnRoot or  
performSyncWorkOnRoot  
}
```

`ensureRootIsScheduled` is where lanes become scheduling decisions:

```

function ensureRootIsScheduled(root, eventTime) {
  const nextLanes = getNextLanes(root, NoLanes);
  if (nextLanes === NoLanes) return; // nothing to do

  const newCallbackPriority = getHighestPriorityLane(nextLanes);
  const existingCallbackPriority = root.callbackPriority;

  if (existingCallbackPriority === newCallbackPriority) return; // already scheduled

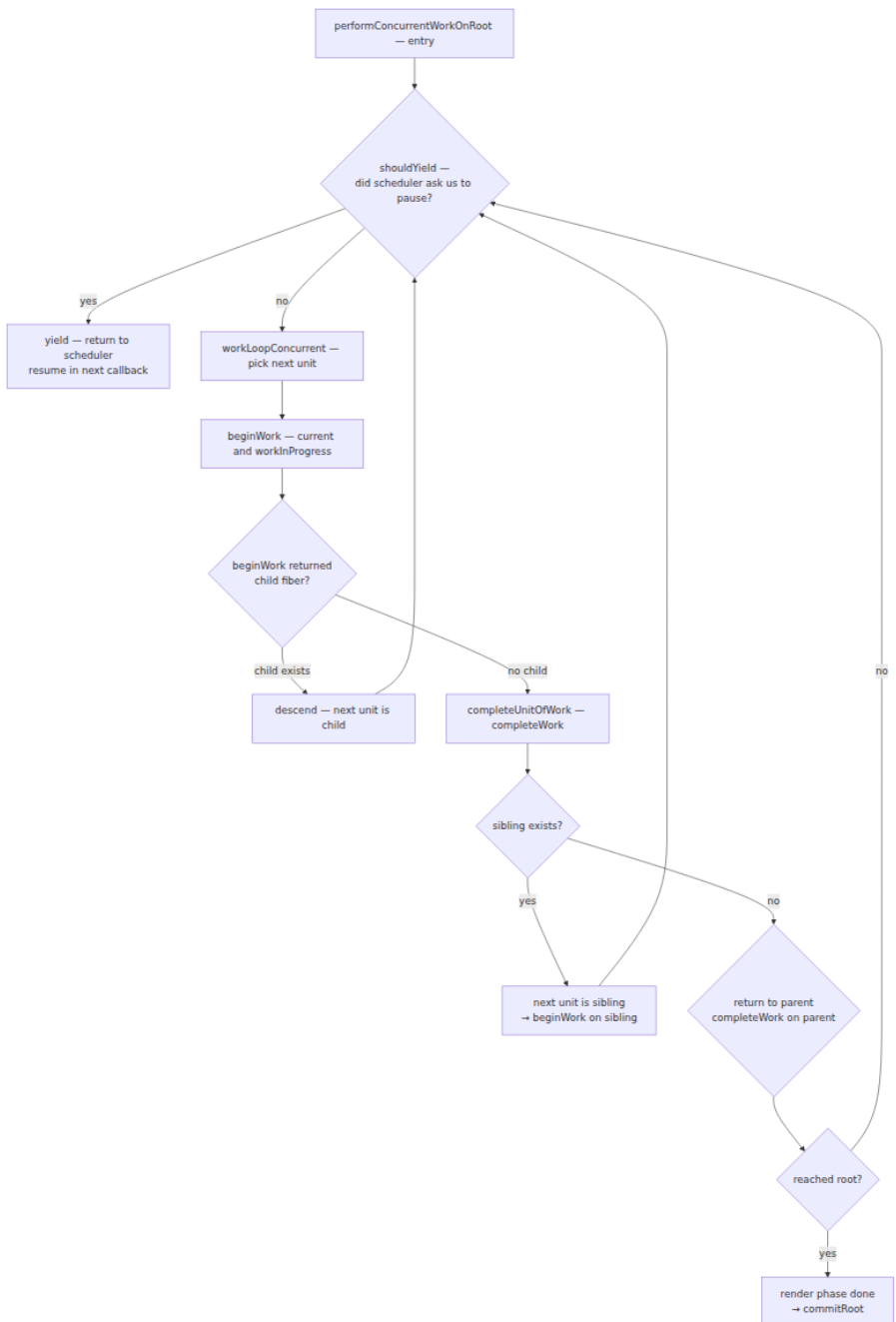
  if (existingCallbackNode !== null) cancelCallback(existingCallbackNode);

  let newCallbackNode;
  if (newCallbackPriority === SyncLane) {
    // Sync work – flush synchronously after microtasks, or in next tick
    scheduleSyncCallback(performSyncWorkOnRoot.bind(null, root));
  } else {
    // Concurrent work – schedule via Scheduler package (cooperative)
    newCallbackNode = scheduleCallback(
      schedulerPriorityToLanePriority(newCallbackPriority),
      performConcurrentWorkOnRoot.bind(null, root)
    );
  }
  root.callbackNode = newCallbackNode;
  root.callbackPriority = newCallbackPriority;
}

```

3.2 The work loop — an iterative depth-first walk

React does not recurse. It walks fibers iteratively using `child / sibling / return`, yielding to the browser scheduler when its time slice expires.



Concrete walk for a small tree — note how the reconciler visits each fiber **twice** (once going down, once coming up), exactly like a tree traversal that builds and then finalizes:

```
// Tree:
//   App
//   └─ Header
//       └─ Title
//       └─ List
//           └─ Item(a)
//           └─ Item(b)

// Walk order (each line is one unit of work):
// 1. beginWork(App)           → returns Header (child)
// 2. beginWork(Header)        → returns Title
// 3. beginWork(Title)          → returns null (leaf, HostComponent)
// 4. completeWork(Title)      → creates/updates DOM text, bubbles flags
//    to Header
// 5. completeWork(Header)     → bubbles subtreeFlags
// 6. beginWork(List)           → reconcileChildren → Item(a), Item(b)
// 7. beginWork(Item a)         → ...
// 8. completeWork(Item a)
// 9. beginWork(Item b)
// 10. completeWork(Item b)
// 11. completeWork(List)
// 12. completeWork(App)       → finishedWork = App WIP; commit
```

```

// Simplified work loop (ReactFiberWorkLoop.js)
function workLoopConcurrent() {
  while (workInProgress !== null && !shouldYield()) {
    performUnitOfWork(workInProgress);
  }
}

function performUnitOfWork(unitOfWork) {
  const current = unitOfWork.alternate;
  let next = beginWork(current, unitOfWork, renderLanes);
  unitOfWork.memoizedProps = unitOfWork.pendingProps;

  if (next === null) {
    next = completeUnitOfWork(unitOfWork);
  }
  workInProgress = next;
}

function completeUnitOfWork(unitOfWork) {
  let completedWork = unitOfWork;
  do {
    const current = completedWork.alternate;
    const returnFiber = completedWork.return;

    // 1. Finalize this fiber (create DOM, compute flags, diff props)
    completeWork(current, completedWork, renderLanes);

    // 2. Bubble flags and lanes to parent
    if (returnFiber !== null) {
      bubbleProperties(completedWork, returnFiber);
    }

    // 3. Try sibling; otherwise walk up
    const siblingFiber = completedWork.sibling;
    if (siblingFiber !== null) return siblingFiber;
    completedWork = returnFiber;
  } while (completedWork !== null);
  return null; // reached root
}

```

3.3 `beginWork` — reconcile this fiber's children

`beginWork` is a giant switch on `workInProgress.tag`. Its job is to **reconcile children**: compare the new React elements returned by the component with the previous child fibers, and produce the WIP child list.


```

function beginWork(current, workInProgress, renderLanes) {
  // Bailout fast path – props and lanes unchanged, skip subtree
  if (
    current !== null &&
    workInProgress.pendingProps === current.memoizedProps &&
    (current.lanes & renderLanes) === NoLanes &&
    (workInProgress.childLanes & renderLanes) === NoLanes
  ) {
    return bailoutOnAlreadyFinishedWork(current, workInProgress, renderLanes);
  }

  switch (workInProgress.tag) {
    case HostRoot: return updateHostRoot(current, workInProgress, renderLanes);
    case HostComponent: return updateHostComponent(current, workInProgress, renderLanes);
    case FunctionComponent: return updateFunctionComponent(current, workInProgress, renderLanes);
    case ClassComponent: return updateClassComponent(current, workInProgress, renderLanes);
    case SuspenseComponent: return updateSuspenseComponent(current, workInProgress, renderLanes);
    case Fragment: return updateFragment(current, workInProgress, renderLanes);
    case ContextProvider: return updateContextProvider(current, workInProgress, renderLanes);
    // ... ~30 tags
  }
}

```

For function components, `updateFunctionComponent` renders the component (calls it) and reconciles the returned elements:

```

function updateFunctionComponent(current, workInProgress, renderLanes) {
  {
    const nextChildren = renderWithHooks(current, workInProgress, Component, props, renderLanes);
    // renderWithHooks sets up the Hooks dispatcher, calls Component(props), collects hooks

    reconcileChildren(current, workInProgress, nextChildren, renderLanes);
  }
  return workInProgress.child;
}

```

`reconcileChildren` dispatches to one of two paths:

```
function reconcileChildren(current, workInProgress, nextChildren,
renderLanes) {
  if (current === null) {
    // Mount – no previous children, just create fibers
    workInProgress.child = mountChildFibers(workInProgress, null, nextC
hildren, renderLanes);
  } else {
    // Update – diff against current child list
    workInProgress.child = reconcileChildFibers(workInProgress,
current.child, nextChildren, renderLanes);
  }
}
```

3.4 Diffing — keyed vs. unkeyed reconciliation

The diff is where React earns its performance. Given the old child fibers and the new React elements, the reconciler must produce the minimal WIP child list with flags (Placement, Update, Deletion) that tell the commit phase what DOM mutations to perform.

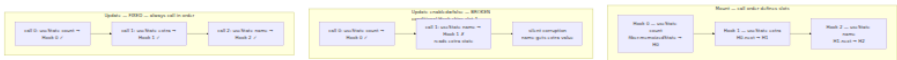
Unkeyed (index-based) reconciliation — default when no `key` is provided. React matches children by position:

```
// Unkeyed: position is identity
// Previous: [<li>A</li>, <li>B</li>, <li>C</li>]
// Next:      [<li>A</li>, <li>C</li>]
// Diff by index: index 0 matches, index 1: B vs C → update B to C,
index 2: delete C
// Result: B's DOM node is mutated to show "C", C's DOM node is
removed.
// If B held state (input, animation), that state incorrectly survives
on "C".
```

Keyed reconciliation — when `key` is provided, React builds a `Map<key, oldFiber>` and matches by key, detecting moves:

```
// Keyed: key is identity
// Previous: [<li key="a">A</li>, <li key="b">B</li>, <li key="c">C</li>]
// Next:      [<li key="b">B</li>, <li key="a">A</li>, <li key="c">C</li>]
// Diff by key: all three keys found → reorder via Placement flag
//              (move, not recreate)
// Result: DOM nodes for a/b are moved, state follows the key.

// reconcileChildFibers fast paths (ReactChildFiber.js):
// 1. Single element → reconcileSingleElement      (key + type
// check)
// 2. Single text node → reconcileSingleTextNode
// 3. Array/iterable → reconcileChildrenArray      (the main diff)
//    3a. Fast path: walk both lists in lockstep while keys+types match
//    3b. On mismatch: build Map of remaining old fibers by key, then
// match
//    3c. Remaining new elements → Placement; remaining old fibers →
// Deletion
```



Concrete demo — an instrumented reconciler trace for a reorder:

```

function Item({ value }) {
  // Each Item has local state (e.g., input), tied to fiber identity
  const [text, setText] = React.useState(value);
  return <li><input value={text} onChange={e => setText(e.target.value)} /> </li>;
}

function List({ items }) {
  // With keys: React moves fibers; input state follows the key
  return <ul>{items.map(it => <Item key={it.id} value={it.label} />)}</ul>;
}

// Initial: items = [{id:'a',label:'Alpha'}, {id:'b',label:'Beta'}, {id:'c',label:'Gamma'}]
// User types "ALPHA!" into first input → Item(a) state: "ALPHA!"

// Reorder: items = [{id:'b',...}, {id:'a',...}, {id:'c',...}]
// Trace (reconcileChildrenArray):
//   oldFiber(a) key=a → Map { a→fiberA, b→fiberB, c→fiberC }
//   new element key=b → Map hit: fiberB → reuse, Placement=5
//   (move), old index 1 → new index 0
//   new element key=a → Map hit: fiberA → reuse, Placement=5
//   (move), old index 0 → new index 1
//   new element key=c → Map hit: fiberC → reuse, no move (old index 2 → new index 2, after moves)
//   commit: DOM nodes for a and b are moved, not recreated; input "ALPHA!" stays with key "a"
// Without keys: React would patch by index – the input containing "ALPHA!" would now display "Beta"

```

A backend engineer's takeaway: keyed reconciliation is **consistent hashing for UI state**. The `key` is the stable identity that lets React distinguish "the same entity in a new position" from "a new entity." Choosing `key={index}` is like using array offset as a cache key — it works until the list mutates, then it silently serves stale state.

3.5 `completeWork` and `subtreeFlags` bubbling

`completeWork` finalizes a fiber after its children are done. For host components it diffs props and creates or updates DOM nodes; for composite components it does bookkeeping.

The critical optimization is **flag bubbling**. Each fiber has `flags` (effects on itself) and `subtreeFlags` (union of its entire subtree's flags). When `completeWork` finishes, it bubbles to the parent:

```

function bubbleProperties(completedWork, returnFiber) {
  let newChildLanes = NoLanes;
  let subtreeFlags = NoFlags;

  let child = completedWork.child;
  while (child !== null) {
    newChildLanes = mergeLanes(newChildLanes, mergeLanes(child.lanes, child.childLanes));
    subtreeFlags |= child.subtreeFlags;
    subtreeFlags |= child.flags;
    child = child.sibling;
  }

  completedWork.childLanes = newChildLanes;
  completedWork.subtreeFlags = subtreeFlags;

  // Also bubble to parent's flags without traversing again at commit
  returnFiber.subtreeFlags |= subtreeFlags;
  returnFiber.childLanes = mergeLanes(returnFiber.childLanes, newChildLanes);
}

```

At commit, React walks only fibers where `subtreeFlags & MutationMask !== 0`, skipping entire subtrees that had no DOM side effects. In a 10,000-node tree where only one leaf changed, the commit walk visits `0(depth + changed_subtree)` fibers rather than the full tree. This is the same pruning that makes Merkle-tree verification efficient: a summary hash at each interior node lets you skip unchanged subtrees.

3.6 The commit phase — not interruptible

Once `performConcurrentWorkOnRoot` produces a `finishedWork` tree, `commitRoot` runs synchronously in three sub-phases:

1. **Before mutation** — `getSnapshotBeforeUpdate`, focus management.
2. **Mutation** — apply `Placement / Update / Deletion` to the DOM (insert, move, remove, set props). This is where `fiber.stateNode` (the DOM node) is actually mutated.
3. **Layout** — `componentDidMount / componentDidUpdate`, `useLayoutEffect` callbacks (fire synchronously after DOM mutation, before paint), ref attachment.

```

function commitRoot(root, finishedWork) {
  const prevExecutionContext = executionContext;
  executionContext |= CommitContext;

  // 1. Before mutation
  commitBeforeMutationEffects(root, finishedWork);

  // 2. Mutation – the only phase that touches the DOM
  commitMutationEffects(root, finishedWork);

  // 3. Swap trees – the atomic commit point
  root.current = finishedWork;

  // 4. Layout effects (useLayoutEffect, componentDidMount)
  commitLayoutEffects(finishedWork, root, lanes);

  // 5. Passive effects (useEffect) – scheduled, not synchronous
  schedulePassiveEffects(finishedWork);

  executionContext = prevExecutionContext;
}

```

The pointer swap `root.current = finishedWork` is the linearization point. Before it, the world sees `current`; after it, the world sees `finishedWork`. The mutation phase has already made the DOM match `finishedWork`, so there is no window where `root.current` and the DOM disagree.

4. Hooks — State as a Call-Order-Indexed Linked List

Hooks let function components hold state without classes. Underneath, they are a **linked list of hook objects attached to the fiber**, indexed by call order. This design explains both how Hooks work and why their rules are non-negotiable.

4.1 The hook linked list

Each function-component fiber's `memoizedState` points to the head of a singly-linked list of `Hook` objects. Each hook holds its own `memoizedState`, `queue`, and `next` pointer.

```

type Hook = {
  memoizedState:
any; // state for useState/useReducer, effect object for useEffect,
etc.
  baseState: any; // base state for reducer coalescing
  baseQueue: Update<any> | null;
  queue: any; // update queue (for useState/useReducer) or
effect queue
  next: Hook | null; // next hook in this component
};

```

On every render, React walks this list in lockstep with the component's hook calls. The first `useState` call reads/writes `hook[0]`, the second reads/writes `hook[1]`, and so on.

```

// How React tracks hooks during render (ReactFiberHooks.js,
simplified)
let currentlyRenderingFiber = null;
let workInProgressHook = null; // tail of new list being built
let currentHook = null; // pointer into old list (from current
fiber)

function updateWorkInProgressHook() {
  // Called by each Hook on re-render to get its Hook object
  if (workInProgressHook === null) {
    // First hook – clone from current's first hook
    currentlyRenderingFiber.memoizedState = workInProgressHook =
      createWorkInProgressHook(currentHook);
  } else {
    // Subsequent hooks – append to new list
    workInProgressHook = workInProgressHook.next =
      createWorkInProgressHook(currentHook.next);
  }
  currentHook = currentHook.next;
  return workInProgressHook;
}

function mountWorkInProgressHook() {
  // Called by each Hook on mount – no current list, create fresh
  const hook = { memoizedState: null, queue: null, next: null, baseStat
e: null, baseQueue: null };
  if (workInProgressHook === null) {
    currentlyRenderingFiber.memoizedState = workInProgressHook = hook;
  } else {
    workInProgressHook = workInProgressHook.next = hook;
  }
  return workInProgressHook;
}

```

4.2 `useState` / `useReducer` dispatch trace

`useState` is sugar over `useReducer` with a basic-state reducer (`(s, a) => typeof a === 'function' ? a(s) : a`). The dispatch path is identical for both.

End-to-end trace of `setCount(c => c + 1)`:


```

// Component
function Counter() {
  const [count, setCount] = React.useState(0);
  return <button onClick={() => setCount(c => c + 1)}>{count}
</button>;
}

// 1. dispatchSetState is the setter returned by useState
function dispatchSetState(fiber, queue, action) {
  const lane = requestUpdateLane(fiber); // DefaultLane for normal
setState
  const update = {
    lane,
    action, // the updater: c => c + 1
    hasEagerState: false,
    eagerState: null,
    next: null,
  };

  // 2. Eager-state optimization – compute next state synchronously if
possible
  // If next state === current state (Object.is), bail out without
scheduling
  const currentState = queue.lastRenderedState;
  const eagerState = queue.lastRenderedReducer(currentState, action);
  update.hasEagerState = true;
  update.eagerState = eagerState;
  if (Object.is(eagerState, currentState)) return; // no-op, skip
render

  // 3. Enqueue into the hook's circular update list
  const pending = queue.pending;
  if (pending === null) {
    update.next = update; // first update points to itself (circular)
  } else {
    update.next = pending.next;
    pending.next = update;
  }
  queue.pending = update;

  // 4. Schedule – this is where lanes and the scheduler enter
  const root = enqueueConcurrentHookUpdate(fiber, queue, update, lane);
  scheduleUpdateOnFiber(fiber, lane, eventTime);
  // → ensureRootIsScheduled → performConcurrentWorkOnRoot
}

// 5. During next render, the reconciler replays the queue
function updateReducer(reducer, initialArg) {
  const hook = updateWorkInProgressHook();
  const queue = hook.queue;

```

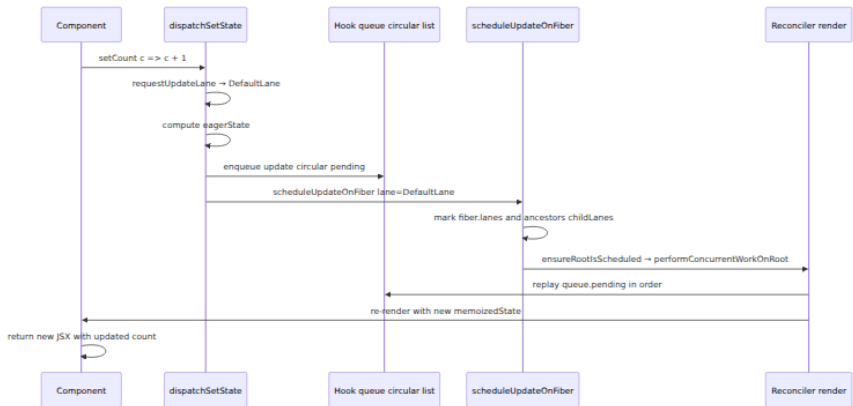
```

let baseState = hook.baseState;
let first = queue.pending;

if (first !== null) {
  // Replay all pending updates in order
  let newState = baseState;
  let update = first.next;
  do {
    const action = update.action;
    newState = update.hasEagerState
      ? update.eagerState
      : reducer(newState, action);
    update = update.next;
  } while (update !== first.next);

  hook.memoizedState = newState;
  hook.baseState = newState;
  queue.pending = null;
  queue.lastRenderedState = newState;
}
return [hook.memoizedState, queue.dispatch];
}

```



Batched updates: multiple `setState` calls inside the same event handler are coalesced into one render. Before React 18, batching only happened inside React event handlers; since React 18, **all** updates — including `setTimeout`, promises, and native event handlers — are batched via `batchedUpdates` and the lane system. The circular queue naturally handles this: N dispatches enqueue N updates, but only one `scheduleUpdateOnFiber` triggers one render that replays all N updates in order.

4.3 `useEffect` — `mount`, `update`, `unmount`

Effects are not part of the render phase. They are **registered** during render and **executed** after commit. This separation is what keeps rendering pure and interruptible.

Each `useEffect` call appends an effect object to `fiber.updateQueue`:

```
type Effect = {
  tag: HookFlags;           // HasEffect | Layout | Passive
  create: () => (() => void) | void; // setup function
  destroy: (() => void) | void;    // cleanup from previous effect
  deps: any[] | null;           // dependency array
  next: Effect;                // circular list
};
```

Three-phase lifecycle:

```
function Counter({ id }) {
  const [count, setCount] = React.useState(0);

  React.useEffect(() => {
    // Phase 1 – mount (or deps changed): create
    console.log(`subscribe ${id} count=${count}`);
    const sub = subscribe(id, count);

    // Phase 2 – cleanup on next effect run or unmount: destroy
    return () => {
      console.log(`unsubscribe ${id} count=${count}`);
      sub.unsubscribe();
    };
  }, [id, count]); // deps array – compared via Object.is per slot

  return <div>{count}</div>;
}

// Timeline:
// Mount:   render → commit (DOM updated) → passive effect flush → create()
// Update (id same, count 0→1): render → commit → flush: destroy(old) → create(new)
// Unmount: commit (Deletion) → flush: destroy()
// Deps unchanged: render → commit → flush: nothing (effect skipped, tag has no HasEffect)
```

How `areHookInputsEqual` decides to skip:

```
function areHookInputsEqual(nextDeps, prevDeps) {
  if (prevDeps === null) return false;
  for (let i = 0; i < prevDeps.length && i < nextDeps.length; i++) {
    if (!Object.is(nextDeps[i], prevDeps[i])) return false;
  }
  return true;
}
// On mount: prevDeps is null → always HasEffect
// On update: Object.is compare per slot → if any differs, HasEffect;
// else skip
```

Effect timing — the distinction between `useLayoutEffect` and `useEffect`:

Hook	When it fires	Blocks paint?	Use for
<code>useLayoutEffect</code>	Synchronously after mutation, before browser paint (inside <code>commitLayoutEffects</code>)	Yes	DOM measurements, synchronous visual corrections
<code>useEffect</code>	Asynchronously after paint (flushed via <code>scheduleCallback</code> with <code>NormalPriority</code>)	No	Subscriptions, data fetching, non-visual side effects

```
// useLayoutEffect - e.g., measure then correct to avoid flicker
function Tooltip({ targetRef }) {
  const tooltipRef = React.useRef(null);
  const [pos, setPos] = React.useState({ top: 0, left: 0 });

  React.useLayoutEffect(() => {
    // Runs before paint – user never sees the wrong position
    const rect = targetRef.current.getBoundingClientRect();
    const tipRect = tooltipRef.current.getBoundingClientRect();
    setPos({ top: rect.bottom + 4, left: rect.left + (rect.width - tipR
ect.width) / 2 });
  }, [targetRef]);

  return <div ref={tooltipRef} style={pos}>tip</div>;
}
```

4.4 Rules of Hooks — the call-order invariant

The hook list is indexed by call order, not by name. On mount, hooks are appended; on update, they are consumed positionally. This makes the following rule a correctness requirement, not a style preference:

Do not call Hooks conditionally, inside loops, or after an early return. Always call them in the same order.

```
// BROKEN – conditional hook violates call-order invariant
function Broken({ enabled }) {
  const [count, setCount] = React.useState(0); // Hook 0 – always runs
  if (enabled) {
    const [extra, setExtra] = React.useState(0); // Hook 1 – only when
    enabled!
  }
  const [name, setName] = React.useState(""); // Hook 1 or 2
  depending on enabled
  // ...
}

// Trace:
// Mount with enabled=true: Hook0:count, Hook1:extra, Hook2:name →
// list [H0,H1,H2]
// Update with enabled=false: code calls useState(count) → H0 , then
// skips extra,
//                               then calls useState(name) → reads H1
// (which is extra's state!)
//                               → name's state is now extra's previous
// state – silent corruption
// Next update: extra's dispatcher called, but H1 is now name's slot –
// state cross-contamination

// CORRECT – always call hooks unconditionally, gate the *behavior*
function Fixed({ enabled }) {
  const [count, setCount] = React.useState(0);
  const [extra, setExtra] = React.useState(0);
  const [name, setName] = React.useState("");
  // Use extra only when enabled – the hook still exists, just unused
  React.useEffect(() => {
    if (!enabled) return;
    // subscribe with extra
  }, [enabled, extra]);
}
```

The static rule is enforced by `eslint-plugin-react-hooks`, which performs control-flow analysis to verify that every hook call dominates the function exit. The runtime invariant is enforced by React's dev-mode

dispatcher, which throws if the number of hooks on update differs from mount.

5. Suspense — Throwing Promises as Control Flow

Suspense inverts the usual data-fetching pattern. Instead of a component checking `if (loading) return <Spinner />`, it **throws a Promise**, and React catches it at the nearest `<Suspense>` boundary. The fallback is shown until the Promise resolves, at which point React retries rendering the suspended tree.

This is unusual — exceptions for control flow are generally discouraged — but it solves a real problem: data dependencies are often discovered **during** rendering, deep inside the tree. Throwing lets any descendant suspend without threading loading state through every intermediate component.

5.1 The throw-Promise protocol

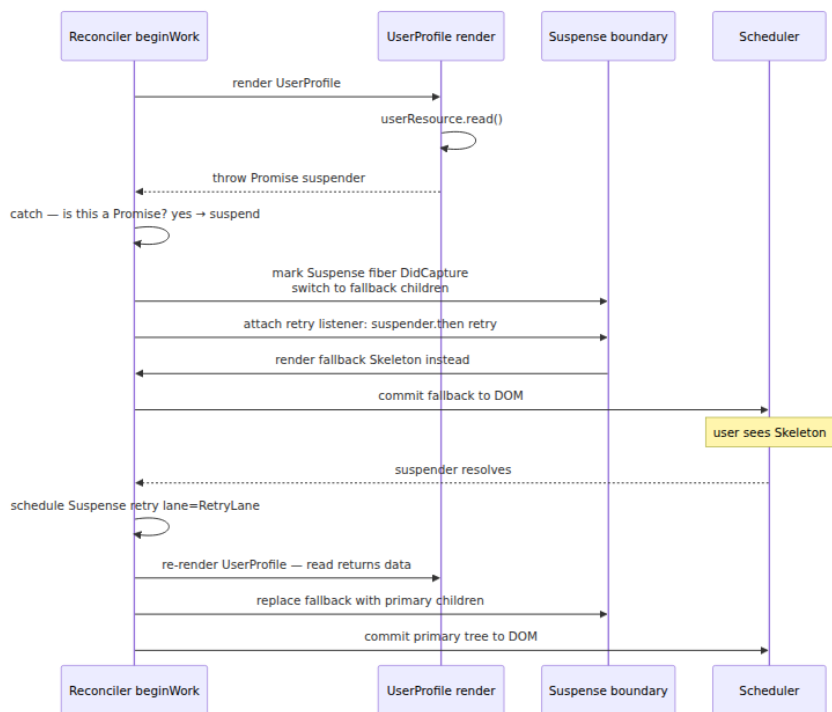
```
// A Suspense-compatible resource (the pattern React docs use for
illustration)
function createResource(promise) {
  let status = "pending";
  let result;
  const suspender = promise.then(
    r => { status = "success"; result = r; },
    e => { status = "error"; result = e; }
  );
  return {
    read() {
      if (status === "pending") throw suspender; // ← suspend: throw
      if (status === "error") throw result; // ← error boundary
      return result; // ← data available
    }
  };
}

// Usage
const userResource = createResource(fetch("/api/user/42").then(r => r.json()));

function UserProfile() {
  const user = userResource.read(); // throws Promise on first render
  return <h1>{user.name}</h1>;
}

function App() {
  return (
    <Suspense fallback={<Skeleton />}>
      <UserProfile />
    </Suspense>
  );
}
```

What happens frame-by-frame:



Internally (simplified from `ReactFiberThrow.js`):


```

function throwException(root, returnFiber, sourceFiber, value,
rootRenderLanes) {
  // value is the thrown Promise
  sourceFiber.flags |= Incomplete;
  sourceFiber.flags |= ShouldCapture;

  // Walk up to find the nearest Suspense boundary
  let workInProgress = returnFiber;
  while (workInProgress !== null) {
    if (workInProgress.tag === SuspenseComponent) {
      const retryLane = ClaimRetryLane();
      // Attach a listener that schedules a retry when the Promise
      settles
      attachSuspenseRetryListeners(workInProgress, value, retryLane);
      workInProgress.flags |= ShouldCapture;
      workInProgress.lanes = mergeLanes(workInProgress.lanes,
      retryLane);
      return createSuspenseFallbackChildren(workInProgress, value);
    }
    if (workInProgress.tag === HostRoot) break;
    workInProgress = workInProgress.return;
  }
  // No boundary found – treat as error (error boundary or fatal)
  throw value;
}

```

Critical detail for backend engineers: the thrown Promise is never caught by user `try/catch`. React intercepts it inside `invokeGuardedCallback` / the reconciler's `try` around `beginWork`. A component that wraps `resource.read()` in `try/catch` and swallows the Promise will break Suspense — `read()` must let the Promise propagate.

5.2 Suspense boundaries, fallback, and nested suspense

Each `<Suspense>` creates a fiber with two child sets: the **primary** children (what you wrote inside) and the **fallback** children. Only one set is visible at a time. Nested boundaries isolate suspension — an inner suspension shows only the inner fallback, not every ancestor's fallback.

```
function App() {
  return (
    <Suspense fallback={<PageSkeleton />>
      <Header /> { /* never suspends – always visible */}
      <Suspense fallback={<UserSkeleton />>
        <UserProfile /> { /* suspends until user fetch resolves */}
      </Suspense>
      <Suspense fallback={<PostsSkeleton />>
        <Posts /> { /* suspends until posts fetch resolves */}
      </Suspense>
    </Suspense>
  );
}
// If UserProfile suspends but Posts does not, only UserSkeleton shows;
// Posts renders immediately. Boundaries are independent commit units.
```

The fiber layout for one boundary:

```
// Suspense fiber children (when suspended):
// SuspenseFiber
//   └─ OffscreenFiber (primary – hidden, dehydrated)
//       └─ UserProfile (suspended, marked Incomplete)
//   └─ Fragment (fallback – visible)
//       └─ UserSkeleton
```

Offscreen is the mechanism that keeps the primary tree mounted but hidden — its **visibility** semantics are reused for **Suspense** fallbacks, for **hidden** subtrees, and for the upcoming **Activity** API.

5.3 Suspense waterfall vs. parallel — the fetch coordination problem

The throw-Promise model makes it easy to write a **waterfall** by accident: each component fetches sequentially because the next component does not even render until the previous suspension resolves.

```

// WATERFALL – sequential, total time = sum of latencies
function WaterfallApp() {
  return (
    <Suspense fallback={<Skeleton />}>
      <UserProfile />    {/* fetch user – suspends 300ms */}
      <UserPosts />     {/* not rendered until UserProfile resolves –
then fetches posts 300ms */}
    </Suspense>
  );
  // Timeline: |--- user 300ms ---|--- posts 300ms ---| = 600ms
  // Network:  —fetch user—————fetch posts—————
}

// PARALLEL – concurrent, total time = max of latencies
// Hoist fetches above Suspense so they start before rendering
function ParallelApp() {
  return (
    <Suspense fallback={<Skeleton />}>
      <UserProfileAndPosts />
    </Suspense>
  );
}

function UserProfileAndPosts() {
  // Both resources start fetching before either read() suspends
  const user = userResource.read(); // if pending, throw – but both
fetches already in flight
  const posts = postsResource.read(); // if this also suspends, React
waits for both
  return <><h1>{user.name}</h1><PostsList posts={posts} /></>;
}

// With parallel initiation:
// Timeline: |--- user 300ms ---|
//           |--- posts 300ms --| = 300ms (overlapped)
// Network:  —fetch user—
//           —fetch posts— (concurrent)

```

The fix is to **initiate fetches before rendering**, not inside `read()` on first call. Modern approaches:

```

// Pattern 1 – initiate at module scope / route loader
const userPromise = fetch("/api/user/42").then(r => r.json());
const postsPromise = fetch("/api/posts?user=42").then(r => r.json());
const userResource = createResource(userPromise);
const postsResource = createResource(postsPromise);

// Pattern 2 – initiate in parent, pass resource down
function App() {
  const [userRes, postsRes] = React.useMemo(() => {
    const u = fetchUser(42);
    const p = fetchPosts(42);
    return [wrapPromise(u), wrapPromise(p)]; // both start immediately
  }, []);
  return (
    <Suspense fallback={<Skeleton />}>
      <Profile userRes={userRes} postsRes={postsRes} />
    </Suspense>
  );
}

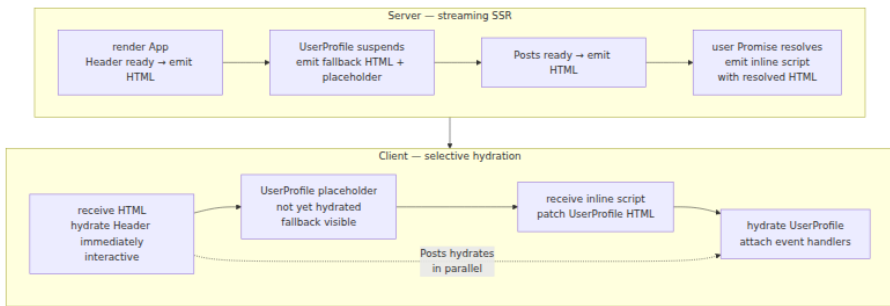
// Pattern 3 – framework-level (Next.js / Remix / TanStack) – route
// loader runs before component tree renders
// The framework initiates all data fetches for a route in parallel,
// then provides them via cache

```

For backend engineers: this is the `N+1 query` problem transposed to the client. The waterfall pattern is the frontend analog of a service that fetches a user, then for each user fetches posts in a loop. The fix is the same — batch or parallelize initiations, and use a cache (React `cache`, or a framework data layer) so repeated `read()` calls do not re-fetch.

5.4 Server Suspense, selective hydration, and `cache`

On the server (React 18+ with streaming SSR), Suspense boundaries are **streaming commit points**. The server renders to HTML, and each Suspense boundary that suspends emits its fallback HTML immediately, then streams the resolved content as an inline `<script>` that patches the HTML when the Promise settles. The client **selectively hydrates**: it can make a suspended subtree interactive before the rest of the page has hydrated.



Selective hydration semantics:

```
// Server
import { renderToPipeableStream } from "react-dom/server";

const stream = renderToPipeableStream(
  <App />,
  {
    onShellReady() { pipeToResponse(stream, response); }, // shell =
    onAllReady() { /* all Suspense boundaries resolved */ },
  }
);

// Client — hydration is per-boundary, not all-or-nothing
import { hydrateRoot } from "react-dom/client";

// React can hydrate high-priority Suspense boundaries first.
// If the user clicks inside a not-yet-hydrated boundary, React
// synchronously hydrates that boundary before handling the event
// (so the handler is available — no lost clicks).
```

The `cache` API (React 18+) complements Suspense by deduplicating fetches during a single server render pass:

```
import { cache } from "react";

// cache() memoizes per-render, not globally – like request-scoped
memoization
export const getUser = cache(async (id) => {
  const res = await fetch(`https://api.example.com/user/${id}`, {
    cache: "no-store" });
  return res.json();
});

// In any component during the same render, getUser(42) hits the same
Promise
// – no duplicate fetches even if 10 components request the same user.
// This is the server analog of DataLoader batching in GraphQL
backends.
function Avatar({ userId }) {
  const user = React.use(getUser(userId)); // React.use() unwraps the
Promise (React 19)
  return <img src={user.avatar} alt={user.name} />;
}
```

6. Concurrent Features — Interrupting Low-Priority Work

The Fiber reconciler can **interrupt** a render in progress, handle a higher-priority update, and then resume or restart the interrupted render. Concurrent features are the public API that exposes this capability: `startTransition`, `useTransition`, `useDeferredValue`, and concurrent rendering itself.

6.1 Concurrent rendering — the scheduler contract

Concurrent rendering means the render phase may run more than once for a single commit, may be interleaved with browser work, and may be abandoned. The contract with components is:

- **Render must be pure.** No side effects, no subscriptions, no DOM mutations during render. Effects belong in `useEffect` / `useLayoutEffect`, which run only after a successful commit.
- **Render may be called multiple times without committing.** An interrupted low-priority render's WIP tree is discarded; its side effects must not have leaked.

- **State updates inside a transition are interruptible; urgent updates are not.** Typing in an input (`SyncLane / InputContinuousLane`) preempts a transition render (`TransitionLane`).

```
// createRoot enables concurrent rendering (React 18+)
import { createRoot } from "react-dom/client";

// Legacy: ReactDOM.render(<App />, container) – synchronous, blocking
// Concurrent: createRoot enables time-slicing and interruption
const root = createRoot(document.getElementById("root"));
root.render(<App />);

// Concurrent root also enables Suspense, transitions, and selective
hydration
```

6.2 Lanes and transitions — priority downgrading

When an update is wrapped in `startTransition`, React assigns it a `TransitionLane` instead of `DefaultLane`. Transition lanes are lower priority, so the scheduler can interrupt them:

```
function SearchApp() {
  const [query, setQuery] = React.useState("");
  const [isPending, startTransition] = React.useTransition();

  function handleChange(e) {
    const next = e.target.value;
    setQuery(next); // urgent – DefaultLane / InputContinuousLane,
    // stays responsive

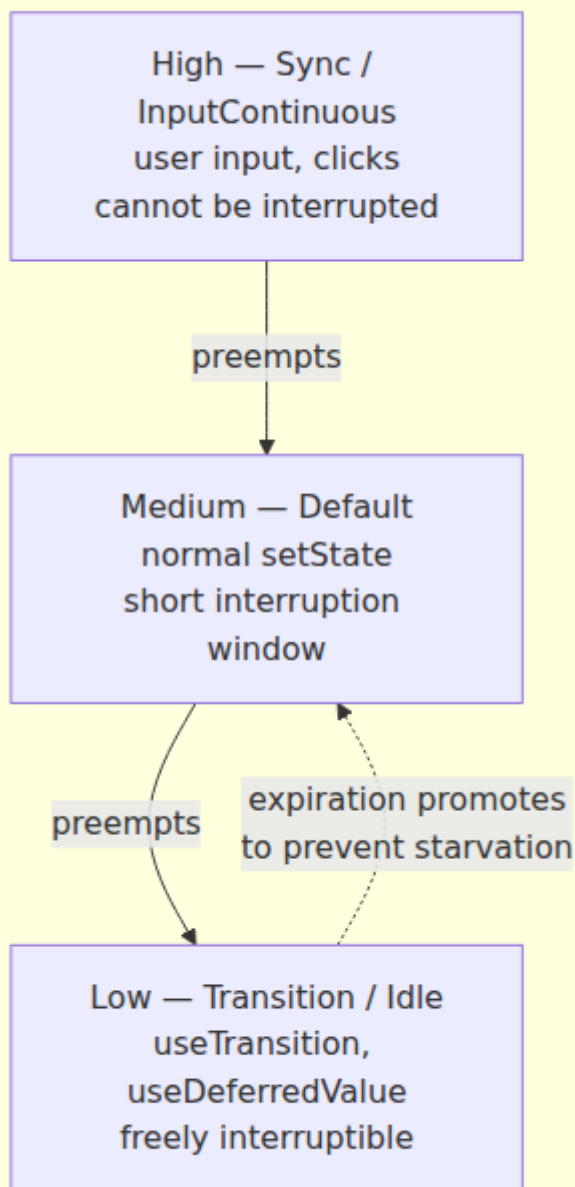
    startTransition(() => {
      // Transition – TransitionLane, interruptible
      setSearchQuery(next); // drives expensive <Results
    query={searchQuery} />
    });
  }

  return (
    <>
      <input value={query} onChange={handleChange} />
      {isPending && <Spinner />} {/* isPending mirrors whether
transition is in flight */}
      <Results query={searchQuery} />
    </>
  );
}
```

What happens when the user types while `Results` is mid-render:

```
// Timeline:
// t0: user types "a" → setQuery("a") on DefaultLane → high-priority
// render, input updates immediately
//           startTransition → setSearchQuery("a") on
// TransitionLane → low-priority render of Results starts
// t1: user types "ab" → setQuery("ab") on DefaultLane → RE-ENTERS
// scheduler at higher priority
//           → interrupt Results render for "a" (discard
// WIP)
//           → commit input "ab" immediately
//           → startTransition → setSearchQuery("ab") on
// new TransitionLane
//           → render Results for "ab" from scratch
// t2: no more input → Results for "ab" completes and commits
// User never sees a frozen input – keystrokes commit eagerly, results
// commit when ready
```


Lane priority and interruption



6.3 useDeferredValue — debouncing without timers

`useDeferredValue` is `useTransition` for a value rather than an update. It returns a deferred copy that lags behind the real value, rendered at `TransitionLane` priority:

```
function SearchWithDeferred() {
  const [query, setQuery] = React.useState("");
  const deferredQuery = React.useDeferredValue(query);
  //           urgent ┐           └ transition lane

  // query updates synchronously (typing stays responsive)
  // deferredQuery updates in a transition – expensive consumers read
  // deferredQuery
  return (
    <>
      <input value={query} onChange={e => setQuery(e.target.value)} />
      /* Results sees deferredQuery – its render is interruptible */
      <ExpensiveResults query={deferredQuery} />
      /* Optional: indicate staleness */
      {query !== deferredQuery && <div style={{ opacity: 0.5 }}><ExpensiveResults query={deferredQuery} /></div>}
    </>
  );
}

// How it works internally (simplified):
// 1. deferredQuery === query on initial render
// 2. When query changes, React schedules a transition update that sets
//    deferredQuery = query
// 3. That transition render can be interrupted by the next query
//    change
// 4. Effect: the expensive subtree re-renders at most once per settled
//    query, not per keystroke
```

Unlike a `setTimeout`-based debounce, `useDeferredValue` is integrated with the scheduler: there is no fixed delay, and the deferred value commits as soon as the main thread is idle and no higher-priority work is pending. On a fast device it may update every frame; on a slow device it naturally throttles — adaptive, not clock-based.

6.4 Concurrent rendering demo — tracing an interruptible render

The following demo logs the reconciler's decision points. Run it in a React 18+ `createRoot` app with `React.Profiler` or the DevTools Profiler to see the same timings visually.

```
// demo-concurrent.jsx – run with `npx vite` (Vite + React 18)
import React from "react";
import { createRoot } from "react-dom/client";

// An intentionally expensive component – simulates a large list or
// chart
function ExpensiveList({ filter }) {
  console.log(`[render] ExpensiveList filter="${filter}"`);
  // Artificial work – in production this would be real layout/
  // computation
  const items = React.useMemo(() => {
    const start = performance.now();
    let filtered = [];
    for (let i = 0; i < 20000; i++) {
      if (String(i).includes(filter)) filtered.push(i);
      // Yield hint – in a real app, React's scheduler yields here
      // automatically
    }
    console.log(`[memo] filtered ${filtered.length} items in ${(perform
ance.now() - start).toFixed(1)}ms`);
    return filtered;
  }, [filter]);

  return <ul>{items.slice(0, 100).map(n => <li key={n}>{n}
</li>)}</ul>;
}

function App() {
  const [query, setQuery] = React.useState("");
  const [isPending, startTransition] = React.useTransition();
  const [filter, setFilter] = React.useState("");

  function handleChange(e) {
    const next = e.target.value;
    setQuery(next); // urgent – input must stay responsive
    startTransition(() => {
      setFilter(next); // transition – ExpensiveList re-render is
      // interruptible
    });
  }

  // Trace: open DevTools console and type quickly – observe:
  // 1. query updates on every keystroke (input never lags)
  // 2. ExpensiveList renders only for the settled filter, not every
  // intermediate query
  // 3. isPending is true while the transition is in flight
  return (
    <div>
      <input value={query} onChange={handleChange} placeholder="filter
0-20000" />

```

```

    {isPending && <span> □ updating...</span>}</div>
    <ExpensiveList filter={filter} />
  );
}

createRoot(document.getElementById("root")).render(
  <React.StrictMode><App /></React.StrictMode>
);

// Expected console trace when typing "12" quickly:
// [render] ExpensiveList filter=""           - initial
// [render] ExpensiveList filter="1"         - transition for "1" starts
// [memo] filtered ...                       - (may be interrupted if
"12" arrives before commit)
// [render] ExpensiveList filter="12"        - transition for "1"
discarded, new transition for "12"
// [memo] filtered ...                       - completes for "12"
// (query input showed "1" then "12" without jank - urgent updates
committed eagerly)

```

Replacing `startTransition` with a direct `setFilter(next)` would make every keystroke a `DefaultLane` update, forcing `ExpensiveList` to render synchronously for every character. On a low-end device that render might exceed the frame budget (16 ms at 60 fps), causing visible input lag — the same head-of-line blocking you see when a backend service does expensive synchronous work on the request path instead of offloading to a background lane.

7. The Distributed-Systems Lens

React's client engine mirrors problems backend engineers solve in distributed systems. Naming the correspondence makes the design decisions legible and helps you apply the same operational discipline to frontend state.

7.1 Cooperative scheduling and priority inversion

Fiber's scheduler is a **cooperative multitasking** executor on a single thread — the browser main thread. Like any cooperative scheduler, it must choose when to yield. React yields via `shouldYield()` (backed by `scheduler`'s `getCurrentTime` and `shouldYieldToHost`), which checks whether there is pending higher-priority work or the frame deadline is

near. This is the browser analog of `Gosched()` in Go or `tokio::task::yield_now()` in Rust.

Priority inversion appears when a low-priority transition holds a resource (e.g., a `Suspense` boundary's fallback is committed) and a high-priority update needs to replace it. React avoids inversion by letting high-priority lanes preempt low-priority renders entirely — the low-priority WIP is discarded and rebuilt after the high-priority commit. There is no lock to hold; fibers are pure values, not mutexes, so preemption is always safe.

7.2 Hydration as replication and consistency

Server-side rendering is **replication**: the server produces HTML (the primary), the client hydrates it (the replica). Consistency hazards are identical to database replication:

Replication concern	SSR / hydration analog
Divergence — replica disagrees with primary	Hydration mismatch — server HTML and client render differ (e.g., <code>Date.now()</code> or <code>Math.random()</code> in render) — React warns and falls back to client render
Lag — replica behind primary	Selective hydration — client hydrates shell first, suspended subtrees later; the user sees fallback until the replica catches up
Atomic cutover	<code>root.current = finishedWork</code> — the commit is the linearization point; the DOM never shows a partial tree
Read-your-writes	<code>useSyncExternalStore</code> / <code>useLayoutEffect</code> — ensure the client reads consistent state after hydration, not a stale server snapshot

The hydration mismatch warning (`Text content does not match server-rendered HTML`) is React's consistency check — like a checksum failure on a replicated log entry. The fix is to make the render deterministic, or to delay the non-deterministic part until after hydration (`useEffect`).

7.3 Suspense as a circuit breaker and bulkhead

A `Suspense` boundary is a **bulkhead**: it isolates failure (suspension) to a subtree. Without boundaries, one slow fetch would block the entire page (the fallback problem). With boundaries, each data dependency is

isolated, and the page degrades gracefully — exactly how bulkheads prevent one slow downstream from cascading to the whole service.

The retry listener (`suspender.then(retry)`) is a **circuit breaker** half-open probe: when the Promise resolves, React retries the suspended subtree. If it suspends again (e.g., a dependent fetch), it re-opens. The `RetryLane` ensures retries do not starve user input — they are low priority, just as a circuit-breaker probe should not contend with live traffic.

7.4 Caching and deduplication — request-scoped memoization

`cache()` on the server is request-scoped memoization — the same scope as a per-request `DataLoader` or a per-RPC `context.Context` cache in a gRPC service. It deduplicates concurrent `getUser(42)` calls within one render pass without leaking state across requests. On the client, the equivalent is a framework data cache (React Query, SWR, Next.js fetch cache) that deduplicates across component instances and across navigations — a shared read-through cache with TTL and revalidation, familiar to any backend engineer who has built a caching layer in front of a database.

7.5 Observability — what to measure

For a production React app treated as a distributed system, instrument the same signals you would for a service:

- **Render duration and commit frequency** — `React.Profiler`'s `onRender` callback, or the DevTools Profiler. Track `actualDuration` vs. `baseDuration` — a ratio near 1.0 means bailouts are working; a high ratio means excessive re-renders.
- **Hydration mismatches** — count `console.error` hydration warnings in production logging; each is a consistency violation.
- **Suspense fallback rate and duration** — how often and how long fallbacks are shown; long fallbacks indicate fetch waterfalls or slow upstreams.
- **Transition pending time** — `isPending` duration from `useTransition`; long pending states mean the main thread is saturated.
- **Interaction to Next Paint (INP)** — the Core Web Vital that captures input responsiveness, directly affected by lane priority and concurrent rendering effectiveness.

Key takeaways

- A fiber is a heap-allocated unit of work with identity (`tag / key / type`), tree pointers (`child / sibling / return`), state (`pendingProps / memoizedProps / memoizedState / updateQueue`), and scheduling metadata (`flags / subtreeFlags / lanes / childLanes / alternate`). The reconciler reads every field on every render.
- React double-buffers the fiber tree: `current` is the committed, on-screen tree; `workInProgress` is the speculative next tree. Only `commitRoot`'s pointer swap `root.current = finishedWork` makes the new tree visible. Interrupted renders discard `workInProgress` with no side effects.
- Lanes are a 31-bit priority bitmask. `SyncLane` and `InputContinuousLane` are urgent and non-interruptible; `TransitionLane` and `IdleLane` are interruptible and subject to expiration. `childLanes` bubbling lets the scheduler skip entire subtrees with no relevant work.
- The reconciler loop is an iterative depth-first walk: `beginWork` descends (reconciles children, may suspend), `completeWork` ascends (finalizes DOM, bubbles `subtreeFlags` and `childLanes`). The render phase is interruptible; the commit phase (before-mutation → mutation → layout) is synchronous and atomic.
- Child reconciliation has two paths: unkeyed (index-based, fast but loses identity on reorder) and keyed (Map-based, detects moves via `key`, preserves state). `key={index}` is a correctness bug for mutable lists — use stable entity IDs.
- `subtreeFlags` is a Merkle-like summary: interior fibers carry the union of their subtree's mutation flags, so `commitRoot` visits only fibers that actually need DOM work.
- Hooks form a call-order-indexed linked list on `fiber.memoizedState`. Each Hook indexes by position, so conditional Hook calls corrupt state across renders. The Rules of Hooks are a structural invariant enforced by `eslint-plugin-react-hooks` and the dev-mode dispatcher.
- `useState / useReducer` dispatch enqueues into a circular update queue, applies an eager-state bailout (`Object.is` check), and schedules via `scheduleUpdateOnFiber` with a lane derived from the

event type. Multiple dispatches in one event batch into one render that replays the queue in order.

- `useEffect` registers effects during render and flushes them after commit: `destroy` of the previous effect, then `create` of the next, compared via `Object.is` on the `deps` array. `useLayoutEffect` fires synchronously before paint; `useEffect` fires asynchronously after paint.
- Suspense's throw-Promise protocol lets any descendant suspend by throwing a Promise, caught at the nearest `<Suspense>` boundary. The boundary swaps primary children for fallback, attaches a `RetryLane` listener, and re-renders when the Promise resolves. Nested boundaries isolate suspension independently.
- Suspense waterfalls happen when fetches are initiated inside `read()` on first render — each level suspends sequentially. The fix is to initiate all fetches before rendering (module scope, route loader, or `React.cache`) so they run in parallel and `read()` either returns or suspends on an already-in-flight Promise.
- On the server, Suspense boundaries are streaming commit points: fallback HTML is emitted immediately, resolved content streams as inline scripts. The client selectively hydrates boundaries in priority order and synchronously hydrates a boundary on interaction if the user clicks before it is ready.
- Concurrent rendering lets the scheduler interrupt a low-priority transition render to handle a higher-priority urgent update, discarding the interrupted WIP. `startTransition / useTransition` downgrade updates to `TransitionLane`; `useDeferredValue` creates a transition-lagged copy of a value. Both keep the UI responsive by ensuring urgent updates (typing, clicks) never wait for expensive renders.
- Treating React as a distributed system clarifies its design: cooperative scheduling with priority queues, double-buffered MVCC for consistent commits, bulkheaded Suspense boundaries, request-scoped `cache` deduplication, and the same observability signals (latency, error rate, consistency checks) you apply to backend services.

Further reading

- React source — `ReactFiber.js`, `ReactFiberWorkLoop.js`, `ReactChildFiber.js`, `ReactFiberHooks.js`, `ReactFiberThrow.js`, `ReactFiberLane.js` ([github.com/facebook/react, packages/react-reconciler](https://github.com/facebook/react/tree/main/packages/react-reconciler)). Reading the reconciler source with this chapter as a map is the fastest way to go deeper.
- React documentation — [React Internals and Suspense/Transitions](#) and [React Server Components](#) — official API reference and conceptual guides.
- Dan Abramov — [A Cartoon Intro to Fiber](#) — the original Fiber architecture overview with diagrams, still accurate on core concepts.
- Andrew Clark — [React Suspense and Time Slicing](#) and [Concurrent UI Patterns](#) — React team's introductions to the Suspense and concurrency model.
- Scheduler package — [scheduler](#) — the cooperative scheduler that Fiber delegates yielding to; read `Scheduler.js` for `shouldYieldToHost` and priority mapping.
- Lin Clark — [A Cartoon Intro to Fiber](#) (talk) — visual walkthrough of the fiber tree and work loop.
- J. S. Choi — [Inside Fiber: an in-depth overview of the new reconciliation algorithm in React](#) — detailed fiber-field walkthrough with examples.
- Kent C. Dodds — [Application State Management with React](#) — practical guidance on when Hook state vs. external stores is appropriate.
- Next.js documentation — [Data Fetching, Caching, and Revalidation](#) — how framework-level caching and Suspense integrate in production.

- Web Incubator CG — [Scheduler API proposal](#) (`scheduler.postTask` , `scheduler.yield`) — the browser primitives that may eventually back `Scheduler` natively.

Chapter 8 — Vue and Svelte

Internals: Reactivity (Proxy vs Signals vs Compile-Time), Virtual DOM vs Compiled Output

What this chapter covers: Two dominant answers to the same question — how does a UI stay consistent with state? Vue 3 answers with a runtime reactive system built on `Proxy`, an effect graph, and a compiler-informed Virtual DOM. Svelte answers by erasing itself at compile time: no `Proxy`, no Virtual DOM, only the imperative DOM mutations the compiler can prove you need. We open both frameworks on the operating table, trace every trap, dependency, patch flag, and generated `$.append` call, and compare what each choice costs in bundle bytes, CPU cycles, and developer ergonomics.

Learning goals:

- Explain Vue 3's Proxy-based reactivity from first principles: `reactive / ref`, the `track / trigger` pair, `WeakMap` dep storage, `ReactiveEffect` scheduling, and why `ref` needs `.value`.
- Distinguish `reactive` vs `ref`, `computed` vs `watch / watchEffect`, and the abandoned Reactivity Transform (`$ref`) — including when each is the correct primitive.
- Trace Svelte's compile-time reactivity through both eras: Svelte 3/4's `$.label` transform and Svelte 5's runes (`$state`, `$derived`, `$effect`), and explain why Svelte needs no runtime `Proxy`.
- Describe Vue's Virtual DOM pipeline — `h()`, `VNode` shape and flags (`PatchFlags`, `ShapeFlags`), `patch` with block optimization and static hoisting — and quantify what the compiler saves the runtime.
- Read Svelte's compiled output as imperative DOM code (`create / mount / update / detach`) and explain why diffing is unnecessary.

- Compare Vue and Svelte side-by-side on runtime cost, bundle size, memory pressure, SSR/hydration, and the debugging trade-offs of implicit magic vs explicit mutation.
- Connect framework choice to distributed-systems realities: edge rendering budgets, hydration cost at CDN scale, independent deployability of micro-frontends, and observability of reactive graphs in production.

1. Two Philosophies, One Problem

Every frontend framework solves the same consistency problem: given mutable application state, how do you keep the DOM in sync without the developer manually calling `element.textContent = newValue` on every mutation?

React (see Chapter 7) answers with re-rendering: state change schedules a new render, the reconciler diffs the result, and the commit phase flushes DOM writes. The framework is always present at runtime.

Vue and Svelte offer two different departures from that model:

Dimension	Vue 3	Svelte 3/4 and 5
Reactivity	Runtime — <code>Proxy</code> traps intercept gets/sets	Compile-time — compiler rewrites assignments into update calls
Dependency tracking	Automatic <code>track</code> on read, <code>trigger</code> on write	Explicit signal graph (<code>\$state</code> / <code>\$derived</code>) or implicit via <code>\$:</code> analysis
Rendering	Virtual DOM with compiler hints (blocks, patch flags)	No Virtual DOM — compiled imperative DOM mutations
Runtime shipped to browser	~40 kB min+gzip (runtime-dom)	~2-5 kB runtime helpers; most logic is compiled away
Mental model	State is a reactive proxy; effects re-run automatically	State is a variable; the compiler instruments writes

For a senior backend engineer, the analogy is direct: Vue is like an ORM with change tracking — it wraps your objects, intercepts mutations, and flushes diffs. Svelte is like a code generator — it rewrites your source at build time so no tracking layer is needed at runtime. Both achieve fine-grained updates; they differ in *when* the work happens and *where* the complexity lives.

Svelte — Compile-Time Reactive

```
let count = $state(0)
```

Compiler rewrites
assignment → signal set

Signal graph
\$derived / \$effect

Compiled DOM mutation
textContent = count

No VDOM
no diff

2. Vue 3 Reactivity — A Proxy Around Plain Objects

2.1 `reactive()` and the Proxy Trap Layer

Vue 3's reactivity is built on a single primitive: `Proxy`. Unlike Vue 2's `Object.defineProperty`, which could only intercept known keys and required `Vue.set` for new properties, `Proxy` intercepts *any* property access on the target, including dynamic keys, array index writes, `in` checks, and `delete`.

The core lives in `packages/reactivity/src/reactive.ts` and `baseHandlers.ts`:

```

// Simplified from Vue 3 source – packages/reactivity/src/reactive.ts
const reactiveMap = new WeakMap(); // target → proxy cache
const targetMap = new WeakMap(); // target → (key → dep Set)

export function reactive(target) {
  if (!isObject(target)) return target;
  // Return cached proxy if already wrapped
  const existing = reactiveMap.get(target);
  if (existing) return existing;

  const proxy = new Proxy(target, {
    get(t, key, receiver) {
      const res = Reflect.get(t, key, receiver);
      track(t, key); // record dependency
      return isObject(res) ? reactive(res) : res; // deep conversion,
// lazy
    },
    set(t, key, value, receiver) {
      const oldValue = t[key];
      const result = Reflect.set(t, key, value, receiver);
      if (hasChanged(value, oldValue)) {
        trigger(t, key); // notify dependents
      }
      return result;
    },
    deleteProperty(t, key) {
      const hadKey = Object.prototype.hasOwnProperty.call(t, key);
      const result = Reflect.deleteProperty(t, key);
      if (hadKey) trigger(t, key);
      return result;
    },
    has(t, key) {
      track(t, key);
      return Reflect.has(t, key);
    },
    ownKeys(t) {
      track(t, ITERATE_KEY); // for ...in /
// Object.keys
      return Reflect.ownKeys(t);
    }
  });

  reactiveMap.set(target, proxy);
  return proxy;
}

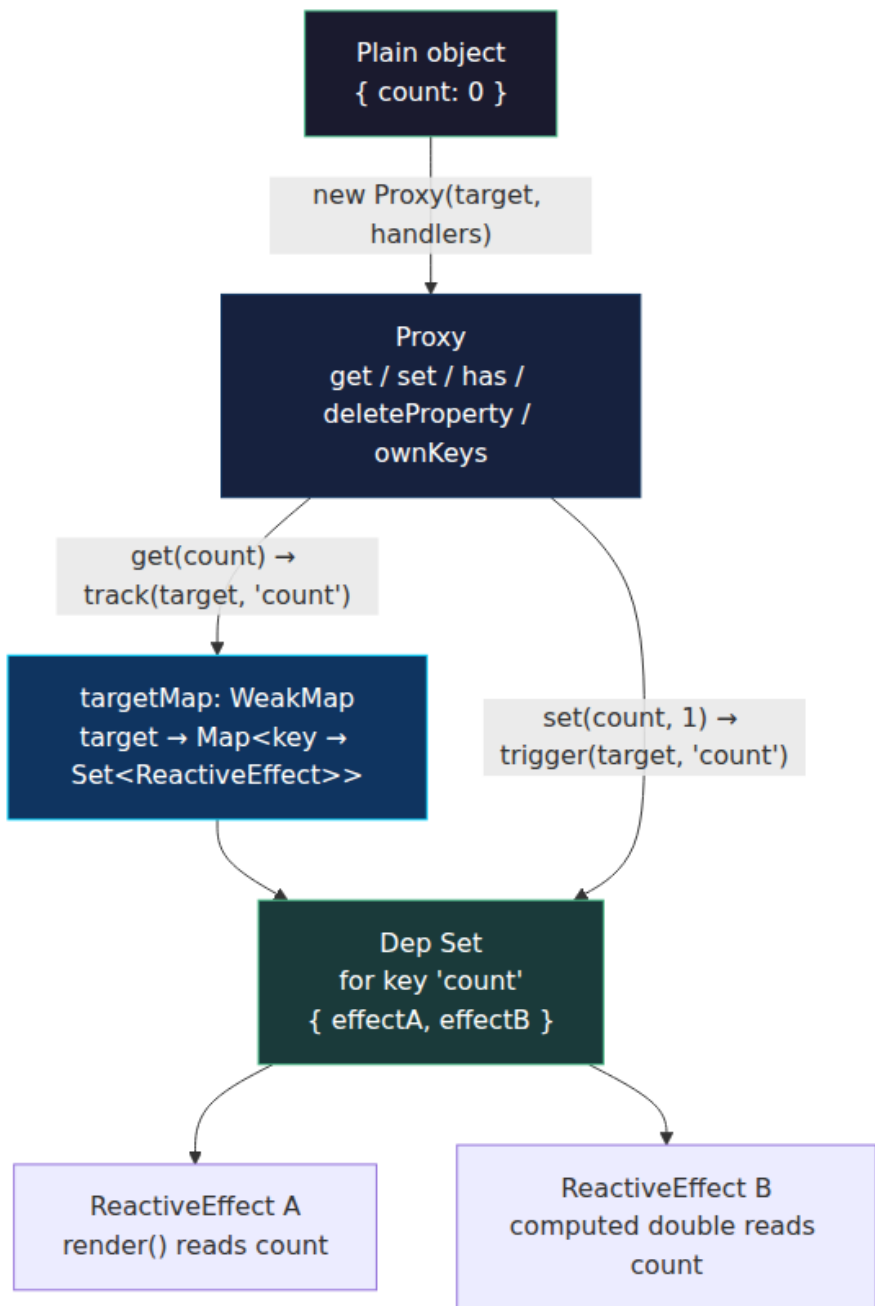
```

Three design decisions matter:

1. **Lazy deep conversion.** `get` wraps nested objects in `reactive()` on first access, not eagerly at creation. A 1,000-key state tree where you

only read 5 keys creates only 5 nested proxies. This is the same lazy-materialization principle behind demand-paged memory.

2. **Proxy identity via WeakMap.** `reactiveMap` ensures `reactive(obj) === reactive(obj)` — identity stability matters because component props are compared by reference. `targetMap` stores the dependency graph without preventing GC of the targets.
3. **Array instrumentation.** Arrays get special handling in `collectionHandlers.ts`. Methods like `push`, `pop`, `splice` are wrapped to pause tracking during internal reads (otherwise `push` would trigger spurious `length` tracking) and to trigger correctly for index and length changes. Without this, `arr.push(1)` would both track and trigger `length` in the same synchronous frame, causing infinite loops.



2.2 ref vs reactive — Why Primitives Need a Box

Proxy only wraps objects. A bare `let count = 0` has no object identity to proxy, so Vue cannot intercept `count = 1`. The solution is `ref` — a single-property reactive box:

```
// packages/reactivity/src/ref.ts - simplified
class RefImpl {
  constructor(value) {
    this._rawValue = value;
    this._value = isObject(value) ? reactive(value) : value;
    this.dep = new Set(); // dep for .value
  }
  get value() {
    trackRefValue(this); // track(this.dep)
    return this._value;
  }
  set value(newVal) {
    if (hasChanged(newVal, this._rawValue)) {
      this._rawValue = newVal;
      this._value = isObject(newVal) ? reactive(newVal) : newVal;
      triggerRefValue(this); // trigger(this.dep)
    }
  }
}

export function ref(value) { return new RefImpl(value); }
```

`reactive` vs `ref` — the rules a senior engineer should internalize:

Concern	<code>reactive(obj)</code>	<code>ref(value)</code>
Wraps	Objects/arrays deeply (lazy)	Any value — primitives via <code>.value</code> box, objects via inner <code>reactive</code>
Access	<code>state.count</code> — no <code>.value</code>	<code>count.value</code> in JS; auto-unwrapped in templates
Reassignment	<code>state = newObj</code> breaks reactivity (variable rebinding is not trapped)	<code>count.value = 1</code> preserves reactivity (mutation of the box)
Destructuring	<code>const { count } = state</code> loses reactivity (bare value)	<code>const c = count</code> keeps reactivity (still a ref object)
Prop passing	Must use <code>toRefs(state)</code> to keep per-key refs	Pass the ref directly

Concern	reactive(obj)	ref(value)
Use when	Grouped domain state (form , user)	Standalone primitives, values that get reassigned, composable return values

```
// reactive vs ref – the pitfalls that bite every team once
import { reactive, ref, toRefs, isRef } from 'vue';

// --- reactive: destructuring severs the proxy link ---
const state = reactive({ count: 0, name: 'Ada' });
let { count } = state; // count is now just 0 – a plain number
count++;               // no trigger – nobody is tracking this local
console.log(state.count); // still 0

// Fix: toRefs preserves per-key reactivity
const { count: countRef } = toRefs(state);
countRef.value++;          // triggers – countRef is a ref linked to
                           // state.count
console.log(state.count); // 1

// --- ref: auto-unwrapping in templates, not in plain JS ---
const count2 = ref(0);
function useCounter() {
  return { count2 }; // caller gets a ref – must use .value in JS
}
const { count2: c } = useCounter();
console.log(isRef(c)); // true
console.log(c.value); // 0 – .value required in script

// In <template>{{ count2 }}</template> – no .value needed, compiler
unwraps
```

Collection types (Map , Set , WeakMap , WeakSet) cannot be proxied via the base handlers because their methods access internal slots ([[MapData]]) that throw if this is a Proxy. Vue provides collectionHandlers that proxy the methods instead — size is tracked via ITERATE_KEY, and get/set/has/delete each track/trigger their respective keys.

2.3 track / trigger / dep / ReactiveEffect — The Engine

Every reactive read calls track, every write calls trigger, and both operate on the same global data structure:

```

// packages/reactivity/src/effect.ts – simplified to essentials
let activeEffect = undefined; // currently executing effect, if any
let shouldTrack = true;

const targetMap = new WeakMap(); // target → key → dep Set

export function track(target, key) {
  if (!shouldTrack || !activeEffect) return;
  let depsMap = targetMap.get(target);
  if (!depsMap) targetMap.set(target, (depsMap = new Map()));
  let dep = depsMap.get(key);
  if (!dep) depsMap.set(key, (dep = new Set()));
  if (!dep.has(activeEffect)) {
    dep.add(activeEffect);
    activeEffect.deps.push(dep); // for cleanup on re-run
  }
}

export function trigger(target, key) {
  const depsMap = targetMap.get(target);
  if (!depsMap) return;
  const dep = depsMap.get(key);
  if (!dep) return;
  // Copy to array – effects may mutate the set during trigger
  const effects = [...dep];
  for (const effect of effects) {
    if (effect.scheduler) effect.scheduler(effect);
    else effect.run();
  }
}

class ReactiveEffect {
  deps = [];
  active = true;
  constructor(fn, scheduler) {
    this.fn = fn;
    this.scheduler = scheduler;
  }
  run() {
    if (!this.active) return this.fn();
    const prev = activeEffect;
    activeEffect = this;
    try { return this.fn(); }
    finally {
      activeEffect = prev;
    }
  }
  stop() {
    if (this.active) {
      for (const dep of this.deps) dep.delete(this);
    }
  }
}

```

```

    this.deps.length = 0;
    this.active = false;
  }
}
}

```

The execution model:

1. A component's `render` (or a `watchEffect` callback) runs inside `effect.run()`, which sets `activeEffect`.
2. Every reactive `get` during that execution calls `track`, adding `activeEffect` to the dep set for that key.
3. On the next `set`, `trigger` finds the dep set and re-runs each effect.
4. Before re-running, the effect cleans up its old deps — so conditional branches that no longer read a key automatically unsubscribe from it.

```

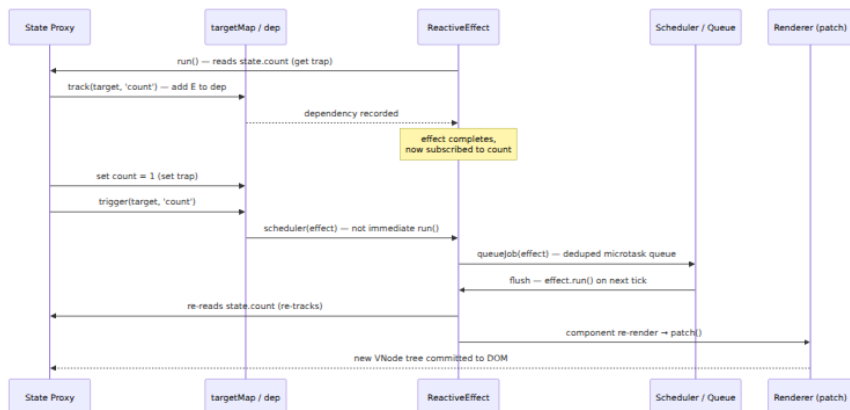
// Effect invalidation – conditional deps are cleaned up automatically
import { reactive, effect } from 'vue';

const state = reactive({ show: true, a: 1, b: 2 });

effect(() => {
  // On first run: tracks show + a
  // On second run (show=false): tracks show + b, dep on 'a' is removed
  console.log(state.show ? state.a : state.b);
});

state.a = 99;    // triggers – effect depends on 'a' (show is true)
state.b = 99;    // does NOT trigger – effect does not depend on 'b'
yet
state.show = false; // triggers, re-runs, now depends on show + b
state.a = 100;    // no longer triggers – dep on 'a' was cleaned up
state.b = 100;    // now triggers

```



Key production detail: effects are **not** re-run synchronously inside `trigger`. The scheduler batches them into a microtask queue (`queueJob` in `runtime-core/src/scheduler.ts`), deduped by effect identity. Ten synchronous mutations to the same reactive object produce one component re-render, not ten. This is the same coalescing principle behind write-ahead log batching.

2.4 `computed` and `watch` — Lazy Effects and Schedulers

```
import { reactive, computed, watch, watchEffect } from 'vue';

const state = reactive({ count: 0, name: 'Ada' });

// computed — lazy, cached, only recomputes when deps change
const doubled = computed(() => state.count * 2);
console.log(doubled.value); // 0 — computed runs on first access
console.log(doubled.value); // 0 — cached, no re-run
state.count = 5;
console.log(doubled.value); // 10 — dirty, recomputes on next access

// watchEffect — immediate, tracks whatever it reads
watchEffect(() => {
  console.log(`count is ${state.count}`);
});
// logs immediately: "count is 5", then on every count change

// watch — explicit source, old/new values, lazy by default
watch(
  () => state.count,
  (newVal, oldVal) => {
    console.log(`${oldVal} → ${newVal}`);
  }
);
state.count = 6; // logs "5 → 6"

watch(
  () => state.name,
  async (newName) => {
    // common pattern: fetch on param change, with cancellation
    const data = await fetch(`/api/users/${newName}`).then(r => r.json());
    // Vue's watch cleanup: onInvalidate runs before next invocation
  }
);
```

Internally, `computed` is a `ReactiveEffect` with `lazy: true` and a `scheduler` that marks it dirty instead of re-running immediately. The next `.value` access re-runs the getter if dirty. `watch` is a `ReactiveEffect` whose scheduler is the component's job queue (so the callback runs after the component has flushed) and whose getter is the watch source function.

Primitive	Eager?	Cached?	Has old/new?	Scheduler
<code>effect / watchEffect</code>	Yes — runs immediately	No	No	Component queue (batched)
<code>computed</code>	No — lazy on first <code>.value</code>	Yes — until dep changes	No (derive, don't compare)	Dirty-flag, no queue
<code>watch(source, cb)</code>	No — lazy until source changes	N/A	Yes	Component queue, with <code>flush: 'pre'/'post'/'sync'</code>

The `flush` option controls *when* the watcher callback fires relative to the component update cycle — `pre` (before DOM patch), `post` (after), `sync` (immediately, rarely used because it defeats batching).

2.5 The Reactivity Transform (`$ref`) — An Experiment Retired

Vue 3.2 shipped an experimental **Reactivity Transform** that let you write `let count = $ref(0)` and use `count` without `.value` — the compiler rewrote bare assignments into `.value` sets:

```
// Authored with reactivity transform (experimental, now deprecated)
let count = $ref(0);
let doubled = $computed(() => count * 2);

function inc() {
  count++; // compiler rewrites → count.value++
  console.log(doubled); // → doubled.value
}
```

The transform was removed in Vue 3.4 (RFC withdrawn). Reasons instructive for any team considering compiler magic:

- **Tooling cost.** Every tool that reads Vue code (TypeScript, ESLint, Vitest, IDE) needed a pre-transform pass to understand `$ref` semantics. The ecosystem never fully adopted it.
- **Mental model split.** Code inside `<script setup>` with the transform behaved differently from the same code in a plain `.ts` file. Two dialects of Vue increased onboarding cost.

- **Destructuring still leaked.** `$ref` values that crossed a function boundary reverted to plain values unless the callee also participated in the transform.

The replacement is ergonomic conventions, not compiler rewriting: `ref` + auto-unwrapping in templates, and composables that return refs explicitly. The lesson generalizes — compiler sugar that requires whole-toolchain cooperation must clear a high bar to justify its maintenance burden.

3. Svelte Reactivity — The Disappearing Framework

3.1 Svelte 3/4 — The `$:` Label Era

Before runes, Svelte had no runtime reactivity primitives at all. Reactivity was the compiler. Any assignment to a top-level `let` in a `.svelte` component was reactive, and `$:` labeled statements declared derived values:

```
<!-- Counter.svelte - Svelte 3/4 -->
<script>
  let count = 0;           // reactive by virtue of being top-level
  let
    $: doubled = count * 2; // re-runs whenever count changes
    $: {
      console.log(`count is ${count}`);
      if (count > 10) alert('high!');
    }
    $: if (count > 5) console.log('more than five');

  function inc() { count += 1; } // assignment triggers update - compiler inserts invalidation
</script>

<button on:click={inc}>{count} × 2 = {doubled}</button>
```

What the compiler did with this (Svelte 3/4 output, simplified):

```

// Svelte 3/4 compiled output - Counter.svelte → Counter.js
(simplified)
import { SvelteComponent, init, safe_not_equal, element, text, listen,
set_data } from 'svelte/internal';

function create_fragment(ctx) {
  let button;
  let t;
  return {
    c() { // create - called once
      button = element('button');
      t = text(`${ctx[0]} × 2 = ${ctx[1]}`);
    },
    m(target, anchor) { // mount - insert into DOM
      target.appendChild(button);
      button.appendChild(t);
      listen(button, 'click', ctx[2]);
    },
    p(ctx, [dirty]) { // update - called when count changes
      if (dirty & 1) set_data(t, `${ctx[0]} × 2 = ${ctx[1]}`);
    },
    d(detaching) { // destroy
      if (detaching) button.remove();
    }
  };
}

function instance($$self, $$props, $$invalidate) {
  let count = 0;
  let doubled;
  // $: doubled = count * 2 → compiler inserts this after every count
  // assignment
  // $$invalidate(0, count = count + 1) in inc()
  $$self.$$update = () => {
    if ($$self.$$dirty & 1) doubled = count * 2;
  };
  function inc() { $$invalidate(0, count += 1); }
  return [count, doubled, inc];
}

```

The mechanism: the compiler wraps every assignment to a reactive variable in `$$invalidate(index, value)`, which marks the component dirty and schedules an update. `$:` blocks become code inside `$$update` that re-runs when their dependencies are dirty. No Proxy, no `WeakMap`, no `track/trigger` — just code generation driven by static analysis of the assignment graph.

The limitation that motivated Svelte 5: reactivity was **component-scoped**. A plain `let count = 0` in a `.js` module was not reactive — only top-level `let` inside `.svelte` files. Sharing reactive state across components required stores (`writable/readable` with subscribe protocol), which reintroduced runtime overhead and boilerplate.

3.2 Svelte 5 Runes — Explicit Signals

Svelte 5 (stable since late 2024) replaces component-scoped magic with **runes** — explicit reactive primitives that work in any `.svelte` or `.svelte.js` file and are compiled away:

```
<!-- Counter.svelte – Svelte 5 with runes -->
<script>
  let count = $state(0);           // reactive signal
  let doubled = $derived(count * 2); // computed – cached, lazy
  let history = $state([]);        // reactive array – no Proxy, just
  signal on reassignment

  $effect(() => {
    console.log(`count is ${count}`);
    // cleanup: runs before next effect invocation and on destroy
    return () => console.log('cleanup');
  });

  function inc() { count += 1; }    // compiler rewrites to signal set
  function pushHistory() { history.push(count); } // see caveat below
</script>

<button onclick={inc}>{count} × 2 = {doubled}</button>
```

```
// Shared reactive state – Svelte 5 .svelte.js (works outside
components)
import { SvelteMap } from 'svelte/reactivity';

// counter.svelte.js
export let count = $state(0);
export let doubled = $derived(count * 2);
export function inc() { count += 1; }

// Any component can import and mutate – no store wrapper needed
// App.svelte: import { count, inc } from './counter.svelte.js'
```

The three runes:

Rune	Role	Vue equivalent	Lazy?
<code>\$state(value)</code>	Reactive signal — write triggers dependents	<code>ref(value)</code>	N/A
<code>\$derived(expr)</code>	Cached derived value	<code>computed(() => expr)</code>	Yes — recomputes only when read after dep changes
<code>\$effect(fn)</code>	Side effect — runs after DOM update, with cleanup	<code>watchEffect / watch</code> with <code>flush: 'post'</code>	No — runs on dep change

Additional runes: `$state.raw` (no deep reactivity — only reassignment triggers, like `shallowRef`), `$derived.by(() => { ... })` for multi-statement derivations, `$effect.pre` (runs before DOM update, like Vue `flush: 'pre'`), and `$inspect` (dev-only logging without subscribing).

The array/object caveat. `$state` with objects/arrays in Svelte 5 uses a Proxy internally for deep reactivity — `history.push(count)` does trigger. But `$state.raw` does not: you must reassign (`history = [...history, count]`). This parallels Vue's `reactive` vs `shallowReactive` distinction. Svelte's proxy for `$state` is an implementation detail of the signal, not the primary reactivity mechanism — the signal graph still drives scheduling, not `track` / `trigger` on arbitrary objects.



What the Svelte 5 compiler actually emits (simplified from real output, `svelte/internal/client`):

```
// Svelte 5 compiled output – Counter.svelte → Counter.svelte.js
(simplified)
import { source, derived, effect, get, set } from 'svelte/internal/
client';
import { append, from_html, set_text } from 'svelte/internal/client';

const template = from_html(` </button>`);

export default function Counter($$anchor) {
  let count = source(0);
  let doubled = derived(() => get(count) * 2);

  effect(() => { console.log(get(count)); });

  function inc() { set(count, get(count) + 1); }

  const button = template();
  const text = button.firstChild;

  effect(() => { set_text(text, `${get(count)} × 2 = ${get(doubled)}
`); });
  button.addEventListener('click', inc);
  append($$anchor, button);
}
```

`source` creates a signal cell (value + version + subscribers), `derived` creates a lazy computed cell, `get` subscribes the current effect, and `set` bumps the version and schedules subscribers. The DOM update is itself an `effect` — no Virtual DOM diff, just a `set_text` inside an effect that re-runs when `count` or `doubled` changes.

3.3 Why Svelte Needs No Runtime Proxy (Mostly)

In Vue, `Proxy` is the *mechanism* — it is how the framework discovers dependencies at runtime. In Svelte, the compiler is the mechanism — it *knows* at build time that `count` is read inside `derived(() => count * 2)` and inside `set_text(...)`, so it emits `get(count)` calls that subscribe to the signal. For primitives and reassigned variables, no Proxy is needed at all.

The Proxy that Svelte 5 *does* use for `$state({ ... })` and `$state([...])` is an optimization for ergonomic deep mutation — it lets `obj.nested.x = 1` trigger without requiring `obj = { ...obj, nested: { ...obj.nested, x: 1 } }`. But the scheduling still flows through the signal graph, not

through `track / trigger` on arbitrary keys. The Proxy is a convenience wrapper around the signal, not the signal itself.



4. Vue Virtual DOM — Compiler-Informed Diffing

Vue *does* use a Virtual DOM — but not the naive kind that diffs every node. The template compiler and the runtime conspire to make patching cheap.

4.1 `h()` and `VNode` Shape — What a Virtual Node Really Is

Every Vue template compiles to `h()` (hyperscript) calls that create `VNode` objects:

```
// Template
// <div class="app">
//   <h1>{{ title }}</h1>
//   <Button :count="count" @click="inc" />
// </div>

// Compiled render function (simplified, Vue 3.4)
import { openBlock, createElementBlock, createElementVNode,
  createVNode, toDisplayString } from 'vue';

export function render(ctx, cache) {
  return (openBlock(), createElementBlock('div', { class: 'app' }, [
    createElementVNode('h1', null, toDisplayString(ctx.title), 1 /*
PatchFlag TEXT */),
    createVNode(Button, { count: ctx.count, onClick: ctx.inc }, null,
8 /* PatchFlag PROPS */, ['count'])
  ]));
}
```

A `VNode` is a plain object — not a DOM node — with this shape (from `runtime-core/src/vnode.ts`):

```

interface VNode {
  __v_isVNode: true;
  type: string | Component | Symbol; // 'div', MyComponent, Fragment,
  Text
  props: Record<string, any> | null;
  key: string | number | null;
  ref: Ref | null;
  children: VNode[] | string | null;
  shapeFlag: number; // bitmask: ELEMENT=1, STATEFUL_COMPONENT=4,
  TEXT_CHILDREN=8, ARRAY_CHILDREN=16, ...
  patchFlag: number; // compiler hint: TEXT=1, CLASS=2, STYLE=4,
  PROPS=8, FULL_PROPS=16, ...
  dynamicProps: string[] | null; // which props are dynamic: ['count']
  el: Element | null; // linked DOM element after mount
  component: ComponentInternalInstance | null;
}

```

Two flag systems carry compiler knowledge to the runtime:

- **ShapeFlags** — what *kind* of node is this? Element vs component vs text vs fragment. Lets `patch` branch without `typeof` checks on every node.
- **PatchFlags** — what *can change* in this node? If the compiler can prove only `textContent` is dynamic (`PatchFlag.TEXT`), `patch` skips diffing `class`, `style`, and `props` entirely.

```

// PatchFlags – packages/shared/src/patchFlags.ts
export const enum PatchFlags {
  TEXT = 1, // dynamic textContent
  CLASS = 1 << 1, // dynamic class
  STYLE = 1 << 2, // dynamic style
  PROPS = 1 << 3, // dynamic props (known keys in
dynamicProps)
  FULL_PROPS = 1 << 4, // props with unknown keys (v-
bind="obj")
  NEED_HYDRATION = 1 << 5, // need hydration
  STABLE_FRAGMENT = 1 << 6, // fragment with stable order
  KEYED_FRAGMENT = 1 << 7, // keyed fragment
  UNKEYED_FRAGMENT = 1 << 8, // unkeyed fragment
  NEED_PATCH = 1 << 9, // non-props patch needed
  DYNAMIC_SLOTS = 1 << 10, // dynamic slots
  DEV_ROOT_FRAGMENT = 1 << 11,
  HOISTED = -1, // static – hoisted, never patches
  BAIL = -2, // diff without optimization
}

```

A `PatchFlag.HOISTED` node is created once outside the render function and reused across renders — zero allocation, zero diff. The compiler hoists every static subtree it can prove does not depend on reactive state.

4.2 `patch()` with Block Optimization and Static Hoisting

Naive Virtual DOM diffs the entire tree on every update — $O(n)$ where n is total nodes. Vue 3's **block optimization** narrows that to $O(d)$ where d is the number of *dynamic* nodes.

How it works:

1. **Blocks.** The compiler groups each template into *blocks* — a block is a subtree rooted at a node with `openBlock()` / `createElementBlock()`. Inside a block, the compiler collects `dynamicChildren` — only the VNodes that have a `patchFlag` (i.e., that can change).
2. **patch fast path.** When a block re-renders, `patch` iterates only `dynamicChildren` instead of all children. A 200-node template with 3 dynamic bindings patches 3 nodes, not 200.
3. **Static hoisting.** Fully static subtrees are hoisted to module scope and patch-flagged `HOISTED`. They are never visited during patch at all.

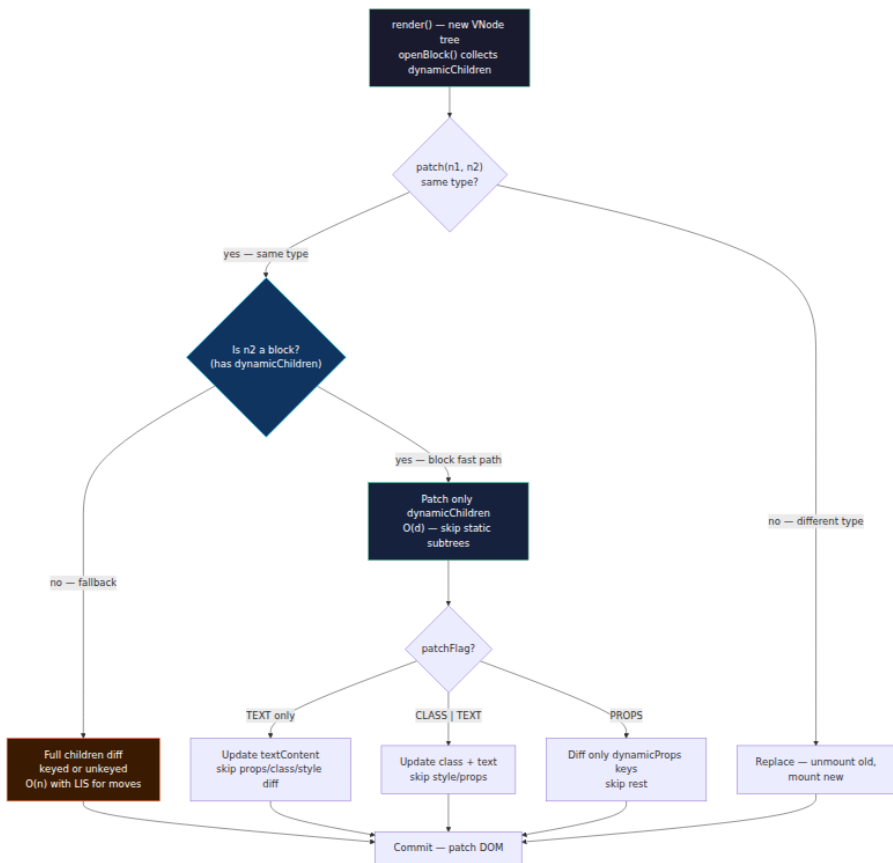

```

// Template with block optimization visible
// <div>
//   <p>Static – never changes</p>           ← hoisted,
PatchFlag.HOISTED
//   <p>{{ count }}</p>                     ← dynamic, PatchFlag.TEXT
//   <div class="card">
//     <span>Static label</span>             ← static within block
//     <span :class="active ? 'on' : 'off'">{{ label }}</span> ← CLASS
| TEXT
//   </div>
// </div>

// Compiled output – note openBlock / createElementBlock /
dynamicChildren
const _hoisted_1 = createElementVNode('p', null, 'Static – never
changes', -1 /* HOISTED */);

export function render(ctx, cache) {
  return (openBlock(), createElementBlock('div', null, [
    _hoisted_1, // reused – no allocation, no diff
    createElementVNode('p', null, toDisplayString(ctx.count),
1 /* TEXT */),
    createElementVNode('div', { class: 'card' }, [
      createElementVNode('span', null, 'Static label', -1 /* HOISTED
inside? no – inside block, but static child skipped by dynamicChildren
*/),
      createElementVNode('span', {
        class: normalizeClass(ctx.active ? 'on' : 'off')
      }, toDisplayString(ctx.label), 3 /* TEXT | CLASS */)
    ])
  ]));
  // block's dynamicChildren = [p(TEXT), span(TEXT|CLASS)] – only 2
  // nodes patched
}

```



The keyed children diff (`patchKeyedChildren` in `runtime-core/src/renderer.ts`) uses the classic longest-increasing-subsequence (LIS) optimization to minimize DOM moves — identical to the algorithm described in Chapter 7 for React's reconciler, but applied only to the dynamic fragment, not the whole tree.

Static hoisting deserves a concrete measurement. For a typical admin dashboard template with ~150 nodes and ~15 dynamic bindings, block optimization reduces patch work by roughly 10x — from visiting 150 VNodes to visiting 15. The compiler pays once at build time to save work on every frame at runtime. This is the same trade-off as building an index to avoid a full table scan.

5. Svelte Compiled Output — No Virtual DOM at All

5.1 What the Compiler Emits

Svelte has no `h()`, no `VNode`, no `patch`. The compiler emits imperative DOM API calls directly. For a Svelte 5 component, the four lifecycle fragments are `create` (build DOM), `mount` (insert), `update` (mutate in place inside effects), and `destroy` (remove):

```
<!-- List.svelte - Svelte 5 -->
<script>
  let items = $state(['apple', 'banana']);
  let filter = $state('');
  let filtered = $derived(items.filter(i => i.includes(filter)));
  function add() { items.push(`item-${items.length}`); }
</script>

<input bind:value={filter} placeholder="filter" />
<button onclick={add}>Add</button>
<ul>
  {#each filtered as item (item)}
    <li>{item}</li>
  {/each}
</ul>
<p>{filtered.length} items</p>
```

Simplified compiled output (Svelte 5, `svelte/internal/client`):

```

// List.svelte → List.svelte.js (simplified, comments added)
import { source, derived, get, set, effect } from 'svelte/internal/client';
import { append, from_html, set_text, template_effect, each } from 'svelte/internal/client';

const row_template = from_html('<li> </li>');
const root_template = from_html('<input
placeholder="filter"><button>Add</button><ul></ul><p> </p>');

export default function List($$anchor) {
  let items = source(['apple', 'banana']);
  let filter = source('');
  let filtered = derived(() => get(items).filter(i => i.includes(get(filter)))));

  function add() {
    const next = [...get(items), `item-${get(items).length}`];
    set(items, next);
  }

  const fragment = root_template();
  const input = fragment.firstChild;
  const button = input.nextSibling;
  const ul = button.nextSibling;
  const p = ul.nextSibling;
  const p_text = p.firstChild;

  // Two-way binding – input ↔ filter signal
  input.addEventListener('input', () => set(filter, input.value));
  effect(() => { if (input.value !== get(filter)) input.value = get(filter); });

  button.addEventListener('click', add);

  // Keyed each – the compiler emits a keyed reconciler, but NOT a VDOM diff
  // It tracks which keys exist and surgically inserts/moves/removes <li> nodes
  each(ul, () => get(filtered), (item) => item, row_template, (li, item) => {
    set_text(li.firstChild, item);
  });

  // Text update – fine-grained effect, not a tree diff
  effect(() => { set_text(p_text, `${get(filtered).length} items`); });

  append($$anchor, fragment);
}

```

Create — once

```
from_html('
    ')
clone template
```

```
from_html('
    ...')
clone root template
```

```
Allocate signal cells
source() / derived()
```

Mount — once

```
append(anchor,
    fragment)
insert into DOM
```

The `each` block is the one place Svelte *does* diff — but it diffs a flat keyed list of DOM nodes directly, not a Virtual DOM tree. The algorithm is a keyed reconciler specialized to a single `{#each}` scope, with no cross-component Virtual DOM.

5.2 Fine-Grained Updates Without Diffing

Because each reactive value's DOM update is its own `effect`, Svelte achieves $O(1)$ updates per changed value — not $O(d)$ like Vue's block patch, and not $O(n)$ like a naive Virtual DOM. Changing `filter` from `""` to `"a"` triggers:

1. `set(filter, "a")` — bump signal version.
2. `filtered` derived — marked dirty, recomputes on next `get`.
3. `each effect` — reads `filtered`, diffs the keyed list, surgically updates `<li>` nodes.
4. `set_text` effect for the `<p>` — reads `filtered.length`, writes one text node.

No Virtual DOM tree is created, no flags are checked, no `dynamicChildren` array is walked. The compiler already proved which DOM writes correspond to which signals.

The cost of this precision is paid at build time: the compiler must correctly identify every reactive dependency. Svelte 5's explicit runes make this easier — `$state` / `$derived` / `$effect` are unambiguous. Svelte 3/4's implicit `let + $:` required whole-component data-flow analysis that occasionally needed `$$invalidate` hints for edge cases like mutating an object property without reassignment (`obj.x = 1` without `obj = obj` did not trigger in Svelte 3).

6. Compiled Output Comparison — The Same Counter, Two Compilers

To make the difference visceral, here is the same counter component compiled by both frameworks, side by side:

```

// =====
// Vue 3 - Counter.vue compiled (render function, ~Vue 3.4)
// Template: <button @click="count++">{{ count }} × 2 = {{ doubled }}</button>
// =====
import { openBlock, createElementBlock, createElementVNode, toDisplayString } from 'vue';

// Static hoisting: none here – everything is dynamic
export function render(ctx, cache) {
  // ctx.count is a ref auto-unwrapped; ctx.doubled is a computed ref
  return (openBlock(), createElementBlock('button', {
    onClick: () => ctx.count++ // triggers Proxy set → scheduler → re-render
  }, toDisplayString(ctx.count) + ' × 2 = ' + toDisplayString(ctx.doubled), 1 /* TEXT */));
  // PatchFlag TEXT – patch only updates textContent, skips props/class/style
}
// Runtime cost per update: create new VNode → patch compares TEXT flag → one textContent write
// VNode allocation: 1 object per render
// Diff work: O(1) – single dynamic node in block

// =====
// Svelte 5 - Counter.svelte compiled (imperative DOM, Svelte 5)
// Source: let count = $state(0); let doubled = $derived(count*2);
//         <button onclick={() => count++}>{count} × 2 = {doubled}</button>
// =====
import { source, derived, get, set, effect } from 'svelte/internal/client';
import { from_html, set_text, append } from 'svelte/internal/client';

const template = from_html('<button> </button>');

export default function Counter($$anchor) {
  let count = source(0);
  let doubled = derived(() => get(count) * 2);
  const button = template();
  const text = button.firstChild;

  // Single effect owns the text update – no VNode, no diff, no flag check
  effect(() => { set_text(text, `${get(count)} × 2 = ${get(doubled)}`); });
  button.addEventListener('click', () => set(count, get(count) + 1));
  append($$anchor, button);
}
// Runtime cost per update: set() bumps version → effect re-runs → one

```

```

set_text call
// VNode allocation: 0 – no Virtual DOM objects
// Diff work: none – compiler wired the exact DOM write

```

Concern	Vue 3 output	Svelte 5 output
DOM update mechanism	New VNode → <code>patch</code> diff with <code>PatchFlag.TEXT</code> → <code>textContent =</code>	<code>effect</code> re-runs → <code>set_text(text, ...)</code> directly
Objects allocated per update	1 VNode + patch traversal state	0 VNodes; effect re-runs in place
Work skipped on update	Static subtrees hoisted; non-TEXT flags skipped	Nothing to skip — only subscribed effects run
Compiler hint	<code>PatchFlag.TEXT</code> on the VNode	No hint needed — the effect is the hint
Event handler	<code>onClick</code> prop on VNode, patched via <code>patchProp</code>	<code>addEventListener</code> once at mount, never patched
Scaling with template size	$O(d)$ — <code>dynamicChildren</code> only	$O(1)$ per changed signal

For a single counter the difference is negligible. For a 500-row table where one cell changes, Vue patches the block's dynamic children (perhaps 500 text nodes if each row has one dynamic binding), while Svelte re-runs the single effect that owns that cell's text node. Both are fast — the practical difference is constant factors and allocation pressure, not algorithmic complexity.

7. Head-to-Head — Runtime Cost, Bundle Size, Ergonomics

7.1 Runtime Cost and Memory

Metric	Vue 3	Svelte 5
Reactive cell	Proxy + WeakMap entry + Set dep per key	source cell: { v, version, subscribers } — ~3 fields
Dependency storage	WeakMap<target, Map<key, Set<effect>>> — grows with observed keys	Per-signal subscriber set — grows with number of effects reading the signal
Update scheduling	Global scheduler queue — deduped microtask flush	Per-signal version bump — effects scheduled via microtask, similar batching
Memory per component instance	VNode tree (retained between renders for diff) + reactive proxies	DOM nodes + signal cells — no VNode tree retained
GC pressure per update	New VNode objects allocated, old tree GC'd	No VNode allocation; signal version bump is a field write

Vue's WeakMap + Set graph is the heavier structure, but it scales well because deps are only created for keys actually read during an effect. Svelte's signal cells are lighter per cell, but every \$state variable allocates one. In practice, neither dominates heap for typical apps — the difference matters in extreme cases: thousands of reactive objects (Vue's WeakMap retains dep sets) vs thousands of signals (Svelte's per-signal bookkeeping).

CPU cost is dominated by different phases:

- **Vue:** Proxy trap overhead on every property access (even non-reactive reads go through the get trap and branch on shouldTrack) plus VNode diff. The trap cost is small per access (~tens of nanoseconds) but accumulates in hot loops that read reactive state repeatedly. markRaw / shallowReactive exist to opt out.

- **Svelte:** `get / set` call overhead inside effects. No proxy trap on plain reads outside effects. The compiler can even inline `get / set` to field accesses in optimized builds.

7.2 Bundle Size

Bundle size is where the compile-time approach wins unambiguously:

App	Vue 3 (runtime-dom, min+gzip)	Svelte 5 (helpers, min+gzip)
Hello world (one component)	~42 kB (runtime always included — VDOM + reactivity + scheduler)	~4 kB (template clone + a few <code>source / effect</code> helpers)
50-component SPA	~42 kB runtime + app code	~6-10 kB helpers + app code (helpers deduped, per-component code is inline DOM ops)
With router + state	+ ~10 kB (vue-router)	+ ~2 kB (svelte routing is user-space, no framework router required)

Numbers are approximate and vary with minifier and treeshaking, but the order of magnitude is stable: Vue ships a runtime that handles every component; Svelte ships helpers and compiles each component into standalone DOM code. The gap narrows as app code grows — for a 500 kB app, 40 kB vs 5 kB is noise. For an embedded widget, a marketing page, or an edge-rendered fragment where every kilobyte counts against a 100 kB budget, it decides the framework.

Svelte's per-component code can be *larger* than Vue's template-compiled render function for very simple components, because each Svelte component inlines its DOM creation. Vue's render function is compact but pays the shared runtime tax once. The crossover where Vue's larger runtime is amortized is typically a mid-size SPA.

7.3 Developer Ergonomics and Debugging

Concern	Vue 3	Svelte 5
Reactivity declaration	<code>ref / reactive</code> explicit; <code>.value</code> ceremony in JS	<code>\$state / \$derived</code> explicit; no <code>.value</code> — reassignment is the mutation
Template unwrapping	Refs auto-unwrapped in templates — no <code>.value</code> in HTML	Signals auto-subscribed in templates — no explicit <code>get</code>
Destructuring	Loses reactivity — <code>toRefs</code> or keep the ref	Loses reactivity if you destructure the value — keep the signal binding
Reactivity leak	Easy to accidentally make everything reactive (deep <code>reactive</code>)	Easy to forget <code>\$state</code> and wonder why assignment does not trigger
DevTools	Vue DevTools shows reactive graph, dep tracking, component inspector, timeline	Svelte DevTools shows signal graph (new in Svelte 5), compiler warnings
Debugging updates	<code>effect</code> + scheduler is indirect — stack trace goes through queue flush	<code>effect</code> inside compiled output — stack trace is closer to the assignment
TypeScript	<code>Ref<number></code> vs <code>number</code> distinction leaks into types; <code>UnwrapRef</code> helpers	<code>\$state</code> returns the value type directly — no wrapper type

Both frameworks have a version of the same pitfall: reactivity that silently does not fire because the developer mutated state outside the reactive boundary. In Vue, that is `state = newObj` (rebinding) or mutating a `markRaw` object. In Svelte 3/4, it was `obj.x = 1` without `obj = obj`; in Svelte 5 with `$state`, deep mutation does trigger (via Proxy), but `$state.raw` does not — and the choice is explicit.

7.4 Effect Invalidation — When Cleanup Matters

Both frameworks need to handle effect cleanup: aborting stale fetches, removing event listeners, cancelling timers. The pattern is structurally identical, but the API surface differs:

```
// Vue 3 – watch with onCleanup
import { ref, watch } from 'vue';

const query = ref('');
watch(query, async (newQuery, oldQuery, onCleanup) => {
  const controller = new AbortController();
  onCleanup(() => controller.abort()); // called before next watch
  invocation

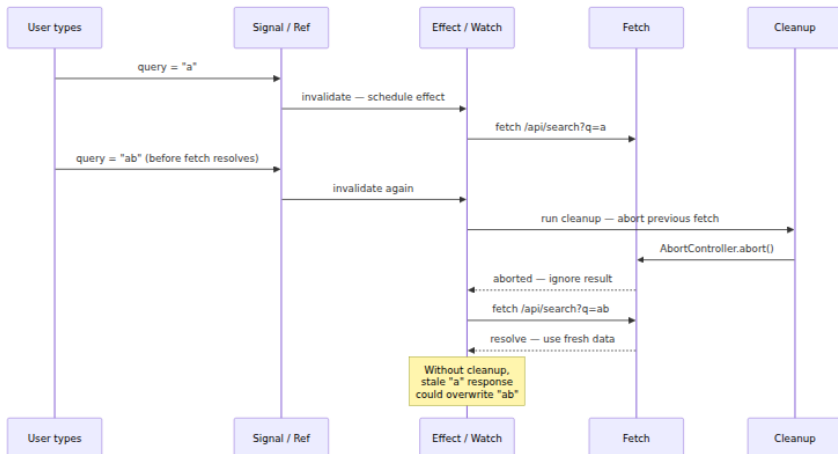
  const res = await fetch(`/api/search?q=${newQuery}`, {
    signal: controller.signal
  });
  const data = await res.json();
  // ... use data – safe because stale request was aborted
}, { flush: 'post' });
```

```
// Svelte 5 – $effect with return cleanup
let query = $state('');

$effect(() => {
  const controller = new AbortController();
  const q = query; // subscribe to query

  fetch(`/api/search?q=${q}`, { signal: controller.signal })
    .then(r => r.json())
    .then(data => { /* ... use data */ });

  return () => controller.abort(); // cleanup before next run + on
  destroy
});
```



The distributed-systems parallel is request cancellation in any concurrent system: without explicit invalidation, stale responses race with fresh ones and the last writer wins — which is not necessarily the freshest data. Both Vue's `onCleanup` and Svelte's `return () => ...` are cooperative cancellation tokens scoped to the effect lifetime.

8. The Distributed-Systems Lens

Framework internals are not just frontend trivia — they determine properties that matter at fleet scale.

8.1 Edge Rendering and Bundle Budgets

When you render at the edge (Cloudflare Workers, Fastly Compute, Vercel Edge Functions), every kilobyte of framework runtime is cold-start latency. A Vue SSR bundle carries the full `runtime-dom` plus the server renderer (`@vue/server-renderer`); a Svelte SSR bundle is the compiled component plus a small streaming helper. For an edge function with a 1 MB limit and a 50 ms CPU budget, the difference between 42 kB and 4 kB of framework overhead is measurable.

Svelte's compiled output also treeshakes more aggressively at the edge: unused components contribute zero runtime, not just zero render calls. Vue's runtime is monolithic — you pay for `Teleport`, `Suspense`, and `KeepAlive` even if you use none of them (though `runtime-dom` treeshaking is improving).

8.2 Hydration Cost — Why It Matters for CDN Scale

Hydration is the process of making server-rendered HTML interactive. Both frameworks hydrate, but the cost differs:

- **Vue hydration** walks the server-rendered DOM and the client VNode tree in parallel, matching nodes and attaching reactivity. The VNode tree must be created on the client even though the DOM already exists — allocation and matching are $O(n)$ in the number of nodes.
- **Svelte hydration** reuses the compiled DOM-creation code but skips actual creation when a DOM node already exists (via `hydrate` markers). Because there is no VNode tree, there is nothing to allocate and diff — the compiler knows which effects to attach to

which existing nodes. Hydration is closer to $O(d)$ in the number of dynamic bindings.

For a CDN-cached page served to millions of users, hydration cost is multiplied by every page load. A 200 ms hydration on a low-end device becomes a Core Web Vitals (INP) regression at scale. Svelte's lighter hydration is one reason it is popular for content-heavy sites (blogs, docs, marketing) where SSR is the primary rendering mode.

This connects directly to Chapter 9's discussion of islands architecture and partial hydration: both Vue (via `defineAsyncComponent` + `Suspense`) and Svelte (via `{#await}` and dynamic `import()`) support loading interactivity per island, but Svelte's per-island cost is lower because each island carries no shared runtime.

8.3 Micro-Frontends and Independent Deployability

In a micro-frontend architecture where teams deploy fragments independently, framework runtime sharing matters:

- **Vue micro-frontends** typically share a singleton `vue` runtime via module federation or import maps. Version skew (team A on 3.3, team B on 3.4) can break because the reactivity internals (`targetMap`, effect scheduler) are global singletons. Pinning a single Vue version across teams is an operational constraint.
- **Svelte micro-frontends** have no shared runtime singleton — each fragment carries its own compiled DOM code plus a small shared helper. Version skew is less dangerous because there is no global reactive graph. Two fragments built with different Svelte versions can coexist on the same page without sharing state. The trade-off is dedup: if ten fragments each bundle `svelte/internal/client` helpers, you pay the helper cost ten times unless you externalize it.

8.4 Observability of the Reactive Graph

Debugging reactivity in production is a distributed-tracing problem applied to the client. When a component re-renders unexpectedly, you need to answer: which signal changed, which effect was invalidated, and what triggered the mutation?

- **Vue** exposes `getCurrentScope`, `ReactiveEffect` tracking, and the Vue DevTools timeline that records every `trigger` with its target, key, and effect stack. In production, `app.config.warnHandler` and

`effectScope` let you instrument the graph programmatically. The `WeakMap`-based dep graph is inspectable at runtime.

- **Svelte 5** exposes `$inspect` (dev-only, logs signal reads/writes) and the `svelte` DevTools signal graph. Because the signal graph is built at compile time, the mapping from signal to effect is more static and easier to visualize — but also less inspectable at runtime without dev-mode instrumentation.

For both, the operational lesson is the same as for any event-driven system: instrument the edges (what triggered the effect), not just the nodes (that an effect ran). A reactive graph without trigger provenance is as opaque as a microservice trace without span attribution.

9. Choosing Between Them

There is no universal winner. The choice depends on where the complexity budget should be spent:

Choose Vue 3 when...	Choose Svelte 5 when...
Team already has Vue expertise and ecosystem (Vuetify, Nuxt, Pinia)	Starting greenfield and bundle size / edge performance is a primary constraint
App is a large SPA where 42 kB runtime is amortized	App is content-heavy, SSR-first, or widget/embedded where every kB counts
Dynamic, deeply nested state with frequent structural changes (large <code>reactive</code> trees)	State is mostly flat signals or moderate-depth objects
Need runtime reactivity for dynamic plugins / user-defined schemas (Proxy handles unknown keys)	Compiler can see all reactive declarations at build time (closed-world assumption)
Shared component library across micro-frontends with a single Vue version	Independently deployed fragments where runtime isolation matters
DevTools timeline and ecosystem maturity are priorities	Compile-time guarantees and minimal runtime are priorities

A hybrid architecture is also viable and increasingly common: Svelte for marketing/docs/edge-rendered islands, Vue (or React) for the authenticated app shell. The two can coexist on the same page via Web Components or iframe islands — the same isolation strategy used for micro-frontends.

Key Takeaways

- Vue 3's reactivity is a **runtime Proxy system**: `reactive` wraps objects in `Proxy`, `ref` boxes primitives, `track / trigger` maintain a `WeakMap<target, Map<key, Set<effect>>>` graph, and a scheduler batches effect re-runs into a deduped microtask queue. One `Proxy` per object, one `Set` per observed key, one `ReactiveEffect` per component render or `watch`.
- `ref` vs `reactive` is a choice about identity: `reactive` is for grouped state where the proxy identity is stable; `ref` is for primitives and values that get reassigned. Destructuring either without `toRefs` severs reactivity — the most common Vue bug in code review.
- `computed` is a lazy, cached `ReactiveEffect` with a dirty flag; `watch / watchEffect` are eager effects with a scheduler and optional `flush` timing. The Reactivity Transform (`$ref`) was retired because compiler sugar that requires whole-toolchain cooperation must clear a high bar.
- Svelte's reactivity is **compile-time code generation**: the compiler rewrites `let x = $state(0)` into `source(0)` signal cells, `$derived` into lazy computed cells, and `$effect` into subscribed effects. No `WeakMap`, no `Proxy` traps on every read — just `get / set` calls the compiler proved are needed. Svelte 3/4's `$:` labels worked the same way with `$$invalidate` and `$$update`.
- Vue's Virtual DOM is **compiler-informed**: `PatchFlags` and `ShapeFlags` let `patch` skip static subtrees, `openBlock / createElementBlock` collect `dynamicChildren` so `patch` is $O(d)$ not $O(n)$, and `HOISTED` nodes are never visited. This narrows the classic Virtual DOM cost without abandoning the abstraction.
- Svelte has **no Virtual DOM**: the compiler emits imperative `set_text`, `set_attribute`, and `each` reconciliation calls directly, each inside its own `effect`. Updates are $O(1)$ per changed signal

with zero VNode allocation. The cost is paid at build time in compiler complexity.

- Bundle size reflects the philosophy: Vue ships ~42 kB of runtime that handles every component; Svelte ships ~4 kB of helpers and compiles each component into inline DOM code. The gap matters at the edge and for widgets, and narrows for large SPAs where app code dominates.
- Both frameworks handle effect cleanup identically in spirit — Vue's `onCleanup` and Svelte's `return () => ...` are cooperative cancellation tokens that prevent stale async results from overwriting fresh state. Without them, concurrent mutations race.
- At fleet scale, framework choice affects edge cold start, hydration cost ($O(n)$ VNode matching vs $O(d)$ effect attachment), micro-frontend version isolation, and the observability of the reactive graph. Instrument trigger provenance, not just effect execution.

Further Reading

- Vue 3 Reactivity — source: `packages/reactivity/src/reactive.ts`, `effect.ts`, `ref.ts`, `computed.ts` (github.com/vuejs/core).
- Vue 3 Renderer — `packages/runtime-core/src/renderer.ts` (patch, block optimization), `packages/compiler-core/src/codegen.ts` (PatchFlags, hoisting).
- Vue RFCs — Reactivity Transform withdrawal: github.com/vuejs/rfcs/discussions/431; Vapor mode (no-VDOM future): github.com/vuejs/rfcs/discussions/377.
- Svelte 5 Runes — official docs: [\\$state](https://svelte.dev/docs/svelte/$state), [\\$derived](https://svelte.dev/docs/svelte/$derived), [\\$effect](https://svelte.dev/docs/svelte/$effect).
- Svelte compiler output — `packages/svelte/src/compiler/phases/3-transform/` and `packages/svelte/src/internal/client/` (github.com/sveltejs/svelte).
- Rich Harris — "Rethinking Reactivity" (2019, original Svelte talk): youtube.com/watch?v=AdNJ3fydeao.
- Rich Harris — "Svelte 5 and Runes" (2024): svelte.dev/blog/runes.
- Evan You — "Vue 3 Reactivity in Depth" (official docs): vuejs.org/guide/extras/reactivity-in-depth.

- Evan You — "Compiler-Informed Virtual DOM" (Vue 3 deep dive): vuejs.org/guide/extras/rendering-mechanism.
- Svelte vs Vue bundle analysis — `rollup-plugin-visualizer` / `esbuild --metafile` for per-framework measurement on your own app (no single benchmark is authoritative; measure your bundle).

Chapter 9 — Rendering at Scale: SSR, SSG, ISR, Streaming SSR, Hydration, Islands, and Partial Hydration

What this chapter covers: Every modern frontend framework must answer one question: where does HTML come from and when does it become interactive? This chapter traces the full rendering spectrum — client-side rendering (CSR), server-side rendering (SSR), static generation (SSG), incremental static regeneration (ISR), streaming SSR with Suspense, and the post-render phase where HTML becomes an application (hydration). We then examine the escape hatches that make scale tractable: islands architecture, partial hydration, and resumability. Along the way we ground each abstraction in concrete runtime mechanics — `renderToString` vs `renderToPipeableStream`, `hydrateRoot` reconciliation, Next.js App Router and React Server Components (RSC), and Astro's island directives — and connect every choice to the realities of CDNs, cache invalidation, and multi-service backends.

Learning goals:

- Distinguish CSR, SSR, SSG, ISR, and streaming SSR by where HTML is produced, when it is produced, and what is cached where — and choose correctly for a given page's data freshness, personalization, and traffic profile.
- Explain SSR internals from first principles: synchronous `renderToString` vs streaming `renderToPipeableStream` / `renderToReadableStream`, backpressure, and why the latter unlocks Suspense.
- Implement SSG and ISR in Next.js (Pages and App Router), including `revalidate`, `fallback`, on-demand revalidation with

revalidatePath / revalidateTag , and fetch cache semantics — and reason about their consistency model (stale-while-revalidate).

- Wire streaming SSR end-to-end with React Suspense , understand chunk framing and selective hydration, and debug the resulting out-of-order delivery.
- Describe hydration precisely — what hydrateRoot does, why it re-executes component trees, how mismatches are detected, and how to diagnose and fix them without suppressHydrationWarning as a crutch.
- Compare islands architecture (Astro, Fresh) and partial hydration against full-page hydration, and explain Qwik's resumability as a qualitatively different strategy that eliminates hydration entirely.
- Quantify performance trade-offs with real metrics (TTFB, FCP, LCP, TTI, INP) and connect rendering choices to distributed-systems concerns: CDN cache hit ratios, origin load, thundering herds on revalidation, and independent deployability.

1. The Rendering Spectrum: Where and When HTML Is Born

Every rendering strategy is a placement decision along two axes: **where** the HTML bytes are assembled (browser vs server vs build) and **when** they are assembled (request time vs build time vs background revalidation). Get either axis wrong and you pay in latency, origin cost, or stale data.

Strategy	Where HTML is built	When	Cacheable at CDN?	Data freshness	Interactivity
CSR (SPA)	Browser	On navigation	Only shell + JS	Live (client fetch)	After JS download + hydration
SSR (renderToString)	Server	Per request	Conditionally (short TTL / Vary)	Per-request fresh	After full HTML + hydration
Streaming SSR	Server	Per request, chunked	Same as SSR, but TTFB wins	Per-request fresh, progressive	Selective / prioritized hydration

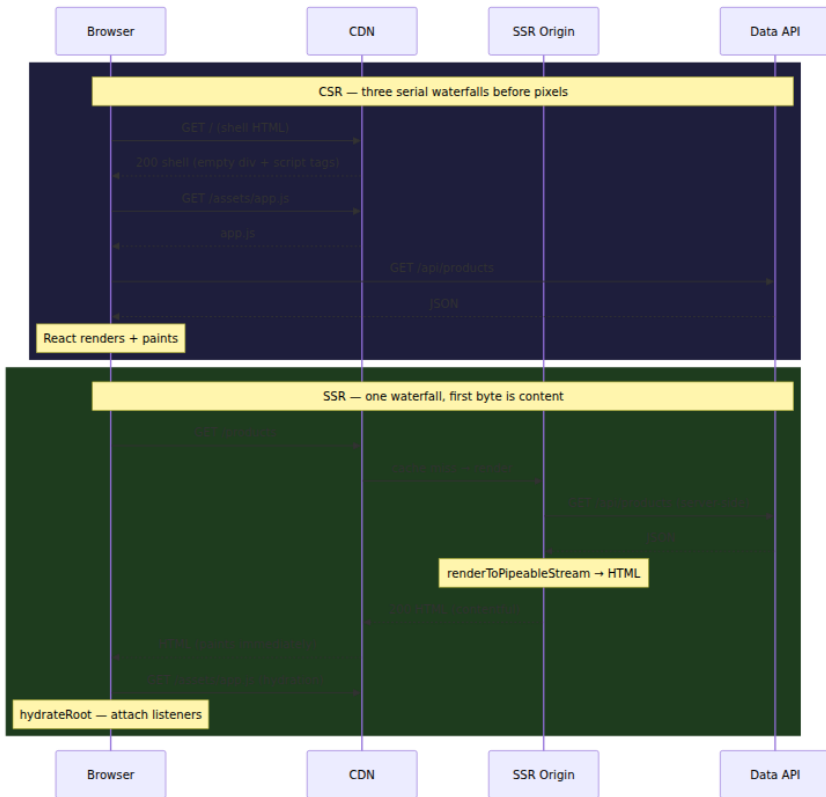
Strategy	Where HTML is built	When	Cacheable at CDN?	Data freshness	Interactivity
SSG	Build worker	At build	Aggressively (immutable until deploy)	Stale until next deploy	After hydration
ISR	Build + server	Build + background revalidate	Stale-while-revalidate	Bounded staleness	After hydration
Islands / Partial Hydration	Build + server + browser (per island)	Build + per-island hydration	Static shell cached, islands vary	Shell stale, islands live	Island-by-island

For a backend engineer the analogy is precise: CSR is like a thick client that calls your APIs directly — the edge serves only static assets and your API fleet bears every read. SSR moves the aggregation to the server tier (a BFF / SSR origin) and ships assembled HTML. SSG/ISR push assembly into the build and CDN tier, turning the origin from a per-request renderer into a background revalidator. Islands push the decision down to the component level: each island declares its own hydration contract.

There is no globally optimal strategy. A marketing landing page, a personalized feed, and a real-time dashboard have different freshness, cardinality, and compute requirements. Scale forces you to mix them — often on the same route.

1.1 CSR vs SSR: The Waterfall That Users Actually Experience

In CSR the browser downloads a near-empty shell, then JavaScript, then data, then renders. In SSR the server does the data fetch and render before the first byte reaches the browser. The network waterfall differs fundamentally:



Two observations matter at scale:

1. **TTFB vs completeness trade-off.** CSR TTFB is fast (CDN serves a tiny shell) but *meaningful* paint is late — after JS and data fetches. SSR TTFB is slower (origin must fetch data and render) but first contentful paint (FCP) arrives with that first byte. Users perceive SSR as faster even when total bytes are larger, because the browser can paint and the user can read before JavaScript arrives.
2. **Origin load.** CSR moves load to your API fleet and the user's device. SSR concentrates load on the SSR origin. Without caching, SSR is an amplification layer — one page request fans out to N API calls server-side. This is the core scaling problem the rest of this chapter solves.

2. SSR in Depth: From `renderToString` to Streaming

2.1 The Naive Model: `renderToString`

React 16-era SSR was synchronous and blocking:

```
// server-legacy.tsx – synchronous SSR (blocking)
import { renderToString } from "react-dom/server";
import App from "../App";

export function handleRequest(req: Request, res: Response) {
  // 1. Fetch all data BEFORE rendering – waterfall is serial
  const data = await fetchProduct(req.params.id); // blocks
  const html = renderToString(<App data={data} />);

  res.setHeader("Content-Type", "text/html");
  res.end(`<!DOCTYPE html>
    <html><head><link rel="stylesheet" href="/assets/app.css"></head>
    <body>
      <div id="root">${html}</div>
      <script>window.__DATA__ = ${JSON.stringify(data)}</script>
      <script src="/assets/app.js"></script>
    </body></html>`);
}
```

`renderToString` renders the entire tree to a string in memory, then sends it. It is simple, debuggable, and compatible with any streaming-incompetent proxy — but it has three scaling liabilities:

- **All-or-nothing TTFB.** The slowest `await` before `renderToString` delays the first byte for every user. One slow downstream API stalls the entire page.
- **Memory pressure.** The full HTML string is buffered in memory before flushing. For large pages (catalog, search results) this allocates and holds O(page size) on the SSR origin per concurrent request — GC pressure under load.
- **No Suspense.** `renderToString` cannot suspend. If a component throws a promise (Suspense), the legacy API has no way to emit a fallback and resume later.

For low-traffic internal tools this is fine. For a high-traffic storefront it is a bottleneck.

2.2 The Streaming Model: `renderToPipeableStream` and `renderToReadableStream`

React 18 replaced the blocking model with streaming renderers that write HTML incrementally as data resolves, interleaving Suspense fallbacks with eventual content.

```
// server-streaming.tsx – streaming SSR with Suspense + backpressure
import { renderToPipeableStream } from "react-dom/server";
import { DataProvider } from "../data";
import App from "../App";

export function handleRequest(req: Request, res: Response) {
  let didError = false;

  const { pipe, abort } = renderToPipeableStream(
    <DataProvider>
      <App />
    </DataProvider>,
    {
      bootstrapScripts: ["/assets/app.js"],
      // Called when the shell (non-Suspense content) is ready to
      stream
      onShellReady() {
        res.statusCode = didError ? 500 : 200;
        res.setHeader("Content-Type", "text/html");
        pipe(res); // Node Writable – respects backpressure
      },
      onShellError(err) {
        console.error("Shell error", err);
        res.statusCode = 500;
        res.setHeader("Content-Type", "text/html");
        res.end("<!doctype html><p>Internal Server Error</p>");
      },
      onError(err) {
        didError = true;
        console.error("Recoverable render error", err);
        // Stream continues – error boundary / fallback is emitted
      },
    }
  );

  // Safety: abort suspended work if client disconnects
  req.on("close", () => abort());
  // Optional hard timeout – prevent dangling renders under downstream
  stalls
  setTimeout(abort, 10_000);
}
```

The Edge / Web Streams variant is symmetrical:

```
// server-edge.tsx – Edge runtime (Cloudflare Workers / Vercel Edge)
import { renderToReadableStream } from "react-dom/server";

export default {
  async fetch(req: Request): Promise<Response> {
    const stream = await renderToReadableStream(<App />, {
      bootstrapScripts: ["/assets/app.js"],
      onError(err) { console.error(err); },
    });
    return new Response(stream, {
      headers: { "Content-Type": "text/html" },
    });
  },
};
```

What streaming buys you at scale:

- **Earlier TTFB.** `onShellReady` fires as soon as everything *outside* Suspense boundaries is ready. The browser receives the shell (header, nav, layout, fallbacks) while slow islands are still fetching server-side. Paint starts before the slowest API responds.
- **Backpressure awareness.** `pipe(res)` respects Node stream backpressure — if the client is on a slow link, the renderer pauses rather than buffering unbounded HTML in origin memory. This is the same flow-control principle as TCP windowing applied to HTML generation.
- **Resilience.** A single failing Suspense boundary emits its error fallback; the rest of the page still streams. With `renderToString`, one throw fails the whole page.

The cost is complexity: chunked transfer encoding, inline `<script>` placeholders that the client runtime uses to swap fallbacks for resolved content, and the fact that some intermediaries (older WAFs, misconfigured proxies) buffer chunked responses — negating the benefit. Validate your CDN and proxy chain actually forwards chunks; otherwise you have added complexity for no TTFB gain.

2.3 Next.js: From `getServerSideProps` to React Server Components

Next.js is the clearest record of how the community's mental model shifted.

Pages Router (pre-13) — explicit per-page SSR/SSG:

```
// pages/products/[id].tsx — Pages Router
import type { GetServerSideProps, GetStaticProps, GetStaticPaths }
from "next";

// SSR — runs on every request, origin does data fetch + render
export const getServerSideProps: GetServerSideProps = async ({ params }) => {
  const product = await fetch(`https://api.internal/products/${params!.id}`).then(r => r.json());
  if (!product) return { notFound: true };
  return { props: { product } }; // serialized as JSON into HTML
};
export default function ProductPage({ product }: { product: Product }) {
  return <ProductDetail product={product} />;
};

// SSG + ISR — rendered at build, revalidated in background
export const getStaticPaths: GetStaticPaths = async () => {
  const ids = await fetchTopProductIds(); // e.g. top 1000 at build time
  return { paths: ids.map(id => ({ params: { id } })), fallback: "blocking" };
};
export const getStaticProps: GetStaticProps = async ({ params }) => {
  const product = await fetch(`https://api.internal/products/${params!.id}`).then(r => r.json());
  return { props: { product }, revalidate: 60 }; // ISR: regenerate at most every 60s
};
```

App Router (13+) — Server Components by default, `fetch` cache controls rendering:

```

// app/products/[id]/page.tsx – App Router + RSC
// This file is a React Server Component – it runs ONLY on the server,
// never ships to the browser, and can await directly.

export const revalidate =
60; // route-level ISR: background revalidate every 60s
// Alternatives: export const dynamic = 'force-dynamic' // always SSR
//                export const dynamic = 'force-static' // always SSG

async function getProduct(id: string) {
  // Next.js extends fetch with cache + revalidation semantics:
  const res = await fetch(`https://api.internal/products/${id}`, {
    // Per-fetch control – more granular than route-level revalidate
    next: { revalidate: 60, tags: ["products"] },
    // cache: 'no-store' → always dynamic (SSR)
    // cache: 'force-cache' → static (SSG)
  });
  if (!res.ok) throw new Error(`Failed to fetch product ${id}`);
  return res.json();
}

export default async function ProductPage({ params }: { params: { id: string } }) {
  // No getServerSideProps, no props serialization – just await.
  const product = await getProduct(params.id);
  return (
    <div>
      <h1>{product.name}</h1>
      {/* Client Component boundary – only this subtree hydrates */}
      <AddToCartButton productId={product.id} />
      {/* Streaming: slow part wrapped in Suspense – shell streams
immediately */}
      <Suspense fallback={<ReviewsSkeleton />}>
        <Reviews productId={product.id} />
      </Suspense>
    </div>
  );
}

// On-demand revalidation – call from webhook / admin mutation
// app/api/revalidate/route.ts
import { revalidateTag, revalidatePath } from "next/cache";
export async function POST(req: Request) {
  const { tag, path } = await req.json();
  if (tag) revalidateTag(tag); // purge all fetches tagged "products"
  if (path) revalidatePath(path); // purge a specific route
  return Response.json({ revalidated: true });
}

```

The App Router collapses three previously distinct concepts (SSR, SSG, ISR) into a single caching model: every `fetch` declares its freshness contract, and the framework derives the rendering strategy. A `fetch` with `cache: 'no-store'` makes the route dynamic; a `fetch` with `revalidate` makes it ISR; no `fetch` at all makes it static. This is more compositional than the Pages Router's per-page mode switch and maps cleanly onto CDN semantics — but it demands that developers understand HTTP caching, not just React.

Next.js configuration that matters at scale:

```

// next.config.js
/** @type {import('next').NextConfig} */
const config = {
  // Standalone output for container deploys – no next start dependency
  // on node_modules
  output: "standalone",

  // ISR / fetch cache backed by Redis for multi-instance consistency
  // (self-hosted)
  // On Vercel this is automatic; self-hosted you wire it:
  // experimental: { incrementalCacheHandlerPath: require.resolve('./
  cache-handler.js') },

  // Streaming requires no extra flag since Next 13 – but compression +
  // chunked encoding do:
  compress: true,

  // CDN + cache headers – App Router respects fetch revalidate, but
  // static assets need explicit policy
  async headers() {
    return [
      {
        source: "/assets/:path*",
        headers: [{ key: "Cache-Control", value: "public, max-
        age=31536000, immutable" }],
      },
    ];
  },

  // Split vendor chunks for hydration cost control – large hydration
  // JS delays TTI
  webpack(cfg) {
    cfg.optimization.splitChunks = {
      chunks: "all",
      cacheGroups: {
        framework: { test: /[\\/]node_modules[\\/](react|react-dom|
        scheduler)[\\/]$/, name: "framework", priority: 40 },
        lib: { test: /[\\/]node_modules[\\/]$/, name: "lib", priority: 3
        0 },
      },
    };
    return cfg;
  },
};
export default config;

```

3. SSG and ISR: Pre-render Once, Revalidate in the Background

3.1 SSG — Shift Rendering Left to Build Time

Static generation renders pages at build time and emits HTML files that any CDN can serve as immutable assets. For content with low write frequency (marketing pages, docs, blog posts) this is optimal: origin cost is zero at request time, cache hit ratio is ~100%, and global latency is the CDN edge RTT.

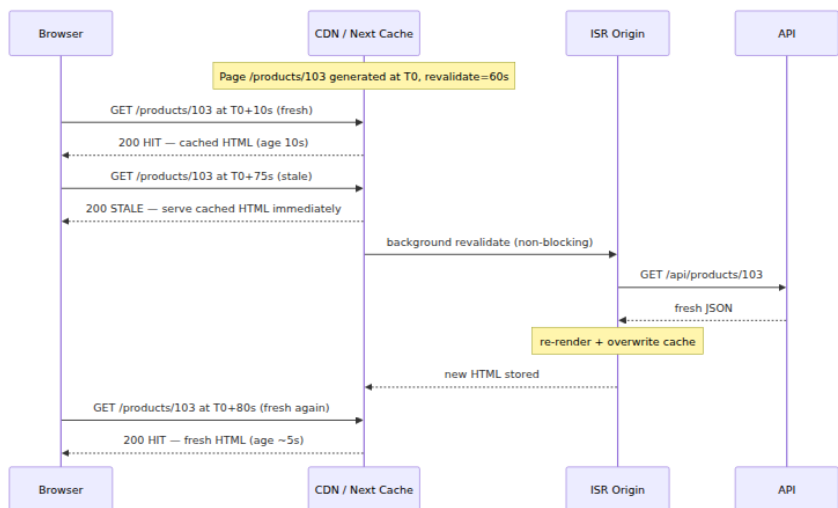
```
# Build-time SSG — output is plain HTML + JSON data files
$ next build
Route (app)                                Size      First Load JS
├ ○ /                                       5.2 kB      87 kB
├ ○ /about                                 1.1 kB      83 kB
└ ● /products/[id] (ISR: 60s)             8.4 kB      91 kB # ● =
ISR, ○ = static
└ λ /api/revalidate                       0 B         0 B # λ =
server function

# Emitted artifacts (self-hosted output: standalone)
$ ls .next/server/app/products/
103.html 104.html 105.html # one HTML file per pre-rendered param
$ ls .next/static/chunks/
framework-*.js lib-*.js app-*.js
```

The scaling constraint is cardinality. Pre-rendering 10 million product pages at build time is infeasible — build duration, storage, and the fact that most pages are never visited make exhaustive SSG wasteful. ISR exists to solve this.

3.2 ISR — Stale-While-Revalidate as a Consistency Model

ISR borrows directly from HTTP's `stale-while-revalidate` (RFC 5861). The first request after a page becomes stale is served **immediately** from the stale cache while a background regeneration is triggered. Subsequent requests get the fresh page once regeneration completes.



This is an **eventually consistent** model with bounded staleness. Readers never block on writes. The write path (revalidation) is asynchronous and decoupled from the read path — the same pattern as a database read replica with async replication or a CQRS projection that lags behind the write model. The bound is `revalidate` seconds plus regeneration time.

Three `fallback` modes handle the case where a path was not generated at build time:

fallback	Behavior on first request for unknown path	Use when
<code>false</code>	404 immediately	Closed set (e.g. <code>/docs/[slug]</code> from known files)
<code>true</code>	Serve fallback shell, then client-side fill + cache	Large set, want fast TTFB even on miss
<code>"blocking"</code>	SSR on first request (no fallback), then cache	Large set, SEO-sensitive, prefer complete HTML on first hit

In the App Router, `fallback` is replaced by `dynamicParams` :

```
// app/products/[id]/page.tsx
export const dynamicParams =
true; // true → blocking-like: unknown ids render on demand then cache
// dynamicParams = false → 404 on unknown ids (like fallback: false)

export async function generateStaticParams() {
  // Controls which params are pre-rendered at build – same as
  getStaticPaths
  return (await fetchTopIds()).map(id => ({ id }));
}
```

3.3 On-Demand Revalidation — Closing the Staleness Window

Time-based revalidation bounds staleness but cannot react to writes. When a product price changes, waiting up to 60 seconds to reflect it may violate business requirements. On-demand revalidation lets the write path explicitly invalidate the read path:

```
// When product 103 is updated via admin API / CMS webhook:
// 1. Tag-based – most granular, App Router idiomatic:
await fetch("https://app.example.com/api/revalidate", {
  method: "POST",
  body: JSON.stringify({ tag: "products" }),
  // Server handler calls revalidateTag("products")
});

// 2. Path-based – coarser, Pages Router compatible:
// revalidatePath("/products/103")

// 3. Self-hosted cache handler – must propagate across instances:
// If you run N SSR origins behind a load balancer without shared
// cache,
// each origin must receive the invalidation (pub/sub) or share a
// cache (Redis).
```

The distributed-systems problem is cache coherence. With a single Vercel-like platform, the cache is global. Self-hosted with N replicas and local filesystem caches, an on-demand revalidation that hits replica A does not purge replica B — users see inconsistent pages depending on which replica the load balancer chose. Solutions in order of robustness:

1. **Shared cache backend** (Redis, S3, or a purpose-built incremental cache handler) so all replicas read/write the same store.
2. **Pub/sub invalidation** — publish revalidation events to all replicas via Redis pub/sub, NATS, or similar.

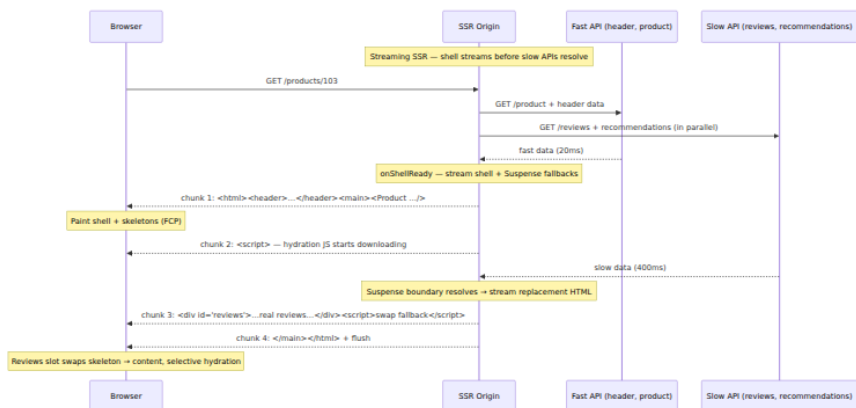
3. **CDN purge** — issue a CDN cache purge (CloudFront invalidation, Fastly purge key, Cloudflare cache tag) so the CDN refetches from any replica.

Without one of these, ISR self-hosted at scale exhibits the same split-brain that any replicated cache without invalidation does.

4. Streaming SSR: Suspense, Chunks, and Selective Hydration

4.1 Why Streaming Exists

Without streaming, the SSR waterfall is serial: fetch all data → render → send. The 95th-percentile API dominates TTFB. With streaming, Suspense boundaries let the server send the shell early and fill holes as data arrives — overlapping data fetching, rendering, and network transfer.



The key mechanism is the **template + inline script** pattern. Each Suspense boundary emits a fallback with a unique ID and, when its data resolves, an inline `<script>` that moves the resolved HTML into place. The client React runtime listens for these swaps and hydrates each boundary independently — this is *selective hydration*.

4.2 Suspense Boundaries as Scheduling Primitives

```
// app/products/[id]/page.tsx – streaming with prioritized Suspense
import { Suspense } from "react";

// Server Components can be async – they suspend implicitly when
awaiting
async function Reviews({ productId }: { productId: string }) {
  // This fetch suspends the Reviews boundary until it resolves
  const reviews = await fetch(`https://api.internal/products/${productId}/reviews`, {
    next: { revalidate: 30 },
  }).then(r => r.json());
  return <ReviewList reviews={reviews} />;
}

async function Recommendations({ productId }: { productId: string }) {
  const recs = await fetch(`https://api.internal/products/${productId}/recs`);
  return <Carousel items={recs} />;
}

export default function ProductPage({ params }: { params: { id: string } }) {
  return (
    <div>
      /* Shell – streams immediately, no suspension */
      <ProductHeader id={params.id} />

      /* Each Suspense boundary is an independent streaming unit.
         Slow boundaries do NOT block the shell or each other. */
      <Suspense fallback=<ReviewsSkeleton />>
        <Reviews productId={params.id} />
      </Suspense>

      <Suspense fallback=<CarouselSkeleton />>
        <Recommendations productId={params.id} />
      </Suspense>

      /* Client island – hydrates independently, can hydrate before
         reviews resolve */
      <Suspense fallback={null}>
        <AddToCartHydrationBoundary productId={params.id} />
      </Suspense>
    </div>
  );
}
```

Streaming + Suspense gives the server a **priority scheduler** that mirrors React's client scheduler:

- Shell content has highest priority — it streams first.
- Each Suspense boundary resolves independently — no head-of-line blocking.
- On the client, React hydrates boundaries in priority order (user-interacted boundaries first via selective hydration) rather than top-to-bottom.

SSR Origin — Streaming Chunks

Chunk 1 — Shell
header + layout +
fallbacks
onShellReady → pipe()

Chunk 2 —
bootstrapScripts
app.js + hydration
runtime

Chunk 3 — Reviews
resolved
...
+ swap script

Chunk 4 —
Recommendations
resolved
...

4.3 Operational Realities of Streaming

Streaming is not free. Validate these before committing a route to it:

- **Proxy buffering.** Some corporate proxies, WAFs, and older Nginx configs buffer chunked responses until `Content-Length` is known — check `proxy_buffering off` and `chunked_transfer_encoding on` where applicable.
 - **Compression.** Streaming + gzip/brotli interact: the compressor must flush per chunk. Most CDNs handle this, but self-hosted Nginx `gzip` with default buffering may delay chunks.
 - **Error handling.** Errors inside a Suspense boundary after the shell has been sent cannot change the HTTP status code (headers already flushed). Your monitoring must distinguish shell errors (5xx status) from boundary errors (200 with embedded error fallback) — they look identical at the CDN layer unless you emit a header or metric.
-

5. Hydration: Attaching Interactivity to Server HTML

5.1 What `hydrateRoot` Actually Does

Hydration is often described as "attaching event listeners." That undersells it. `hydrateRoot` re-executes your component tree on the client, builds a Fiber tree, and **reconciles** it against the server-rendered DOM — reusing existing DOM nodes where they match and warning where they diverge.

```
// client.tsx – hydration entry
import { hydrateRoot } from "react-dom/client";
import App from "./App";

// hydrateRoot: React takes over existing DOM inside #root.
// It does NOT re-render from scratch – it walks the existing DOM and
// attaches.
hydrateRoot(
  document.getElementById("root")!,
  <App data={window.__DATA__} />,
  {
    onRecoverableError(err, info) {
      // Mismatches and recoverable render errors land here – log them
      console.error("Recoverable hydration error", err, info.components
    tack);
    reportToObservability(err, info);
  },
}
);

// Contrast:
// createRoot(document.getElementById("root")!).render(<App />)
// → discards server HTML, renders from scratch (CSR). No hydration,
// no mismatch check.
// hydrateRoot(...)
// → reuses server HTML, warns on mismatch, preserves DOM state
// (focus, scroll, input values).
```

The cost of hydration is proportional to the size of the hydrated tree — every component function re-executes, every hook re-initializes, and the reconciler walks the DOM. Hydrating a 5,000-node product page is $O(\text{nodes})$ CPU on the main thread, blocking user interaction (TTI) until complete. This is why full-page hydration scales poorly and why islands/partial hydration exist (Section 6).

Selective hydration (React 18+) improves the UX without reducing the CPU cost: React hydrates boundaries in priority order and can interrupt low-priority hydration to handle a user event (e.g., clicking an "Add to Cart" button hydrates that boundary first). TTI for the *interacted* island improves; total hydration time does not shrink.

5.2 Hydration Mismatches: Causes, Detection, Debugging

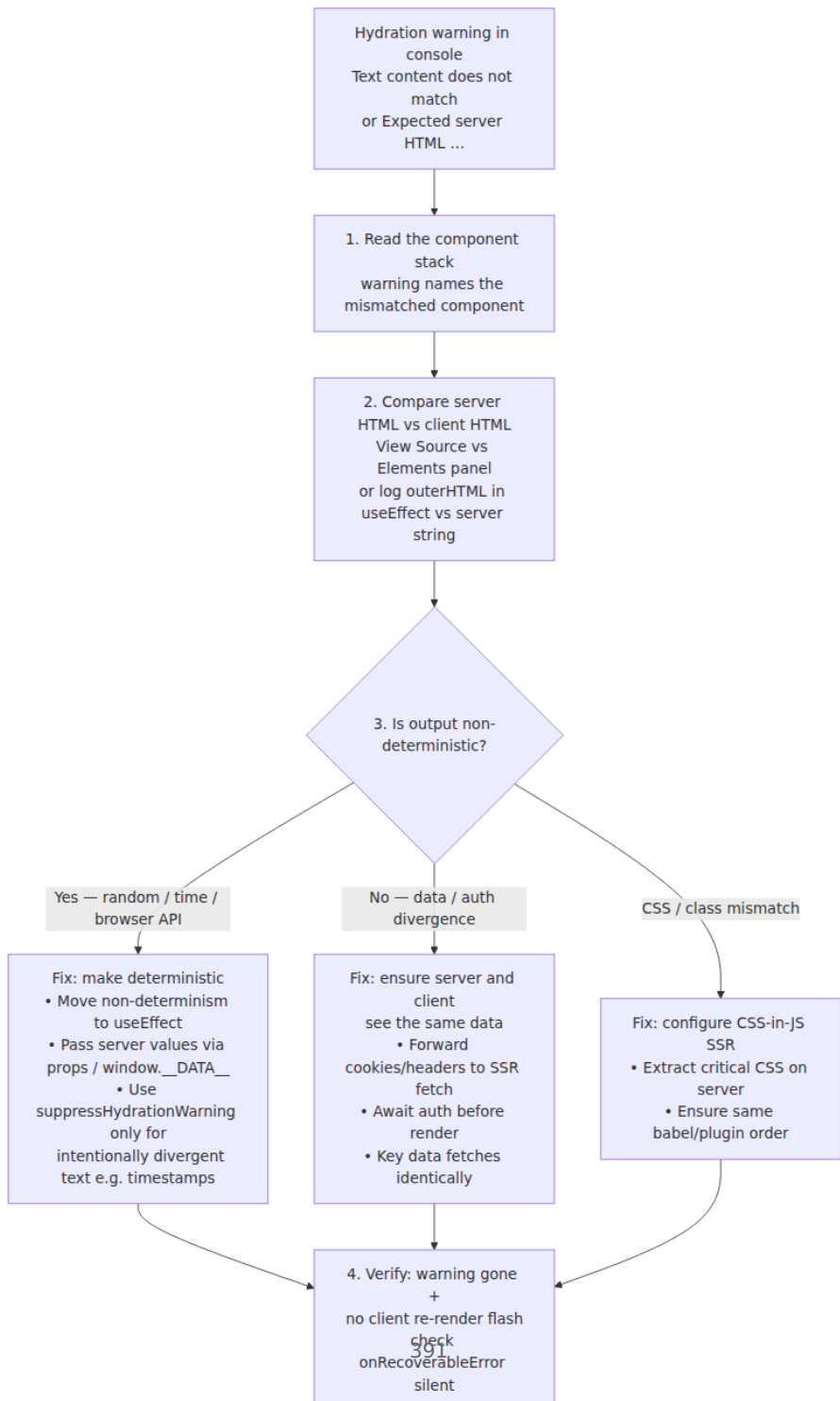
A mismatch means the server-rendered HTML and the client's first render produced different output for the same component. React detects this during hydration by comparing the expected DOM (from the client

render) with the actual DOM (from the server). In development it logs a warning; in production it attempts to recover by re-rendering the mismatched subtree on the client (discarding server HTML for that subtree).

Common causes — every one is a **non-determinism** between server and client:

Cause	Example	Why it mismatches
Random / time	<code>Math.random()</code> , <code>Date.now()</code> , <code>new Date().toLocaleString()</code>	Server and client generate different values
Browser-only globals	<code>window.innerWidth</code> , <code>localStorage</code> , <code>navigator</code>	Undefined on server, defined on client
User-specific data	Auth state, cookies not forwarded to SSR	Server renders logged-out, client renders logged-in
Locale / timezone	<code>Intl.DateTimeFormat</code> without explicit locale	Server locale (e.g. <code>en-US</code> container) vs client locale
CSS-in-JS ordering	Emotion/styled-components without SSR setup	Class name hashes differ between server/client
Streaming + non-deterministic Suspense	Data fetch not keyed consistently	Fallback vs content differs between renders

Debugging workflow:



Concrete fix — the most common mismatch, a timestamp:

```
// ❌ Mismatches – server and client render different strings
function LastUpdated({ ts }: { ts: number }) {
  return <time>{new Date(ts).toLocaleString()}</time>;
  // Server: "8/21/2026, 12:00:00 AM" (UTC container)
  // Client: "8/21/2026, 8:00:00 AM" (local timezone) → mismatch
}

// ✅ Deterministic – format on server, hydrate with same string
function LastUpdatedFixed({ ts, formatted }: { ts: number; formatted: string }) {
  // formatted is produced server-side with explicit locale/timezone
  return <time suppressHydrationWarning dateTime={new Date(ts).toISOString()}>
    {formatted}
  </time>;
  // Or defer formatting to client only:
}

function LastUpdatedClientOnly({ ts }: { ts: number }) {
  const [label, setLabel] = useState<string | null>(null);
  useEffect(() => {
    // Runs only on client, after hydration – no mismatch possible
    setLabel(new Date(ts).toLocaleString());
  }, [ts]);
  if (label === null) return <time>{/* server placeholder */}</time>;
  return <time>{label}</time>;
}
```

Rules of thumb for mismatch-free hydration:

1. **Server and client must be pure functions of the same props.**
Any divergence is a bug. Treat `hydrateRoot`'s second argument as a consistency check — if it would render different HTML than the server did, hydration will warn and the client will pay for a re-render.
2. **Defer browser-only work to `useEffect`.** Effects do not run during SSR and do not participate in hydration comparison — they run after hydration commits.
3. **Instrument `onRecoverableError`.** In production, mismatches do not log to the console — they silently re-render the subtree. Without `onRecoverableError` telemetry you will not know you are shipping wasted client work and layout shifts.

6. Islands, Partial Hydration, and Resumability

Full-page hydration treats every component as interactive, even when 90% of the page is static content that will never need JavaScript. Islands architecture inverts this: the page is mostly static HTML with isolated *islands* of interactivity that hydrate independently.

6.1 Astro Islands — Declarative Hydration Directives

Astro renders every component to static HTML by default (zero JS). Only components annotated with a `client:*` directive ship JavaScript and hydrate:

```

---
// src/pages/products/[id].astro ☐ Astro island page
import Layout from "../../layouts/Layout.astro";
import ProductDetail from "../../components/ProductDetail.astro"; // static ☐ no JS
import AddToCart from "../../components/AddToCart.tsx"; // React island
import Reviews from "../../components/Reviews.tsx"; // React island
import Recommendations from "../../components/Recommendations.tsx";

const { id } = Astro.params;
const product = await fetch(☐https://api.internal/products/${id}☐).then(
  (r => r.json());
// Astro SSG/SSR: this page can be pre-rendered (SSG) or server-rendered (SSR)
// via ☐export const prerender = true☐ or ☐output: "server"☐ in astro.config.mjs
---

<Layout title={product.name}>
  <☐-- Static ☐ no JS shipped, no hydration -->
  <ProductDetail product={product} />

  <☐-- Island: hydrates immediately on page load (high priority) -->
  <AddToCart productId={product.id} client:load />

  <☐-- Island: hydrates when visible (below the fold) -->
  <Reviews productId={product.id} client:visible />

  <☐-- Island: hydrates when idle (lowest priority, non-critical) -->
  <Recommendations productId={product.id} client:idle />

  <☐-- Island: hydrates on media query (e.g. mobile-only drawer) -->
  <☐-- <MobileDrawer client:media="(max-width: 768px)" /> -->

  <☐-- Island: never hydrates on client ☐ server-only (e.g. heavy chart rendered as image) -->
  <☐-- <ExpensiveChart data={data} server:only /> -->
</Layout>

```

```
// astro.config.mjs – Astro configuration at scale
import { defineConfig } from "astro/config";
import react from "@astrojs/react";
import svelte from "@astrojs/svelte"; // multiple frameworks per page – islands are framework-agnostic

export default defineConfig({
  output: "server", // "static" | "server" | "hybrid"
  adapter: await import("@astrojs/node").then(m => m.default({ mode: "s
tandalone" })),
  // Hybrid: pre-render marketing pages, SSR dynamic routes
  // src/pages/about.astro → prerender = true → SSG
  // src/pages/products/[id].astro → prerender = false → SSR

  integrations: [react(), svelte()],

  // Vite under the hood – same chunk-splitting concerns as Next.js
  vite: {
    build: {
      rollupOptions: {
        output: { manualChunks: { framework: ["react", "react-dom"] } }
      },
    },
  },
});
```

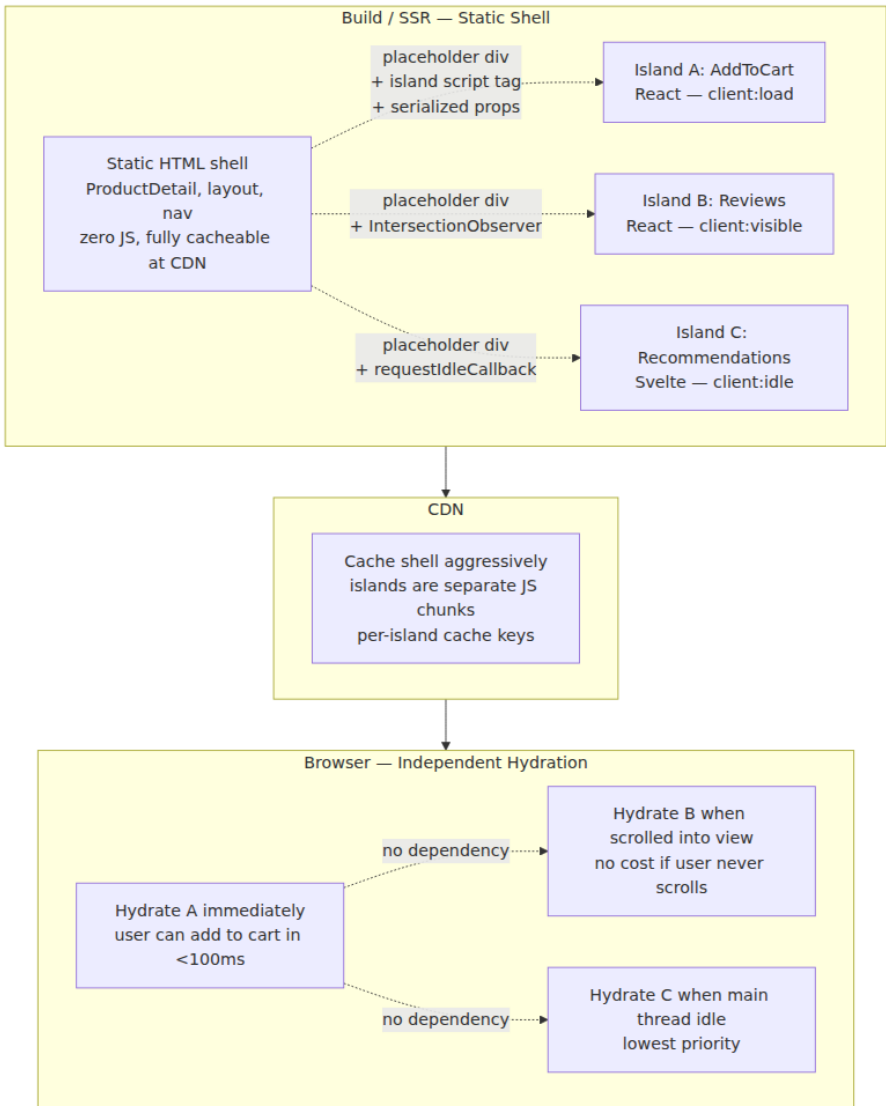
Hydration directives form a **priority and trigger matrix**:

Directive	When JS loads	When hydration runs	Use for
<code>client:load</code>	Eagerly with page	Immediately	Above-the-fold, critical interactivity
<code>client:idle</code>	Eagerly	<code>requestIdleCallback</code>	Below-the-fold, non-critical
<code>client:visible</code>	Lazily	<code>IntersectionObserver</code> fires	Offscreen content
<code>client:media</code>	Lazily	Media query matches	Responsive-only components

Directive	When JS loads	When hydration runs	Use for
<code>client:only="react"</code>	Eagerly	Immediately, no SSR	Browser-only components (maps, editors)

The scaling win is multiplicative: each `client:visible` island below the fold saves JS download, parse, and hydration on initial load. For a content-heavy page with one critical island (add-to-cart) and five deferred islands, the initial JS payload can drop by 70-80% — directly improving TTI and INP.

6.2 Islands Architecture — How It Fits Together



Key differences from full-page hydration:

- **Failure isolation.** A JS error in the Reviews island does not break AddToCart. With full-page `hydrateRoot`, an uncaught error during hydration can leave the entire page non-interactive.

- **Independent versioning.** Islands can be deployed and cached independently — the shell at `v42` can serve an island JS chunk at `v43` as long as the props contract is respected. This mirrors micro-frontend deployability.
- **Framework agnostic.** Astro islands can mix React, Svelte, Vue, and Solid on the same page — each island is its own framework runtime, scoped to its DOM subtree. The shell pays no framework cost.

Fresh (Deno) follows the same model with `islands/` directory convention — any component under `islands/` is an island, everything else is static.

6.3 Resumability: Qwik's Alternative to Hydration

Hydration — even partial, even selective — re-executes component code on the client to rebuild the state that was already computed on the server. Qwik observes that this is wasted work: the server already knew the component tree, props, and listeners. Instead of re-executing, Qwik *serializes* that knowledge into HTML and *resumes* on the client without re-execution.

Resumability (Qwik) — Resume

Server: render HTML
execute components

Serialize: HTML + QRLs
listener URLs + state
in HTML attributes

Client: NO re-execution
listeners are QRL stubs
onClick fetches handler
chunk

Interactive
cost = $O(\text{interacted islands})$

In Qwik, every event handler is a **QRL** (Qwik URL) — a lazy reference to a code chunk, serialized as an attribute:

```
// Qwik component – looks like React, behaves differently
import { component$, useSignal, $ } from "@builder.io/qwik";

export const Counter = component$(() => {
  const count = useSignal(0);
  // $( ) marks the handler as a separate chunk – not bundled with the component
  const increment = $(() => count.value++);

  // Rendered HTML includes: <button on:click="./chunk-abc.js#increment[0]">
  // No JS executes on load – clicking fetches chunk-abc.js and resumes state.
  return <button onClick$={increment}>Count: {count.value}</button>;
});

// Build output – handler is a separate file, fetched only on interaction
// dist/build/counter-abc.js → export const increment = () => { ... }
```

Comparison on a 100-component page where the user interacts with one counter:

Metric	Full hydration (React)	Partial hydration (Astro)	Resumability (Qwik)
JS downloaded before any interaction	Entire app bundle (e.g. 150 kB)	Shell + client:load islands (e.g. 30 kB)	~1 kB Qwik loader
JS executed before interaction	All 100 components re-execute	client:load islands re-execute	Zero component re-execution
JS on first interaction	Already loaded	Already loaded (if island hydrated) or load on trigger	Fetch one QRL chunk (e.g. 2 kB)
Time to interactive (TTI)	Hydration of whole tree completes	Hydration of critical islands completes	Loader parsed — effectively instant
State continuity	Rebuilt by re-execution	Rebuilt per island	Resumed from serialized state

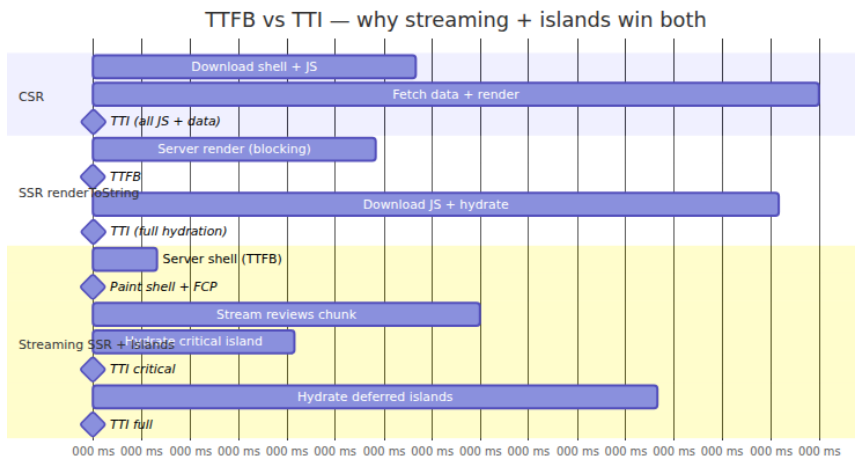
Resumability is not a free upgrade — it requires the framework to own serialization of state, listeners, and even lexical scope (closures must be serializable). Qwik's optimizer does this at build time via Rust-based transforms. For teams already on React, islands + partial hydration capture most of the benefit with less migration cost. For greenfield, latency-critical surfaces (e-commerce PDP, search), resumability's O(interaction) cost is unmatched.

7. Performance at Scale: TTFB, TTI, and the Distributed Systems Lens

7.1 Metrics That Actually Matter

Rendering choices move different Web Vitals in opposite directions. Optimizing one without measuring the others is how teams ship fast TTFB and slow INP.

Metric	What it measures	Rendering lever
TTFB (Time to First Byte)	Server + network latency to first HTML byte	SSR streaming helps; SSG/ISR helps most (CDN hit)
FCP (First Contentful Paint)	First DOM content painted	SSR/SSG win over CSR; streaming shell paints early
LCP (Largest Contentful Paint)	Largest element (hero, product image) painted	SSR helps if LCP element is server-rendered; streaming with Suspense can <i>delay</i> LCP if hero is inside a suspended boundary
TTI (Time to Interactive)	Main thread idle + listeners attached	Islands / resumability win; full hydration loses
INP (Interaction to Next Paint)	Latency of worst interaction	Less hydration JS → less main-thread contention → better INP
CLS (Cumulative Layout Shift)	Visual stability	Suspense fallbacks must reserve space; mismatches cause shifts



Three lessons from the timeline:

1. **Streaming moves TTFB left** (shell streams before slow data) and **islands move TTI left** (only critical islands hydrate before interactive). Together they close the TTFB-TTI gap that pure SSR leaves open.
2. **LCP can regress with streaming** if the LCP element is inside a Suspense boundary — the browser cannot paint it until the boundary resolves. Place LCP-critical content (hero image, product title) *outside* Suspense or in a high-priority boundary.
3. **TTI is not a single number** with selective hydration and islands. Report TTI-critical (time until primary CTA is interactive) and TTI-full separately — business metrics correlate with the former.

7.2 The Distributed Systems Lens

Rendering at scale is a distributed caching and consistency problem with a CDN, N SSR origins, a data API fleet, and browsers as the final replica.

Cache hit ratio is the dominant cost lever. Every SSR cache miss is an origin render that fans out to API calls. At 10k RPS with a 70% hit ratio, the origin handles 3k renders/s. At 95% hit ratio, 500 renders/s — a 6× reduction in origin fleet size. SSG and ISR exist to push the hit ratio toward 95%+ for cacheable pages. Measure hit ratio per route (`Cache-Status` , `X-Cache` , CDN analytics) and treat a drop as an incident.

Stale-while-revalidate and the thundering herd. When a popular ISR page expires, the first request triggers a background revalidation. If 1k requests arrive during the regeneration window and your cache does not coalesce them, you get 1k concurrent origin renders for the same page — a self-inflicted DDoS. Mitigations:

- **Request coalescing / singleflight** at the cache layer — only one regeneration per key at a time; others serve stale.
- **Soft TTL + hard TTL** — serve stale up to hard TTL while revalidating, only block when past hard TTL.
- **Origin concurrency limits** — cap concurrent renders per route; queue or shed load beyond the cap.

This is the same pattern as cache stampede protection in any distributed cache (see Volume 6, Chapter 3 — Caching Strategies).

Invalidation propagation and consistency. As noted in Section 3.3, ISR with N replicas and local caches is eventually consistent with no bound unless invalidations are broadcast. For price, inventory, or compliance-sensitive content, prefer on-demand revalidation with a shared cache or explicit CDN purge over time-based revalidation alone. Model it as you would any replicated state: define the consistency SLA (e.g., "price updates visible within 5 seconds globally"), instrument staleness, and alert when violated.

Independent deployability. Islands and micro-frontends let teams deploy rendering units independently — the shell at v42 can serve an island at v43. This requires a props contract (schema, versioned) and backward-compatible island interfaces. Treat island props like an API schema: version them, validate at build, and canary island JS changes separately from shell changes. Astro's per-island JS chunks make canarying natural; full-page hydration bundles make it all-or-nothing.

Observability per rendering tier:

```
# What to measure per route, per rendering strategy
# CDN layer
#   cache_hit_ratio, cache_age_seconds, revalidation_concurrency
# Origin layer
#   ssr_render_duration_p50/p95, render_error_rate, concurrent_renders
#   suspense_boundary_resolve_p95, streaming_chunk_count
# Browser layer (Real User Monitoring)
#   TTFB, FCP, LCP, INP, CLS, hydration_duration,
hydration_mismatch_count
#   island_hydration_duration{island="AddToCart"},
qrl_fetch_latency{chunk="..."}
```

If you measure only origin latency and not LCP/INP, you will optimize TTFB at the expense of the metrics users and search ranking actually reward.

Key takeaways

- CSR, SSR, SSG, ISR, and streaming SSR are placement decisions along *where* and *when* HTML is assembled. No single strategy is optimal for every page — mix them, often on the same route, guided by data freshness and traffic.
- `renderToString` is synchronous and blocking — simple but TTFB-bound by the slowest data fetch and memory-heavy under concurrency. `renderToPipeableStream` / `renderToReadableStream` stream the shell early, respect backpressure, and enable Suspense-driven progressive delivery.
- Next.js App Router collapses SSR/SSG/ISR into a unified `fetch` cache model (`cache` , `revalidate` , `tags`). Route-level `revalidate` and per-fetch `next.revalidate` replace the Pages Router's `getServerSideProps` / `getStaticProps` mode switch — understand HTTP caching to use it correctly.
- ISR is `stale-while-revalidate` with bounded staleness. Time-based `revalidate` bounds staleness passively; on-demand `revalidateTag` / `revalidatePath` closes the window on writes. Self-hosted ISR with N replicas requires a shared cache or broadcast invalidation to avoid split-brain.
- Streaming SSR via Suspense boundaries lets the server send the shell before slow data resolves, with inline scripts that swap fallbacks for content. Selective hydration then hydrates boundaries

in priority order — but proxies/WAFs that buffer chunked responses negate the benefit, and errors after headers flush cannot change the status code.

- `hydrateRoot` re-executes the component tree and reconciles against server DOM — it is $O(\text{tree size})$ CPU on the main thread. Mismatches are non-determinisms between server and client (random, time, browser globals, unforwarded auth); defer browser-only work to `useEffect` and instrument `onRecoverableError` to catch silent production mismatches.
- Islands architecture (Astro `client:*`, Fresh) makes most of the page static HTML and hydrates only interactive islands, each with its own trigger (`load` / `idle` / `visible` / `media`). This isolates failures, enables per-island caching and deployability, and can cut initial JS by 70%+.
- Qwik's resumability eliminates hydration entirely by serializing listeners and state as QRLs in HTML — the client resumes without re-execution, paying $O(\text{interacted})$ rather than $O(\text{tree})$ cost. It is the strongest TTI optimization but requires framework-level serialization.
- Performance is multi-dimensional: streaming helps TTFB/FCP, islands/resumability help TTI/INP, and Suspense placement can hurt LCP if the hero is suspended. Measure all of TTFB, FCP, LCP, TTI, INP, and CLS per route — and treat rendering as a distributed caching problem with hit ratio, stampede protection, and consistency SLAs.

Further reading

- React 18 — Rendering APIs: `renderToPipeableStream`, `renderToReadableStream`, `hydrateRoot`, and Suspense for SSR. <https://react.dev/reference/react-dom/server> and <https://react.dev/reference/react-dom/client/hydrateRoot>
- Next.js — App Router, Data Fetching and Caching, `fetch` extensions, `revalidateTag` / `revalidatePath`, and Route Segment Config (`revalidate`, `dynamic`, `dynamicParams`). <https://nextjs.org/docs/app/building-your-application/caching> and <https://nextjs.org/docs/app/api-reference/functions/revalidateTag>

- Next.js — `next.config.js` reference and `output: "standalone"` for container deploys. <https://nextjs.org/docs/app/api-reference/next-config-js>
- Astro — Islands architecture and `client:*` directives. <https://docs.astro.build/en/concepts/islands/>
- Fresh (Deno) — Islands architecture and partial hydration. <https://fresh.deno.dev/docs/concepts/islands>
- Qwik — Resumability vs hydration, QRLs, and the optimizer. <https://qwik.dev/docs/concepts/resumable/> and <https://qwik.dev/docs/advanced/optimizer/>
- HTTP — `stale-while-revalidate` and `stale-if-error` (RFC 5861) and Cache-Control extensions. <https://httpwg.org/specs/rfc5861.html>
- Web Performance — Core Web Vitals (LCP, INP, CLS), TTFB, and RUM vs lab data. <https://web.dev/vitals/> and <https://developer.mozilla.org/en-US/docs/Web/Performance>
- Vercel — Incremental Static Regeneration and On-Demand Revalidation deep dive (platform reference for ISR semantics that generalize to self-hosted). <https://vercel.com/docs/incremental-static-regeneration>

Chapter 10 — State Management, Data Fetching, and Caching (TanStack Query, SWR, Zustand/Jotai, Cache Invalidation)

What this chapter covers: The two hard problems every production frontend eventually faces — keeping remote data consistent with the server without drowning in `useEffect` fetch logic, and keeping local UI state predictable without turning every component into a prop-drilling relay. We dissect server state and client state as fundamentally different consistency problems, then open the four tools senior teams actually standardize on: TanStack Query for canonical server-state caching (query keys, `staleTime` vs. `gcTime`, deduplication, retries, infinite queries, optimistic updates), SWR for its stale-while-revalidate simplicity and when its trade-offs bite, Zustand for minimal store-based client state, and

Jotai for atom-graph fine-grained reactivity. Along the way we trace cache invalidation as a distributed invalidation problem, compare normalized vs. denormalized caches, and build an offline mutation queue that survives tab close and network partition.

Learning goals:

- Distinguish server state from client state by ownership, mutability, staleness, and invalidation semantics — and choose the correct primitive for each.
- Explain TanStack Query's cache model: `queryKey` as a hierarchical address, `staleTime` vs. `gcTime` (formerly `cacheTime`), deduplication, retry with exponential backoff, background refetch triggers, and garbage collection.
- Implement infinite queries (`useInfiniteQuery`), optimistic updates with rollback, and dependent/parallel query patterns without waterfalling.
- Compare TanStack Query and SWR on cache key design, revalidation triggers, mutation ergonomics, and middleware — and know when SWR's smaller API surface is the right call.
- Build client stores with Zustand (imperative store, selectors, subscriptions, middleware) and atom graphs with Jotai (primitive atoms, derived atoms, async atoms, atom dependency DAG).
- Design cache invalidation: `invalidateQueries` scoping, `refetchOnWindowFocus` / `refetchOnReconnect` / `refetchInterval` , tag-based and predicate invalidation, and the invalidation cascade.
- Contrast normalized vs. denormalized caches: why TanStack Query and SWR are denormalized by default, when normalization (entity tables, `normalizr`-style) pays off, and how Apollo Client's normalized cache differs.
- Implement an offline mutation queue with optimistic application, persistent outbox, conflict detection, and background sync replay — the frontend analog of a write-ahead log.

1. Two Kinds of State, Two Consistency Models

Every frontend bug that survives code review traces back to the same category error: treating server state as if it were client state.

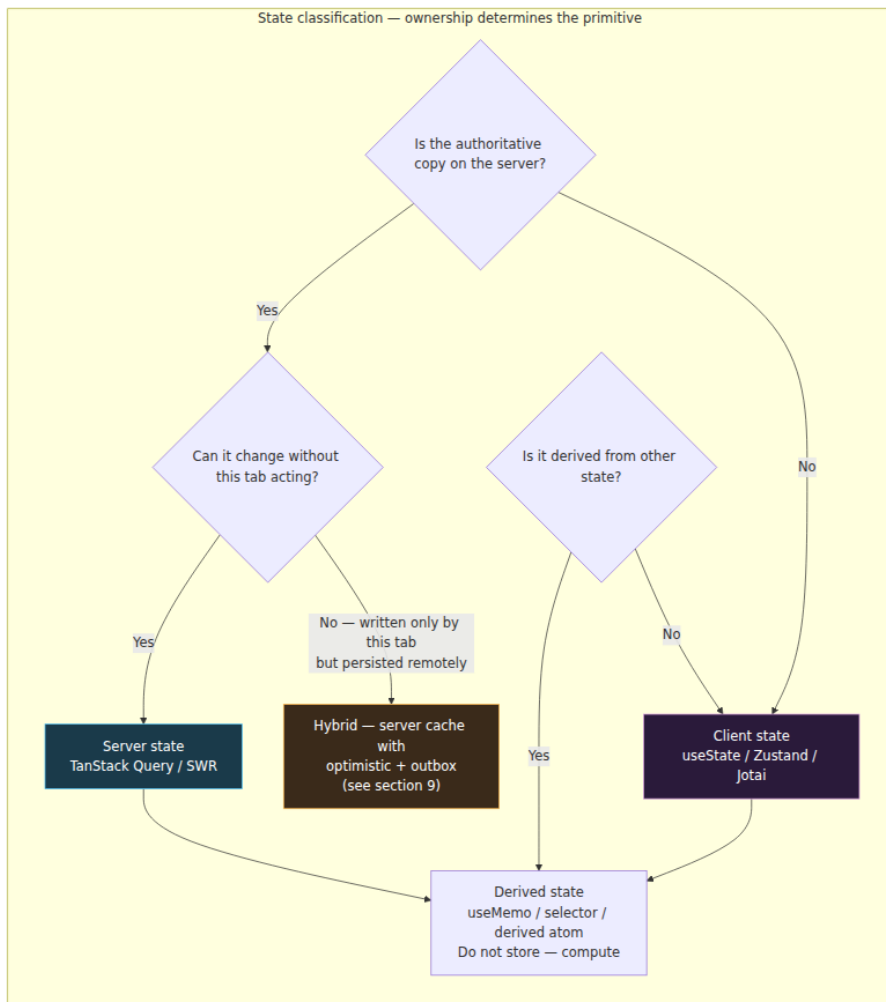
Property	Client state	Server state
Owner	The browser — this tab owns it	The server — the browser holds a cached replica
Mutability	Synchronous, local — <code>setState</code> is authoritative	Asynchronous, remote — local copy is stale the instant it is fetched
Source of truth	In-memory store / component	Database behind an API; other clients mutate it concurrently
Consistency hazard	Lost updates within the tab (rare)	Stale reads, write conflicts, concurrent writers (constant)
Invalidation	Explicit — reducer or setter	Implicit — TTL, focus, reconnect, push, poll, mutation
Persistence	Ephemeral unless persisted	Durable, but the cache is ephemeral

A backend engineer already has the vocabulary for this. Client state is thread-local memory. Server state is a cache over a remote database with an unknown number of concurrent writers and no invalidation channel unless you build one. The entire design space of TanStack Query and SWR is a read-through cache with background revalidation. The entire design space of Zustand and Jotai is an in-process store with subscriptions.

Getting the boundary wrong produces two characteristic failure modes:

- 1. Server state in a global store.** You `fetch('/api/users')` in a `useEffect`, `dispatch({ type: 'SET_USERS', payload })` into Redux/Zustand, and now you own the staleness problem yourself: when do you refetch? How do you deduplicate concurrent mounts? How do you garbage-collect? You have reimplemented half of TanStack Query, worse.
- 2. Client state in the server cache.** You stash `isDropdownOpen` or `draftComment` as a query with `staleTime: Infinity`. It works until someone calls `invalidateQueries()` broadly and wipes UI state, or until you try to persist it and discover the query cache is not a state machine.

The rule is mechanical: **if the data has an authoritative copy on the server and can change without this tab acting, it is server state and belongs in a server-state cache. Otherwise it is client state and belongs in a store or component state.**



A practical test when reviewing a PR: look at each `useState` and each store slice and ask *would this value be wrong if another user mutated the same entity one second ago?* If yes, it should be a query, not state. Conversely, if a query key contains UI-only concerns (sort order, selected

tab, ephemeral form draft), it should be a store atom that *parameterizes* the query, not part of the cached value.

1.1 Where the old patterns break

The naive fetch-in-effect pattern fails on every property a cache must provide:

```
// Anti-pattern – every concern is manual and subtly wrong
function UsersList() {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<Error | null>(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    fetch("/api/users")
      .then((r) => {
        if (!r.ok) throw new Error(String(r.status));
        return r.json();
      })
      .then((data) => {
        if (!cancelled) setUsers(data);
      })
      .catch((e) => {
        if (!cancelled) setError(e);
      })
      .finally(() => {
        if (!cancelled) setLoading(false);
      });
    return () => { cancelled = true; };
  }, []); // empty deps – never refetches, even on window focus or
  reconnect

  // Missing: deduplication (two mounts = two fetches)
  // Missing: retry with backoff (one transient 503 = permanent error)
  // Missing: garbage collection (users stays in memory after unmount
  forever)
  // Missing: background revalidation (stale after first render,
  forever)
  // Missing: dependent query coordination (fetch posts after users?
  waterfall)
}
```

Each `// Missing` comment is a feature TanStack Query or SWR provides by default. Reimplementing them correctly requires roughly 300 lines of

careful cache, timer, and subscription logic — which is precisely what those libraries are.

2. TanStack Query — A Read-Through Cache for Server State

TanStack Query (v5, formerly React Query) is not a data-fetching library. It is a **cache with a fetcher interface**. Its core abstraction is `QueryCache` — an in-memory `Map<QueryKey, QueryEntry>` where each entry holds `data`, `error`, `status`, `fetchStatus`, `dataUpdatedAt`, and timers for staleness and garbage collection. Every hook (`useQuery`, `useInfiniteQuery`, `useQueries`) is a subscription to one or more entries in that cache.

2.1 `queryKey` — The Hierarchical Address

The `queryKey` is the cache address. It is an array, compared by deep structural equality (via `dequal`-style hashing), where prefix matching determines invalidation scope.

```

import { QueryClient, useQuery, useQueryClient } from "@tanstack/react-
query";

// Key design – hierarchical, serializable, stable
const userKeys = {
  all: ["users"] as const,
  lists: () => [...userKeys.all, "list"] as const,
  list: (filters: { role?: string; page?: number }) =>
    [...userKeys.lists(), filters] as const,
  details: () => [...userKeys.all, "detail"] as const,
  detail: (id: string) => [...userKeys.details(), id] as const,
};

// Usage – each key is a distinct cache entry
function useUsers(filters: { role?: string }) {
  return useQuery({
    queryKey: userKeys.list(filters), // e.g. ["users", "list",
    {role:"admin"}]
    queryFn: () => fetchUsers(filters), // only called on cache
    miss or stale revalidation
  });
}

function useUser(id: string) {
  return useQuery({
    queryKey: userKeys.detail(id), // ["users", "detail", "42"]
    queryFn: () => fetchUser(id),
    enabled: !!id, // dependent query –
    disabled until id is truthy
  });
}

// Invalidation by prefix – one call wipes the subtree
function useInvalidateUsers() {
  const qc = useQueryClient();
  return {
    invalidateAll: () => qc.invalidateQueries({ queryKey:
userKeys.all }),
    invalidateLists: () => qc.invalidateQueries({ queryKey: userKeys.li
sts() }),
    invalidateOne: (id: string) => qc.invalidateQueries({ queryKey: use
rKeys.detail(id) }),
  };
}

```

Three rules that prevent the most common production bugs:

1. **Keys must be serializable.** The key is hashed to a string for Map lookup. Non-serializable members (class instances, functions) hash

unstably and cause phantom misses. Use primitives, plain objects, and arrays.

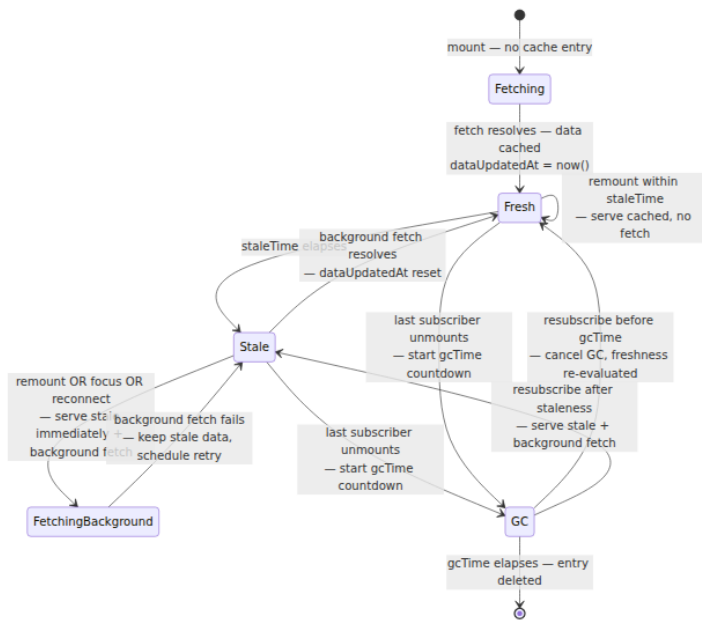
- 2. **Include every variable that affects the fetch in the key.** If `queryFn` reads `filters.role` but the key is `["users"]`, two different filter values alias to the same cache entry. The key must be a pure function of the fetcher's inputs.
- 3. **Use the factory pattern.** The `userKeys` factory above makes prefix invalidation trivial and prevents typos. Without it, `invalidateQueries({ queryKey: ["users"] })` and `useQuery({ queryKey: ["user"] })` silently fail to match — a one-character namespace bug that no type checker catches unless the factory is typed.

The `queryKey` hashing also explains why inline object literals without memoization still work: TanStack Query hashes by value, not reference. `{ role: "admin" }` on two renders produces the same hash, so no extra fetch is triggered. But inline *arrays* that include unstable values (e.g., `new Date()`) will thrash the cache.

2.2 `staleTime` vs. `gcTime` — Two Timers, Two Concerns

This is the most misconfigured surface in TanStack Query. The two timers control orthogonal lifecycles:

Timer	Old name (v4)	Default	What it controls
<code>staleTime</code>	<code>staleTime</code>	<code>0</code> (immediately stale)	How long cached data is considered fresh . Fresh data is returned without a background fetch. Stale data is returned immediately <i>and</i> refetched in the background.
<code>gcTime</code>	<code>cacheTime</code>	5 minutes	How long an unused cache entry (zero active subscribers) is retained before garbage collection. After GC, the next mount is a hard cache miss.



Configuration guidance for backend-minded teams:

```

import { QueryClient } from "@tanstack/react-query";

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 30_000,          // 30s – tune per resource volatility
      gcTime: 5 *
60_000,          // 5 min – keep unused entries for quick back-nav
      retry: 3,                  // transient failure tolerance (see 2.4)
      retryDelay: (attempt) => Math.min(1000 * 2 ** attempt, 30_000),
      refetchOnWindowFocus: true,
      refetchOnReconnect: true,
      refetchOnMount: true,     // respects staleTime – only refetches
    if stale
    },
  },
});

// Per-query overrides – match staleness to mutation frequency
const queries = {
  // Reference data – rarely changes, safe to keep fresh longer
  useCountries: () =>
    useQuery({
      queryKey: ["countries"],
      queryFn: fetchCountries,
      staleTime: 60 * 60_000,    // 1 hour
      gcTime: 24 * 60 * 60_000,
    }),

  // Hot data – changes frequently, tolerate extra fetches for
  freshness
  useOrderBook: (symbol: string) =>
    useQuery({
      queryKey: ["orderbook", symbol],
      queryFn: () => fetchOrderBook(symbol),
      staleTime: 2_000,          // 2s – frequent revalidation
      refetchInterval: 5_000,    // poll every 5s while mounted
    }),

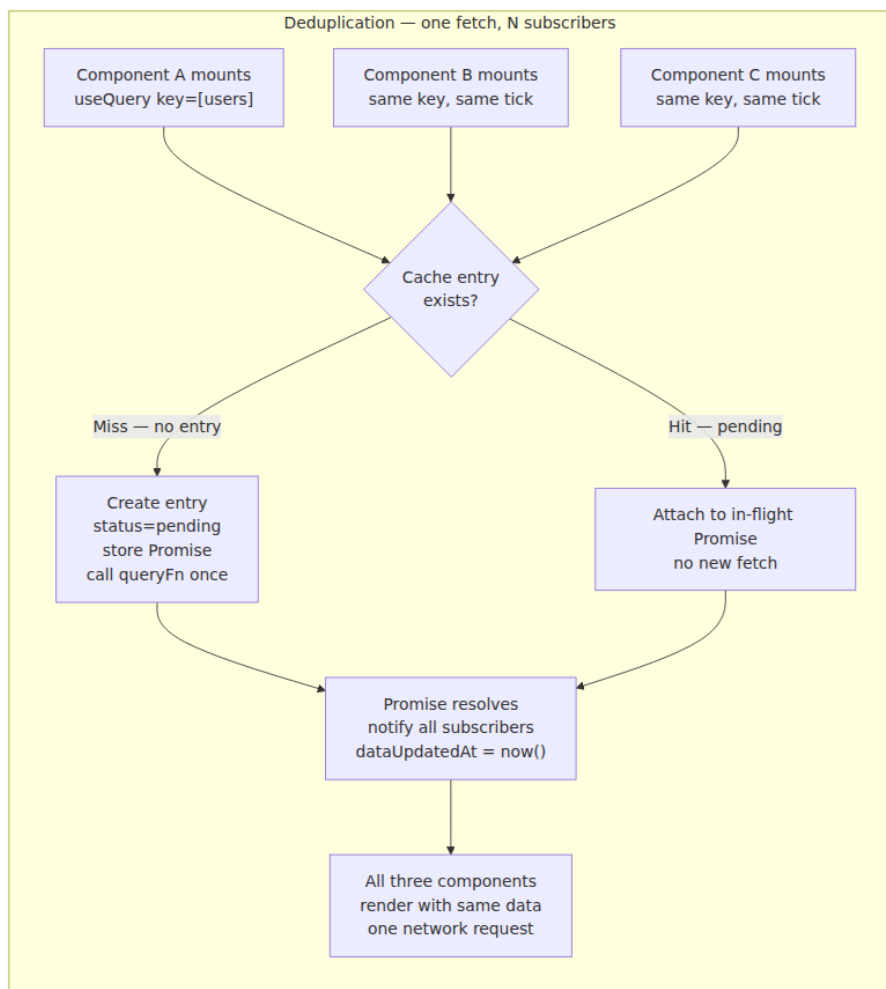
  // User-specific – fresh on navigation, but don't refetch on every
  focus
  useCurrentUser: () =>
    useQuery({
      queryKey: ["me"],
      queryFn: fetchMe,
      staleTime: 5 * 60_000,
      refetchOnWindowFocus: false, // avoid noisy refetch on alt-tab
    }),
};

```

The back-nav case is why `gcTime` matters. With `gcTime: 0`, navigating from a list to a detail and back triggers a full refetch and a loading spinner — even though `staleTime` was satisfied. With `gcTime: 5m`, the list data is still in the cache and renders synchronously on back-nav, then revalidates in the background if stale. This is the same trade-off as a CPU cache vs. main memory: `staleTime` is the coherence window, `gcTime` is the eviction policy.

2.3 Deduplication and Request Coalescing

When three components mount simultaneously and call `useQuery({ queryKey: ["users"], queryFn: fetchUsers })`, TanStack Query issues **one** network request. The mechanism is straightforward: the first subscriber triggers `fetch`, subsequent subscribers with the same key within the same tick attach to the in-flight `Promise` stored on the cache entry.



This replaces the manual `let inflight: Promise | null` singleton that teams otherwise scatter across service modules. It also handles the race where a component unmounts mid-fetch: the fetch continues (it is a cache-level operation, not a component-level one), and the result is still cached for the next subscriber.

Key detail: deduplication is keyed on the **hashed queryKey**, not on the `queryFn` reference. Two hooks with the same key but different `queryFn` still deduplicate to one fetch — the first `queryFn` wins. This is why `queryFn` should always be a pure function of the key.

Beyond single-key dedup, `useQueries` and prefetching handle fan-out without waterfalling:

```
// Parallel – N queries, N cache entries, maximally concurrent
function useUsersParallel(ids: string[]) {
  return useQueries({
    queries: ids.map((id) => ({
      queryKey: userKeys.detail(id),
      queryFn: () => fetchUser(id),
    })),
  });
}

// Prefetch on hover – warm the cache before navigation
function UserLink({ id }: { id: string }) {
  const qc = useQueryClient();
  return (
    <a
      href={`\`/users/${id}\`}
      onMouseEnter={() =>
        qc.prefetchQuery({
          queryKey: userKeys.detail(id),
          queryFn: () => fetchUser(id),
          staleTime: 30_000,
        })
      }
    >
      View {id}
    </a>
  );
}

// Dependent query – second fetch waits for first, but is still cached/
// deduped
function useUserPosts(userId: string) {
  const userQ = useQuery({
    queryKey: userKeys.detail(userId),
    queryFn: () => fetchUser(userId),
  });
  const postsQ = useQuery({
    queryKey: ["posts", { userId }],
    queryFn: () => fetchPostsByUser(userId),
    enabled: !!userQ.data, // gated – no fetch until user resolves
  });
  return { userQ, postsQ };
}
```

2.4 Retries — Exponential Backoff with Correct Defaults

TanStack Query retries **queries** but not **mutations** by default — the right call, since mutations are non-idempotent.

```
useQuery({
  queryKey: ["users"],
  queryFn: fetchUsers,
  retry: 3,
  retryDelay: (attempt) => Math.min(1000 * 2 ** attempt, 30_000),
  // attempt 0 -> 1000ms, attempt 1 -> 2000ms, attempt 2 -> 4000ms
});

// Conditional retry – don't retry on 4xx, only on network/5xx
useQuery({
  queryKey: ["users"],
  queryFn: fetchUsers,
  retry: (failureCount, error) => {
    if (error instanceof ApiError && error.status >= 400 && error.statu
s < 500) return false;
    return failureCount < 3;
  },
});

// Mutations – never retry by default; opt in only for idempotent
mutations
const mutation = useMutation({
  mutationFn: createUser,
  retry: false, // correct default – creating a user twice is not safe
});

// Idempotent mutation – safe to retry
const idempotentMutation = useMutation({
  mutationFn: (vars: { id: string; name: string }) =>
    putUser(vars), // PUT is idempotent
  retry: 2,
});
```

The retry timer is per-query-entry, not per-component. If a query fails and two components are subscribed, both see `status: 'error'` and both benefit from the single retry schedule.

2.5 Infinite Queries — Cursor Pagination as a Cache Problem

`useInfiniteQuery` models paginated data as a single cache entry whose `data` is `{ pages: TPage[], pageParams: unknown[] }`. Each `fetchNextPage()` call appends a page; the cache entry grows monotonically until invalidated or garbage-collected.

```

type Project = { id: string; name: string };
type ProjectsPage = { items: Project[]; nextCursor: string | null };

function useInfiniteProjects(filters: { teamId: string }) {
  return useInfiniteQuery({
    queryKey: ["projects", "infinite", filters],
    queryFn: ({ pageParam, signal }) =>
      fetchProjects({ ...filters, cursor: pageParam as string | undefined, signal },
        initialPageParam: undefined as string | undefined,
        getNextPageParam: (lastPage) => lastPage.nextCursor ?? undefined, /
        / undefined = no more pages
        getPreviousPageParam: (firstPage) => undefined, // cursor
        pagination - no backward param
        staleTime: 30_000,
      });
}

// Component - renders flattened pages, fetches on scroll
function ProjectsList({ teamId }: { teamId: string }) {
  const {
    data,
    fetchNextPage,
    hasNextPage,
    isFetchingNextPage,
    status,
  } = useInfiniteProjects({ teamId });

  // IntersectionObserver trigger
  const sentinelRef = React.useRef<HTMLDivElement>(null);
  React.useEffect(() => {
    if (!hasNextPage) return;
    const el = sentinelRef.current;
    if (!el) return;
    const io = new IntersectionObserver(
      ([entry]) => {
        if (entry.isIntersecting && !isFetchingNextPage) fetchNextPage(
);
      },
      { rootMargin: "400px" }
    );
    io.observe(el);
    return () => io.disconnect();
  }, [hasNextPage, isFetchingNextPage, fetchNextPage]);

  if (status === "pending") return <Skeleton />;
  const all = data!.pages.flatMap((p) => p.items);
  return (
    <>
      <ul>{all.map((p) => <li key={p.id}>{p.name}</li>)}</ul>

```

```

    <div ref={sentinelRef} />
    {isFetchingNextPage && <Skeleton />}
    {!hasNextPage && <p>End of list.</p>}
  </>
);
}

```

Operational details that matter at scale:

- **pageParam is opaque.** The cache does not interpret it; `getNextPageParam` extracts it from the last page's response. For offset pagination, return `pages.length`; for cursor pagination, return `lastPage.nextCursor`.
- **Refetch refetches all pages.** `invalidateQueries({ queryKey: ["projects"] })` re-fetches every page sequentially via `queryFn` with each stored `pageParam`. For long lists this is expensive — prefer targeted invalidation or `setQueryData` patching (see section 2.6).
- **select for derived views.** Flattening `pages.flatMap(...)` on every render is O(total items). Use `select` to memoize:

```

const { data: projects } = useInfiniteProjects({ teamId });
const flat = React.useMemo(() => projects?.pages.flatMap((p) =>
p.items) ?? [], [projects]);

// Or inside the query – cached and referentially stable across renders
const q = useInfiniteQuery({
  // ...
  select: (data) => data.pages.flatMap((p) => p.items),
});

```

- **Bidirectional infinite scroll** (`getPreviousPageParam` + `fetchPreviousPage`) is supported but rare. Most backends only need forward pagination; implementing backward fetch requires the API to return `prevCursor`.

2.6 Optimistic Updates — Applying Writes Before They Commit

Optimistic updates make mutations feel instant by applying the expected result to the cache synchronously, then reconciling with the server response. The pattern has four phases: snapshot, optimistically patch, commit-or-rollback, reconcile.

```

type Todo = { id: string; title: string; done: boolean };

function useToggleTodo() {
  const qc = useQueryClient();

  return useMutation({
    mutationFn: ({ id, done }: { id: string; done: boolean }) =>
      patchTodo(id, { done }), // PUT /todos/:id

    // 1. Snapshot + optimistic patch – runs synchronously before the
    // network request
    onMutate: async ({ id, done }) => {
      // Cancel outgoing refetches so they don't overwrite the
      // optimistic value
      await qc.cancelQueries({ queryKey: ["todos"] });

      // Snapshot previous value for rollback
      const previous = qc.getQueryData<Todo[]>(["todos"]);

      // Optimistically update – synchronous, UI reflects immediately
      qc.setQueryData<Todo[]>(["todos"], (old) =>
        old ? old.map((t) => (t.id === id ? { ...t, done } : t)) : old
      );

      return { previous };
    },

    // 2. Rollback on failure – restore snapshot
    onError: (_err, _vars, context) => {
      if (context?.previous) {
        qc.setQueryData(["todos"], context.previous);
      }
    },

    // 3. Reconcile – refetch to converge with server truth
    onSettled: () => {
      qc.invalidateQueries({ queryKey: ["todos"] });
    },
  });
}

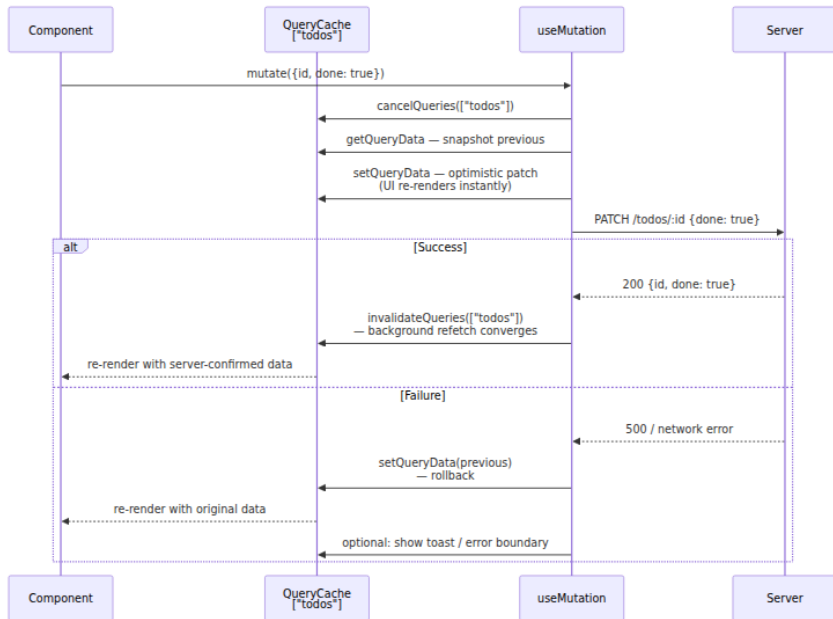
// Usage – no loading spinner on the checkbox; the toggle is instant
function TodoItem({ todo }: { todo: Todo }) {
  const { mutate } = useToggleTodo();
  return (
    <label>
      <input
        type="checkbox"
        checked={todo.done}
        onChange={() => mutate({ id: todo.id, done: !todo.done })}
      />
    </label>
  );
}

```

```

    />
    {todo.title}
  </label>
);
}

```



Key invariants:

- **Always cancelQueries before setQueryData.** Without cancellation, an in-flight background refetch can resolve after the optimistic patch and overwrite it with stale server data — a lost-update bug.
- **Snapshot via getQueryData, not closure.** The closure's data may be stale if another mutation already patched the cache. getQueryData reads the current cache entry atomically.
- **Prefer onSettled invalidation over manual setQueryData on success.** The server may have applied side effects (timestamps, derived fields) that the optimistic patch did not anticipate. Invalidation converges to truth. For latency-sensitive cases, patch on success *and* invalidate — the patch hides the refetch latency.
- **Optimistic updates are not transactions.** Two concurrent optimistic mutations to the same entity can interleave. TanStack

Query serializes `onMutate` calls via `cancelQueries`, but if two mutations patch overlapping fields, the second snapshot includes the first optimistic value. On rollback, restoring the second snapshot does not undo the first mutation — use per-entity `setQueryData` or a proper store for contended entities.

For creates (no `id` yet), generate a temporary client id and replace it on success:

```
function useCreateTodo() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (vars: { title: string }) => postTodo(vars),
    onMutate: async ({ title }) => {
      await qc.cancelQueries({ queryKey: ["todos"] });
      const previous = qc.getQueryData<Todo[]>(["todos"]);
      const temp: Todo = { id: `temp-${Date.now()}`, title, done:
false };
      qc.setQueryData<Todo[]>(["todos"], (old) => (old ? [...old,
temp] : [temp]));
      return { previous, tempId: temp.id };
    },
    onSuccess: (created, _vars, ctx) => {
      // Replace temp entry with server-confirmed entity (real id)
      qc.setQueryData<Todo[]>(["todos"], (old) =>
old ? old.map((t) => (t.id === ctx?.tempId ? created : t)) : ol
d
      );
    },
    onError: (_e, _v, ctx) => {
      if (ctx?.previous) qc.setQueryData(["todos"], ctx.previous);
    },
  });
}
```

3. SWR — Stale-While-Revalidate, Minimal Surface

SWR (from Vercel, `swr` on npm) implements the HTTP cache directive `stale-while-revalidate` as a React hook. The name is the algorithm: return stale data immediately, revalidate in the background, then return fresh data.


```

import useSWR from "swr";
import useSWRInfinite from "swr/infinite";
import useSWRMutation from "swr/mutation";

// Basic – fetcher is (key) => Promise<data>
const fetcher = (url: string) => fetch(url).then((r) => {
  if (!r.ok) throw new Error(String(r.status));
  return r.json();
});

function Profile({ id }: { id: string }) {
  const { data, error, isLoading, isValidating, mutate } = useSWR(
    id ? `/api/users/${id}` : null, // null key = disabled (like
    enabled: false)
    fetcher,
    {
      revalidateOnFocus: true,
      revalidateOnReconnect: true,
      dedupingInterval: 2000, // dedup window – default 2s
      errorRetryCount: 3,
      // No staleTime/gcTime distinction – SWR's cache is simpler
    }
  );
  if (isLoading) return <Skeleton />;
  if (error) return <ErrorBox error={error} />;
  return <div>{data.name}</div>;
}

// Mutation – explicit mutate call, no built-in optimistic helper
function useUpdateName(id: string) {
  const { mutate } = useSWR(`/api/users/${id}`, fetcher);
  return async (name: string) => {
    // Optimistic – manually patch, then revalidate
    await mutate(
      async (current: any) => {
        await fetch(`/api/users/${id}`, {
          method: "PATCH",
          body: JSON.stringify({ name }),
        });
        return { ...current, name }; // return new data for cache
      },
      {
        optimisticData: (current: any) => ({ ...current, name }),
        rollbackOnError: true,
        revalidate: true,
      }
    );
  };
}

```

```

// Infinite – key is a function of page index and previous page
function ProjectsInfinite({ teamId }: { teamId: string }) {
  const { data, size, setSize, isLoading } = useSWRInfinite(
    (index, prev) => {
      if (prev && !prev.nextCursor) return null; // no more pages –
disable
      const cursor = prev?.nextCursor ?? "";
      return `/api/projects?team=${teamId}&cursor=${cursor}&page=${inde
x}`;
    },
    fetcher
  );
  const items = data?.flatMap((p) => p.items) ?? [];
  return (
    <
      <ul>{items.map((p) => <li key={p.id}>{p.name}</li>)}</ul>
      <button onClick={() => setSize(size + 1)}>Load more</button>
    </>
  );
}

```

3.1 TanStack Query vs. SWR — When Each Fits

Dimension	TanStack Query	SWR
Cache key	Structured array <code>["users", { id }]</code> with prefix invalidation	String (or array serialized to string); invalidation by key string or predicate
Freshness control	Two knobs: <code>staleTime</code> + <code>gcTime</code> — precise coherence vs. eviction	One knob: <code>dedupingInterval</code> — dedup window only; no explicit freshness TTL
GC	Explicit — entry deleted after <code>gcTime</code> with no subscribers	Implicit — cache persists in memory; no per-entry GC timer
Dedup	Per-key, tick-coalesced, cache-level	Per-key, <code>dedupingInterval</code> window (default 2s)
Retry	Configurable count + backoff + predicate	<code>errorRetryCount</code> + <code>errorRetryInterval</code> ; less expressive predicate

Dimension	TanStack Query	SWR
Optimistic updates	First-class <code>onMutate</code> / <code>onError</code> / <code>onSettled</code> with <code>cancelQueries</code> + snapshot	<code>mutate</code> with <code>optimisticData</code> + <code>rollbackOnError</code> — manual but workable
Infinite	<code>useInfiniteQuery</code> with <code>getNextPageParam</code> + flattened <code>pages</code>	<code>useSWRInfinite</code> with key function <code>(index, prev) => key \ null</code>
Dependent queries	<code>enabled</code> gate	<code>null</code> key gate — idiomatic and concise
DevTools	Dedicated DevTools panel (query inspector, cache explorer)	No official DevTools; relies on React DevTools
Bundle size	~13 kB min+gzip	~4 kB min+gzip
Ecosystem	<code>persistQueryClient</code> , <code>createPersister</code> , framework adapters	<code>swr</code> middleware, cache provider API

For backend teams, the decision heuristic is:

- **Choose TanStack Query** when cache correctness matters more than bundle size: multiple teams sharing a cache, complex invalidation, background sync, offline support, or when you need DevTools to debug stale-data incidents. This is the default for non-trivial SPAs.
- **Choose SWR** when the app is small, the data is mostly read-only, or you are already in the Vercel/Next.js ecosystem and want the lightest possible revalidation layer. SWR is also the right call when the fetcher is already a thin wrapper over `fetch` and you do not need hierarchical invalidation.

Both are denormalized caches. Neither normalizes entities (see section 7).

4. Client State — Zustand and Jotai

Client state — UI toggles, form drafts, selection, ephemeral filters — belongs in a synchronous, local store with subscriptions. Two designs dominate modern React.

4.1 Zustand — A Minimal External Store

Zustand is a ~1 kB external store built on `useSyncExternalStore`. There is one store object, one `set` function, and selectors for granular subscriptions. No provider, no context, no boilerplate.

```

import { create } from "zustand";
import { devtools, persist, subscribeWithSelector } from "zustand/
middleware";
import { immer } from "zustand/middleware/immer";

// Store – single source of truth for client state
type Filters = { query: string; role: "all" | "admin" | "member";
page: number };
type UIState = {
  filters: Filters;
  selectedIds: Set<string>;
  isCommandPaletteOpen: boolean;
  // Actions – colocated with state, no separate reducer file
  setQuery: (q: string) => void;
  setRole: (r: Filters["role"]) => void;
  toggleSelect: (id: string) => void;
  toggleCommandPalette: () => void;
  reset: () => void;
};

const initialFilters: Filters = { query: "", role: "all", page: 1 };

export const useUIStore = create<UIState>()(
  devtools(
    persist(
      subscribeWithSelector(
        immer((set) => ({
          filters: initialFilters,
          selectedIds: new Set<string>(),
          isCommandPaletteOpen: false,

          setQuery: (query) =>
            set((s) => {
              s.filters.query = query;
              s.filters.page = 1; // reset pagination on filter change
            })),

          setRole: (role) =>
            set((s) => {
              s.filters.role = role;
              s.filters.page = 1;
            })),

          toggleSelect: (id) =>
            set((s) => {
              if (s.selectedIds.has(id)) s.selectedIds.delete(id);
              else s.selectedIds.add(id);
            })),

          toggleCommandPalette: () =>

```

```

        set((s) => {
            s.isCommandPaletteOpen = !s.isCommandPaletteOpen;
        })),

        reset: () =>
            set((s) => {
                s.filters = { ...initialFilters };
                s.selectedIds = new Set();
            })),
    )),
    { name: "ui-store", partialize: (s) => ({ filters:
s.filters }) } // persist only filters
    ),
    { name: "UIStore" }
    )
);

// Selectors – each component subscribes to a slice; unrelated changes
don't re-render
function SearchInput() {
    const query = useUIStore((s) => s.filters.query);
    const setQuery = useUIStore((s) => s.setQuery);
    return <input value={query} onChange={(e) => setQuery(e.target.value)}
    />;
}

function RoleFilter() {
    const role = useUIStore((s) => s.filters.role);
    const setRole = useUIStore((s) => s.setRole);
    return (
        <select value={role} onChange={(e) => setRole(e.target.value as Fil
ters["role"])>
            <option value="all">All</option>
            <option value="admin">Admin</option>
            <option value="member">Member</option>
        </select>
    );
}

// Wiring client state to server state – store drives query key
function UsersPage() {
    const filters = useUIStore((s) => s.filters);
    const { data, isPending } = useQuery({
        queryKey: ["users", "list", filters],
        queryFn: () => fetchUsers(filters),
        placeholderData: (prev) => prev, // keep previous page visible
        while fetching next
    });
    // ...
}

```

```
// Subscriptions outside React – for imperative integrations
(analytics, WebSocket handlers)
const unsub = useUIStore.subscribe(
  (s) => s.filters,
  (filters, prev) => {
    console.log("filters changed", prev, "->", filters);
    // e.g., push to URL search params for shareable links
    const params = new URLSearchParams(filters as any);
    history.replaceState(null, "", `?${params}`);
  },
  { equalityFn: (a, b) => JSON.stringify(a) === JSON.stringify(b) }
);
```

How Zustand subscriptions work internally (simplified):

```
// Simplified Zustand core – useSyncExternalStore over a plain object
function createStore<T>(initializer: (set: SetState<T>, get:
GetState<T>) => T) {
  let state: T;
  const listeners = new Set<() => void>();
  const set: SetState<T> = (partial) => {
    const next = typeof partial === "function" ? (partial as any)
(state) : partial;
    if (Object.is(next, state)) return;
    state = { ...state, ...next };
    listeners.forEach((l) => l()); // notify all subscribers
synchronously
  };
  const get: GetState<T> = () => state;
  const subscribe = (listener: () => void) => {
    listeners.add(listener);
    return () => listeners.delete(listener);
  };
  state = initializer(set, get);
  // Hook – selector + equality check prevents re-render when slice is
unchanged
  return (selector: (s: T) => any = (s) => s) => {
    return useSyncExternalStore(subscribe, () => selector(state), ()
=> selector(state));
  };
}
```

Trade-offs vs. Redux Toolkit:

Concern	Zustand	Redux Toolkit
Boilerplate	Zero — store is a hook	Slices, reducers, configureStore, provider

Concern	Zustand	Redux Toolkit
DevTools	Via <code>devtools</code> middleware — time-travel	Built-in, more mature
Middleware	Composable (<code>persist</code> , <code>immer</code> , <code>subscribeWithSelector</code>)	<code>createListenerMiddleware</code> , <code>redux-persist</code>
Selector stability	Manual — <code>useShallow</code> or custom <code>equalityFn</code>	<code>createSelector</code> (reselect) memoized
Team scaling	Lightweight for small/medium apps; can fragment if teams create many stores	Single-store discipline scales to large teams

For senior backend teams, the key discipline with Zustand is **store cohesion**: one domain per store, not one store per component. A `useUIStore` for filters/selection and a `useEditorStore` for draft state is correct. A `useButtonStore` for one button's `isOpen` is not — that is `useState`.

4.2 Jotai — Atoms as a Dependency Graph

Jotai inverts the model: instead of one store with selectors, there are many **atoms** — minimal units of state — composed into a directed acyclic graph. Derived atoms recompute when their dependencies change; components subscribe to individual atoms and re-render only when that atom's value changes.


```

import { atom, createStore } from "jotai";
import { atomWithQuery, atomWithMutation } from "jotai-tanstack-query";

// Primitives – each atom is an independent read/write node
const queryAtom = atom("");
const roleAtom = atom<"all" | "admin" | "member">("all");
const pageAtom = atom(1);

// Derived (read-only) – recomputes when any dependency changes
const filtersAtom = atom((get) => ({
  query: get(queryAtom),
  role: get(roleAtom),
  page: get(pageAtom),
}));

// Derived (read-write) – custom read + write
const searchAtom = atom(
  (get) => get(queryAtom),
  (_get, set, value: string) => {
    set(queryAtom, value);
    set(pageAtom, 1); // side effect on write – reset pagination
  }
);

// Async derived – Jotai suspends while the promise is pending
// (Suspense-compatible)
const usersAtom = atom(async (get) => {
  const filters = get(filtersAtom);
  const res = await fetch(`/api/users?${new URLSearchParams(filters as any)}`);
  if (!res.ok) throw new Error(String(res.status));
  return res.json() as Promise<User[]>;
});

// Action atom – write-only, for mutations
const toggleRoleAtom = atom(null, (get, set) => {
  const current = get(roleAtom);
  set(roleAtom, current === "admin" ? "member" : "admin");
});

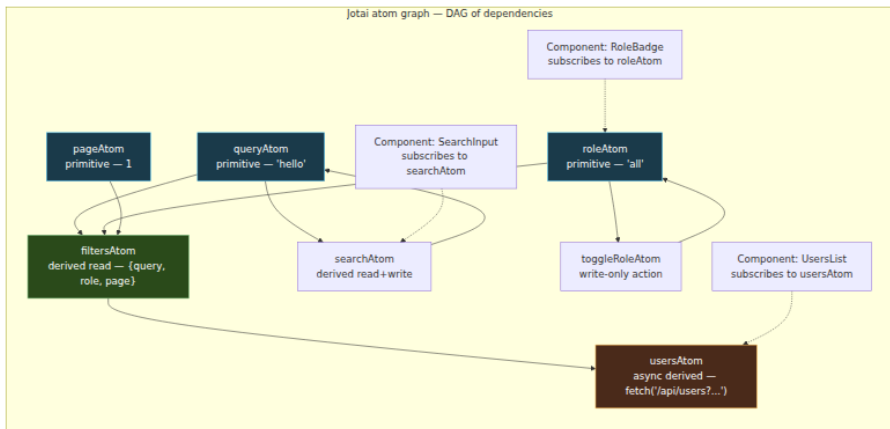
// Usage – each component subscribes to exactly the atoms it reads
import { useAtom, useAtomValue, useSetAtom } from "jotai";

function SearchInput() {
  const [query, setQuery] = useAtom(searchAtom); // subscribes to
  queryAtom only
  return <input value={query} onChange={(e) => setQuery(e.target.value)} />;
}

```

```
function RoleBadge() {
  const role =
  useAtomValue(roleAtom); // subscribes to roleAtom only – not queryAtom
  return <span>{role}</span>;
}

function UsersList() {
  const users = useAtomValue(usersAtom); // suspends until fetch
  resolves
  return <ul>{users.map((u) => <li key={u.id}>{u.name}</li>)}</ul>;
}
```



How Jotai's subscription model differs from Zustand's:

Property	Zustand	Jotai
Granularity	Store-level with selector slicing	Atom-level — each atom is a subscription node
Derivation	Manual — <code>useMemo</code> or selector composition	First-class — <code>atom((get) => ...)</code> tracks deps automatically
Async	Outside the store — combine with TanStack Query	Inside the graph — <code>atom(async (get) => ...)</code> suspends
Write isolation	Any <code>set</code> notifies all listeners (filtered by selector)	Only atoms that depend on the written atom re-render

Property	Zustand	Jotai
Mental model	Centralized state (like a single-table DB)	Graph of signals (like a spreadsheet or reactive stream DAG)

Backend analogy: Zustand is a single document store with secondary indexes (selectors). Jotai is a normalized entity graph with computed views — closer to a materialized-view DAG where each view subscribes to its base tables.

When to choose which:

- **Zustand** when client state is flat and imperative: filters, toggles, form drafts, selection sets. The store reads like a service class.
- **Jotai** when client state is derived and fine-grained: computed filters, cross-dependent toggles, async state that should suspend, or when you need per-atom subscription to avoid re-render cascades in large component trees.
- **Either** can drive TanStack Query keys. The pattern `store state → queryKey → useQuery` is identical; only the store primitive differs.

5. Cache Invalidation — The Hard Part

Phil Karlton's quip holds for frontend caches: invalidation is where correctness is won or lost. TanStack Query gives you four invalidation mechanisms; using the wrong one causes either stale reads or thundering-herd refetches.

5.1 invalidateQueries — Scoped Invalidation by Key Prefix

```
const qc = useQueryClient();

// Exact match – only ["users","detail","42"]
qc.invalidateQueries({ queryKey: ["users", "detail", "42"], exact:
true });

// Prefix match – all keys starting with ["users"] (default)
qc.invalidateQueries({ queryKey: ["users"] });

// Predicate – arbitrary logic over the query key and query state
qc.invalidateQueries({
  predicate: (query) =>
    query.queryKey[0] === "projects" && query.state.dataUpdatedAt < Dat
e.now() - 60_000,
});

// Invalidate and refetch type control
qc.invalidateQueries({ queryKey: ["users"] });
// marks stale + refetches active queries
qc.invalidateQueries({ queryKey: ["users"] }, { cancelRefetch: false })
; // don't cancel in-flight fetches
```

The `exact` flag is the most common source of silent no-ops. `invalidateQueries({ queryKey: ["users"] })` with `exact: true` matches *only* the key `["users"]`, not `["users", "detail", "42"]`. The default `exact: false` is prefix matching and is almost always what you want for cache busting.

5.2 Automatic Revalidation Triggers

Trigger	Default	What it does
<code>refetchOnMount</code>	<code>true</code> (if stale)	Refetches when a component using the query mounts and the data is stale
<code>refetchOnWindowFocus</code>	<code>true</code>	Refetches all active stale queries when the window regains focus (<code>visibilitychange</code> + <code>focus</code> events)

Trigger	Default	What it does
<code>refetchOnReconnect</code>	<code>true</code>	Refetches when the browser fires <code>online</code> after being <code>offline</code>
<code>refetchInterval</code>	<code>false</code>	Polls at a fixed interval while the query is mounted
<code>refetchIntervalInBackground</code>	<code>false</code>	Continues polling even when the tab is not focused

```
// Polling for hot data – only while the component is mounted and the
tab is visible
useQuery({
  queryKey: ["buildStatus", buildId],
  queryFn: () => fetchBuildStatus(buildId),
  refetchInterval: (query) =>
    query.state.data?.status === "running" ? 2000 : false, // stop
    polling when build finishes
  refetchIntervalInBackground: false,
});

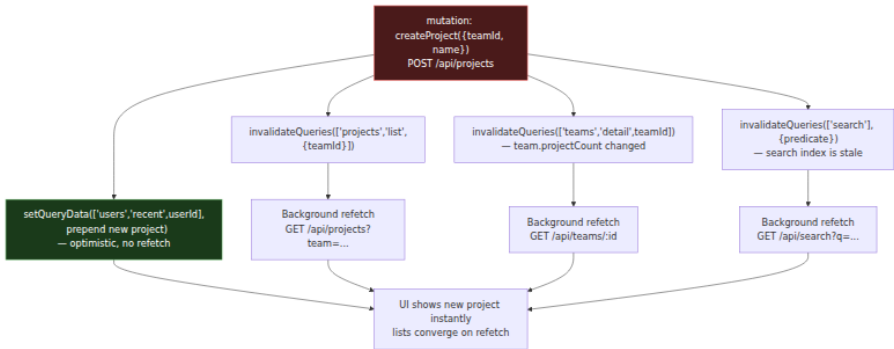
// Disable focus refetch for noisy resources (avoids fetch storm on
alt-tab)
useQuery({
  queryKey: ["analytics", dateRange],
  queryFn: () => fetchAnalytics(dateRange),
  refetchOnWindowFocus: false,
  staleTime: 5 * 60_000,
});
```

Tune `refetchOnWindowFocus` per query, not globally. Global `refetchOnWindowFocus: false` silences the most useful staleness recovery mechanism for data that is cheap to refetch. Global `true` with many active queries causes a thundering herd on focus — each stale query fires simultaneously. TanStack Query batches focus refetches in a single microtask, but the server still sees N concurrent requests. Mitigate with staggered `staleTime` or by disabling focus refetch for expensive queries.

5.3 The Invalidation Cascade

A single mutation often affects multiple cache entries: creating a project invalidates the project list, the team dashboard, and the user's recent-

activity feed. The cascade must be explicit — the cache does not know about entity relationships.



Encapsulate the cascade in the mutation, not in the component:

```
function useCreateProject(teamId: string) {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (vars: { name: string }) => postProject({ teamId, ...vars }),
    onSuccess: (created) => {
      // Precise invalidation – only the affected team
      qc.invalidateQueries({ queryKey: ["projects", "list", { teamId }]});
      qc.invalidateQueries({ queryKey: ["teams", "detail", teamId]});
      // Optimistic prepend for instant feedback – no refresh needed for the creator
      qc.setQueryData<Project[]>(["projects", "list", { teamId }], (old) =>
        old ? [created, ...old] : [created]
      );
      // Broad invalidation for derived data – search is denormalized
      qc.invalidateQueries({ queryKey: ["search"] });
    },
  });
}
```

Alternatives that reduce cascade breadth:

- **setQueryData patching** instead of invalidation — updates the cache entry in place without a network round-trip. Use when the mutation response contains the full updated entity and no derived fields need recomputation.

- **Tag-based invalidation** (Next.js `fetch` cache, RTK Query tags) — the server declares which tags a mutation invalidates, and the framework maps tags to cache entries. TanStack Query does not have built-in tags, but the `predicate` option achieves the same with manual bookkeeping.
- **Optimistic list operations** — for reordering or deletion, patching the list is cheaper and faster than refetching it.

6. Normalized vs. Denormalized Caches

TanStack Query and SWR are **denormalized** caches: each query key holds an independent copy of the data. If `GET /users/42` and `GET /projects/99` both embed `{ id: "42", name: "Ada" }` as a project owner, that user object is duplicated in two cache entries. Mutating the user's name and invalidating `["users", "detail", "42"]` leaves `["projects", "detail", "99"]` stale.

A **normalized** cache (Apollo Client, Relay, Redux Toolkit's `createEntityAdapter` pattern) stores entities in a flat table keyed by `__typename:id` and queries hold references:

Denormalized (TanStack Query / SWR):

```
["users","detail","42"] → { id:"42", name:"Ada", role:"admin" }
["projects","detail","99"] → { id:"99", name:"Atlas", owner:
{ id:"42", name:"Ada" } }
— duplicate, can diverge —▲
```

Normalized (Apollo Client):

```
entities["User:42"] → { id:"42", name:"Ada", role:"admin" } ←
single copy

queries["users:42"] ———▲
queries["projects:99"] → { id:"99", owner: ———▲ } (reference, not
copy)
```

Dimension	Denormalized (TanStack Query / SWR)	Normalized (Apollo / Relay)
Write path	Patch or invalidate each affected key manually	Write to entity table — all referencing queries update automatically
Read path	<code>useQuery</code> returns the cached value directly — no join	Cache resolves references on read — extra indirection
Consistency	Eventual — requires explicit cascade (section 5.3)	Strong within the cache — one write, all views converge
Complexity	Low — no schema, no type policy, no GC for entities	High — requires <code>__typename</code> / <code>id</code> on every entity, type policies, field merging
When it wins	REST APIs, heterogeneous backends, small entity overlap	GraphQL APIs with deep entity reuse, many writers to the same entities

For teams on REST, normalization can be added manually with `normalizr` or a lightweight entity table, but the cost is rarely justified unless the same entities appear in many queries:


```

// Manual entity table – normalize on write, denormalize on read
type EntityTable = Map<string, User>;
const userTable: EntityTable = new Map();

function normalizeUsers(users: User[]) {
  for (const u of users) userTable.set(u.id, u);
  return users.map((u) => u.id); // store only ids in the query cache
}

// Query holds ids, not objects – single source of truth per entity
function useUsersNormalized() {
  const qc = useQueryClient();
  return useQuery({
    queryKey: ["users", "normalized"],
    queryFn: async () => {
      const users: User[] = await fetchUsers();
      return normalizeUsers(users);
    },
    select: (ids) => ids.map((id) => userTable.get(id)!), //
    // denormalize on read
  });
}

// Mutation – write to table, all queries referencing that id see the
// update
function updateUserInTable(user: User) {
  userTable.set(user.id, user);
  // No invalidateQueries needed for normalized reads – select() re-
  // runs
}

```

The trade-off is the same as database normalization: normalized caches eliminate update anomalies but add read-path joins and require discipline (every entity must have a stable id, every write must go through the table). For most REST frontends, the pragmatic choice is a denormalized cache with disciplined invalidation — and to adopt a normalized cache only when the same entities are fetched via many overlapping queries and stale-owner bugs recur.

7. Background Sync and Refetch Semantics

Background sync is the mechanism that keeps the cache eventually consistent with the server without blocking the UI. Three primitives compose it:

1. **Stale-while-revalidate** — the default. The cache returns stale data synchronously and fetches in the background. The component never shows a loading spinner on revalidation — `isFetching` is `true` while `data` is still the stale value.
2. **Polling** — `refetchInterval` for hot data. The query refetches on a timer while mounted. Use the functional form to stop polling when the data reaches a terminal state.
3. **Push invalidation** — WebSocket/SSE/server-sent event that calls `invalidateQueries` or `setQueryData` when the server pushes an update. This is the only mechanism that avoids polling.

```
// Push invalidation via WebSocket – server tells the client what
// changed
function useRealtimeInvalidation() {
  const qc = useQueryClient();
  React.useEffect(() => {
    const ws = new WebSocket("wss://api.example.com/events");
    ws.onmessage = (event) => {
      const msg = JSON.parse(event.data);
      switch (msg.type) {
        case "user:updated":
          qc.setQueryData(["users", "detail", msg.id],
            msg.payload); // no fetch – server sent the entity
          break;
        case "project:created":
          qc.invalidateQueries({ queryKey: ["projects", "list", { teamId: msg.teamId }]});
          break;
        case "build:status":
          qc.setQueryData(["buildStatus", msg.buildId], msg.payload);
          break;
      }
    };
    return () => ws.close();
  }, [qc]);
}
```

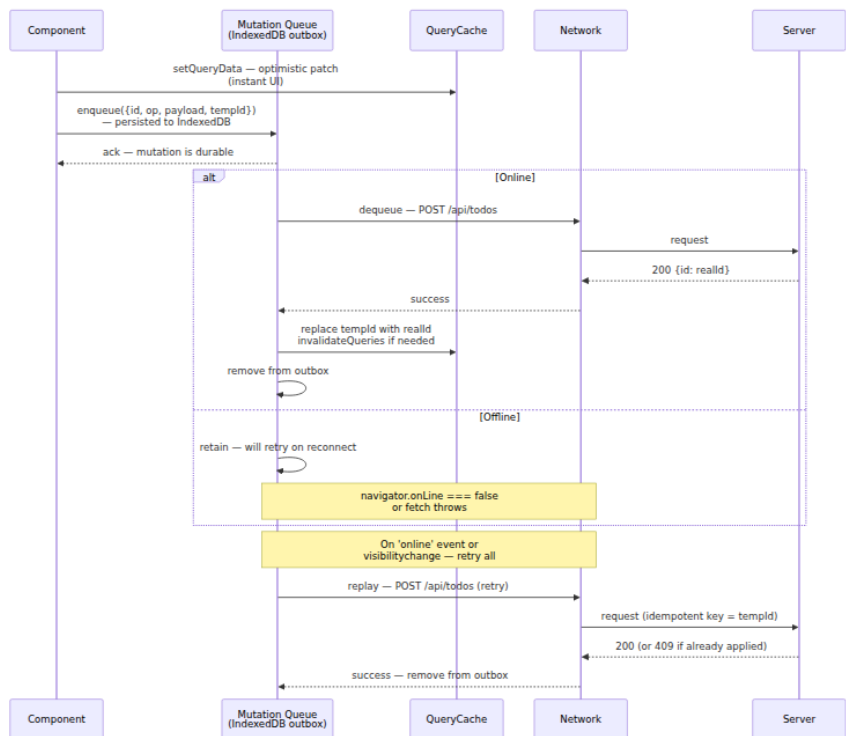
Distinguish `isPending` (no data yet — initial fetch) from `isFetching` (any fetch in flight, including background revalidation). Rendering a spinner

on `isFetching` causes the UI to flash on every background refetch — the same mistake as showing a loading bar on every cache revalidation in a backend service.

```
function UsersList() {
  const { data, isPending, isFetching, error } = useQuery({
    queryKey: ["users"],
    queryFn: fetchUsers,
  });
  if (isPending) return <Skeleton />;           // no data yet – show
skeleton
  if (error) return <ErrorBox error={error} />;
  return (
    <div style={{ opacity: isFetching ? 0.7 : 1 }}>  {/* subtle
revalidation indicator */}
      <ul>{data.map((u) => <li key={u.id}>{u.name}</li>)}</ul>
      {isFetching && <span> updating...</span>}
    </div>
  );
}
```

8. Offline Mutations Queue — A Write-Ahead Log for the Frontend

When the user mutates data while offline (or during a transient partition), the frontend must not lose the write. The solution is an **outbox** — a persistent queue of mutations that replays when connectivity returns. This is the frontend analog of a write-ahead log (WAL) in a database: writes are durably recorded before they are acknowledged, and replay is idempotent.



8.1 Implementation

The queue must survive page reload and tab close, so it cannot live in memory. `IndexedDB` (via `idb-keyval` or similar) or `localStorage` with a size cap are the durable stores. Each entry carries an idempotency key so replay is safe.

```

// Offline queue – durable outbox with replay and conflict handling
import { get as idbGet, set as idbSet } from "idb-keyval";

type OutboxEntry = {
  localId: string;           // client-generated, used as Idempotency-
  // Key header
  op: "create" | "update" | "delete";
  url: string;
  payload: unknown;
  createdAt: number;
  attempts: number;
};

const OUTBOX_KEY = "mutation-outbox";

// Durable queue – survives reload
async function enqueue(entry: OutboxEntry) {
  const box: OutboxEntry[] = (await idbGet(OUTBOX_KEY)) ?? [];
  await idbSet(OUTBOX_KEY, [...box, entry]);
}

async function dequeue(localId: string) {
  const box: OutboxEntry[] = (await idbGet(OUTBOX_KEY)) ?? [];
  await idbSet(OUTBOX_KEY, box.filter((e) => e.localId !== localId));
}

async function peekAll(): Promise<OutboxEntry[]> {
  return (await idbGet(OUTBOX_KEY)) ?? [];
}

// Replay – idempotent, with exponential backoff
async function replayOutbox(qc: QueryClient) {
  const entries = await peekAll();
  for (const entry of entries) {
    try {
      const res = await fetch(entry.url, {
        method: entry.op === "create" ? "POST" : entry.op ===
"update" ? "PATCH" : "DELETE",
        headers: {
          "Content-Type": "application/json",
          "Idempotency-Key":
entry.localId, // server must honor this – see below
        },
        body: JSON.stringify(entry.payload),
      });
    }
    if (res.status === 409) {
      // Already applied – server recognized the idempotency key
      await dequeue(entry.localId);
      continue;
    }
  }
}

```

```

    if (!res.ok) throw new Error(String(res.status));
    const result = await res.json();
    // Converge the cache – replace temp id or patch entity
    if (entry.op === "create") {
      qc.setQueryData<Todo[]>(["todos"], (old) =>
        old ? old.map((t) => (t.id === entry.localId ? result :
t)) : old
        );
    }
    await dequeue(entry.localId);
  } catch (err) {
    // Transient failure – increment attempts, back off
    const box = await peekAll();
    const next = box.map((e) =>
      e.localId === entry.localId ? { ...e, attempts: e.attempts +
1 } : e
    );
    await idbSet(OUTBOX_KEY, next);
    if (entry.attempts >= 5) {
      console.error(`Outbox entry ${entry.localId} failed after 5
attempts – manual intervention needed`, err);
      // Optionally: surface to UI as a persistent error banner
    }
    // Stop replay on first failure to preserve ordering within the
    outbox
    break;
  }
}
}

// Hook – wraps a mutation with outbox durability
function useOfflineMutation() {
  const qc = useQueryClient();

  // Replay on reconnect and on mount (covers reload while offline)
  React.useEffect(() => {
    replayOutbox(qc);
    const onOnline = () => replayOutbox(qc);
    const onVisible = () => {
      if (document.visibilityState === "visible") replayOutbox(qc);
    };
    window.addEventListener("online", onOnline);
    document.addEventListener("visibilitychange", onVisible);
    return () => {
      window.removeEventListener("online", onOnline);
      document.removeEventListener("visibilitychange", onVisible);
    };
  }, [qc]);

  return useMutation({
    mutationFn: async (vars: { title: string }) => {

```

```

    const localId = `temp-${Date.now()}-${
Math.random().toString(36).slice(2, 8)}`;
    const entry: OutboxEntry = {
      localId,
      op: "create",
      url: "/api/todos",
      payload: vars,
      createdAt: Date.now(),
      attempts: 0,
    };

    // Optimistic – instant UI
    qc.setQueryData<Todo[]>(["todos"], (old) =>
      old ? [...old, { id: localId, title: vars.title, done:
false }] : [{ id: localId, title: vars.title, done: false }]
    );

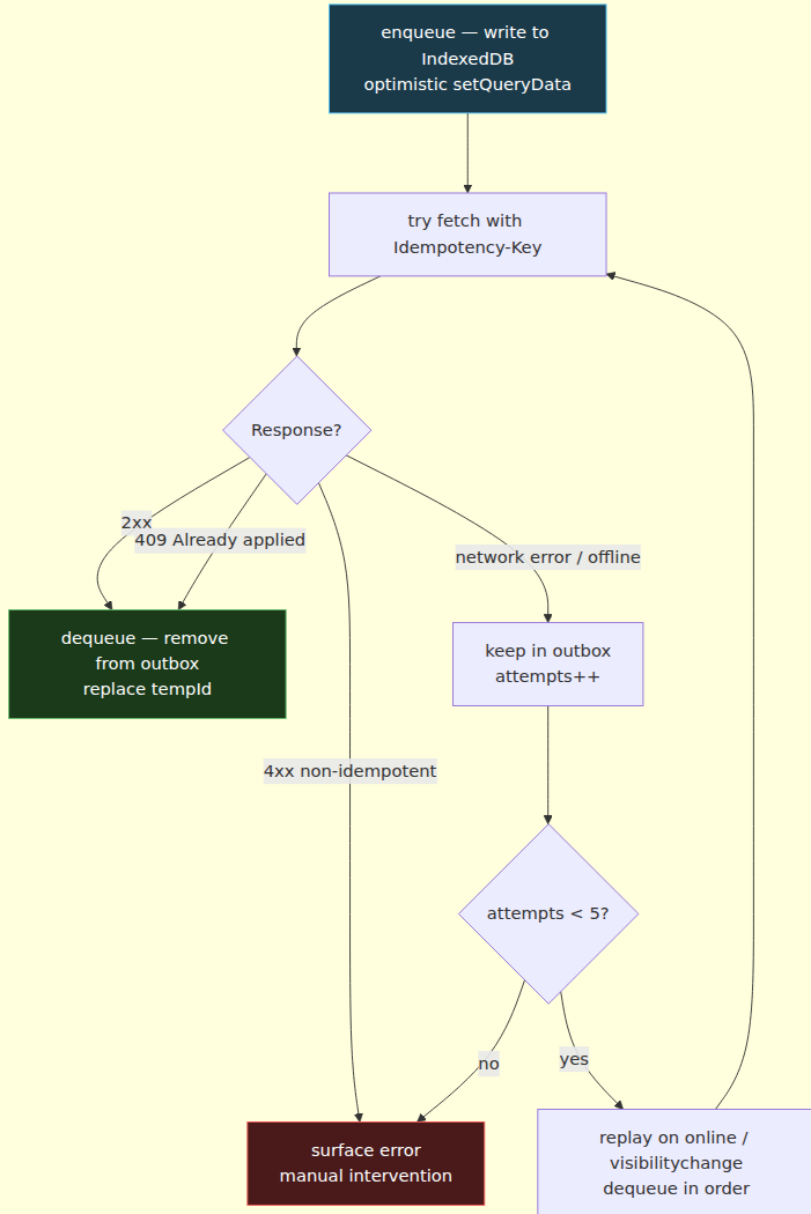
    // Durable – survives reload
    await enqueue(entry);

    // Attempt immediately – if offline, replay will handle it later
    try {
      const res = await fetch(entry.url, {
        method: "POST",
        headers: { "Content-Type": "application/json", "Idempotency-
Key": localId },
        body: JSON.stringify(vars),
      });
      if (!res.ok) throw new Error(String(res.status));
      const result = await res.json();
      qc.setQueryData<Todo[]>(["todos"], (old) =>
        old ? old.map((t) => (t.id === localId ? result : t)) : old
      );
      await dequeue(localId);
      return result;
    } catch (err) {
      // Network failure – leave in outbox for replay; keep
      optimistic UI
      if (!navigator.onLine || err instanceof TypeError) {
        console.warn("Offline – mutation queued for replay", localId)
;
        return { id: localId, title: vars.title, done: false } as Tod
o;
      }
      // Non-network error – rollback
      qc.setQueryData<Todo[]>(["todos"], (old) => old?.filter((t) =>
t.id !== localId) ?? []);
      await dequeue(localId);
      throw err;
    }
  },

```

```
});  
}
```


Offline queue — state machine per entry



Backend contract the queue depends on:

- **Idempotency-Key header.** The server must store the key and return the original response on replay (or `409 / 200` with the existing entity). Without this, replay creates duplicates. This is the same contract as Stripe's `Idempotency-Key` or S3's multipart upload idempotency.
- **Ordering.** The queue replays in FIFO order. If operations on the same entity must be ordered (create then update), the queue must not reorder them. The `break` on first failure in `replayOutbox` preserves ordering — a failed entry blocks later entries.
- **Conflict detection.** If another client mutated the same entity while this client was offline, the server should return `409 Conflict` with the current entity. The client can then surface a merge dialog or apply last-writer-wins. Without conflict detection, offline replay silently overwrites concurrent writes — a lost-update anomaly.

TanStack Query's `persistQueryClient` plugin (with an `idb` persister) handles the *read* side of offline — persisting the query cache to IndexedDB so cached data survives reload. The outbox above handles the *write* side. Together they make the app usable offline with eventual consistency on reconnect — the same guarantees as a mobile app with a local SQLite WAL.

9. Putting It Together — Architecture of a Production Data Layer

A production frontend composes these primitives into a layered data architecture. Each layer has one owner and one consistency model.

Conventions that keep this architecture maintainable across teams:

1. **One query-key factory per domain.** `userKeys`, `projectKeys`, `buildKeys` — each factory owns its namespace. No inline keys.
2. **Store drives query, not the reverse.** `filters` lives in Zustand/Jotai; the query key *reads* it. Never write server data into the client store.
3. **Mutations own their invalidation.** The `useCreateProject` hook invalidates `["projects"]` and `["teams"]` — the component that calls `mutate` does not decide what to invalidate.

4. **staleTime and gcTime are tuned per resource, not globally.**
Reference data gets long `staleTime`; hot data gets short `staleTime` + `refetchInterval`; user-specific data disables `refetchOnWindowFocus`.
5. **Optimistic updates are opt-in per mutation**, with snapshot + rollback. Not every mutation needs them — only latency-sensitive writes where the expected result is predictable.
6. **The outbox is opt-in per mutation** and requires server idempotency. Do not add it to every mutation; add it to writes where data loss is unacceptable.

10. The Distributed-Systems Lens

Frontend state management is distributed systems engineering with the browser as a node.

Distributed-systems concern	Frontend analog	What to do
Cache coherence	<code>staleTime</code> / <code>gcTime</code> / <code>invalidateQueries</code>	Treat the query cache as a read-through cache with TTL. Tune TTL per resource volatility, not globally.
Write-ahead log	Offline outbox in IndexedDB	Durably record mutations before acknowledging; replay with idempotency keys.
Idempotency	<code>Idempotency-Key</code> header on replay	Server must deduplicate by key; client must generate a stable key per mutation.
Conflict resolution	Concurrent offline edits to the same entity	Server returns <code>409</code> with current state; client merges or prompts. Last-writer-wins is the default, not the goal.

Distributed-systems concern	Frontend analog	What to do
Thundering herd	<code>refetchOnWindowFocus</code> with many stale queries	Stagger <code>staleTime</code> , disable focus refetch for expensive queries, batch WebSocket invalidations.
Deduplication	Per-key request coalescing	Rely on the cache's dedup; do not add a second dedup layer in the fetcher.
Eventual consistency	Optimistic update + background revalidation	Apply locally, converge via refetch or push. The UI is eventually consistent with the server.
Backpressure	Infinite query with <code>IntersectionObserver</code> + <code>fetchNextPage</code>	Paginate on the server, fetch on demand, keep <code>pages</code> bounded or virtualize.
Observability	Query DevTools, <code>isFetching</code> / <code>isPending</code> counters, outbox depth	Instrument cache hit rate, revalidation latency, and outbox queue length — the same RED metrics as any cache.

Every decision in this chapter maps to a decision you have already made on the backend: TTL vs. explicit invalidation, normalized vs. denormalized storage, optimistic concurrency vs. pessimistic locking, WAL durability, and idempotent replay. The browser is not a special case — it is a cache node with a UI attached.

Key takeaways

- Server state and client state have different owners, mutability, and invalidation semantics. Server state belongs in a server-state cache (TanStack Query / SWR); client state belongs in a local store (Zustand / Jotai) or component state. Mixing them recreates staleness or invalidation bugs.

- TanStack Query's cache is a `Map<QueryKey, Entry>` where the `queryKey` array is the hierarchical address. Design keys as a factory (`userKeys.list(filters)`) to make prefix invalidation reliable and typo-proof.
- `staleTime` controls freshness (when to revalidate); `gcTime` controls eviction (when to delete an unused entry). Fresh data is served without a fetch; stale data is served immediately plus a background fetch. `gcTime` preserves back-nav without a spinner.
- Deduplication is automatic and per-key: N simultaneous subscribers to the same key trigger one fetch. Do not add manual dedup in the fetcher; include every variable that affects the fetch in the key.
- Retries use exponential backoff and should be conditional — retry network/5xx, not 4xx. Mutations do not retry by default because they are non-idempotent; only idempotent mutations (PUT) should opt in.
- `useInfiniteQuery` stores `{ pages, pageParams }` as a single cache entry. `getNextPageParam` extracts the cursor; `fetchNextPage` appends. Refetch refetches all pages — prefer `setQueryData` patching for large lists.
- Optimistic updates follow snapshot → patch → commit-or-rollback → reconcile. Always `cancelQueries` before `setQueryData` to avoid lost updates from in-flight refetches; snapshot via `getQueryData`, not closure.
- SWR implements stale-while-revalidate with a smaller API: string keys, `dedupingInterval` instead of `staleTime / gcTime`, and manual `mutate` with `optimisticData`. Choose it for small or read-heavy apps; choose TanStack Query when hierarchical invalidation, DevTools, and precise freshness control matter.
- Zustand is a single external store with selector subscriptions; Jotai is an atom DAG with per-atom subscriptions and first-class derived/async atoms. Both drive query keys; the choice is flat/imperative (Zustand) vs. graph/derived (Jotai).
- Invalidation is explicit and cascading: one mutation may invalidate many keys. Encapsulate the cascade in the mutation hook. Prefer `setQueryData` patching when the response contains the full entity; use `invalidateQueries` with `prefix` or `predicate` when derived data is stale.
- `refetchOnWindowFocus` / `onReconnect` / `interval` and WebSocket push are the four revalidation triggers. Tune per query — global focus

refetch causes a thundering herd; per-query polling with a functional `refetchInterval` stops when data reaches a terminal state.

- Denormalized caches (TanStack Query / SWR) duplicate entities across keys and require explicit cascade; normalized caches (Apollo) store one copy per entity with references and converge automatically. For REST, denormalized with disciplined invalidation is the pragmatic default.
- The offline outbox is a write-ahead log: durably enqueue to IndexedDB, apply optimistically, replay with `Idempotency-Key` on `online / visibilitychange`, handle 409 as already-applied, preserve FIFO ordering, and surface conflicts for merge. Pair with `persistQueryClient` for offline reads.

Further reading

- TanStack Query documentation — [Queries, Mutations, Invalidation, Infinite Queries, Optimistic Updates](#) — canonical reference for `queryKey`, `staleTime / gcTime`, `invalidateQueries`, and `useInfiniteQuery`.
- TanStack Query source — `QueryCache`, `QueryObserver`, `MutationCache` on GitHub (github.com/TanStack/query) — read the cache map, dedup, and GC timer implementation directly.
- SWR documentation — [SWR: React Hooks for Data Fetching](#) — stale-while-revalidate semantics, `useSWRInfinite`, `useSWRMutation`, and cache provider API.
- Vercel — [SWR vs. React Query discussion](#) — trade-off comparison from the SWR authors.
- Jotai documentation — [Jotai: Primitive and Flexible State Management](#) — atoms, derived atoms, async atoms, and `jotai-tanstack-query` integration.
- Zustand documentation — [Zustand: Bear Necessities for State Management](#) — store creation, middleware (`persist`, `immer`, `subscribeWithSelector`), and `useSyncExternalStore` semantics.
- Kent C. Dodds — [Application State Management with React](#) — when to use server cache vs. client store vs. component state.
- TkDodo (Dominik Dorfmeister) — [TanStack Query blog series](#) — deep dives on `staleTime` vs. `cacheTime / gcTime`, query keys, and optimistic updates.

- Apollo Client documentation — [Normalized Cache](#) — how a normalized entity cache with `__typename:id` and type policies differs from denormalized caches.
- Mozilla — [IndexedDB API](#) and [idb-keyval](#) — the durable store backing the offline outbox and `persistQueryClient`.
- Stripe API — [Idempotent Requests](#) — the Idempotency-Key contract the offline queue depends on for safe replay.

Chapter 11 — Testing, Linting, and Tooling for Frontend at Scale (Vitest, Playwright, ESLint, TypeScript)

What this chapter covers: The frontend tooling stack that replaces Jest, Cypress, TSLint, and ad-hoc TypeScript configs when a codebase grows past one team and one deploy per week. We dissect Vitest's Vite-native runner and its worker pool, Playwright's browser-context isolation and trace architecture, ESLint's flat-config system with type-aware rules, TypeScript's project-references and incremental-build machinery, visual regression pipelines, and the CI sharding and report-merge patterns that keep the whole system under ten minutes. Every section ends in real config you can copy into a monorepo tomorrow.

Learning goals:

- Choose and configure the DOM environment for Vitest (happy-dom vs jsdom vs real browser) and explain the performance and fidelity trade-off.
- Explain Vitest's worker model (threads, forks, vmThreads, vmForks), snapshot update flow, and V8 vs Istanbul coverage collection.
- Design Playwright suites with browser contexts, fixtures, parallel workers, sharding, and trace-viewer debugging.
- Author ESLint flat config with `typescript-eslint`, type-aware rules, and per-glob overrides that scale across packages.
- Operate TypeScript at monorepo scale: project references, `incremental` / `composite` / `isolatedModules`, and the build-mode (`tsc -b`) dependency graph.

- Integrate visual regression (Percy, Chromatic) without flaking the pipeline.
- Build a CI matrix that shards Vitest and Playwright, merges JUnit / HTML / blob reports, and gates deploys on type, lint, and test results.

1. Why Frontend Tooling Breaks at Scale

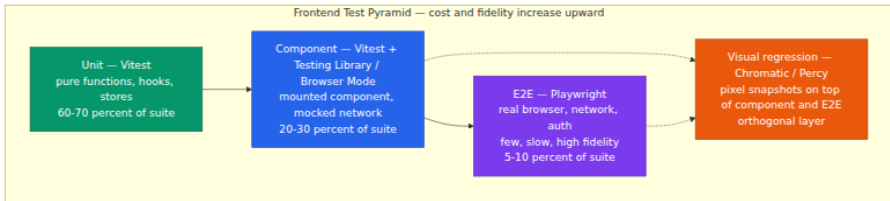
A single-team SPA can afford Jest with jsdom, `eslint .`, `tsc --noEmit`, and a Cypress job that runs serially. At 50 engineers, 4000 test files, and 15 deploys per day those choices become the bottleneck.

Symptom at scale	Root cause	What this chapter replaces it with
<code>jest</code> cold start 40 s, watch mode stale	Jest resolves and transforms outside Vite; no shared pipeline	Vitest reuses Vite's transform and dev server
E2E suite 35 min wall-clock	One Cypress runner, one browser, serial specs	Playwright workers + sharded CI matrix
<code>eslint</code> takes 3 min, rules conflict per package	Legacy <code>.eslintrc</code> cascading, no typed rules	Flat config with per-glob overrides, <code>typescript-eslint</code>
<code>tsc --noEmit</code> takes 90 s on every PR	Single <code>tsconfig.json</code> , no incremental graph	Project references + <code>incremental</code> + <code>tsc -b</code>
Visual bugs ship despite green tests	No pixel-level assertion	Percy / Chromatic snapshot pipeline
CI is flaky, retries mask real failures	No trace, no report merge	Playwright trace viewer, Vitest <code>--merge-reports</code> , blob reporters

Backend engineers will recognize every row as a distributed-systems problem: work that was serial must become parallel, shared state must be isolated, and feedback loops must be incremental and cacheable.

1.1 The frontend test pyramid

The pyramid for frontend is not the classic backend unit → integration → e2e. The middle layer is component tests — a mounted React/Vue/Svelte component asserted with DOM queries — and the cost curve is steeper because real browsers are involved.



The rule of thumb: if a bug can be caught below the browser boundary, catch it there. Reserve Playwright for flows that require navigation, auth, cross-origin iframes, or real layout. Everything else belongs in Vitest.

2. Vitest — A Vite-Native Test Runner

Vitest is not a fork of Jest. It reuses the Jest-compatible API (`describe`, `it`, `expect`, `vi`) but replaces the runner, transformer, resolver, and watcher with Vite. That single decision explains most of its wins: transforms are cached by Vite, path aliases and plugins are shared, and HMR-style watch invalidation is free.

2.1 Architecture

```
Source file → Vite plugin pipeline (resolve → load → transform)
              → Vitest runner (collect tests → schedule workers → run)
              → Reporter / Coverage / Snapshot manager
```

Jest's pipeline parses and transforms each file in its own `jest-runtime` sandbox. Vitest delegates to `vite dev server` in transform mode, so `vite.config.ts` aliases, CSS modules, and import handling work identically in app and test. The consequence: a monorepo with a custom Vite plugin (say, an SVG-to-component transform) does not need a second Jest transformer.

2.2 happy-dom vs jsdom vs Real Browser

Vitest can run tests in four environments, configured per project or per file:

Environment	What it is	DOM fidelity	Speed	When to use
node	No DOM at all	None	Fastest	Pure utilities, stores, parsers
happy-dom	JS implementation of DOM, no layout engine	Good for queries, no layout / <code>getComputedStyle</code> limits	~2-3x faster than jsdom	Unit and component tests that only need DOM queries
jsdom	JS implementation of DOM + more complete CSSOM	Higher fidelity, supports <code>getComputedStyle</code> , <code>innerText</code>	Slower, heavier	Component tests that assert styles or measure elements
browser (Playwright/ WebDriver)	Real Chromium/ Firefox/WebKit	Full fidelity	Slowest, requires browser binary	Tests that need layout, focus, clipboard, or real event dispatch

For a senior backend analogy: happy-dom is to jsdom as an in-memory fake is to a heavier emulator. Both satisfy the DOM interface, but only the real browser satisfies the rendering contract.

```
// vitest per-file environment override
/**
 * @vitest-environment happy-dom
 */
import { test, expect } from 'vitest';
import { render } from '@testing-library/react';
import { Button } from './Button';

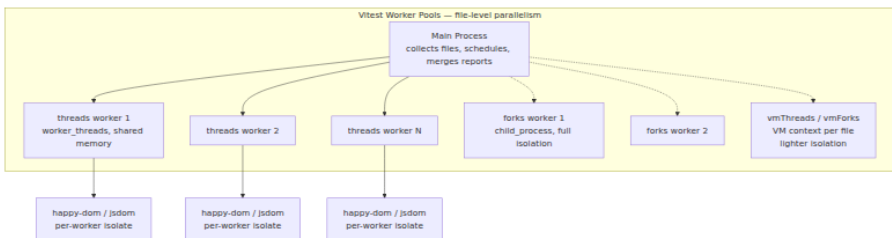
test('renders label', () => {
  const { getByRole } = render(<Button label="Save" />);
  expect(getByRole('button', { name: 'Save' })).toBeInTheDocument();
});
```

Default guidance for a large codebase: set `environment: 'happy-dom'` globally, override to `jsdom` for the handful of suites that need `getComputedStyle` or `Range`, and promote layout-sensitive suites to `browser` mode rather than faking more of the DOM.

Browser mode itself deserves attention. Vitest's `browser.enabled: true` launches a real browser via Playwright or WebDriver and executes tests inside it. The test file still imports from `vitest`, but `expect` and DOM APIs run in the browser context and results are serialized back. This closes the fidelity gap without moving the test to a separate Playwright project.

2.3 Workers: threads, forks, vmThreads, vmForks

Vitest parallelizes at the file level. Each test file is assigned to a worker; inside a file, tests run serially by default (configurable). Four pool strategies exist:



Pool	Mechanism	Isolation	Startup cost	Best for
<code>threads</code>	<code>node:worker_threads</code>	Shared heap, <code>vm</code> sandbox per file	Low	Default for most suites; fastest
<code>forks</code>	<code>node:child_process</code>	Full process isolation	Higher	Suites that mutate <code>process.env</code> , global state, or native addons
<code>vmThreads</code>	<code>worker_threads</code> + <code>vm</code> context	Per-file VM context, no <code>require</code> cache sharing	Low-medium	Need per-file global isolation without fork cost
<code>vmForks</code>	<code>child_process</code> + <code>vm</code> context	Per-file VM context + process isolation	Highest	Maximum isolation for leaking suites

Practical rules at scale:

- Default to `pool: 'threads'`. Move leaking suites to `forks` with `poolOptions.forks.singleFork: false` so they get their own process.
- Set `poolOptions.threads.singleThread: false` and let Vitest size the pool to `os.availableParallelism() - 1`. Override with `--poolOptions.threads.maxThreads` in CI if the runner is shared.
- Isolate files that touch `process.env` or register global handlers with `sequence.concurrent: false` or by placing them in a separate Vitest project (see below).
- `isolate: true` (default) re-creates the environment per file. Setting `isolate: false` is faster but leaks globals — only safe for pure unit files.

2.4 Snapshots

Vitest supports three snapshot forms:

1. **Inline snapshots** — `expect(value).toMatchInlineSnapshot()` writes the snapshot into the source file on `--update`.
2. **File snapshots** — `expect(value).toMatchSnapshot()` writes to `__snapshots__/TestName.snap`.
3. **Custom serializers** — `expect.addSnapshotSerializer()` for normalizing unstable fields (timestamps, ids).

Snapshot flow:

```
test run → serialize value → compare to stored snapshot
          → if mismatch: fail, show diff
          → if --update: overwrite snapshot file
          → CI: never auto-update; fail on mismatch
```

At scale, snapshots are a liability if misused. Serialize the minimal stable shape, not the entire component tree. Normalize non-deterministic fields:

```
expect.addSnapshotSerializer({
  test: (val) => val && typeof val === 'object' && 'id' in val,
  serialize: (val) => JSON.stringify({ ...val, id: '[ID]', createdAt: '[DATE]' }),
});
```

Store snapshots next to tests, check them in, and review them as code. A snapshot update that touches 200 files is a signal that the serializer is too broad.

2.5 Coverage: V8 vs Istanbul

Vitest delegates coverage to either `@vitest/coverage-v8` or `@vitest/coverage-istanbul`:

Provider	Mechanism	Accuracy	Speed	Notes
v8	V8's built-in coverage (via <code>c8</code>)	Native, handles ESM, no instrumentation	Fast, no AST rewrite	Default; requires <code>node --coverage</code> compatible

Provider	Mechanism	Accuracy	Speed	Notes
<code>istanbul</code>	Babel-style instrumentation (<code>istanbul-lib-instrument</code>)	Works everywhere, supports <code>/* istanbul ignore */</code>	Slower, rewrites source	Needed when custom Babel transforms must be covered

For Vite + TypeScript + ESM codebases, `v8` is almost always correct. It avoids double-transform and respects Vite's pipeline. Enable with `coverage.provider: 'v8'` and `coverage.reportsDirectory`.

Coverage at scale is a gate, not a goal. Set `coverage.thresholds` per project and enforce `lines / branches / functions` on changed files via CI diff, not a global percentage that encourages gaming.

2.6 Authoritative `vitest.config.ts`

The config below is a realistic monorepo setup: two Vitest projects (unit and browser), typed, with coverage, snapshot, and worker tuning. It assumes the monorepo runs `vitest --project unit --project browser` or `vitest --run` for all.

```
// vitest.config.ts
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';
import tsconfigPaths from 'vite-tsconfig-paths';

export default defineConfig({
  plugins: [react(), tsconfigPaths()],
  test: {
    // Global defaults – overridden per project below
    globals: true,
    css: true,
    mockReset: true,
    restoreMocks: true,
    clearMocks: true,

    // File-level parallelism: threads by default, forks for leaking
    suites
    pool: 'threads',
    poolOptions: {
      threads: {
        maxThreads: 8,
        minThreads: 2,
        singleThread: false,
        isolate: true,
      },
      forks: {
        maxForks: 4,
        singleFork: false,
      },
    },
  },

  // Sharding is configured via CLI (--shard=1/4), not here.
  // Merge reports across shards with --merge-reports.

  coverage: {
    provider: 'v8',
    reportsDirectory: './coverage',
    reporter: ['text', 'lcov', 'html', 'json'],
    thresholds: {
      lines: 70,
      branches: 65,
      functions: 70,
      statements: 70,
    },
    exclude: [
      '**/__tests__/**',
      '**/*.test.{ts,tsx}',
      '**/generated/**',
      '**/*.stories.*',
    ],
  },
});
```

```

    // v8 ignores are respected; istanbul-style comments also work
    via transform
  },

  // Snapshot config
  snapshotFormat: {
    printBasicPrototype: false,
  },

  // Projects let one vitest invocation run heterogeneous suites
  projects: [
    {
      extends: true,
      test: {
        name: 'unit',
        include: ['packages/*/src/**/*.test.ts', 'packages/**/*.spec.ts', 'packages/**/*.spec.tsx'],
        exclude: ['**/*.browser.test.*', '**/e2e/**'],
        environment: 'happy-dom',
        setupFiles: ['./test/setup/unit.ts'],
        sequence: { concurrent: false },
      },
    },
    {
      extends: true,
      test: {
        name: 'browser',
        include: ['packages/*/src/**/*.browser.test.ts', 'packages/**/*.browser.test.tsx'],
        environment: 'browser',
        browser: {
          enabled: true,
          provider: 'playwright',
          instances: [{ browser: 'chromium' }],
          headless: true,
        },
        setupFiles: ['./test/setup/browser.ts'],
      },
    },
  ],

  reporters: ['default', 'junit'],
  outputFile: {
    junit: './reports/junit-vitest.xml',
  },
});

```

Key details:

- `plugins: [react(), tsconfigPaths()]` is shared between app and test — no second transform config.

- `projects` replaces the old `workspace` field (Vitest 2+). Each project can have its own `environment`, `browser`, and `include`.
- `poolOptions.threads.isolate: true` prevents cross-file leakage in `threads` mode. Flip to `false` only for a known-pure project.
- Coverage `thresholds` fail the run; pair with `coverage.reportOnFailure: true` in CI so the HTML report is still emitted on failure.

3. Component Testing — The Middle of the Pyramid

Component tests mount a real component, interact with it via user-event, and assert via DOM queries. The stack is Vitest + `@testing-library/react` (or Vue/Svelte equivalents) + `happy-dom` or `browser` mode.

```
// packages/ui/src/Button.browser.test.tsx – runs in real browser
import { describe, it, expect } from 'vitest';
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { Button } from './Button';

describe('Button', () => {
  it('emits onClick with correct payload', async () => {
    user = userEvent.setup();
    const onClick = vi.fn();
    render(<Button label="Save" onClick={onClick} />);

    await user.click(screen.getByRole('button', { name: 'Save' }));
    expect(onClick).toHaveBeenCalledTimes(1);
  });

  it('is disabled when loading', () => {
    render(<Button label="Save" loading />);
    expect(screen.getByRole('button', { name: 'Save' })).toBeDisabled();
  });
});
```

Principles that keep component suites fast and stable at scale:

- Query by role and accessible name (`getByRole('button', { name })`), not by class or test id. This couples tests to the accessibility contract, not the implementation.

- Use `userEvent` over `fireEvent`. `userEvent` dispatches the full sequence (`pointerdown` → `mousedown` → `focus` → `mouseup` → `click`) that a real interaction triggers.
- Mock network at the `fetch` / `msw` layer, not at the component prop layer, so the loading/error/empty states are exercised.
- Keep component tests hermetic: no real network, no real timers unless `vi.useFakeTimers()` is explicit, no shared DOM between files (Vitest isolates by default).

Browser mode vs happy-dom for components: if your component uses `ResizeObserver`, `IntersectionObserver`, `scrollIntoView`, or CSS-dependent logic, promote that file to browser mode. Faking those APIs in happy-dom creates a mock that diverges from the real layout engine and hides bugs.

4. Playwright — E2E at Scale

Playwright is the successor to Puppeteer and Cypress for cross-browser E2E. Its architecture is a Node controller that speaks CDP (Chromium), Juggler (Firefox), or WebKit IPC to browser instances.

4.1 Browser contexts — the isolation primitive

A browser context is Playwright's equivalent of an incognito profile: isolated cookies, storage, permissions, and viewport, but sharing a single browser process. Creating a context is milliseconds; launching a browser is seconds.

Playwright multiplexes contexts over a single browser process — `browser.newContext()` creates an isolated incognito profile (cookies, storage, permissions) in milliseconds, while `browser.launch()` costs seconds. The runner assigns one context per test (via the built-in page fixture) so every test starts from a clean slate without paying the browser-launch tax.

```
// Isolated contexts – no state leakage between tests
import { test, expect } from '@playwright/test';

test('buyer sees checkout', async ({ browser }) => {
  const buyerCtx = await browser.newContext({ storageState: 'buyer.json'
});
  const page = await buyerCtx.newPage();
  await page.goto('/checkout');
  await expect(page.getByRole('heading', { name: 'Checkout' })).toBeVisible();
  await buyerCtx.close();
});

test('admin sees dashboard', async ({ browser }) => {
  const adminCtx = await browser.newContext({ storageState: 'admin.json'
});
  const page = await adminCtx.newPage();
  await page.goto('/admin');
  await expect(page.getByRole('heading', { name: 'Admin' })).toBeVisible();
  await adminCtx.close();
});
```

Most tests never call `browser.newContext` directly. Playwright's built-in `page` fixture creates a fresh context per test and tears it down, which is why E2E tests are isolated by default.

4.2 Fixtures — dependency injection for tests

Fixtures are Playwright's replacement for global `beforeEach` setup. Each fixture is a factory that can depend on other fixtures, is scoped to `test` or `worker`, and is torn down automatically.

```
// fixtures.ts
import { test as base, expect } from '@playwright/test';

type Fixtures = {
  authenticatedPage: import('@playwright/test').Page;
  apiHelper: { createOrder: (input: unknown) => Promise<string> };
};

export const test = base.extend<Fixtures>({
  // Worker-scoped fixture: one auth per worker, not per test
  authenticatedPage: async ({ browser }, use) => {
    const ctx = await browser.newContext({ storageState: 'playwright/.a
uth/user.json' });
    const page = await ctx.newPage();
    await use(page);
    await ctx.close();
  },

  apiHelper: async ({ request }, use) => {
    const helper = {
      createOrder: async (input: unknown) => {
        const res = await request.post('/api/orders', { data: input });
        expect(res.ok()).toBeTruthy();
        const { id } = await res.json();
        return id as string;
      },
    };
    await use(helper);
  },
});

export { expect } from '@playwright/test';
```

Fixture scoping matters for performance: `scope: 'worker'` fixtures are created once per worker and shared across tests in that worker. Use it for expensive auth or DB seeding. Default `scope: 'test'` gives isolation.

4.3 Parallel workers and sharding

Playwright runs specs in parallel across workers. Each worker owns a browser process and runs one spec file at a time (tests within a file run serially by default, `fullyParallel: true` allows intra-file parallelism).

Sharding splits the spec list deterministically so CI machines can run disjoint subsets:

```
# CI matrix - 4 shards, each runner executes one
npx playwright test --shard=1/4
npx playwright test --shard=2/4
npx playwright test --shard=3/4
npx playwright test --shard=4/4
```

After sharding, merge the blob reports:

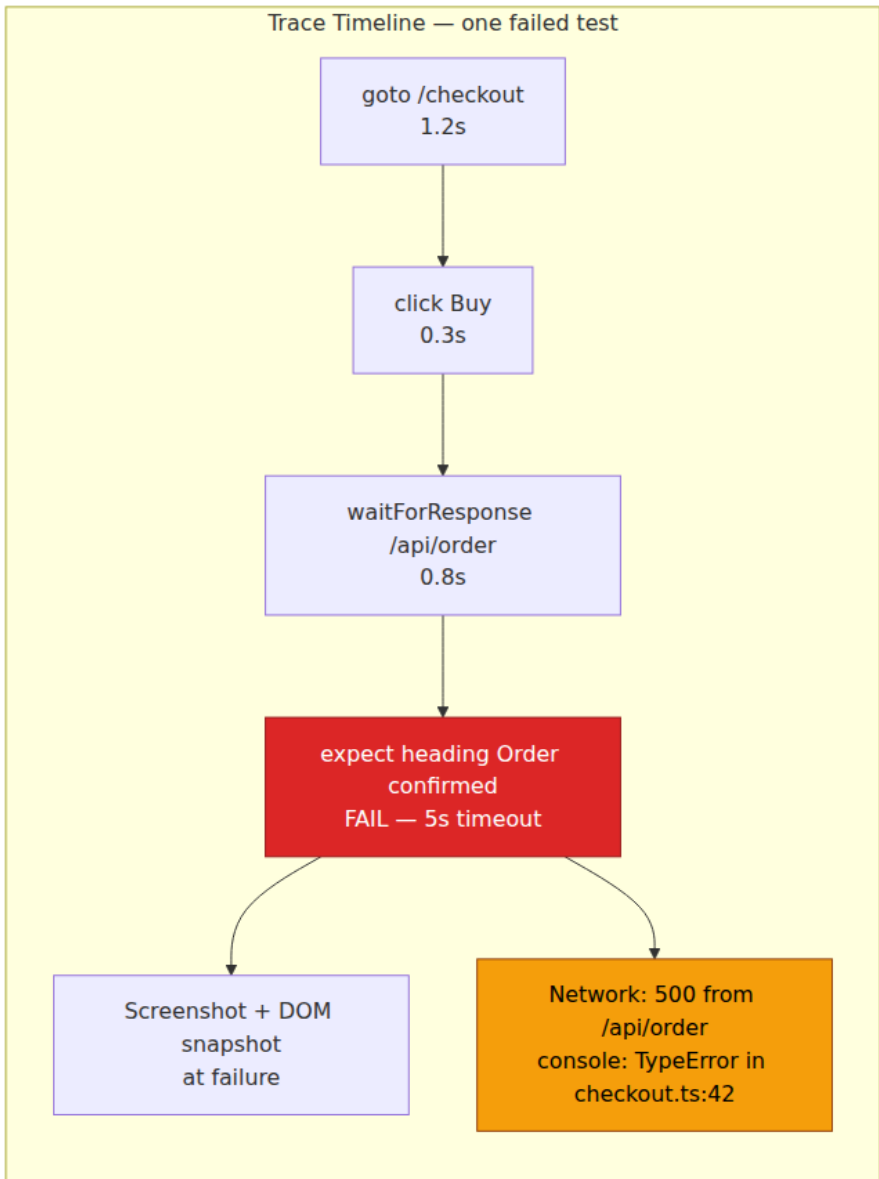
```
# Each shard writes a blob
# playwright.config.ts: reporter: [['blob', { outputDir: 'blob-
report' }]]
# After all shards finish:
npx playwright merge-reports ./blob-report --reporter html
```

The blob reporter stores raw test results and traces per shard. `merge-reports` combines them into a single HTML report with a unified trace viewer — the pattern that scales to 30-minute suites on 4-8 CI runners.

4.4 Trace viewer

`trace: 'on-first-retry'` is the recommended setting. On failure (or retry), Playwright captures a trace ZIP containing:

- Screenshots per action
- DOM snapshots per action
- Network log (request/response headers and bodies)
- Console messages
- Source location of each `expect`
- Timeline with action durations



Open with `npx playwright show-trace trace.zip` or from the HTML report. At scale, upload traces as CI artifacts so any engineer can debug a flaky failure without re-running locally. Never set `trace: 'on'` globally — it doubles artifact size and slows every test.

4.5 Authoritative playwright.config.ts

```

// playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './e2e',
  testMatch: '**/*.e2e.ts',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 4 : undefined, // let CI matrix + shard
  control this; locally auto
  timeout: 30_000,
  expect: { timeout: 7_000 },

  reporter: process.env.CI
    ? [
        ['blob', { outputDir: 'blob-report' }],
        ['junit', { outputFile: 'reports/junit-playwright.xml' }],
        ['github'],
      ]
    : [['html', { open: 'never' }], ['list']],

  use: {
    baseURL: process.env.BASE_URL ?? 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    actionTimeout: 10_000,
    navigationTimeout: 15_000,
  },

  // Isolated projects: each gets its own worker pool, like Vitest
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
      dependencies: ['setup'],
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
      dependencies: ['setup'],
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
      dependencies: ['setup'],
    },
  ],
});

```



```

    name: 'mobile',
    use: { ...devices['Pixel 7'] },
    dependencies: ['setup'],
  },
  // Setup project runs once per worker pool – handles auth
  {
    name: 'setup',
    testMatch: /\.*\setup\.ts/,
    teardown: 'cleanup',
  },
  {
    name: 'cleanup',
    testMatch: /\.*\teardown\.ts/,
  },
],

webServer: {
  command: 'pnpm dev --port 3000',
  url: 'http://localhost:3000/health',
  reuseExistingServer: !process.env.CI,
  timeout: 120_000,
},
});

```

Notes for scale:

- `projects` with `dependencies` and `teardown` is the auth pattern. `setup` logs in once, writes `playwright/.auth/user.json`, and downstream projects load it via `use.storageState`.
- `blob` reporter is mandatory for sharded CI. Without it, shards overwrite each other's HTML report.
- `forbidOnly: !!process.env.CI` prevents a `test.only` from silently skipping the suite in CI.
- `workers: undefined` locally lets Playwright size to cores; in CI, the shard matrix controls parallelism — do not hardcode workers in CI.

5. ESLint — Flat Config and typescript-eslint at Scale

ESLint's legacy `.eslintrc` system cascaded configs by directory and merged `extends` arrays with opaque precedence. The flat config (`eslint.config.ts` / `eslint.config.js`, ESLint 9+) replaces it with a single ordered array of config objects, each declaring its own `files`, `ignores`, `languageOptions`, `rules`, and `plugins`. Order matters: later entries override earlier ones for matching files.

5.1 Flat config graph

eslint.config.ts — ordered array, last match wins

1 · Base JS
files: `**/*.{js,mjs,cjs}`
`@eslint/js` recommended

2 · TypeScript
files: `**/*.{ts,tsx}`
`typescript-eslint`
recommended
`parserOptions.projectService`

3 · Type-aware rules
files: `**/*.{ts,tsx}`
requires type info
`no-floating-promises`,
etc.

4 · React / JSX
files: `**/*.tsx`
`eslint-plugin-react-`
`hooks`, `jsx-a11y`

The key insight for backend engineers: flat config is a middleware chain, not a tree. There is no implicit parent lookup. A file's config is the merge of every array entry whose `files` glob matches it, in array order. This makes the system predictable and debuggable — `npmx eslint --print-config path/to/file.ts` prints the resolved config for a file.

5.2 typescript-eslint — parser, plugin, and project service

`typescript-eslint` is the monorepo that replaces `@typescript-eslint/parser` + `@typescript-eslint/eslint-plugin` (the old split). It provides:

- A parser that produces an ESTree-compatible AST with type information.
- Rule sets: `recommended`, `strict`, `stylistic`, and type-aware (`recommendedTypeChecked`, `strictTypeChecked`).
- `projectService` — a lightweight TypeScript program that answers type queries without requiring `parserOptions.project` to list every `tsconfig.json`.

Type-aware rules (e.g., `no-floating-promises`, `no-misused-promises`, `await-thenable`, `consistent-type-imports`) need the type checker. They are 3-10x slower than syntactic rules. At scale, run them in a separate ESLint invocation or as a CI job so the fast syntactic pass stays under 10 seconds for editor feedback.

5.3 Authoritative eslint flat config

```

// eslint.config.ts
import eslint from '@eslint/js';
import tseslint from 'typescript-eslint';
import reactHooks from 'eslint-plugin-react-hooks';
import jsxA11y from 'eslint-plugin-jsx-a11y';
import vitest from '@vitest/eslint-plugin';
import playwright from 'eslint-plugin-playwright';
import testingLibrary from 'eslint-plugin-testing-library';
import importX from 'eslint-plugin-import-x';
import prettier from 'eslint-config-prettier';

export default tseslint.config(
  // 1 · Ignores – must be first, global
  {
    ignores: [
      '**/dist/**',
      '**/build/**',
      '**/coverage/**',
      '**/blob-report/**',
      '**/.next/**',
      '**/node_modules/**',
      '**/*.generated.*',
    ],
  },

  // 2 · Base JS – all files
  eslint.configs.recommended,

  // 3 · TypeScript – all TS/TSX files
  ...tseslint.configs.recommended,
  ...tseslint.configs.stylistic,
  {
    files: ['**/*.ts,tsx'],
    languageOptions: {
      parserOptions: {
        // projectService: no need to list tsconfig.json files
        projectService: true,
        tsconfigRootDir: import.meta.dirname,
      },
    },
    plugins: {
      'import-x': importX,
    },
    rules: {
      // Enforce type-only imports where possible – helps
      // isolatedModules
      '@typescript-eslint/consistent-type-imports': [
        'error',
        { prefer: 'type-imports', fixStyle: 'inline-type-imports' },
      ],
    },
  },
);

```

```

    '@typescript-eslint/no-unused-vars': [
      'error',
      { argsIgnorePattern: '^_', varsIgnorePattern: '^_', caughtError
s: 'none' },
    ],
    '@typescript-eslint/no-explicit-any': 'warn',
    'import-x/no-cycle': ['error', { maxDepth: 8 }],
    'import-x/no-duplicates': 'error',
  },
},
},

// 4 · Type-aware rules – only where they pay for themselves
// Run this as a separate CI job if wall-clock matters: `eslint --
config eslint.typeaware.config.ts`
{
  files: ['packages/*/src/**/*.{ts,tsx}'],
  extends: [...tseslint.configs.recommendedTypeChecked],
  rules: {
    '@typescript-eslint/no-floating-promises': 'error',
    '@typescript-eslint/no-misused-promises': [
      'error',
      { checksVoidReturn: { attributes: false } },
    ],
    '@typescript-eslint/await-thenable': 'error',
    '@typescript-eslint/no-unnecessary-condition': 'warn',
  },
},

// 5 · React / JSX
{
  files: ['**/*.tsx'],
  plugins: {
    'react-hooks': reactHooks,
    'jsx-ally': jsxAlly,
  },
  rules: {
    ...reactHooks.configs.recommended.rules,
    ...jsxAlly.configs.recommended.rules,
    // jsx-ally recommended includes many rules; tighten for prod:
    'jsx-ally/no-autofocus': 'warn',
  },
},

// 6 · Test files – relax rules that conflict with test ergonomics
{
  files: ['**/*.{test,spec}.*', '**/__tests__/**', '**/e2e/**'],
  plugins: {
    vitest,
    playwright,
    'testing-library': testingLibrary,
  },
},

```

```

    rules: {
      ...vitest.configs.recommended.rules,
      ...testingLibrary.configs['flat/recommended'].rules,
      '@typescript-eslint/no-explicit-any': 'off',
      '@typescript-eslint/no-non-null-assertion': 'off',
      'jsx-ally/click-events-have-key-events': 'off',
    },
  },
},

// 7 · Playwright E2E – playwright rules only on e2e files
{
  files: ['e2e/**/*.ts,tsx'],
  ...playwright.configs['flat/recommended'],
  rules: {
    'playwright/no-wait-for-timeout': 'error',
    'playwright/prefer-web-first-assertions': 'error',
    'playwright/no-force-option': 'warn',
  },
},

// 8 · Prettier must be last – disables formatting rules that
// conflict with prettier
prettier,
);

```

Operational notes:

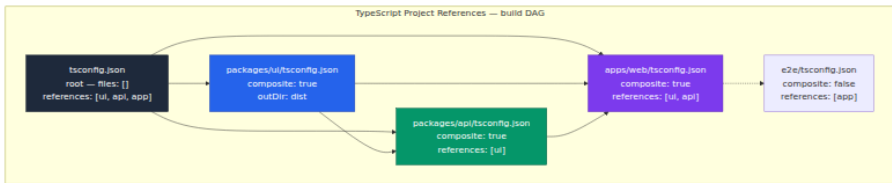
- `projectService: true` (via `tseslint`) replaces `parserOptions.project: ['./tsconfig.json']`. It discovers the owning `tsconfig.json` per file using TypeScript's own resolution, which is correct for project references and avoids the glob-enumeration bottleneck.
- Splitting type-aware rules into a separate config file (`eslint.typeaware.config.ts`) and running it as a parallel CI job cuts PR lint time from ~60s to ~15s for the fast path. The type-aware job can be `continue-on-error` on PRs and required on `main`.
- `eslint-config-prettier` (or `prettier` via `tseslint.config`) must be last so it disables `eslint` formatting rules. Do not run `prettier` as an ESLint rule — run it as a separate formatter.
- Use `npx eslint --print-config packages/ui/src/Button.tsx` to debug why a rule fires or does not fire.

6. TypeScript at Scale

TypeScript's value at scale is not syntax — it is the type checker as a distributed build system. The features that matter beyond a single package are project references, incremental builds, `isolatedModules`, and the interaction between type-aware lint and the checker.

6.1 Project references — the monorepo build graph

A single `tsconfig.json` that includes every file in the monorepo typechecks correctly but is serial and uncacheable: any file change invalidates the whole program. Project references partition the program into composite projects with explicit `references` edges. `tsc -b` (build mode) walks the DAG, builds leaves first, and skips projects whose inputs have not changed.



```
// tsconfig.json — root, no files of its own
{
  "files": [],
  "references": [
    { "path": "../packages/ui" },
    { "path": "../packages/api" },
    { "path": "../apps/web" }
  ]
}
```

```
// packages/ui/tsconfig.json – leaf project
{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": {
    "composite": true,           // required for referenced projects
    "declaration": true,        // emit .d.ts so dependents can
    // typecheck without source
    "declarationMap": true,      // jump-to-definition goes to source,
    // not .d.ts
    "outDir": "./dist",
    "rootDir": "./src",
    "incremental": true,         // per-project .tsbuildinfo
    "tsBuildInfoFile": "./dist/.tsbuildinfo"
  },
  "include": ["src/**/*"],
  "references": []
}
```

```
// packages/api/tsconfig.json – depends on ui
{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": {
    "composite": true,
    "declaration": true,
    "declarationMap": true,
    "outDir": "./dist",
    "rootDir": "./src",
    "incremental": true,
    "tsBuildInfoFile": "./dist/.tsbuildinfo"
  },
  "include": ["src/**/*"],
  "references": [{ "path": "../ui" }]
}
```

```
// tsconfig.base.json – shared options
{
  "compilerOptions": {
    "strict": true,
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "verbatimModuleSyntax": true,
    "isolatedModules": true,
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true,
    "resolveJsonModule": true,
    "declaration": true,
    "sourceMap": true,
    "noEmitOnError": false
  }
}
```

Build and check commands:

```
# Build all projects in DAG order, incrementally, with caching
tsc -b

# Verbose – see which projects are up-to-date vs rebuilding
tsc -b --verbose

# Force rebuild (e.g., after tsconfig change)
tsc -b --force

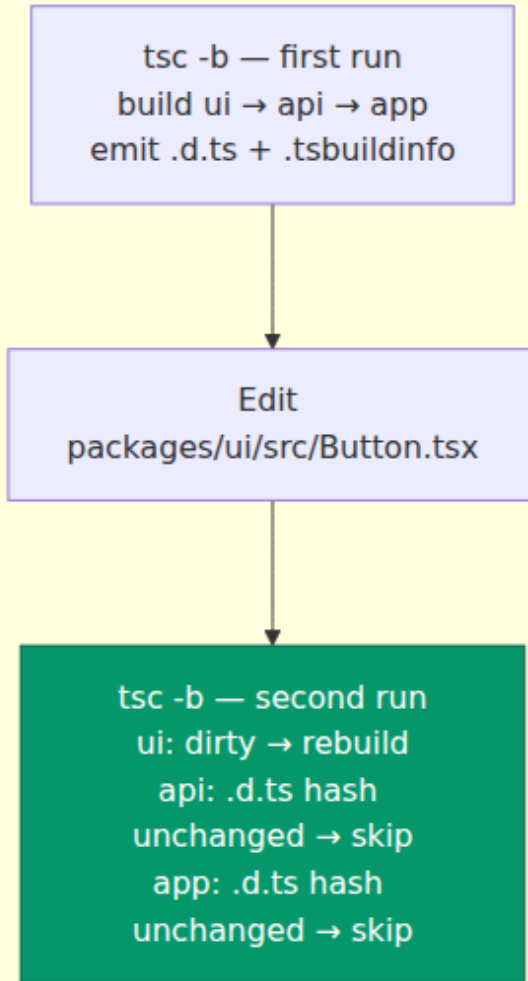
# Typecheck without emitting – still respects references (TS 5+)
tsc --noEmit
# or per-project: tsc -p packages/ui/tsconfig.json --noEmit
```

6.2 isolatedModules, incremental, and verbatimModuleSyntax

Three flags that are non-negotiable at scale:

Flag	What it enforces	Why it matters
<code>isolatedModules: true</code>	Each file must be independently transpilable without cross-file type info	Vite, esbuild, and swc transpile per-file in parallel. Without this, <code>const enum</code> , <code>export =</code> , and type-only elision behave differently between <code>tsc</code> and the bundler, causing prod-only bugs
<code>verbatimModuleSyntax: true</code>	<code>import type</code> / <code>export type</code> must be explicit; value imports are never erased	Prevents the bundler from silently dropping a value import that <code>tsc</code> thought was type-only, and makes the <code>consistent-type-imports</code> lint rule enforceable
<code>incremental: true</code> + <code>composite: true</code>	Emit <code>.tsbuildinfo</code> and <code>.d.ts</code> so dependents can skip re-checking	Without this, project references degrade to serial builds with no cache

Incremental Build — second run after one file changes



The `.tsbuildinfo` file stores the shape of every source file, its dependencies, and the emit output so the next `tsc -b` can skip projects whose inputs hash identically. On the second `tsc -b`, only the dirty project rebuilds — dependents whose `.d.ts` output hash has not changed are skipped. Cache `.tsbuildinfo` in CI (`actions/cache` keyed on

`pnpm-lock.yaml` + `tsconfig` hashes) to cut typecheck from 90 s to ~15 s on no-op PRs:

6.3 Type-aware lint as a second checker

Type-aware ESLint rules run the same type checker as `tsc`, but per file via `projectService`. They catch:

- `no-floating-promises` — an unhandled `Promise` that `tsc` allows but that loses errors at runtime.
- `no-misused-promises` — passing an `async` function where a `void` callback is expected (e.g., `array.forEach(async ...)`).
- `await-thenable` / `no-unnecessary-condition` — dead code and redundant guards.

Because they invoke the checker, they belong in the same cost bucket as `tsc -b`. Two viable strategies:

1. **Single job:** `tsc -b --noEmit` then `eslint` with type-aware rules. Simple, but wall-clock is sum of both.
2. **Parallel jobs:** `tsc -b` on one runner, `eslint` (type-aware) on another, both reading the same `.tsbuildinfo`. Faster, but requires `projectService` so `eslint` does not need to rebuild the program from scratch.

The second strategy is the backend pattern: parallelize independent checks that share a cache.

6.4 Editor and CI integration

For editors, `typescript-eslint` with `projectService` is fast enough for on-save feedback if type-aware rules are limited to the ~10 that catch real bugs. For CI, run the full `recommendedTypeChecked` set.

```
# CI – fast lint (no type info, <15s) – required
pnpm eslint .

# CI – type-aware lint (needs checker, ~30-60s) – required on main,
advisory on PR
pnpm eslint --config eslint.typeaware.config.ts .

# CI – typecheck (no emit, respects references)
pnpm tsc -b

# Local – single command that does all three, for pre-push
pnpm check # runs tsc -b && eslint . && eslint --config
eslint.typeaware.config.ts .
```

7. Visual Regression — Percy and Chromatic

Unit and E2E tests assert behavior; visual regression asserts pixels. A component can pass every DOM assertion and still render broken because of a CSS change three packages away. Visual regression closes that gap by screenshotting rendered UI and diffing against a baseline.

7.1 How it works

The pipeline is: render stories or pages (Storybook or a Playwright-driven preview URL, one URL per variant) → screenshot via the Percy agent or Chromatic CLI (per viewport and browser) → diff against the last accepted baseline stored in the cloud (with anti-aliasing ignored and a per-story threshold) → surface in a review UI that blocks the PR until a human approves or denies the diff.

Two dominant tools:

Tool	Rendering source	Diff model	Review flow	Best for
Chromatic	Storybook stories (or Playwright via <code>chromatic --playwright</code>)	Per-story snapshot, per-viewport, browser farm	GitHub check, per-story approve, auto-accept on <code>main</code>	Component libraries, design systems

Tool	Rendering source	Diff model	Review flow	Best for
Percy (BrowserStack)	Any URL (Storybook, deployed preview, Playwright)	Per-page snapshot, viewport matrix, cross-browser	GitHub check, per-snapshot approve	E2E pages, marketing sites, full-page regression

Both follow the same lifecycle: render → screenshot → upload → diff against baseline → report as a GitHub check → human reviews diffs with changed pixels highlighted → approve updates baseline.

7.2 Chromatic with Storybook

```
# Install
pnpm add -D chromatic

# Run – builds Storybook, uploads stories, diffs
npx chromatic --project-token=$CHROMATIC_PROJECT_TOKEN \
  --branch-name=$GITHUB_HEAD_REF \
  --exit-zero-on-changes false

# Playwright mode – no Storybook needed
npx chromatic --playwright --project-token=$CHROMATIC_PROJECT_TOKEN
```

```
// .storybook/preview.ts – stable visual tests
import type { Preview } from '@storybook/react';

const preview: Preview = {
  parameters: {
    chromatic: {
      // Per-story viewports – avoids combinatorial explosion
      viewports: [320, 768, 1280],
      // Delay snapshot until fonts and images settle
      delay: 500,
      // Ignore anti-aliasing noise
      diffThreshold: 0.02,
    },
  },
};

export default preview;
```


7.3 Percy with Playwright

```
// e2e/visual.e2e.ts
import { test } from '@playwright/test';
import percySnapshot from '@percy/playwright';

test('homepage visual baseline', async ({ page }) => {
  await page.goto('/');
  await page.waitForLoadState('networkidle');
  await percySnapshot(page, 'homepage', {
    widths: [375, 768, 1280],
    percyCSS: `.ad-banner { display: none; }`, // hide flaky regions
  });
});
```

```
# Percy wraps the test run
npx percy exec -- npx playwright test e2e/visual.e2e.ts
```

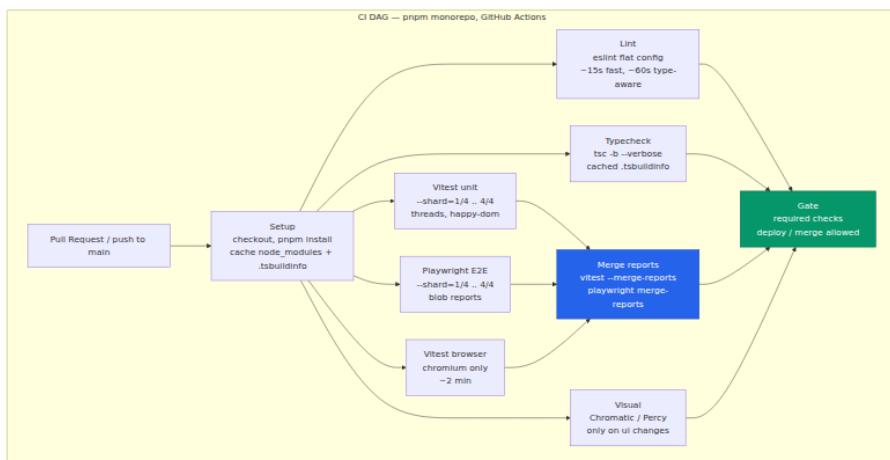
7.4 Operational rules

- **Scope narrowly.** Snapshot 200 stories, not 2000 pages. Prioritize the design system and the top 20 user-facing pages. Combinatorial viewports × browsers explodes cost and flake.
- **Stabilize before screenshotting.** Wait for `networkidle`, fonts loaded (`document.fonts.ready`), and animations settled. Use `chromatic.delay` or `page.waitForLoadState`.
- **Mask flake.** Hide timestamps, avatars, ads, and animated regions with `percyCSS` or `chromatic.diffIncludeAntiAliasing: false`. A visual suite that flakes on every run will be ignored.
- **Gate, don't block blindly.** Require visual review on PRs that touch `packages/ui` or `apps/web/styles`, advisory elsewhere. The goal is to catch CSS regressions early, not to gate every docs change.

8. CI Matrix — Shard, Merge, Gate

At scale, CI is a distributed system: a DAG of jobs that must be parallel, cacheable, and whose outputs merge deterministically. The pattern below keeps typecheck + lint + unit + component + E2E + visual under 10 minutes for a 4000-test monorepo.

8.1 The DAG



8.2 Sharding Vitest and Playwright

Both runners shard deterministically by hashing spec file paths, so shard 1/4 on one runner is disjoint from shard 2/4 on another without coordination.

```

# .github/workflows/ci.yml – excerpt, Vitest sharding
jobs:
  vitest:
    strategy:
      matrix:
        shard: [1, 2, 3, 4]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: pnpm
      - run: pnpm install --frozen-lockfile
      - name: Cache TS build info
        uses: actions/cache@v4
        with:
          path: |
            **/dist/.tsbuildinfo
            **/.tsbuildinfo
          key: tsbuild-${{ hashFiles('pnpm-lock.yaml',
'tsconfig*.json', 'packages/*/tsconfig.json') }}

      - name: Run Vitest shard ${{ matrix.shard }}/4
        run: pnpm vitest run --shard=${{ matrix.shard }}/4 --
reporter=blob --outputFile.blob=blob-report/vitest-$
${{ matrix.shard }}.blob

      - uses: actions/upload-artifact@v4
        with:
          name: vitest-blob-${{ matrix.shard }}
          path: blob-report/vitest-${{ matrix.shard }}.blob

  playwright:
    strategy:
      matrix:
        shard: [1, 2, 3, 4]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: pnpm
      - run: pnpm install --frozen-lockfile
      - run: npx playwright install --with-deps chromium

      - name: Run Playwright shard ${{ matrix.shard }}/4

```

```
run: pnpm playwright test --shard=${{ matrix.shard }}/4
env:
  BASE_URL: http://localhost:3000

- uses: actions/upload-artifact@v4
  if: always()
  with:
    name: playwright-blob-${{ matrix.shard }}
    path: blob-report/
- uses: actions/upload-artifact@v4
  if: failure()
  with:
    name: playwright-traces-${{ matrix.shard }}
    path: playwright-report/
```

8.3 Merging reports

Shards produce disjoint outputs that must merge before the gate job can evaluate them. Both runners support first-class merge.

```

merge-reports:
  needs: [vitest, playwright]
  runs-on: ubuntu-latest
  if: always()
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 22
        cache: pnpm
    - run: pnpm install --frozen-lockfile

    - name: Download all Vitest blobs
      uses: actions/download-artifact@v4
      with:
        pattern: vitest-blob-*
        path: blob-report/
        merge-multiple: true

    - name: Download all Playwright blobs
      uses: actions/download-artifact@v4
      with:
        pattern: playwright-blob-*
        path: blob-report/
        merge-multiple: true

    - name: Merge Vitest reports
      run: |
        pnpm vitest --merge-reports --reporter=junit --reporter=html
        # Produces reports/junit-vitest.xml + html report
        # Coverage is merged via c8/v8: vitest already wrote per-
shard coverage;
        # merge with: pnpm vitest run --coverage --merge-reports

    - name: Merge Playwright reports
      run:

npx playwright merge-reports ./blob-report --reporter html --reporter
junit

    - name: Publish merged HTML
      uses: actions/upload-artifact@v4
      with:
        name: merged-html-reports
        path: |
          html-report/
          playwright-report/

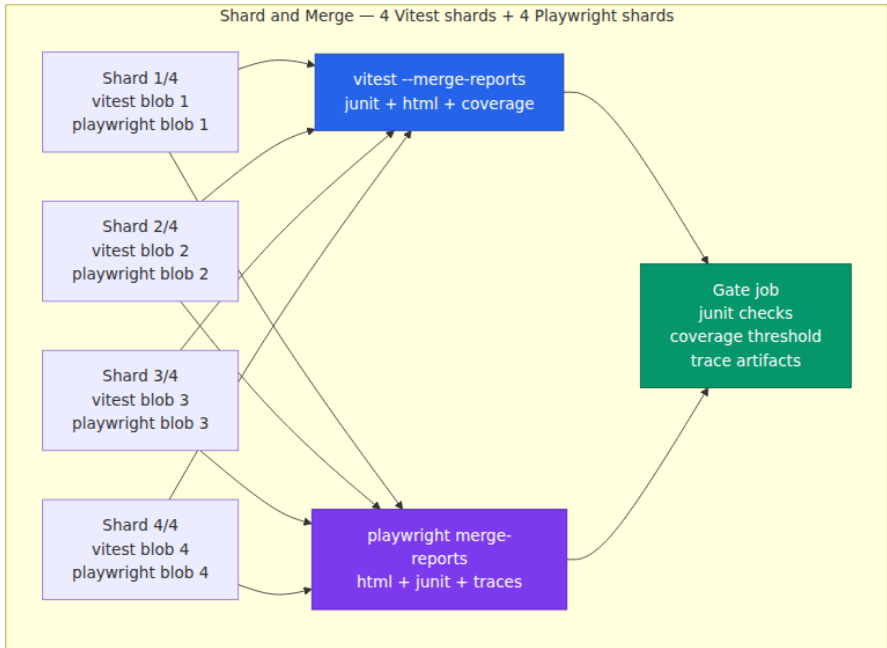
    - name: Publish JUnit for GitHub checks
      uses: mikepenz/action-junit-report@v4

```

```

with:
  report_paths: |
    reports/junit-vitest.xml
    playwright-report/junit.xml
  check_name: Tests
  fail_on_failure: true

```



Key details:

- `--reporter=blob` writes a binary report per shard that includes traces and attachments. Do not use `html` per shard — shards overwrite each other's HTML.
- Vitest's `--merge-reports` and Playwright's `merge-reports` are idempotent and order-independent. Both accept a directory of blobs.
- Coverage merging: `v8` coverage from shards is written per shard; `c8` report `--reporter=lcov` or `vitest --coverage --merge-reports` merges them. Emit `lcov.info` once for Codecov/Coveralls upload.
- JUnit XML is the lingua franca for CI checks. Both runners emit it; the gate job uploads it via `action-junit-report` so failures annotate the PR diff.

8.4 Gating and required checks

```
gate:
  needs: [lint, typecheck, merge-reports, visual]
  runs-on: ubuntu-latest
  if: always()
  steps:
    - name: Check all required jobs passed
      run: |
        # needs.*.result is success/skipped/failure
        if [[ "${{ needs.lint.result }}" != "success" ]]; then echo
"lint failed"; exit 1; fi
        if [[ "${{ needs.typecheck.result }}" != "success" ]]; then
echo "typecheck failed"; exit 1; fi
        if [[ "${{ needs.merge-reports.result }}" != "success" ]];
then echo "tests failed"; exit 1; fi
        # visual is advisory on PRs that don't touch ui
        echo "All gates passed"
```

Configure `gate` as the sole required check in branch protection. This is the backend pattern: a single aggregator job that fans in all shard results, so the PR status is one green or red check, not N shard checks that humans must interpret.

8.5 Caching that actually helps

Cache	Key	Hit rate	Savings
<code>pnpm store / node_modules</code>	<code>hashFiles('pnpm-lock.yaml')</code>	High on PRs	30-60 s install → 5 s
<code>.tsbuildinfo</code> per project	<code>hashFiles('pnpm-lock.yaml', 'tsconfig*.json')</code>	High on no-op PRs	90 s tsc -b → 15 s
Playwright browser binaries	<code>playwright version</code> + OS	High	60 s install → 5 s
Vite transform cache	<code>hashFiles('vite.config.ts', 'pnpm-lock.yaml')</code>	Medium	10 s Vitest cold start → 3 s

Cache	Key	Hit rate	Savings
ESLint cache (--cache)	<code>hashFiles('eslint.config.ts', 'pnpm-lock.yaml')</code>	High	15 s → 3 s on no- op

Always use `actions/cache` with `restore-keys` fallbacks so a partial hit still helps. Cache `blob-report` is never worth caching — it is per-run output.

9. Putting It Together — A Minimal Monorepo

The configs in this chapter are designed to work together. The dependency graph between tools is itself a DAG: TypeScript project references feed both ESLint (via `projectService`) and Vite (via path resolution); Vite feeds Vitest; Playwright and visual tools run independently but all converge on the CI gate.

Repository layout:


```

.
├─ tsconfig.json                # root  ─ files: [], references: [ui
├─ , api, app]
├─ tsconfig.base.json          # shared compilerOptions (isolatedMo
├─ dules, etc.)
├─ vitest.config.ts            # projects: [unit (happy-dom),
├─ browser (chromium)]
├─ playwright.config.ts        # projects: [chromium, firefox,
├─ webkit, setup]
├─ eslint.config.ts            # flat config  ─ ordered array, last
├─ match wins
├─ eslint.typeaware.config.ts  # type-aware rules  ─ parallel CI job
├─ packages/
├─ │ ─ ui/
├─ │ │ ─ tsconfig.json        # composite, references: []
├─ │ │ ─ src/Button.tsx
├─ │ ─ api/
├─ │ │ ─ tsconfig.json        # composite, references: [ui]
├─ │ │ ─ src/client.ts
├─ apps/web/
├─ │ ─ tsconfig.json          # composite, references: [ui, api]
├─ │ ─ src/App.tsx
├─ e2e/
├─ │ ─ tsconfig.json          # no composite, references: [app]
├─ │ ─ checkout.e2e.ts
├─ test/setup/
├─ │ ─ unit.ts                # vi.mock, jest-dom matchers
├─ │ ─ browser.ts              # browser setup
├─ .github/workflows/ci.yml    # shard matrix + merge-reports + gat
e

```

Local development – fast feedback

```

pnpm tsc -b --watch           # typecheck incrementally on save
pnpm eslint . --cache          # fast lint, cached
pnpm vitest --project unit     # watch mode, HMR-style invalidation
pnpm playwright test --ui
# Playwright UI mode – time-travel, trace, pick locator

```

Pre-push – full check

```

pnpm check                    # tsc -b && eslint . && eslint --config
                              eslint.typeaware.config.ts .
pnpm vitest run --coverage
pnpm playwright test

```

CI – sharded, merged, gated (see section 8)

Key takeaways

- The frontend test pyramid is unit (Vitest, pure) → component (Vitest + Testing Library, mounted) → E2E (Playwright, real browser), with visual regression as an orthogonal pixel layer. Push bugs as low in the pyramid as fidelity allows.
- Vitest wins by reusing Vite's transform. Default to `happy-dom` for speed, promote layout-sensitive suites to `jsdom` or `browser` mode, and size `threads` / `forks` pools to the machine. Use `projects` to run heterogeneous suites in one invocation.
- Coverage `v8` is correct for Vite+ESM; `istanbul` only when custom instrumentation is needed. Treat thresholds as per-project gates, not global vanity metrics. Snapshots are code — review them and normalize non-determinism.
- Playwright's primitive is the browser context, not the browser. Fixtures are dependency injection with `test` vs `worker` scope; `trace: 'on-first-retry'` plus the trace viewer is the primary debugger for flaky E2E. Shard with `--shard` and merge with `blob` + `merge-reports`.
- ESLint flat config is an ordered array — last match wins, no implicit cascading. Use `typescript-eslint` with `projectService` so type-aware rules discover `tsconfig.json` per file. Split type-aware lint into a parallel CI job to keep the fast path under 15 seconds.
- TypeScript at scale means project references (`composite: true`, `declaration: true`), `tsc -b` for DAG-ordered incremental builds, and `isolatedModules` + `verbatimModuleSyntax` so the bundler and checker agree. Cache `.tsbuildinfo` in CI and use it as the shared build cache between `tsc` and type-aware lint.
- Visual regression (Chromatic, Percy) screenshots stories or pages and diffs against a baseline. Scope to the design system and top pages, stabilize before screenshotting, mask flake, and gate only on relevant paths.
- CI is a DAG: shard Vitest and Playwright across N runners, merge `blob` reports into unified JUnit/HTML, publish traces as artifacts, and gate on a single aggregator job. Cache `node_modules`, `.tsbuildinfo`, and `eslint --cache` — everything else is per-run output.

Further reading

- Vitest documentation — Config, Projects, Browser Mode, Coverage, Reporters. <https://vitest.dev/config/> , <https://vitest.dev/guide/browser.html>
- Playwright documentation — Fixtures, Test Sharding, Trace Viewer, Blob Reporter. <https://playwright.dev/docs/test-fixtures> , <https://playwright.dev/docs/test-sharding> , <https://playwright.dev/docs/trace-viewer>
- ESLint Flat Config and typescript-eslint — Configuration Files, Project Service, Typed Linting. <https://eslint.org/docs/latest/use/configure/configuration-files> , <https://typescript-eslint.io/getting-started/typed-linting>
- TypeScript Project References and Build Mode. <https://www.typescriptlang.org/docs/handbook/project-references.html>
- TypeScript `isolatedModules` and `verbatimModuleSyntax` . <https://www.typescriptlang.org/tsconfig#isolatedModules> , <https://www.typescriptlang.org/tsconfig#verbatimModuleSyntax>
- Chromatic and Percy documentation. <https://www.chromatic.com/docs/> , <https://docs.percy.io/docs/percy-specific-functionality>
- GitHub Actions — Caching, Artifacts, Required Status Checks. <https://docs.github.com/en/actions/using-workflows/caching-dependencies-to-speed-up-workflows>

Chapter 12 — Production Frontend: Observability, Performance (Core Web Vitals, Lighthouse), and Deployment (CDN, Edge, RSC)

What this chapter covers: The last mile of frontend engineering — getting code to users fast, measuring whether it stays fast, and recovering when it does not. We begin with **Core Web Vitals** — LCP, CLS, and INP — the browser-native metrics that correlate with user engagement and search ranking, and explain how lab measurements (Lighthouse, WebPageTest) differ from field data (Chrome UX Report, RUM). We then build a

production observability stack: the `web-vitals` JavaScript library for metric collection, Sentry session replay for visual debugging, and OpenTelemetry RUM for end-to-end tracing that correlates frontend spans with backend traces. On the deployment side we cover CDN edge caching — cache-control headers, fingerprinting strategies, and long-term immutable assets — then push compute to the edge with Cloudflare Workers and Vercel Edge Functions. We trace React Server Component streaming from server to client, showing how the wire protocol delivers HTML and RSC payloads over a single connection. Finally we address rollback: canary deploys, feature flags, and progressive rollout at the edge, with concrete configs for Lighthouse CI, edge workers, and CDN cache rules.

Learning goals:

- Define LCP (Largest Contentful Paint), CLS (Cumulative Layout Shift), and INP (Interaction to Next Paint) — what each measures, how the browser computes it, and the threshold that separates "good" from "poor."
- Explain the lab-vs-field divide: why Lighthouse scores on a developer laptop never match Chrome UX Report data from real users, and when to trust each.
- Instrument a production app with the `web-vitals` JavaScript library, Sentry session replay, and OpenTelemetry RUM to produce a correlated observability pipeline.
- Configure CDN cache headers — `Cache-Control`, `stale-while-revalidate`, content hashing, and immutable long-term caching — with the trade-offs of each strategy.
- Deploy edge logic with Cloudflare Workers and Vercel Edge Functions, understanding the V8 isolate model and its limits compared to serverless containers.
- Trace React Server Component streaming: the RSC wire protocol, `use` on the client, and how partial HTML hydration works over a streaming HTTP response.
- Design canary rollouts and rollback strategies for frontend deployments — percentage-based routing, feature flags, and version-pinned caches.
- Integrate Lighthouse CI into a CI/CD pipeline to gate merges on performance regressions.

1. Core Web Vitals — The Metrics That Matter

Google's Core Web Vitals are a set of three metrics that measure the user-perceived quality of a web page. They are not arbitrary benchmarks — each one is tied to a specific phase of the page lifecycle, and Google has publicly stated that they influence search ranking. As a backend engineer, think of them as SLIs for your frontend: user-facing, measurable, and actionable.

1.1 Largest Contentful Paint (LCP)

What it measures: The time from when the user begins loading the page to when the largest visible content element (image, video poster, or block-level text) finishes rendering in the viewport.

Why it matters: LCP is the closest browser-native metric to "the page feels loaded." It correlates with the user's perception that content has arrived, as opposed to TTFB (which measures server response) or FCP (which only measures the first pixel).

Eligible elements (in the LCP candidate hierarchy): 1. `<img>` elements (including `<svg>` inside `<img>`) 2. `<video>` poster images 3. CSS `background-image` on block-level elements 4. `<text>` elements in SVG 5. Block-level text nodes containing inline content

Thresholds:

Rating	LCP (mobile)	LCP (desktop)
Good	≤ 2.5 s	≤ 2.5 s
Needs improvement	2.5–4.0 s	2.5–4.0 s
Poor	> 4.0 s	> 4.0 s

What delays LCP (and what to do):

Delay source	Mechanism	Mitigation
Slow server response (TTFB)	Backend compute, DB queries, cold start	Edge compute, CDN, warm starts

Delay source	Mechanism	Mitigation
Render-blocking resources	<code><link rel="stylesheet"></code> , synchronous <code><script></code>	<code>media="print"</code> swap, <code>async</code> / <code>defer</code> , critical CSS inlining
Slow resource load	Large images, no preload	<code><link rel="preload"></code> for hero image, responsive <code>srcset</code> , modern formats (WebP/AVIF)
Client-side rendering	SPA shell loads empty, JS hydrates	Server-side rendering (see Vol 19, Ch 9 — SSR), streaming SSR, or static generation
Layout shift delaying paint	Ads, dynamic embeds push LCP element down	Reserve space with aspect-ratio or min-height

LCP timing diagram:

1.2 Cumulative Layout Shift (CLS)

What it measures: The sum of all unexpected layout shifts that occur during the entire lifespan of the page. A layout shift is "unexpected" if it happens without user interaction (click, tap, keypress) and involves elements that move more than a threshold.

Why it matters: Layout shift is the metric most correlated with user frustration — you click a button and a banner loads above it, pushing the button down. The user accidentally taps the wrong thing.

The CLS score formula:

$$\text{layout_shift_score} = \text{impact_fraction} \times \text{distance_fraction}$$

- **impact_fraction:** The total area of unstable elements relative to the viewport.
- **distance_fraction:** How far the unstable element moved relative to the viewport height.

CLS is the sum of these scores across the page session, but Chrome applies **session windows** (a maximum 5-second gap between shifts,

capped at the page lifetime) and reports the **maximum session window** — not the lifetime total. This prevents a long-lived tab from accumulating a misleadingly high score.

Thresholds:

Rating	CLS
Good	≤ 0.1
Needs improvement	0.1–0.25
Poor	> 0.25

Common culprits and fixes:

Shift source	Mechanism	Fix
Images without dimensions	Browser doesn't know size until loaded	Always set <code>width</code> and <code>height</code> (or <code>aspect-ratio</code>); the browser reserves space
Dynamic content injection	Banners, ads, cookie notices push content down	<code>min-height</code> or placeholder skeletons with fixed size
Web fonts causing FOIT/ FOUT	FOUT swaps font after load, changing line heights	<code>font-display: optional</code> or <code>font-display: swap</code> with size-adjusted fallbacks
Late-loading CSS/JS	Additional styles change layout after initial paint	Inline critical CSS, defer non-critical

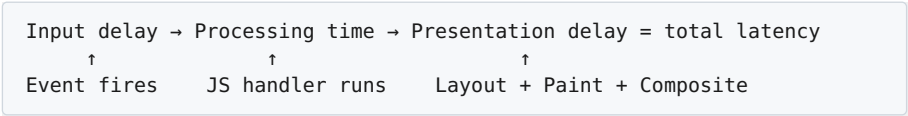
1.3 Interaction to Next Paint (INP)

What it measures: The maximum (or near-maximum) latency of any interaction during the page session. An "interaction" is a pair of `keydown` / `keyup`, `mousedown` / `mouseup`, or `pointerdown` / `pointerup` events. INP measures the time from the event to when the browser paints the next frame after processing the event handler.

Why it matters: INP replaced First Input Delay (FID) in March 2024 because FID only measured the *first* interaction and only measured input

delay (not processing time or render time). INP captures the full latency of *every* interaction, which better represents responsiveness.

The INP timeline for one interaction:



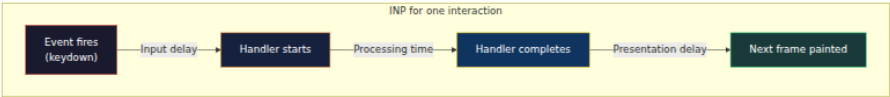
- **Input delay:** Time between the event and when the main thread is available to run the handler. If the main thread is busy with another task, this grows.
- **Processing time:** Time the event handler's JavaScript runs. Long tasks (Figure 2 — Event Loop, Chapter 2) are the primary cause.
- **Presentation delay:** Time between handler completion and the next painted frame. Large DOM mutations or expensive layout/paint work increase this.

Thresholds:

Rating	INP
Good	≤ 200 ms
Needs improvement	200–500 ms
Poor	> 500 ms

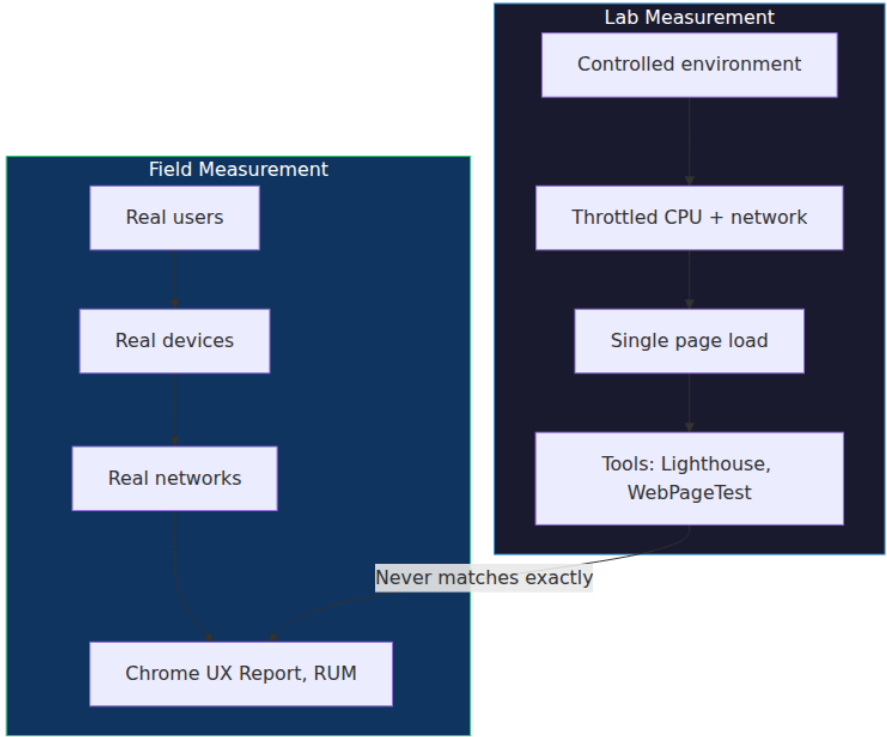
What hurts INP:

Problem	Mechanism	Mitigation
Long JavaScript tasks	Synchronous rendering, large state updates	Break work into chunks (<code>scheduler.yield()</code>), <code>requestIdleCallback</code> , <code>useTransition</code> in React
Layout thrashing	Reading <code>offsetHeight</code> after writing styles	Batch reads before writes, use <code>ResizeObserver</code>
Heavy event handlers	Complex React state updates on click	Debounce, defer to microtask, virtualize lists



1.4 Lab vs. Field — Why the Numbers Differ

A backend engineer is familiar with this distinction: synthetic benchmarks (lab) vs. production traffic (field). Core Web Vitals have the same split.



Dimension	Lab (Lighthouse)	Field (CrUX / RUM)
Environment	Developer machine or CI server, throttled to simulate slow 3G + 4x CPU throttling	Real devices (Moto G, iPhone 13, Pixel 7), real networks (4G, 3G, Wi-Fi)

Dimension	Lab (Lighthouse)	Field (CrUX / RUM)
Sample size	One page load (or a few)	Thousands to millions of sessions over 28-day rolling window
What it's good for	Debugging, regression detection, pre-merge gating	Understanding real user experience, identifying slow pages
What it misses	Variability across devices, networks, geographies, concurrent usage	Cannot inspect DOM, replay interactions, or test specific scenarios
Metric precision	Exact, reproducible	Statistical: 75th percentile over 28 days

Chrome UX Report (CrUX) is Google's field data pipeline: Chrome periodically reports page load metrics for users who opt in to usage statistics. It provides 75th-percentile (p75) values for LCP, CLS, INP, FCP, TTFB, and TTFB per origin or per URL. CrUX data is available via: - PageSpeed Insights API - `chrome-ux-report` BigQuery dataset - The `CrUX API` (per-origin, per-metric)

When to use each:

- **Lab (Lighthouse):** In CI/CD to gate merges. If LCP regresses by 20%, block the PR. In development to debug a slow page.
- **Field (CrUX):** In dashboards to track production health. If p75 LCP exceeds 2.5 s for 3 consecutive weeks, file an issue.
- **RUM (your own):** For granular data CrUX cannot give — per-page, per-user-segment, per-geography, correlated with error rates and backend traces.

2. Observability — The Full Stack from Browser to Backend

2.1 The `web-vitals` JavaScript Library

The `web-vitals` library (maintained by Google) provides a consistent API for measuring LCP, CLS, INP, FCP, TTFB, and Interaction to Next Paint. It uses `PerformanceObserver` internally and handles edge cases (late-loading

images, session windows, attribution data) that raw PerformanceObserver entries do not.

```
// npm install web-vitals
import { onLCP, onCLS, onINP, onFCP, onTTFB } from 'web-vitals';

function sendToAnalytics(metric: {
  name: string;
  value: number;
  rating: 'good' | 'needs-improvement' | 'poor';
  delta: number;
  entries: PerformanceEntry[];
  id: string;
  navigationType: 'navigate' | 'reload' | 'back-forward' | 'prerender';
}) {
  // Example: send to your analytics endpoint
  const body = JSON.stringify({
    name: metric.name,
    value: metric.value,
    rating: metric.rating,
    delta: metric.delta,
    id: metric.id,
    navigationType: metric.navigationType,
    // Attribution data for debugging
    ...(metric.name === 'LCP' && metric.entries.length > 0 && {
      element: (metric.entries[0] as LargestContentfulPaint).element?.tag
    })),
    url: (metric.entries[0] as LargestContentfulPaint).url,
  });

  // Use sendBeacon for reliability – survives page unload
  if (navigator.sendBeacon) {
    navigator.sendBeacon('/api/vitals', body);
  } else {
    fetch('/api/vitals', { body, method: 'POST', keepalive: true });
  }
}

// Register observers – each fires once per metric session window
onLCP(sendToAnalytics); // Largest Contentful Paint
onCLS(sendToAnalytics); // Cumulative Layout Shift
onINP(sendToAnalytics); // Interaction to Next Paint
onFCP(sendToAnalytics); // First Contentful Paint
onTTFB(sendToAnalytics); // Time to First Byte
```

Key design decisions in this snippet:

1. **sendBeacon for reliability.** Page unload events (`beforeunload`) kill in-flight `fetch` requests. `sendBeacon` queues the payload in the browser's networking queue and guarantees delivery even after navigation. This is the same durability guarantee you would get from writing to a WAL before responding — the telemetry is persisted before the process exits.
2. **Rating per metric.** The library classifies each metric into good/needs-improvement/poor based on the thresholds from §1. This lets you filter dashboards and alert on "poor" ratings specifically.
3. **Attribution mode.** For LCP, the attribution data tells you *which element* was the LCP candidate and *which resource* caused the delay. Without attribution, you know LCP is slow but not why. The attribution callback provides `element`, `url`, `timeToFirstByte`, `resourceLoadDelay`, `resourceLoadTime`, and `elementRenderDelay` — breaking the LCP timeline into segments.
4. **delta vs. value.** `value` is the absolute metric value. `delta` is the change since the last report for this metric name. For CLS, `delta` is what matters — it tells you whether the layout shifted *just now* or was already shifted from before.

2.2 Sentry Session Replay

Sentry's session replay captures a DOM recording of a user's session — a visual playback of what they saw and did. This is invaluable for debugging because you can *see* the layout shift, the slow interaction, the error — not just read a metric number.

```
import * as Sentry from '@sentry/react';

Sentry.init({
  dsn: 'https://examplePublicKey@o0.ingest.sentry.io/0',
  integrations: [
    Sentry.replayIntegration({
      // Mask sensitive inputs
      maskAllInputs: true,
      // Block external iframes
      blockAllMedia: true,
      // Only record sessions that had an error
      onErrorSampleRate: 1.0,
      // 10% of normal sessions
      replaysSessionSampleRate: 0.1,
      // Always record if an error occurs
      replaysOnErrorSampleRate: 1.0,
    }),
  ],
  // Correlate with backend traces
  tracesSampleRate: 0.1,
});
```

The replay pipeline:

Privacy: `maskAllInputs` replaces user-entered text with `*` characters. `blockAllMedia` prevents recording of `<img>` and `<video>` elements that may contain PII. The Sentry replay library uses a custom serialization format, not raw DOM snapshots, which reduces upload size and avoids leaking HTML structure.

Correlation: When Sentry captures an error *during* a replay session, the error event links to the replay. You can click the error, open the replay, and watch the user's session from 5 seconds before the error to understand what led to it. This is analogous to tailing logs around a backend error — but visual.

2.3 OpenTelemetry RUM — Full-Stack Tracing

OpenTelemetry (OTel) provides a vendor-neutral API for distributed tracing. The `opentelemetry-web` SDK collects browser spans (navigation, resource loads, user interactions) and exports them to a collector that merges them with backend traces — giving you end-to-end visibility from the browser click to the database query.

```

// npm install @opentelemetry/sdk-trace-web
//             @opentelemetry/api
//             @opentelemetry/exporter-trace-otlp-http

import { WebTracerProvider } from '@opentelemetry/sdk-trace-web';
import { BatchSpanProcessor } from '@opentelemetry/sdk-trace-base';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http';
import { Resource } from '@opentelemetry/resources';
import { ATTR_SERVICE_NAME } from '@opentelemetry/semantic-conventions';
import { registerInstrumentations } from '@opentelemetry/instrumentation';
import { DocumentLoadInstrumentation } from '@opentelemetry/instrumentation-document-load';
import { FetchInstrumentation } from '@opentelemetry/instrumentation-fetch';
import { XMLHttpRequestInstrumentation } from '@opentelemetry/instrumentation-xml-http-request';

const provider = new WebTracerProvider({
  resource: new Resource({
    [ATTR_SERVICE_NAME]: 'my-frontend',
  }),
});

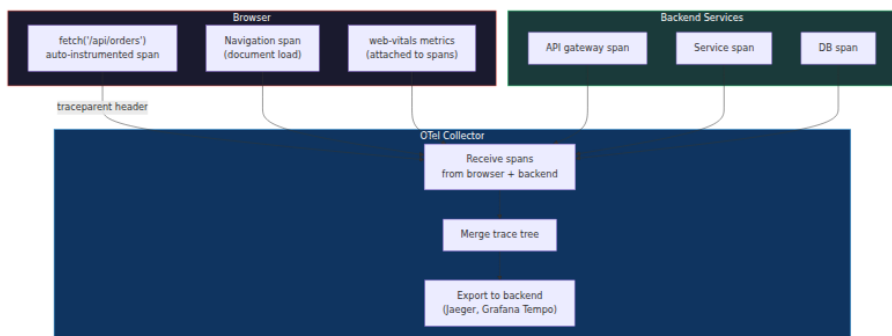
// Batch and export to OTel collector
provider.addSpanProcessor(
  new BatchSpanProcessor(
    new OTLPTraceExporter({
      url: 'https://otel-collector.internal:4318/v1/traces',
    })
  )
);

// Auto-instrument document load, fetch, and XMLHttpRequest
registerInstrumentations({
  instrumentations: [
    new DocumentLoadInstrumentation(),
    new FetchInstrumentation(),
    new XMLHttpRequestInstrumentation(),
  ],
});

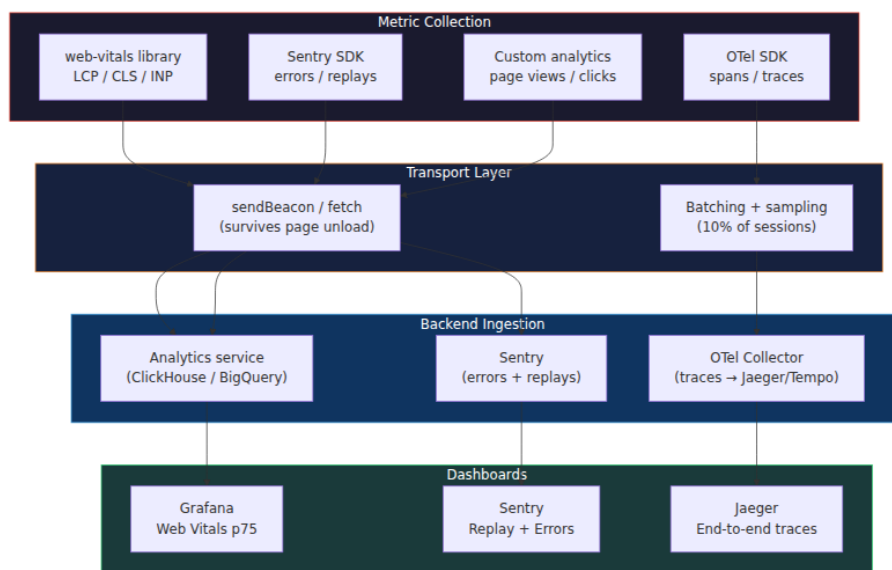
provider.register();

// Now every `fetch()` call automatically creates a span
// with trace context propagated via `traceparent` header,
// correlating frontend and backend traces

```



The RUM collection pipeline:



RUM sampling: You cannot send telemetry from 100% of sessions — the overhead (network, CPU, storage) is prohibitive. The standard approach is deterministic sampling: assign each session a hash, and send telemetry for sessions where `hash(session_id) % 100 < sample_rate`. For high-traffic sites, 1-10% is typical. For error replays, `onErrorSampleRate: 1.0` ensures you capture every error session regardless of the general sample rate.

3. Lighthouse and Chrome UX Report

3.1 Lighthouse — Lab Measurement

Lighthouse is a tool that loads a page in a headless Chrome instance with throttled CPU (4x slowdown) and network (simulated 3G), measures performance metrics, and produces a scored report. It runs entirely in the lab — no real users involved.

What Lighthouse measures beyond Core Web Vitals:

Metric	Description
Total Blocking Time (TBT)	Lab proxy for INP — sum of long tasks (>50 ms) during the load phase
Speed Index	How quickly the page visually loads (painted pixels over time)
Largest Contentful Paint (LCP)	Same as field, but in lab conditions
Cumulative Layout Shift (CLS)	Same as field, but in lab conditions
First Contentful Paint (FCP)	Time to first painted pixel
Time to Interactive (TTI)	When the main thread is quiet enough for input response

Lighthouse CI in a CI/CD pipeline:


```

# .github/workflows/lighthouse.yml
name: Lighthouse CI

on:
  pull_request:
    branches: [main]

jobs:
  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install dependencies
        run: npm ci

      - name: Build
        run: npm run build

      - name: Serve and run Lighthouse
        uses: treosh/lighthouse-ci-action@v12
        with:
          urls: |
            http://localhost:3000/
            http://localhost:3000/dashboard
          configPath: ./lighthouseci.json
          uploadArtifacts: true

      - name: Assert performance budgets
        run: |
          npx lhci autorun --assertions \
            "largest-contentful-paint:warn" \
            "cumulative-layout-shift:error" \
            "interactive:warn" \
            "total-blocking-time:error"

```

```
// lighthouserc.json
{
  "ci": {
    "collect": {
      "numberOfRuns": 3,
      "startServerCommand": "npm run preview",
      "startServerReadyPattern": "Local:",
      "url": "http://localhost:4173/"
    },
    "assert": {
      "assertions": {
        "largest-contentful-paint": ["error", { "maxNumericValue":
2500 }],
        "cumulative-layout-shift": ["error", { "maxNumericValue": 0.1 }
],
        "interactive": ["warn", { "maxNumericValue": 3500 }],
        "total-blocking-time": ["error", { "maxNumericValue": 200 }],
        "resource-summary:script:size": ["warn", { "maxNumericValue": 3
00000 }],
        "resource-summary:total-byte-size": ["warn", { "maxNumericValue
": 1000000 }]
      }
    },
    "upload": {
      "target": "lhci",
      "serverBaseUrl": "https://your-lhci-server.internal"
    }
  }
}
```

Why `numberOfRuns: 3`: Lighthouse results vary between runs due to system load and random network jitter. Running 3 times and taking the median filters out outliers. This is the same statistical technique you would use for any benchmark: multiple runs, discard min/max.

Performance budgets: The `assertions` block in `lighthouserc.json` defines hard limits. If any metric exceeds the budget, the CI step fails and the PR cannot merge. This is your frontend SLO enforced at deploy time — analogous to API latency SLOs in backend CI.

3.2 Chrome UX Report (CrUX)

CrUX is Google's field dataset, collected from real Chrome users who have opted in to usage statistics. It provides 28-day rolling p75 values for Core Web Vitals, available per origin or per page.

Access methods:

Method	Granularity	Use case
CrUX API (REST)	Per-origin, monthly	Dashboard, automated monitoring
CrUX BigQuery	Per-page, daily	Deep analysis, segment breakdown
PageSpeed Insights	Per-page (uses CrUX + Lighthouse)	Debugging, one-off checks
<code>gathering_strategy</code> in <code>crux-api</code>	Per-page, monthly	Production dashboards

CrUX data structure:

```
{
  "record": {
    "key": {
      "origin": "https://example.com",
      "url": "https://example.com/product/123"
    },
    "metrics": {
      "largest_contentful_paint": {
        "histogram": [
          { "start": 0, "end": 1000, "density": 0.52 },
          { "start": 1000, "end": 2500, "density": 0.31 },
          { "start": 2500, "end": 4000, "density": 0.12 },
          { "start": 4000, "density": 0.05 }
        ],
        "percentiles": { "p75": 2100 }
      },
      "cumulative_layout_shift": {
        "histogram": [
          { "start": 0, "end": 0.1, "density": 0.68 },
          { "start": 0.1, "end": 0.25, "density": 0.22 },
          { "start": 0.25, "density": 0.10 }
        ],
        "percentiles": { "p75": 0.08 }
      }
    }
  }
}
```

The p75 metric: CrUX reports the 75th percentile — meaning 75% of real users experienced a metric value *at or below* this number. The p75 is chosen because it is sensitive enough to detect regressions (unlike p50,

which misses tail slowness) but not so sensitive that a few slow sessions dominate the signal (unlike p95 or p99). For a backend engineer, this is the same trade-off you make when choosing between median and p99 latency for SLIs.

4. Deployment — CDN, Edge, and RSC

4.1 CDN Edge Caching

A CDN (Content Delivery Network) caches your static assets at edge nodes close to users. For frontend, this is the single highest-impact optimization: a 500 KB JavaScript bundle served from a CDN node 10 ms away loads 10x faster than one served from your origin server 100 ms away.

Cache-Control header strategies:

Strategy	Header	Use case	Risk
Immutable long-term	<code>Cache-Control: public, max-age=31536000, immutable</code>	Content-hashed assets (<code>main.a1b2c3.js</code>)	None — URL changes when content changes
Revalidation	<code>Cache-Control: public, max-age=3600, must-revalidate</code>	API responses, non-hashed assets	Must have origin server for revalidation
Stale-while-revalidate	<code>Cache-Control: public, max-age=3600, stale-while-revalidate=86400</code>	Semi-dynamic content, HTML shell	User may see stale content for up to SWR window
No cache	<code>Cache-Control: no-store</code>	Sensitive data, auth responses	Every request hits origin

Fingerprinting: The `immutable` strategy requires content-hashed filenames. Your build tool (Vite, Webpack, RSPack) generates filenames like `main.a1b2c3d4.js` where `a1b2c3d4` is a hash of the file contents. When the content changes, the hash changes, the URL changes, and the browser fetches the new file. Old cached files are never invalidated —

they simply become unreferenced. This is analogous to immutable artifacts in a CI/CD pipeline: you never modify an artifact in place, you publish a new version.

```
# nginx CDN configuration for frontend assets
# Hashed assets – immutable, long-term cache
location ~* \.[a-f0-9]{8}\.(js|css|woff2|avif|webp)$ {
    add_header Cache-Control "public, max-age=31536000, immutable";
    add_header Vary "Accept-Encoding";
    gzip on;
    gzip_types text/css application/javascript font/woff2;
    brotli on;
    brotli_types text/css application/javascript font/woff2;
}

# HTML shell – short cache, revalidate
location = /index.html {
    add_header Cache-Control "public, max-age=60, stale-while-
revalidate=3600";
    add_header Vary "Accept-Encoding";
}

# API responses – no cache
location /api/ {
    add_header Cache-Control "no-store";
    proxy_pass http://backend;
}
```

4.2 Edge Compute — Cloudflare Workers and Vercel Edge

CDNs cache. Edge compute *runs code* at the edge node. This is the frontend equivalent of running your API gateway at the edge — you can do request routing, A/B testing, authentication, and even server-side rendering without hitting your origin.

Cloudflare Workers use V8 isolates — not containers, not VMs. An isolate is a lightweight execution environment with a ~5 ms cold start (vs. ~100–500 ms for a Lambda container). The trade-off: Workers have a 128 MB memory limit and 30-second CPU time limit (free tier) or 30 seconds (paid). They cannot run native binaries or access the filesystem.

```

// Cloudflare Worker – edge A/B testing and cache headers
export default {
  async fetch(request, env) {
    const url = new URL(request.url);

    // Serve static assets from KV (Cloudflare's edge key-value store)
    if (url.pathname.match(/\. (js|css|woff2|avif|webp|png|jpg|svg)$/))
    {
      const asset = await env.ASSETS.get(url.pathname, 'arrayBuffer');
      if (asset) {
        return new Response(asset, {
          headers: {
            'Content-Type': getContentType(url.pathname),
            'Cache-Control': 'public, max-age=31536000, immutable',
            'CDN-Cache-Control': 'max-age=31536000',
          },
        });
      }
    }

    // A/B test: route 50% of users to variant
    const bucket = await env.AB_BUCKET.get(request.headers.get('cf-ray'));
    const isVariant = bucket && parseInt(bucket, 16) % 2 === 0;

    if (isVariant) {
      url.hostname = 'variant.example.com';
    }

    // Forward to origin with trace context
    const response = await fetch(url.toString(), {
      headers: {
        ...Object.fromEntries(request.headers),
        'x-ab-variant': isVariant ? 'B' : 'A',
      },
    });

    // Add security and performance headers
    const newHeaders = new Headers(response.headers);
    newHeaders.set('X-Frame-Options', 'DENY');
    newHeaders.set('X-Content-Type-Options', 'nosniff');
    newHeaders.set('Referrer-Policy', 'strict-origin-when-cross-origin');

    if (url.pathname === '/') {
      newHeaders.set('Cache-Control', 'public, max-age=60, stale-while-revalidate=3600');
    }

    return new Response(response.body, {

```

```

        status: response.status,
        headers: newHeaders,
    });
},
};
};

function getContentType(pathname) {
    const ext = pathname.split('.').pop();
    const types = {
        js: 'application/javascript',
        css: 'text/css',
        woff2: 'font/woff2',
        avif: 'image/avif',
        webp: 'image/webp',
        png: 'image/png',
        jpg: 'image/jpeg',
        svg: 'image/svg+xml',
    };
    return types[ext] || 'application/octet-stream';
}

```

Vercel Edge Functions run on the same V8 isolate runtime (via the `edge-runtime` package). They integrate with Next.js's `middleware.ts` and `edge` runtime:

```

// middleware.ts - Vercel Edge
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export const config = {
    matcher: ['/dashboard/:path*', '/api/:path*'],
};

export function middleware(request: NextRequest) {
    // Edge-level auth check
    const token = request.cookies.get('session-token');
    if (!token && request.nextUrl.pathname.startsWith('/dashboard')) {
        return NextResponse.redirect(new URL('/login', request.url));
    }

    // A/B test via header
    const variant = request.cookies.get('ab-variant') || 'A';
    const response = NextResponse.next();
    response.headers.set('x-variant', variant);
    return response;
}

```

Edge vs. serverless containers:

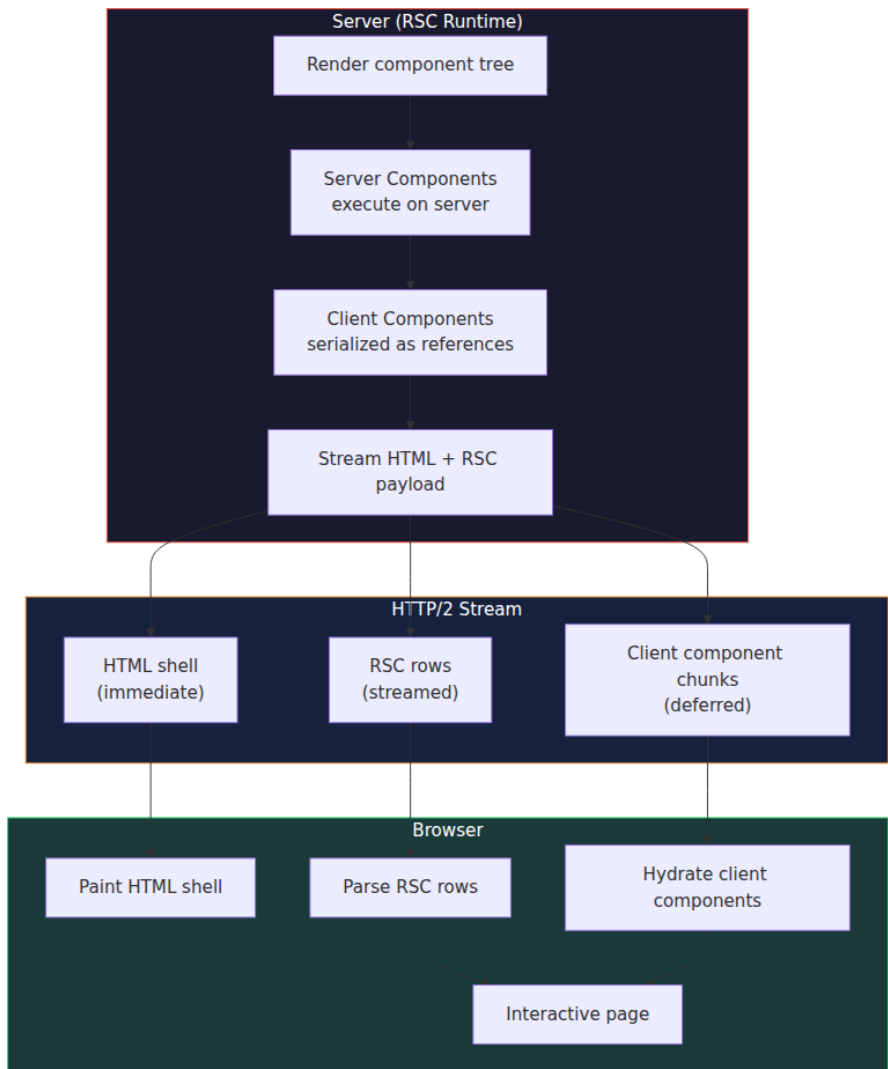
Dimension	Edge (V8 isolates)	Serverless containers (Lambda, Cloud Run)
Cold start	~1-5 ms	~100-500 ms
Memory limit	128 MB	10 GB
CPU limit	30 s (continuous)	15 min
Network	50+ edge locations	1-3 regions
State	KV/R2 (eventual consistency)	Full DB access
Best for	Routing, caching, SSR, auth	Heavy compute, DB queries, ML inference

4.3 React Server Component (RSC) Streaming

React Server Components (RSC) render on the server and stream HTML + RSC payload to the client. The key insight: the server does *not* send a full HTML document and then a separate hydration bundle. Instead, it streams a hybrid response — initial HTML for the shell, then RSC payloads for deferred components, all over a single HTTP connection.

The RSC wire protocol:

1. **Server renders** the component tree. Server Components execute entirely on the server; Client Components are serialized as references.
2. **HTML stream** begins immediately — the shell (layout, header, nav) is sent first so the browser can start painting.
3. **RSC payload** is appended as a series of "rows" in a custom format (not HTML, not JSON — a streaming binary protocol). Each row represents a component boundary, a suspense boundary resolution, or a reference to a client component.
4. **Client hydration** reads the RSC payload, resolves client component references, and attaches event handlers — but does *not* re-render server components.



Server Component that streams:

```

// app/dashboard/page.tsx – Server Component by default
// This runs on the server, NOT in the browser
async function DashboardPage() {
  // Server-side data fetch – no loading spinner, no waterfall
  const orders = await fetchOrders(); // runs during render, streamed
  to client
  const user = await getUser();        // parallel with orders

  return (
    <main>
      <h1>Welcome, {user.name}</h1>

      {/* This Client Component hydrates after the RSC payload arrives
*/}
      <Suspense fallback={<OrderListSkeleton />}>
        <OrderList initialData={orders} />
      </Suspense>

      {/* Another server-rendered section – no client JS needed */}
      <section>
        <h2>Recent activity</h2>
        <ActivityFeed data={user.recentActivity} />
      </section>
    </main>
  );
}

```

```
// components/OrderList.tsx - Client Component (interactive)
'use client';

import { useState } from 'react';

export function OrderList({ initialData }: { initialData: Order[] }) {
  const [filter, setFilter] = useState<string>('all');

  const filtered = filter === 'all'
    ? initialData
    : initialData.filter(o => o.status === filter);

  return (
    <div>
      <select value={filter} onChange={e => setFilter(e.target.value)}>
        <option value="all">All</option>
        <option value="pending">Pending</option>
        <option value="shipped">Shipped</option>
      </select>
      {filtered.map(order => (
        <div key={order.id}>{order.id} ☐ {order.status}</div>
      ))}
    </div>
  );
}
```

Why RSC streaming matters for performance:

The old model (SSR + hydration) required the server to render the *entire* page, send it as HTML, then send a *second* payload (the JavaScript bundle) for hydration. The user saw the page quickly (good LCP) but could not interact with it until hydration completed (bad INP).

RSC streaming solves this: - **Shell paints immediately** (good LCP) — the HTML shell arrives without waiting for data. - **Data streams in** — server components resolve on the server and stream their output. No client-side `useEffect` fetch, no loading spinners for above-the-fold content. - **Client components hydrate incrementally** — only the components that need interactivity get hydrated, and they hydrate as soon as their RSC chunk arrives. - **No full hydration waterfall** — the browser does not need to download and execute the entire client bundle before any interaction works.

This is analogous to streaming responses from a backend API: instead of waiting for the full response to buffer, you send chunks as they become

available. The frontend equivalent is streaming HTML and RSC rows instead of buffering the entire page.

5. Cache Headers in Depth

5.1 The Cache-Control Directive Matrix

Directive	Meaning	Frontend use
<code>public</code>	Any cache may store the response	CDN, browser, proxy
<code>private</code>	Only the user's browser may cache	Authenticated pages, personal data
<code>no-store</code>	No cache may store the response	Sensitive data, auth tokens
<code>no-cache</code>	Cache may store but must revalidate before use	HTML shell (revalidate on every load)
<code>max-age=N</code>	Fresh for N seconds from response time	Static assets
<code>s-maxage=N</code>	Fresh for N seconds in shared (CDN) cache	CDN-specific TTL
<code>immutable</code>	Response will never change (do not revalidate)	Content-hashed assets
<code>stale-while-revalidate=N</code>	Serve stale for up to N seconds while revalidating in background	Semi-dynamic content
<code>must-revalidate</code>	Never serve stale; always revalidate	Default behavior when <code>max-age</code> expires

5.2 Long-Term Caching Strategy

The ideal for static assets: **immutable + content hashing + `max-age=31536000`**. This means:

1. The build produces `main.a1b2c3d4.js`.
2. The CDN caches it for 1 year.

3. The browser caches it for 1 year.
4. When you deploy a new version, the build produces `main.e5f6g7h8.js` — a different URL.
5. Old cached files are never invalidated, never revalidated, never refetched.

The risk: If you use non-hashed filenames (`main.js`) with long `max-age` , browsers will serve the old version until the cache expires. This is the "zombie cache" problem — users see stale code with no way to force an update. Content hashing eliminates this entirely.

The HTML shell problem: Your `index.html` cannot be content-hashed (it is the entry point). It needs a short `max-age` with `stale-while-revalidate` :

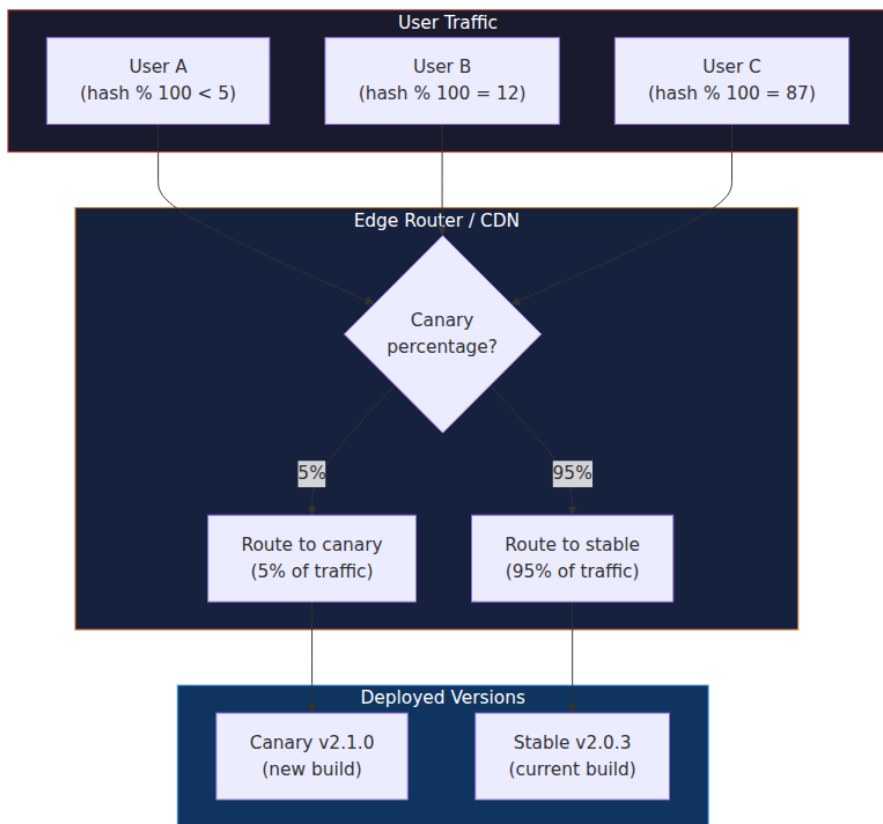
```
Cache-Control: public, max-age=60, stale-while-revalidate=3600
```

This means: the CDN serves the cached HTML for up to 1 hour (stale), but every 60 seconds it revalidates in the background. Users always get a fast response (cache hit), and after at most 60 seconds they get the updated HTML pointing to the new hashed bundles.

6. Canary Deployments and Rollback

6.1 Canary Rollout Topology

A canary deploy sends a small percentage of traffic to the new version before rolling out to everyone. For frontend, this is typically done at the CDN or edge layer — the edge worker or CDN configuration routes requests based on a hash of the user ID or session.



Cloudflare Workers canary implementation:

```

// Edge worker with canary routing
export default {
  async fetch(request, env) {
    const url = new URL(request.url);

    // Hash the user's IP or a cookie for deterministic routing
    const userId = request.headers.get('cf-connecting-ip') || request.headers.get('x-forwarded-for') || '';
    const hash = await hashString(userId);
    const bucket = hash % 100;

    // Canary: 5% of users get the new version
    const canaryThreshold = parseInt(await env.CANARY_THRESHOLD.get('threshold') || '5');

    if (bucket < canaryThreshold) {
      // Canary – serve from canary asset bucket
      const asset = await env.CANARY_ASSETS.get(url.pathname, 'arrayBuffer');
      if (asset) {
        return new Response(asset, {
          headers: {
            'Cache-Control': 'public, max-age=31536000, immutable',
            'X-Served-By': 'canary',
          },
        });
      }
    }

    // Stable – serve from production asset bucket
    const asset = await env.PROD_ASSETS.get(url.pathname, 'arrayBuffer');
    if (asset) {
      return new Response(asset, {
        headers: {
          'Cache-Control': 'public, max-age=31536000, immutable',
          'X-Served-By': 'stable',
        },
      });
    }

    // Fallback to origin
    return fetch(request);
  },
};

async function hashString(str) {
  const encoder = new TextEncoder();
  const data = encoder.encode(str);
  const hashBuffer = await crypto.subtle.digest('SHA-256', data);

```

```
const hashArray = Array.from(new Uint8Array(hashBuffer));
return hashArray.reduce((acc, byte) => (acc * 31 + byte) % 1000000,
0);
}
```

6.2 Rollback Strategies

Strategy	Mechanism	Time to rollback	Trade-off
CDN version swap	Update CDN origin pointer to previous build	Seconds	Requires CDN API access
Edge worker threshold	Set <code>CANARY_THRESHOLD</code> to 0	Seconds	Canary traffic sees old version until KV propagates
Git revert + redeploy	Revert commit, trigger CI/CD	Minutes	Full rebuild, but clean history
Feature flag kill switch	Disable feature in flag service	Seconds	Requires flag service integration
Immutable asset revert	Point HTML shell at old hashed filenames	Minutes	Requires rebuild with old filenames

The critical insight for frontend rollback: Because static assets are content-hashed and immutable, rolling back means serving the old HTML shell (which references the old hashed bundles). The old bundles are still in the CDN cache (or can be re-fetched from the origin). This is why the `stale-while-revalidate` strategy for the HTML shell is so important — it lets you swap the shell in seconds without purging CDN caches.

Rollback without downtime:

Normal Deployment

Deploy v2.1.0

HTML shell points to
main.e5f6g7h8.js

Canary 5% → v2.1.0
Stable 95% → v2.0.3

regression detected

Rollback (set threshold to 0)

Edge worker stops
routing canary

All traffic → v2.0.3

Old HTML shell
still cached at edge

7. Lighthouse CI Integration

7.1 Full CI/CD Pipeline

```

# .github/workflows/frontend-deploy.yml
name: Frontend Deploy

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install and build
        run: |
          npm ci
          npm run build

      - name: Lighthouse CI
        uses: treosh/lighthouse-ci-action@v12
        with:
          urls: |
            http://localhost:4173/
          configPath: ./lighthouseci.json

      - name: Upload build artifacts
        run: |
          # Upload hashed assets to R2/S3 for CDN
          aws s3 sync dist/ s3://my-cdn-assets/ \
            --cache-control "public, max-age=31536000, immutable" \
            --exclude "index.html"

          # Upload HTML shell with short cache
          aws s3 cp dist/index.html s3://my-cdn-assets/index.html \
            --cache-control "public, max-age=60, stale-while-
revalidate=3600"

      - name: Update edge worker canary
        run: |
          # Set canary threshold to 5% for new version
          npx wrangler kv:key put --binding=CONFIG threshold 5

      - name: Monitor canary
        run: |
          # Wait 10 minutes, check error rates
          sleep 600
          ERROR_RATE=$(curl -s "https://analytics.internal/api/error-
rate?version=canary&window=10m" | jq '.rate')
          if (( $(echo "$ERROR_RATE > 0.01" | bc -l) )); then
            echo "Canary error rate too high: $ERROR_RATE"

```

```
        npx wrangler kv:key put --binding=CONFIG threshold 0
        exit 1
    fi

- name: Full rollout
  if: success()
  run: |
    npx wrangler kv:key put --binding=CONFIG threshold 100
```

7.2 Performance Budget Enforcement

```

// scripts/assert-vitals.ts
// Run after Lighthouse CI to check RUM data against budgets
import { chromium } from 'playwright';

const BUDGETS = {
  LCP: 2500,    // ms
  CLS: 0.1,    // score
  INP: 200,    // ms
  FCP: 1800,   // ms
  TTFB: 800,   // ms
} as const;

async function assertPerformance(url: string) {
  const browser = await chromium.launch();
  const page = await browser.newPage();

  const metrics: Record<string, number> = {};

  await page.goto(url, { waitUntil: 'networkidle' });

  // Extract Core Web Vitals from the page
  const vitals = await page.evaluate(() => {
    return new Promise<Record<string, number>>>((resolve) => {
      const results: Record<string, number> = {};

      // Use PerformanceObserver to get final metrics
      const observer = new PerformanceObserver((list) => {
        for (const entry of list.getEntries()) {
          if (entry.entryType === 'largest-contentful-paint') {
            results.LCP = entry.startTime;
          }
          if (entry.entryType === 'layout-shift' && !(entry as any).hadRecentInput) {
            results.CLS = (results.CLS || 0) + (entry as any).value;
          }
        }
      });

      observer.observe({ type: 'largest-contentful-paint', buffered: true });
      observer.observe({ type: 'layout-shift', buffered: true });

      // Give the page a moment to finalize metrics
      setTimeout(() => resolve(results), 3000);
    });
  });

  console.log('Measured metrics:', vitals);

  let failed = false;

```

```

for (const [metric, budget] of Object.entries(BUDGETS)) {
  const actual = vitals[metric];
  if (actual !== undefined && actual > budget) {
    console.error(`FAIL: ${metric} = ${actual.toFixed(0)} (budget: ${
{budget}})`);
    failed = true;
  }
}

if (failed) {
  process.exit(1);
}

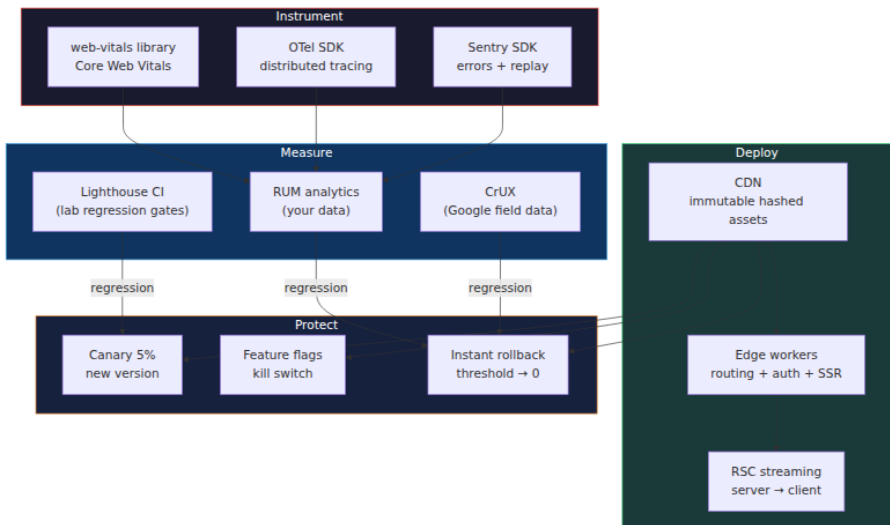
console.log('All metrics within budget');
await browser.close();
}

assertPerformance(process.argv[2] || 'http://localhost:4173/');

```

8. Putting It All Together — The Production Frontend Stack

A backend engineer will recognize this architecture as the same layered approach you use for backend observability and deployment: instrument, measure, deploy safely, rollback quickly.



The production frontend feedback loop:

1. **Instrument** — `web-vitals`, OTel, and Sentry collect metrics, traces, and errors from real user sessions.
2. **Measure** — Dashboards show p75 LCP, CLS, INP per page, per geography, per device. CrUX data validates your RUM data against Google's independent measurement.
3. **Gate** — Lighthouse CI runs on every PR, asserting performance budgets. A regression in LCP > 2500 ms blocks the merge.
4. **Deploy** — Hashed static assets go to the CDN with immutable cache headers. The HTML shell gets `stale-while-revalidate`. Edge workers handle routing, auth, and A/B testing.
5. **Canary** — 5% of users get the new version. Error rates and Web Vitals are monitored in real time.
6. **Rollback** — If metrics degrade, the edge worker threshold is set to 0 in seconds. No CDN cache purge needed — the old assets are already cached.

This is the same deployment discipline you apply to backend services: canary, monitor, rollback. The difference is that frontend rollback is faster (CDN cache is already warm with the old version) and the blast radius is smaller (no database migrations to undo).

Key Takeaways

- **Core Web Vitals are SLIs for your frontend.** LCP measures perceived load time, CLS measures visual stability, and INP measures interactivity. The thresholds (2.5 s, 0.1, 200 ms) are not arbitrary — they are tied to user engagement data at Google scale.
- **Lab and field measurements serve different purposes.** Use Lighthouse in CI to gate merges; use CrUX and RUM in dashboards to monitor production health. Never conflate the two.
- **The `web-vitals` library is the right abstraction.** It handles session windows, attribution, and edge cases that raw `PerformanceObserver` does not. Always send telemetry via `sendBeacon` to survive page unload.

- **Content-hashed immutable assets with long `max-age` are the gold standard.** The HTML shell is the only asset that needs short cache + `stale-while-revalidate`. Everything else is immutable.
 - **Edge compute (V8 isolates) replaces serverless containers for latency-sensitive logic.** A/B testing, auth checks, and SSR at the edge give you 1–5 ms cold starts and 50+ locations.
 - **RSC streaming eliminates the hydration waterfall.** Server components render on the server and stream their output; client components hydrate incrementally. This is streaming responses for the frontend.
 - **Canary deploys with instant rollback are non-negotiable.** Set a threshold, monitor metrics, and roll back in seconds. The old version is already cached at the edge.
-

Further Reading

- [Web Vitals — web.dev](https://web.dev/) — Official documentation for Core Web Vitals
- [web-vitals library — GitHub](https://github.com/web-vitals/web-vitals) — Google's JavaScript library for measuring Core Web Vitals
- [Chrome UX Report — developer.chrome.com](https://developer.chrome.com/ux-reports/) — Field data from real Chrome users
- [Lighthouse CI — GitHub](https://github.com/GoogleChrome/lighthouse-ci) — CI integration for Lighthouse
- [Sentry Session Replay — docs.sentry.io](https://docs.sentry.io/) — Visual replay of user sessions
- [OpenTelemetry JS — opentelemetry.io](https://opentelemetry.io/) — Browser instrumentation for distributed tracing
- [Cloudflare Workers — developers.cloudflare.com](https://developers.cloudflare.com/workers/) — Edge compute on V8 isolates
- [Vercel Edge Functions — vercel.com/docs](https://vercel.com/docs) — Edge compute for Next.js
- [React Server Components — react.dev](https://react.dev/) — Official RSC documentation
- [Cache-Control — MDN Web Docs](https://developer.mozilla.org/en-US/docs/Web/HTTP/Cache/directive) — HTTP cache directive reference
- [Lighthouse Performance Scoring — web.dev](https://web.dev/lighthouse-performance-scoring/) — How Lighthouse calculates scores